Acknowledgement

As I bring this project idea to fulfilment, I am sincerely appreciative for the support, direction, and encouragement I have received throughout this academic journey. From the formulation of concepts to the development of this proposal, I have been fortunate to benefit from the support of many exceptional individuals whose contributions have been important. First and foremost, I would want to convey my profound gratitude to my lecturers at ESU, Kandy, whose constant dedication, competence, and assistance have substantially expanded my comprehension of the subject matter. I am particularly thankful to Ms. Rasika, whose mentoring has been crucial in developing my approach to this research and assuring the quality of my proposal. I would also like to offer my appreciation to my classmates and peers, whose participation, constructive criticism, and encouragement have inspired me to develop my ideas and confront obstacles with confidence. Your support has been a tremendous source of inspiration during this journey. I am extremely grateful to my family, whose constant conviction in my talents and ongoing encouragement have given me with the strength and confidence to pursue my academic aspirations. Your affection and backing have been the cornerstone of all my triumphs. Finally, I would like to acknowledge London Metropolitan University for providing the curriculum and modules that have guided my study, and ESU, Kandy, for offering the facilities, resources, and academic environment necessary to produce this proposal. I also convey my thanks to everyone who has contributed, directly or indirectly, to my intellectual and personal development during this endeavor.

Contents

Course Submission Cover Sheet ..... 1 Module: xxx ..... 1 Assignment no: 001 ..... 1 Weigh ng: 100% of the module mark 1

Deadline: 3PM Wednesday the xxth of January 2025 1

Acknowledgement 8

1. Introduction 17

1.1 Background of the Smart Event Management and Ticketing System 17

1.2 Purpose of the System 17

1.3 Scope of the System ..... 18 1.4 Target Users..... 18
2. System Requirement Analysis 20
2.1 Functional Requirements 20
2.1.1 Home Page Features 20
2.1.2 Member Features 20
2.1.3 Guest Features 21
2.1.4 Admin Features ..... 21 2.1.5 Staff Features 21
2.2 Non-Functional Requirements..... 22
2.2.1 Security 22
2.2.2 Performance 22
2.2.3 Usability 22
2.2.4 Scalability 22
2.3 System Use Case Description 23
2.4 User Roles and Permissions 24
3. Database Design - Oracle 26
3.1 Database Overview 26
3.2 Entity Identification 28
3.3 ER Diagram with Explanation 32
3.4 Database Relationships 33
3.5 Data Dictionary 34
3.6 SQL Developer Data Modeler 41
3.6.1 Logical Model (zoom for clarity) 41
3.6.2 Physical Model (zoom for clarity) 42
3.6.3 Table Conversion Output 42
4. Database Implementation - SQL Server 44
4.1 Database Schema Implementation 44
4.2 Table Creation Scripts 46

4.3 Data Models with Constraints 52
4.4 Insert Statements (Sample Data Population) 64
4.5 Select Queries and Data Retrieval ..... 88 4.6 Database Execution 92
5. Web Application Development – ASP.NET MVC 103
5.1 System Architecture Overview 103
5.2 Project Structure (MVC Pattern) 104
5.3 Home Page 105
5.4 Member Module 108
5.4.1 Member Registration 108
5.4.2 Member Login 110
5.4.3 Browse & Search Events 111
5.4.4 Ticket Booking System 115
5.4.5 Review Submission 117
5.4.5 Inquiry Submission 118
5.4.7 Profile Page 119
5.4.8 My Bookings Page: 119
5.5 Guest Module 121
5.5.1 Browse Events 121
5.5.2 Browse Event Reviews 122
5.5.3 Restricted Event Search 122
5.5.4 Inquiry Submission 123
5.6 Admin Module 124
5.6.1 Admin Homepage 124
5.6.2 Add/Update Venues 125
5.6.3 Add/Update Categories 126
5.6.4 Add/Update Events..... 126
5.6.5 Review Moderation 128
5.6.8 Inquiries Handling 129

5.7 Staff Module 129

5.7.1 Verify Check-ins 130

5.8 UI Design and Navigation Flow 130

6. Testing and Validation 132

6.1 Testing Strategy 132

6.2 Functional Testing 132

6.3 Database Testing 134

6.4 Test Cases Table ..... 135 7 Reflection and Further Discussion 156

7.1 Challenges During Development..... 156

7.2 Lessons Learned 156

7.3 System Limitations. 157

7.4 Potential Future Improvements 158

8.Conclusion. ..... 159 9. Appendix 160

& Navigation Flow .....Error! Bookmark not defined.

Table of Figures

Figure 1: Usecase diagram ..... 23 Figure 2: ERD diagram ..... 32 Figure 3: Logical model ..... 41 Figure 4: Relational model ..... 42 Figure 5: Table conversion output ..... 43 Figure 6: SQL Server ..... 44 Figure 7: SQL Server Interface ..... 44 Figure 8: Table creation script - User ..... 46 Figure 9: Table creation script -Venue ..... 46 Figure 10:Table creation script - Category ..... 47 Figure 11:Table creation script -Event ..... 47 Figure 12:Table creation script -Event category ..... 48 Figure 13:Table creation script -Event session ..... 48 Figure 14:Table creation script -Ticket type ..... 49 Figure 15:Table creation script - Booking..... 49 Figure 16:Table creation script -Booked

ticket ..... 50 Figure 17:Table creation script -Payment ..... 50 Figure 18:Table creation script -Review ..... 51 Figure 19:Table creation script -Inquiry ..... 51 Figure 20: Data model - User ..... 52 Figure 21:Data model - User..... 53
Figure 22:Data model - Venue ..... 53 Figure 23:Data model - Category ..... 54 Figure 24: Data model - Event..... 54 Figure 25:Data model - Event ..... 55 Figure 26:Data model - Event category ..... 55 Figure 27: Data model - Event session 56
Figure 28: Data model - Event session ..... 56 Figure 29:Data model - Ticket type ..... 57 Figure 30:Data model - Booked ticket ..... 57 Figure 31:Data model - Booking ..... 58 Figure 32: Data model - Booking ..... 58 Figure 33: Data model - Payment ..... 59 Figure 34: Data model - Review ..... 59 Figure 35:Data model - Inquiry ..... 60 Figure 36: Data model - Inquiry ..... 60 Figure 37: DbSeeder File ..... 64 Figure 38: DbSeeder File 65
Figure 39: DbSeeder File 65
Figure 40: DbSeeder File 66
Figure 41: DbSeeder File ..... 66 Figure 42: DbSeeder File ..... 67 Figure 43: DbSeeder File 67
Figure 44: DbSeeder File 68
Figure 45: DbSeeder File ..... 68 Figure 46: DbSeeder File ..... 69 Figure 47: DbSeeder File 69
Figure 48: DbSeeder File ..... 70 Figure 49: DbSeeder File ..... 70 Figure 50: DbSeeder File 71
Figure 51: DbSeeder File 71
Figure 52: DbSeeder File 72
Figure 53: DbSeeder File 72
Figure 54: DbSeeder File 73
Figure 55: DbSeeder File ..... 73 Figure 56: DbSeeder File ..... 74 Figure 57: DbSeeder File 74

Figure 58: DbSeeder File 75
Figure 59: DbSeeder File ..... 75 Figure 60: Output Table. ..... 76 Figure 61:Output Table ..... 76 Figure 62: Output Table. 77
Figure 63: Output Table. 77
Figure 64: Output Table. 78
Figure 65: Output Table. 78
Figure 66: Output Table. 79
Figure 67: Output Table. ..... 79 Figure 68:Output Table. ..... 80 Figure 69: Output Table. 80
Figure 70: Output Table. 81
Figure 71: Output Table. ..... 81 Figure 72: INSERT data. ..... 82 Figure 73: Output Table. ..... 82 Figure 74: INSERT data. ..... 83 Figure 75:Output Table ..... 83 Figure 76: INSERT data. ..... 84 Figure 77:Output Table ..... 84 Figure 78: INSERT data. 85
Figure 79: Output Table ..... 85 Figure 80: INSERT data. ..... 86 Figure 81:Output Table ..... 86 Figure 82: INSERT data. 87
Figure 83: Output Table ..... 87 Figure 84:Select Queries and Data Retrieval 88
Figure 85:Select Queries and Data Retrieval 88
Figure 86:Select Queries and Data Retrieval 89
Figure 87:Select Queries and Data Retrieval 89
Figure 88:Select Queries and Data Retrieval 90
Figure 89:Select Queries and Data Retrieval 90
Figure 90:Select Queries and Data Retrieval ..... 91 Figure 91: Database creation ..... 92 Figure 92: Update connection string ..... 93 Figure 93: Create model classes ..... 93 Figure 94: Update ApplicationDbContext ..... 94 Figure 95:: Update ApplicationDbContext 94
Figure 96:: Update ApplicationDbContext 95
Figure 97:: Update ApplicationDbContext ..... 95 Figure 98: Update ApplicationDbContext 96

Figure 99: Update ApplicationDbContext ..... 96 Figure 100: Update ApplicationDbContext 97
Figure 101: Update ApplicationDbContext 97
Figure 102: Update ApplicationDbContext 98
Figure 103: Update ApplicationDbContext 98
Figure 104: Update ApplicationDbContext 99
Figure 105: Update ApplicationDbContext ..... 99 Figure 106: Run migrations ..... 100 Figure 107:Update database 100
Figure 108:Update database ..... 100 Figure 109Update database ..... 101 Figure 110:Update database 101
Figure 111:Update database 102
Figure 112:Update database ..... 102 Figure 113:ASP.NET MVC ..... 104 Figure 114:Homepage 105
Figure 115:Homepage 106
Figure 116:Homepage ..... 106 Figure 117: Homepage..... 107 Figure 118: Member Registration 108
Figure 119: Member Registration ..... 109 Figure 120: Member Login ..... 110 Figure 121:Events ..... 111 Figure 122:Filter events ..... 112 Figure 123: Event Details ..... 114 Figure 124: Selecting tickets ..... 115 Figure 125: Booking - before payment ..... 115 Figure 126: Booking - after payment ..... 116 Figure 127: Review submission - if not attended the event. ..... 117 Figure 128: Review submission - if attended the event ..... 117 Figure 129:Inquiry Submission ..... 118 Figure 130: Profile page ..... 119 Figure 131: Summary of bookings ..... 119 Figure 132: Booking details ..... 120 Figure 133: Browse events ..... 121 Figure 134: Browse event reviews 122
Figure 135: Restricted search ..... 122 Figure 136: Inquiry submission ..... 123 Figure 137: Admin homepage ..... 124 Figure 138: Add/update venues ..... 125 Figure 139: Add/update categories ..... 126 Figure 140: Add/update events ..... 126 Figure 141: Filter events

..... 127 Figure 142: Review Moderation ..... 128 Figure 143: Inquiries handling ..... 129 Figure 144: Verify check-ins ..... 130 Table of Tables

Table 1: User roles and permissions. ..... 25 Table 2: Data dictionary - User ..... 34 Table 3:Data dictionary - Venue ..... 35 Table 4:Data dictionary - Category ..... 35 Table 5:Data dictionary - Event ..... 36 Table 6:Data dictionary -Event category ..... 36 Table 7: Data dictionary - Event session ..... 36 Table 8: Data dictionary - Ticket type..... 37 Table 9:Data dictionary - Booking ..... 37 Table 10:Data dictionary -Booked Tickets ..... 38 Table 11: Data dictionary - Payment ..... 38 Table 12: Data dictionary - Review ..... 39 Table 13: Data dictionary - Inquiry ..... 40 Table 14: Testcase Table 01 ..... 135 Table 15: Testcase Table 02. ..... 136 Table 16: Testcase Table 03 138

Table 17: Testcase Table 04 139
Table 18: Testcase Table 05 140
Table 19: Testcase Table 06 141
Table 20: Testcase Table 07 142
Table 21: Testcase Table 08 143
Table 22: Testcase Table 09 144
Table 23: Testcase Table 10 146
Table 24: Testcase Table 11 148
Table 25: Testcase Table 12 149
Table 26: Testcase Table 13 150
Table 27: Testcase Table 14 151
Table 28: Testcase Table 15 152
Table 29: Testcase Table 16 153
Table 30: Testcase Table 17 154
Table 31: Testcase Table 18 155

1. Introduction

1.1 Background of the Smart Event Management and Ticketing System

In recent years, cultural organizations, councils, and event-hosting institutions have relied more on digital platforms to manage events and engage the public. Music concerts, theatrical

performances, dance showcases, exhibitions, and workshops all require careful organization of venues, timetables, ticket sales, and audience control. Traditional manual or semi-digital systems, such as paper-based ticketing and fragmented web promotion, frequently result in inefficiencies, data discrepancies, and less operational visibility.

The Smart Event Management and Ticketing System is designed to address these issues by offering a centralized and structured digital solution. The system enables the whole management of cultural events, from planning and publication to booking, attendance verification, and post-event evaluation. By combining different event-related activities into a single web-based platform, the solution decreases administrative workload while increasing accuracy and transparency.

Furthermore, the system emphasizes dependability and security through robust database design and role-based access management. By explicitly defining duties for guests, members, staff, and administrators, the platform ensures that sensitive procedures like ticket buying, payment confirmation, and event management are handled in a regulated and safe environment.
This approach promotes effective cultural event administration in a new digital setting.

1.2 Purpose of the System

The Smart Event Management and Ticketing System's major goal is to create an integrated platform for organising, managing, and participating in cultural events. The system intends to make event information more accessible to the public while also providing structured tools for managing bookings, sessions, and attendance.

Another important goal is to provide accurate and secure ticket booking and verification. By enforcing business rules at both the application and database levels, the system guarantees that ticket availability is honored, bookings are consistent, and attendance is accurate. This eliminates the possibility of overbooking, unauthorized admission, and data errors at live events.

In addition, the system is intended to improve communication and participation between event organizers and attendees. Inquiries and reviews enable consumers to communicate with organisers and provide feedback, while moderating methods ensure content quality. Overall, the system contributes to increased operational efficiency, transparency, and customer happiness.

1.3 Scope of the System

The Smart Event Management and Ticketing System addresses the entire lifecycle of cultural event management. This section addresses venue registration, event development, session scheduling, ticket type definition, booking management, attendance verification, and feedback gathering. The system also keeps track of users, inquiries, and reviews to aid administrative decision-making.

The system allows users to take on numerous roles with well defined permissions. Guests can explore events and send queries, members can view pricing and purchase tickets, staff can confirm attendance at event venues, and administrators can manage system data and content. Each role interacts with the system using interfaces specific to their tasks.

The system is developed as a web-based application that can be accessed using common web browsers. While it does not yet allow native mobile applications or real-time payment gateway connection, the architecture is intended to support future upgrades and scalability without requiring large structural modifications.

1.4 Target Users

Guests are unregistered users who utilize the system without authentication. They can look through future and past events, examine event specifics including venue, category, date, and description, and send general inquiries. However, guests are unable to check ticket prices or order tickets, promoting registration while safeguarding transactional functionality.

Members are registered users who have access to enhanced features. Members can see ticket prices, purchase tickets for available event sessions, manage their booking history, and provide reviews for events they've attended. All users that register through the system are automatically allocated to the member role, ensuring controlled and consistent access.

Staff users are internal operational users who manage event-related activities. Their major responsibility is to conduct attendee check-in at event venues using secure verification mechanisms. Staff access is purposely restricted to attendance verification functions in order to maintain security and prevent unauthorised data access.

Administrators are in charge of managing and maintaining the system as a whole. They can set up and administer locations, events, categories, and sessions, as well as moderate reviews, respond to requests, and manage internal user accounts. Administrator access is limited to authorized persons to maintain system integrity and governance.

1.4 Overview of Technologies used

The Smart Event Management and Ticketing System is built using ASP.NET MVC, which adheres to the Model-View-Controller architectural paradigm. This paradigm encourages explicit separation of business logic, user interface, and data management, hence boosting maintainability and scalability. The MVC framework also allows for clear routing, role-based access control, and structured request processing.

During the design process, Oracle Data Modeler is used to create logical and relational database models. It permits the visual depiction of entities, properties, relationships, and constraints, which aids in accurate database planning prior to physical implementation. This modelling approach assures that the conceptual design matches the final relational schema.

The physical database is built on Microsoft SQL Server, with SQL Server Management Studio (SSMS) utilized for database development, management, and testing. SQL Server offers robust support for relational integrity, ACID-compliant transactions, indexing, and performance optimization. Entity Framework Core serves as the object-relational mapper, connecting the application to the database. Additional technologies such as HTML, CSS, Bootstrap, and JavaScript are employed to create a responsive, user-friendly interface, culminating in a strong and scalable web system.

2. System Requirement Analysis

This section specifies both the functional and non-functional requirements for the Smart Event Management and Ticketing System. The requirements specify what the system must do, how it should function, and the limitations under which it operates, ensuring that it satisfies the needs of users and organisations.

2.1 Functional Requirements

2.1.1 Home Page Features

The home page is the principal landing page for all users, including guests, members, administrators, and staff. It clearly defines the system's objective and emphasizes available cultural events in a structured and visually uniform style. Event listings offer key information such as the event title, category, venue, city, session date, and availability status. The home page also serves as a navigation center, where users may access login, registration, event listings, and inquiry tools.

While all users can view event information, pricing and booking actions are restricted according to user role. This guarantees that the home page remains informational while also implementing proper access restriction.

2.1.2 Member Features

Members are registered users who have access to additional features beyond simple event browsing. Members can examine ticket pricing, including the beginning price for current and forthcoming events, and order tickets for available event sessions. Booking feature covers ticket type selection, quantity validation, and confirmation via a unique booking code. The system allows members to manage their personal bookings, including examining booking history and status. The payment status is clearly shown to guarantee transparency, and only confirmed bookings are eligible for entry verification at event venues. Members can also contribute reviews of events they have attended. Review submissions are subject to eligibility requirements and moderation to ensure that only genuine and relevant input is published. Members may also send inquiries about events, payments, or general system usage.

2.1.3 Guest Features

Guests are unregistered users who can use the system without authentication. Guest users can access all available events, including descriptions, venues, categories, and session schedules. This enables potential users to explore the platform before opting to register.

Guest users cannot check ticket pricing or order tickets. To use price or booking capabilities, the user must log in or register as a member. This limitation protects transactional processes while encouraging user registration. Guests can also send requests via email address without creating an account. This feature allows for communication with the organisation while limiting access to booking and payment-related functionality.

2.1.4 Admin Features

Administrators manage and maintain the entire system. Administrators can establish, change, and manage venues, categories, events, and event sessions. This involves setting event data, scheduling sessions, and managing event statuses like active, completed, or cancelled. Administrators also control user-generated content, which includes moderating membersubmitted reviews. Reviews can be allowed, denied, or rejected to guarantee content quality and relevancy. Inquiry

management capabilities enable administrators to effectively see, respond to, and close user questions. Administrators also manage internal user roles such as staff accounts, which ensures system integrity. Administrative access is rigorously limited to ensure that sensitive operations are secure and auditable.

2.1.5 Staff Features

Staff members carry out operational activities connected to event implementation, including attendee verification at event venues. Staff can use the check-in interface to validate bookings during live event sessions. To verify, enter a valid booking code and the last two digits of the member's registered phone number. Before designating a booking as used, the system validates it at the database level to ensure eligibility. This ensures secure and accurate entry handling at events.

2.2 Non-Functional Requirements

2.2.1 Security

The system ensures security by requiring authentication, role-based access control, and database limitations. Only allowed roles can access sensitive elements such as booking, payment confirmation, administrative management, and staff verification. Passwords and sensitive data are kept secure, and database restrictions prevent unauthorized or invalid data alteration.

2.2.2 Performance

The system is intended to provide responsive interaction for common tasks including event browsing, searching, booking, and check-in verification. Efficient querying, indexing, and transactional processing ensure that data retrieval and updates are completed with little latency under normal usage situations.

2.2.3 Usability

The user interface is intended to be intuitive and consistent throughout all modules. Clear navigation, hierarchical layouts, and informative validation messages make it easy to use for both technical and non-technical users. Role-based views ensure that users only see relevant features, eliminating complexity and ambiguity.

2.2.4 Scalability

The system architecture allows for future growth in users, events, and bookings. The usage of a relational database, MVC architecture, and modular design enables the system to extend horizontally and vertically with minimum change. Additional features and connectors can be added without disrupting current functioning.

2.3 System Use Case Description

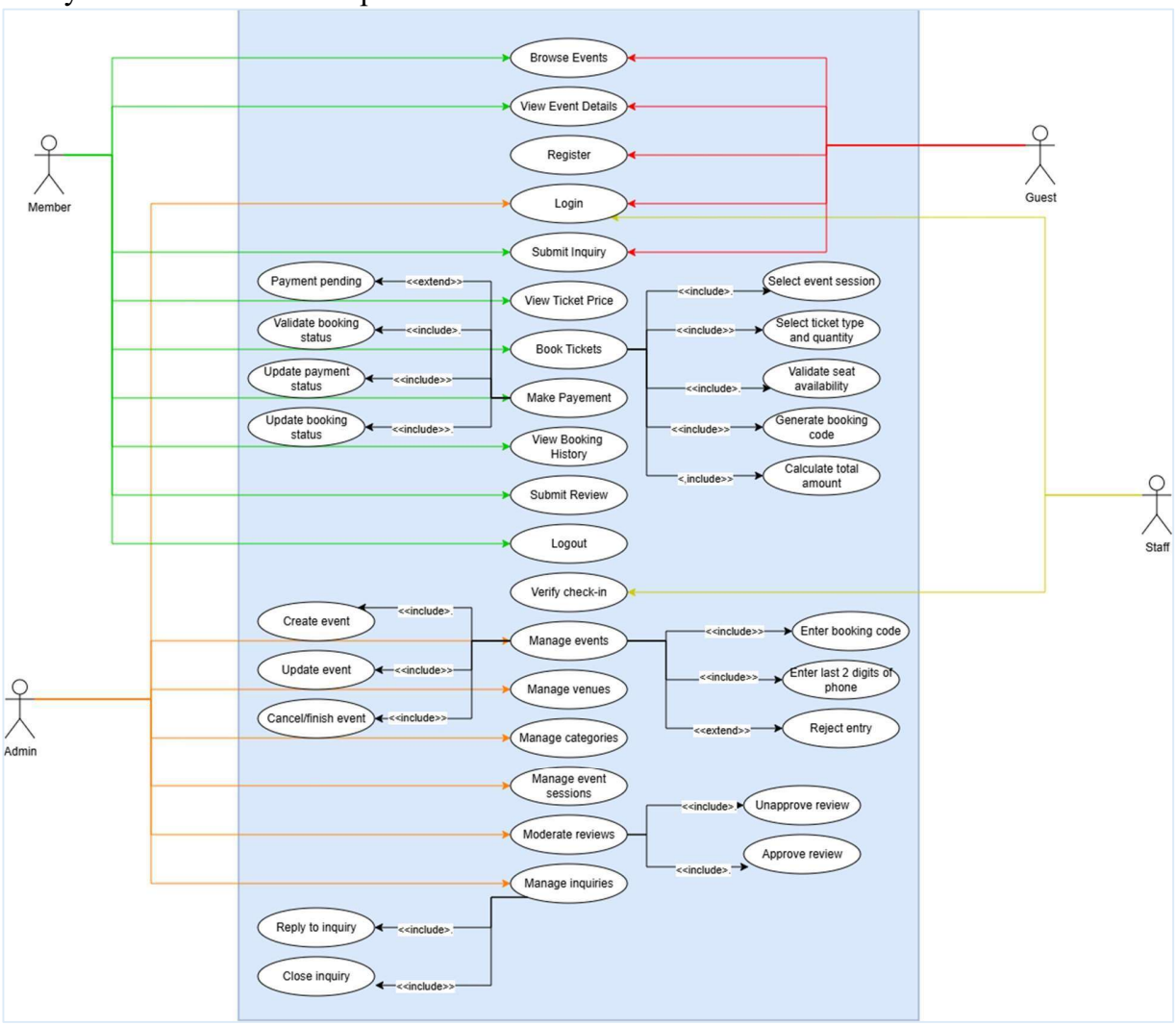


Figure 1: Usecase diagram

The use case diagram depicts the Smart Event Management and Ticketing System's functional behavior by identifying the various user roles and actions available within the system. It gives a high-level perspective of system interactions and clearly defines the system border, ensuring that tasks are segregated according to user entitlements. The graphic enables stakeholders to understand how users interact with the system without focusing on technical implementation details.

The diagram depicts four key actors: Guest, Member, Staff, and Administrator. A Guest is an unregistered user who can explore events, check event details, create an account, log in, and send questions. Guests are purposely denied access to ticket prices and booking functions, ensuring

that only authenticated users can conduct transactions. This separation safeguards key booking and payment functions while enabling the public access to event information.

The Member actor denotes a registered user with full customer capabilities. Members can check ticket prices, purchase tickets, make payments, view booking history, leave reviews, and log out. The booking process uses <<include>> relationships to display mandatory stages such picking an event session, selecting ticket kinds and numbers, confirming seat availability, calculating total cost, and creating a booking code. This structure ensures that bookings adhere to a uniform and controlled workflow.

The Admin and Staff players play operational and management duties. Admin users can manage events, venues, categories, sessions, reviews, and queries while ensuring system governance and content management. Staff members conduct on-site check-in verification at event venues. The verification method involves a booking code and the last two digits of the member's registered phone number, with database validation determining whether the admission is approved or rejected. The usage of <<extend>> relationships mimics conditional behavior such as refusing entrance or dealing with payment-pending circumstances, properly representing real-world operational flows.

2.4 User Roles and Permissions

Feature / Permission	Guest	Member	Staff	Admin
browse events	yes	yes	yes	yes
view event details	yes	yes	yes	yes
view ticket pricing	no	yes	no	yes
book tickets	no	yes	no	no
view booking history	no	yes	no	yes
submit reviews	no	yes	no	no
moderate reviews	no	no	no	yes
submit inquiries	yes	yes	no	no

reply to inquiries	no	no	no	yes
create and manage venues	no	no	no	yes
create and manage events and sessions	no	no	no	yes
perform check-in verification	no	no	yes	no
manage staff accounts	no	no	no	yes

Table 1: User roles and permissions.

3. Database Design - Oracle

3.1 Database Overview

The Smart Event Management and Ticketing System database is designed to support the complete lifecycle of cultural events managed by a metropolitan cultural council, beginning with event creation and publication and extending through ticket sales, attendance management, and post-event engagement. The database supports three core user groups- admins, staff members, and registered users; and guarantees that each group interacts with the system according to well defined duties and access privileges. Administrators oversee event development, approvals, and system-level management, staff members assist operational operations such as verification and query handling, and members participate with the system as customers who browse events, make bookings, post reviews, and contact with the council.

A basic element guiding the database design is the clear separation between stable reference information and continuously changing transactional data. Reference items such as venues and categories are largely static and are utilized across the system to maintain consistency and reduce redundancy. In contrast, transactional entities such as bookings, booked ticket records, and payments update continuously as the system is used. By segregating these diverse sorts of data, the schema stays scalable, easier to maintain, and more efficient for both operational processing and reporting.

Normalization is a crucial aspect of the database design, guaranteeing that each item of information is kept only once and retrieved through well-defined relationships. Instead of duplicating data across tables, the schema relies on foreign key references to link similar things. Events reference their hosting venues and the administrative user who established them, sessions reference their parent events, ticket types reference specific sessions, and booked ticket records reference both the booking and the selected ticket type. Bookings reference the member user who booked the reservation, while optional operational links allow staff users to be recorded as the verifying/check-in users. This normalized structure avoids redundancy, prevents update anomalies, and assures that changes made in one area are reflected consistently throughout the system. As a result, data integrity is protected and reporting accuracy is increased.

Beyond core event and ticketing processes, the database also facilitates member involvement and public interaction. Reviews provide structured comments on events and enable quality assessment and decision-making for future programming, with moderation fields enabling administrators to approve or reject reviews before they become publicly available. Additionally,

the inquiry method enables both registered members and guests to contact the council with questions or issues: member queries link to USER when available, while guest inquiries retain contact email data without requiring an account. Each inquiry retains a clear record of responses made by staff or administrators, enhancing accountability and communication tracking. This integrated strategy ensures that the database not only improves operational efficiency but also enhances transparency and user engagement across the platform.

The database additionally integrates session-level capacity tracking as part of its operational logic. Each event session keeps its own seat-related variables to reflect availability and sales activity properly. By recording total capacity with reserved and sold numbers within the event session entity, the system can manage ticket availability in real time while maintaining a clear audit trail of how seats are assigned. This approach provides dependable booking behavior, prevents over-allocation, and ensures that session availability remains constant across the application.

Overall, the database structure shows a balanced design that blends normalization, scalability, and functional completeness. It offers administrative control, transactional integrity, and user participation within a single unified schema, making it well-suited for administering large-scale cultural events and continuing public interaction in a controlled and dependable manner.

3.2 Entity Identification

1. User

The User entity represents all authenticated accounts in the Smart Event Management and Ticketing System and serves as the foundation for system access control. It saves important identity and authentication information such as complete name, email address, phone number, password, role, account status, and creation timestamp.

The role property distinguishes between administrators, staff members, and registered users, limiting the tasks a user can execute within the system. Administrative users create and manage events, moderate reviews, and reply to requests. Staff members assist with operational tasks such as ticket verification and participant check-in during events. Member users generally interact with the system as customers, browsing events, making reservations, leaving reviews, and submitting inquiries.

By consolidating all user identities and role descriptions into a single entity, the system ensures consistent authentication, streamlined authorization logic, and secure traceability of user actions across bookings, inquiries, reviews, and event management.

2. Venue

The Venue entity maintains information about the physical places where the cultural council hosts events. It collects information such as the venue name, city, optional address lines, contact phone number, and total seating capacity.

Venues are handled as reusable master data, which allows many events to be linked to the same physical site without duplication. This architecture promotes uniform venue management, simplifies maintenance when venue characteristics change, and allows for precise reporting on event allocation and capacity use across locations.

3. Category

The Category entity denotes thematic classifications used to arrange and characterize events such as music, drama, dance, cultural displays, and workshops. Each category has a name and an optional description to enhance clarity and usage.

Categories improve event discoverability by enabling filtering, browsing, and searching within the system. Categories are linked to events via an associative table, allowing for flexible and scalable classification.

4. Event

The Event entity defines the council's core cultural programming. It contains descriptive and administrative data such as the event title, comprehensive description, poster reference, status, associated venue, and the administrative user who established the event.

An event serves as a conceptual container for one or more scheduled sessions, allowing a cultural program to be promoted as a whole while accommodating various performances or showtimes. Events also serve as the primary reference point for category allocations and member reviews, making them an important entity for both operational administration and user participation.

5. Event_Category

The Event_Category entity serves as an associative table, connecting events and categories in a many-to-many relationship. This structure allows each event to belong to several categories, and each category can contain multiple events.

By isolating this relationship into its own entity, the database remains standardized and adaptable. New categories can be allocated or deleted from events without affecting core event or category data, allowing for future reclassification and system expansion.

6. Event_Session

The Event_Session entity represents individual scheduled occurrences of an event and stores session-specific information such as session date, start and end times, operational notes, session status, and seat availability.

Each session is administered independently to ensure proper booking, ticket sales, and attendance tracking. All booking and ticketing processes take place at the session level rather than directly on the event, allowing for exact capacity control, real-time availability management, and dependable operational planning.

7. Ticket_Type

The Ticket_Type entity specifies the various ticket types available for a given event session, such as standard, VIP, or balcony seating. It keeps track of the name, price, and status of each ticket type and ensures that they are unique within the same session.

This structure enables sessions to offer several pricing tiers while retaining consistent ticket definitions. Ticket types are the foundation for all ticket purchases and pricing computations during the booking and payment processes.

8. Bookings

The Bookings entity represents a reservation made by a member user for a particular event session. It stores booking identities, booking dates, booking statuses, total payable amounts, ticket usage indicators, and optional staff verification information.

Each booking is associated with a single user and event session, serving as the central transactional record for ticket line items, payments, and attendance verification. The booking entity is crucial in tracking a reservation's lifecycle, from creation to confirmation and check-in.

9. Booked_Tickets

The Booked_Tickets entity contains individual ticket line items linked with a booking. Each record reflects a certain ticket type purchased in a predetermined number, together with the unit price and calculated line total.

This approach enables a single booking to contain several ticket kinds while retaining accurate financial breakdowns. Separating ticket line items from the booking header increases flexibility, allows for more accurate reporting, and simplifies auditing and reconciliation.

10. Payment

The Payment entity records all financial transactions related with a booking and maintains a strict one-to-one link with the booking entity. It saves the payment amount, method, status, timestamps, and optional notes.

By isolating payment information from booking data, the system offers financial transparency, dependable audit trails, and more extensibility for future payment-related expansions or integrations.

11. Review

The Review entity collects comments offered by member users for events they have attended. It holds rating values, optional written comments, the review date, and administrator moderation information.

Reviews are associated with both the submitting user and the reviewed event, and the system enforces one review per person per event. Before evaluations may be made public, administrative clearance is necessary, which ensures content quality, consistency, and trust in user comments.

12. Inquiry

The Inquiry entity facilitates communication and support messages sent to the council. It enables both registered users and guests by making the user reference optional while still needing contact email information.

The entity keeps track of the inquiry subject, message content, inquiry type, status, reply content, reply timestamp, and the administrative user who made the response. This system guarantees that communication history is clearly tracked, replies are accountable, and inquiries from the public and members are handled appropriately.

3.3 ER Diagram with Explanation

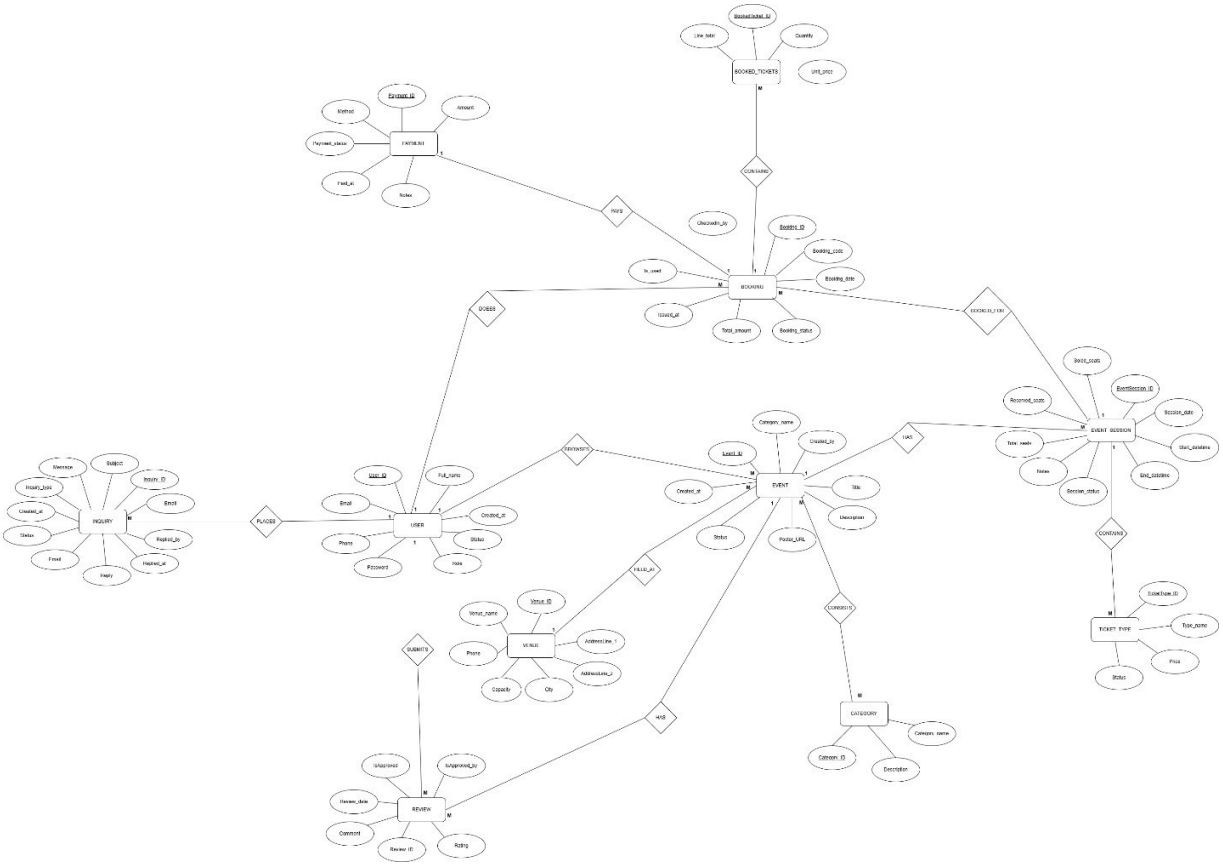


Figure 2: ERD diagram

The ER diagram depicts the logical and relational structure of the database utilized by the Smart Event Management and Ticketing System. It describes the basic entities, their attributes, and their relationships, ensuring that data is structured, normalized, and consistent. This model serves as the foundation for database implementation, enforcing integrity requirements throughout all system operations.

The USER object is fundamental to the model, storing important account information such as name, email, phone number, role, status, and login credentials. The role attribute identifies members, staff, and administrators, allowing for role-based access management at the application and database levels. Users are linked to bookings, reviews, and queries, allowing all interactions to be traced back to a specific account if possible.

Event management is handled via the VENUE, EVENT, EVENT_SESSION, CATEGORY, and TICKET_TYPE entities. A venue can host many events, each with multiple sessions scheduled on

various days and times. Ticket categories are coupled with event sessions, allowing for more flexible price structures. Categories and events are linked using a many-to-many relationship, allowing occurrences to correspond to several classes without data duplication. This approach enables advanced filtering, reporting, and scalability.

Transactional operations are represented by the BOOKING, BOOKED_TICKETS, PAYMENT, REVIEW, and INQUIRY entities. A booking assigns a member to a certain event session and saves the booking status, total cost, and use details. Booked tickets include line item information for each ticket type and number. Payments record financial confirmation and are linked directly to bookings, allowing for safe entry validation. Reviews and inquiries facilitate feedback and communication workflows, while approval and response tracking sections provide moderation and responsibility. Together, these links provide referential integrity, prevent orphan data, and enable dependable booking, payment, and check-in operations.

3.4 Database Relationships

- Venue (1) → Event (M): Each event is linked to a single venue, but a single venue can host several events.
- User (1) → Event (M): One user creates each event, and an administrator user can create multiple events.
- Event (1) → Event_Session (M): A single event may have several planned sessions, each of which represents a specific occurrence.
- Event (M) ↔ Category (M): This allows for flexible classification by connecting events and categories through Event_Category.
- Event_Session (1) → Ticket_Type (M): Several ticket types with various price ranges can be defined for each session.
- User (1) → Booking (M): A member may make more than one booking, over time.
- User (1) → Booking (M): A single staff user can check-in many bookings.
- Event_Session (1) → Booking (M): Multiple bookings can be linked to a session.
- Booking (1) → Booked_Tickets (M): Each booking contains one or more booked ticket line items.
- Booking (1) → Payment (1): A single payment record is linked to exactly one booking.

- User (1) → Review (M): A Member can submit more than one review across several events.
- User (1) → Review (M): A single admin user can moderate many reviews
- Event (1) → Review (M): An event can have more than one review provided by various members.
- User (1) → Inquiry (M): Inquiries are replied by admin users through the replied by relationship.
- User (1) → Inquiry (M): A single user can make many inquiries.

3.5 Data Dictionary

1. User

Column Name	Data Type	Constraints
User_ID	Number(10)	Primary Key, Not Null
Full_name	Varchar2(120)	Not Null
Email	Varchar2(120)	Not Null, Unique
Phone	Varchar2(20)	Not Null
Password	Varchar2(255)	Not Null
Role	Varchar2(20)	Not Null, Check (Admin/Staff/Member)
Status	Varchar2(20)	Not Null, Check (Active/Inactive)
Created_at	Timestamp	Not Null

Table 2: Data dictionary - User

2. Venue

Column Name	Data Type	Constraints

Venue_ID	Number(10)	Primary Key, Not Null
Venue_name	Varchar2(120)	Not Null
AddressLine_1	Varchar2(120)	Nullable
AddressLine_2	Varchar2(120)	Nullable
City	Varchar2(60)	Not Null
Capacity	Number(10)	Not Null
Phone	Varchar2(20)	Nullable

Table 3:Data dictionary - Venue

3. Category

Column Name	Data Type	Constraints
Category_ID	Number(10)	Primary Key, Not Null
Category_name	Varchar2(50)	Not Null
Description	Varchar2(200)	Nullable

Table 4:Data dictionary - Category

4. Event

Column Name	Data Type	Constraints
Event_ID	Number(10)	Primary Key, Not Null
Venue_ID	Number(10)	Foreign Key → VENUE.Venue_ID, Not Null
Created_by	Number(10)	Foreign Key → USER.User_ID, Not Null
Title	Varchar2(150)	Not Null

Description	Varchar2(2000)	Nullable
Poster_URL	Varchar2(255)	Nullable
Status	Varchar2(20)	Not Null, Check (Active/Finished/Cancelled)
Created_at	Timestamp	Not Null

Table 5:Data dictionary - Event

5. Event_Category

Column Name	Data Type	Constraints
Event_ID	Number(10)	Primary Key, Foreign Key → EVENT.Event_ID, Not Null
Category_ID	Number(10)	Primary Key, Foreign Key → CATEGORY.Category_ID, Not Null

Table 6:Data dictionary -Event category

6. Event_Session

Column Name	Data Type	Constraints
Session_ID	Number(10)	Primary Key, Not Null
Event_ID	Number(10)	Foreign Key → EVENT.Event_ID, Not Null
Session_date	Date	Not Null
Start_datetime	Timestamp	Not Null
End_datetime	Timestamp	Nullable
Session_status	Varchar2(20)	Not Null, Check (Active/Cancelled/Finished)
Notes	Varchar2(200)	Nullable
Total_seats	Number(10)	Not Null

Reserved_seats	Number(10)	Not Null, Check (>=0)
Sold_seats	Number(10)	Not Null, Check (>=0)

Table 7: Data dictionary - Event session

7. Ticket_Type

Column Name	Data Type	Constraints
TicketType_ID	NUMBER(10)	Primary Key, Not Null
Session_ID	NUMBER(10)	Foreign Key → EVENT_SESSION.Session_ID, Not Null
Type_name	VARCHAR2(50)	Not Null
Price	NUMBER(10,2)	Not Null, Check (>0)
Status	VARCHAR2(20)	Not Null

Table 8: Data dictionary - Ticket type

8. Booking

Column Name	Data Type	Constraints
Booking_ID	NUMBER(10)	Primary Key, Not Null
User_ID	NUMBER(10)	Foreign Key → USER.User_ID, Not Null
Session_ID	NUMBER(10)	Foreign Key → EVENT_SESSION.Session_ID, Not Null
Booking_code	VARCHAR2(30)	Not Null, Unique
Booking_date	TIMESTAMP	Not Null
Booking_status	VARCHAR2(20)	Not Null, Check (Pending/Confirmed)

Total_amount	NUMBER(10,2)	Not Null, Check (>=0)
Issued_at	TIMESTAMP	Nullable
IsUsed	CHAR(1)	Not Null, Check (Y/N), Default 'N'
CheckedIn_by	NUMBER(10)	Nullable, Foreign Key → USER.User_ID

Table 9:Data dictionary - Booking

9. Booked_Tickets

Column Name	Data Type	Constraints
BookedTicket_ID	Number(10)	Primary Key, Not Null
Booking_ID	Number(10)	Foreign Key → BOOKING.Booking_ID, Not Null
TicketType_ID	Number(10)	Foreign Key → TICKET_TYPE.TicketType_ID, Not Null
Quantity	Number(10)	Not Null, Check (>0)
Unit_price	Number(10,2)	Not Null, Check (>0)
Line_total	Number(10,2)	Not Null, Check (>=0)

Table 10:Data dictionary -Booked Tickets

10. Payment

Column Name	Data Type	Constraints
Payment_ID	NUMBER(10)	Primary Key, Not Null
Booking_ID	NUMBER(10)	Foreign Key → BOOKING.Booking_ID, Not Null, Unique
Amount	NUMBER(10,2)	Not Null, Check (>=0)

Method	VARCHAR2(30)	Not Null, Check (Card/Bank_Transfer/Online_Transfer)
Payment_status	VARCHAR2(20)	Not Null, Check (Pending/Success/Failed)
Paid_at	TIMESTAMP	Nullable
Notes	VARCHAR2(200)	Nullable

Table 11: Data dictionary - Payment

11. Review

Column Name	Data Type	Constraints
Review_ID	Number(10)	Primary Key, Not Null
User_ID	Number(10)	Foreign Key → USER.User_ID, Not Null
Event_ID	Number(10)	Foreign Key → EVENT.Event_ID, Not Null
Rating	Number(10)	Not Null, Check (1–5)
Comment	Varchar2(1000)	Nullable
Review_date	Timestamp	Not Null
IsApproved	Char(1)	Not Null, Check (Y/N), Default 'N'
IsApproved_by	Number(10)	Nullable, Foreign Key → USER.User_ID

Table 12: Data dictionary - Review

12) Inquiry

Column Name	Data Type	Constraints
Inquiry_ID	Number(10)	Primary Key, Not Null
User_ID	Number(10)	Nullable, Foreign Key → USER.User_ID

Email	Varchar2(120)	Not Null
Subject	Varchar2(150)	Not Null
Message	Varchar2(1000)	Not Null
Inquiry_type	Varchar2(30)	Not Null, Check (General/Event/Membership/Technical)
Created_at	Timestamp	Not Null
Status	Varchar2(20)	Not Null, Check (OPEN/CLOSED)
Reply	Varchar2(1000)	Nullable
Replied_at	Timestamp	Nullable
Replied_by	Number(10)	Nullable, Foreign Key → USER.User_ID

Table 13: Data dictionary - Inquiry

The Smart Event Management and Ticketing System database enforces data integrity, business rules, and consistency across all entities via a structured set of constraints. These constraints ensure that recorded data accurately represents real-world actions and avoid the creation of inaccurate or inconsistent records.

Primary Key constraints are applied to all fundamental entities in order to uniquely identify each record within its table. These keys provide entity integrity by allowing every user, venue, category, event, session, booking, ticket type, payment, review, and inquiry to be referenced consistently throughout the database.

Foreign Key constraints are widely used to ensure referential integrity across linked items. They ensure that relationships are valid and meaningful, such as events being linked to existing venues and administrative users, sessions being assigned to valid events, bookings referencing valid users and event sessions, booked ticket records referencing both bookings and ticket types, and payments being linked to an existing booking. Foreign keys also aid operational traceability by connecting staff users to check-in actions and administrators or staff to inquiry responses and review moderation activities. Where system functionality necessitates flexibility, such as guest queries or reviews pending approval, foreign key columns are purposefully defined as nullable to represent optional relationships.

Unique constraints are applied to properties that must remain distinct in order to maintain proper system behavior. These include requiring unique user email addresses to avoid duplicate accounts, unique booking codes for accurate ticket verification, a one-to-one relationship between bookings and payments, distinct ticket type names within the same session, and a single review per user per event. These limitations prevent logical conflicts and enable accurate operational processing.

Check constraints enforce domain rules by limiting column values to predetermined sets or ranges. They are used to manage enumerated values such user roles, account status, event and session status, booking status, payment method and status, inquiry status, and inquiry type. Numeric checks guarantee that quantities, prices, and seat counters are not negative and logically legitimate, whereas rating checks verify that review scores are within the acceptable range. These restrictions incorporate critical business rules directly into the database layer.

Together, the defined Primary Key, Foreign Key, Unique, and Check constraints form a solid basis for the database. They maintain normalization, prevent data anomalies, assure transactional accuracy, and ensure that the physical database implementation meets the functional and operational needs of the Smart Event Management and Ticketing System.

3.6 SQL Developer Data Modeler

3.6.1 Logical Model (zoom for clarity)

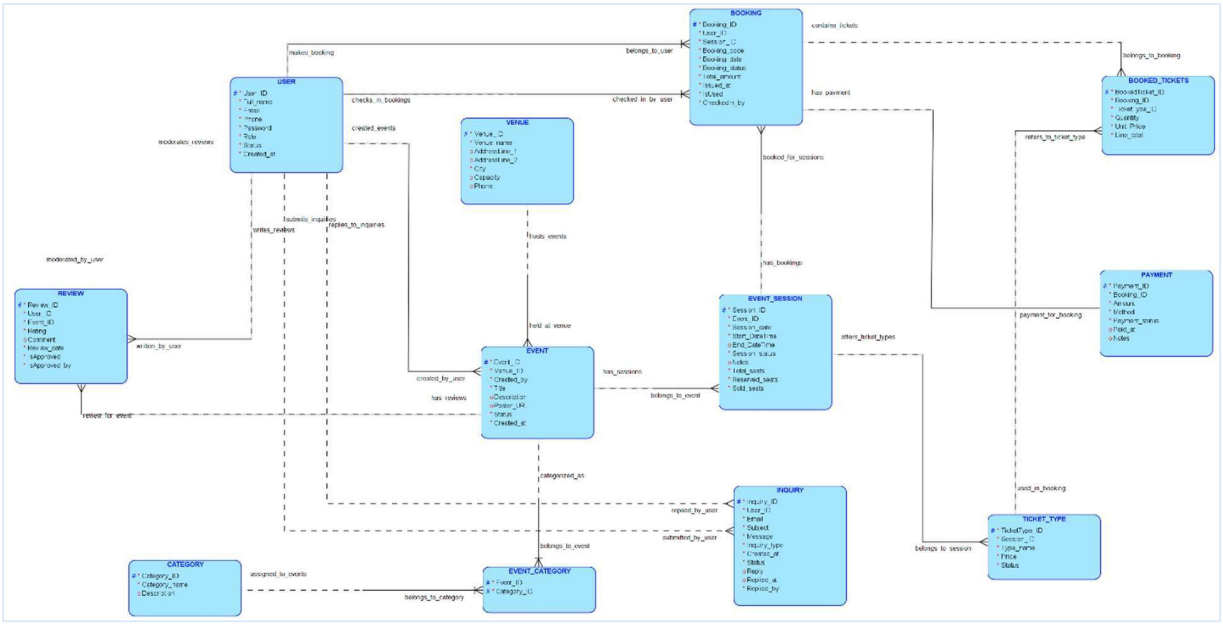


Figure 3: Logical model

3.6.2 Physical Model (zoom for clarity)

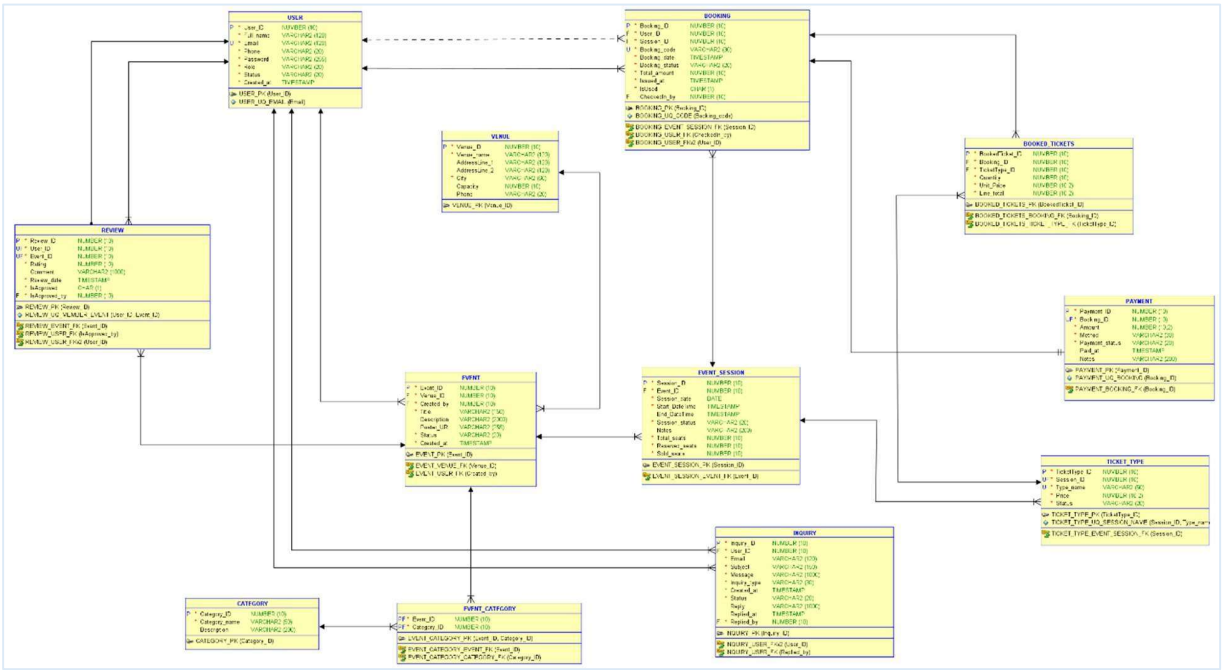


Figure 4: Relational model

3.6.3 Table Conversion Output

 Converting Logical Model to Relational Model

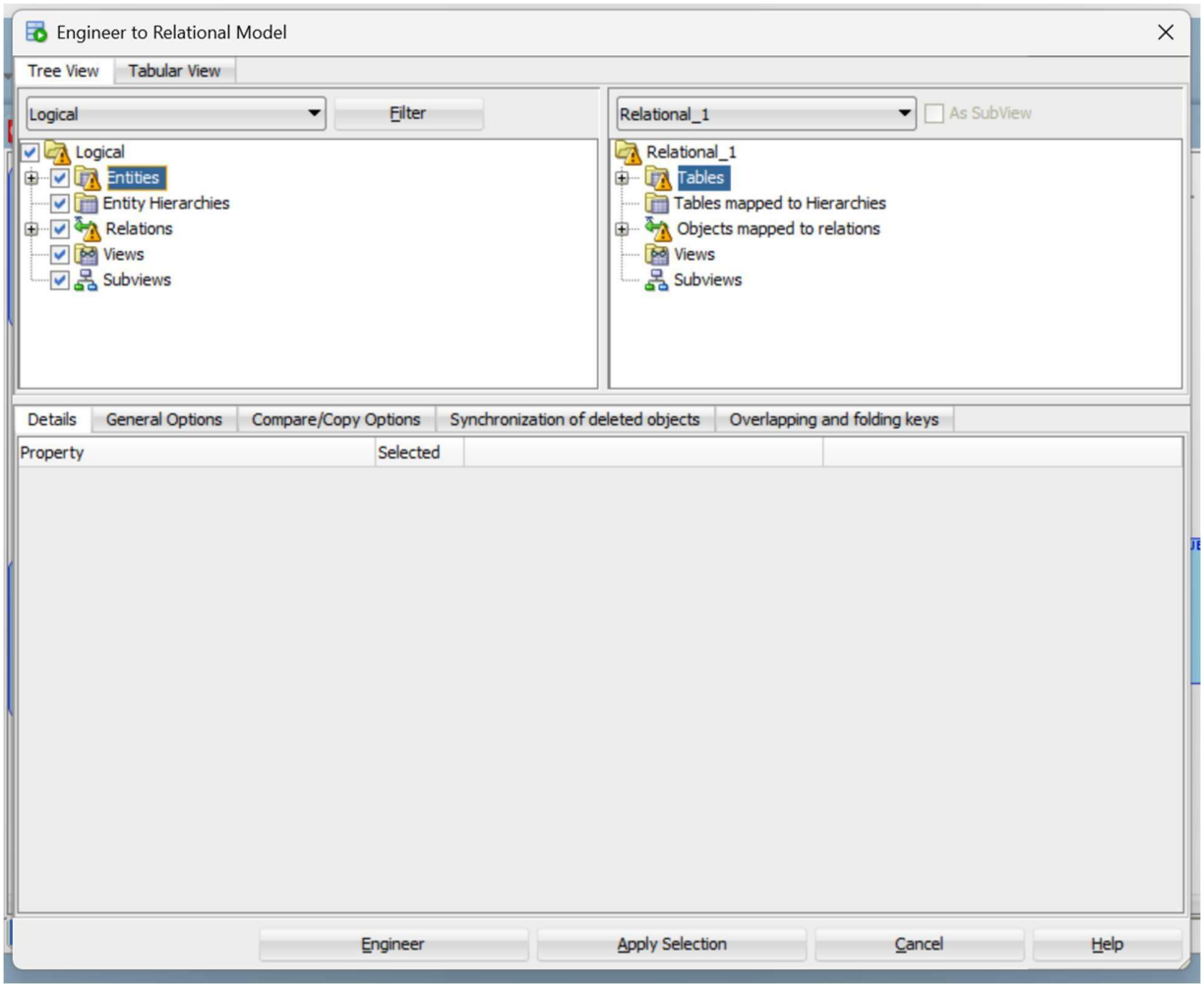


Figure 5: Table conversion output

4. Database Implementation - SQL Server

4.1 Database Schema Implementation

Figure 6: SQL Server

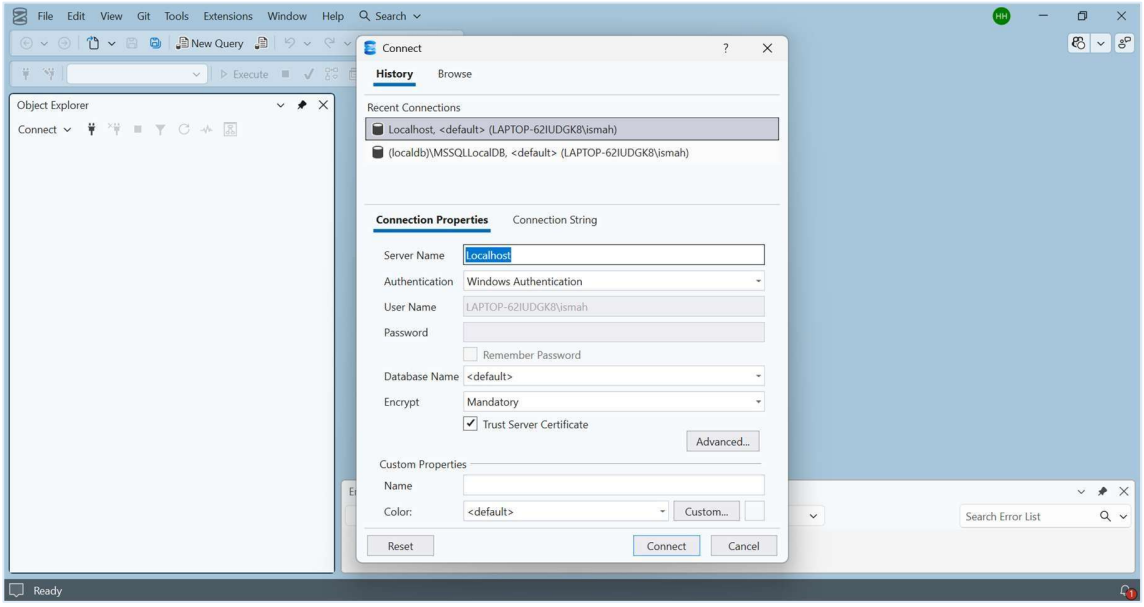


Figure 7: SQL Server Interface

The Smart Event Management and Ticketing System was built on Microsoft SQL Server as the core relational database platform. SQL Server was chosen because it provides strong support for relational data modeling, transaction management, and security, all of which are essential for a booking and payment-based system. The database schema was created using a structured relational

approach, with entities including users, events, sessions, bookings, payments, reviews, and venues represented as normalized tables linked by well-defined relationships.

SQL Server ensures a dependable relational structure by enforcing primary keys, foreign keys, and check constraints, which together protect data integrity throughout the system. These constraints ensure that only genuine records exist, such as bookings from valid event sessions, payments for confirmed bookings, and reviews given by registered members. The database also benefits from SQL Server's ACID-compliant transaction capability, which is notably useful during essential processes such as seat bookings and payment processing, as it ensures consistency and prevents data corruption in concurrent access circumstances.

In terms of efficiency, SQL Server has effective indexing and query optimization algorithms that allow for quick retrieval of frequently requested data such as forthcoming events, available sessions, member bookings and reviews. Another significant advantage is that SQL Server enables authentication, authorization, and role-based access control, which allows the application to restrict data access based on user roles such as admin, staff, and member. Furthermore, SQL Server Management Studio (SSMS) served as the primary administrative and development tool, providing a user-friendly environment for authoring queries, troubleshooting database logic, and viewing execution results.

4.2 Table Creation Scripts

1. User Table

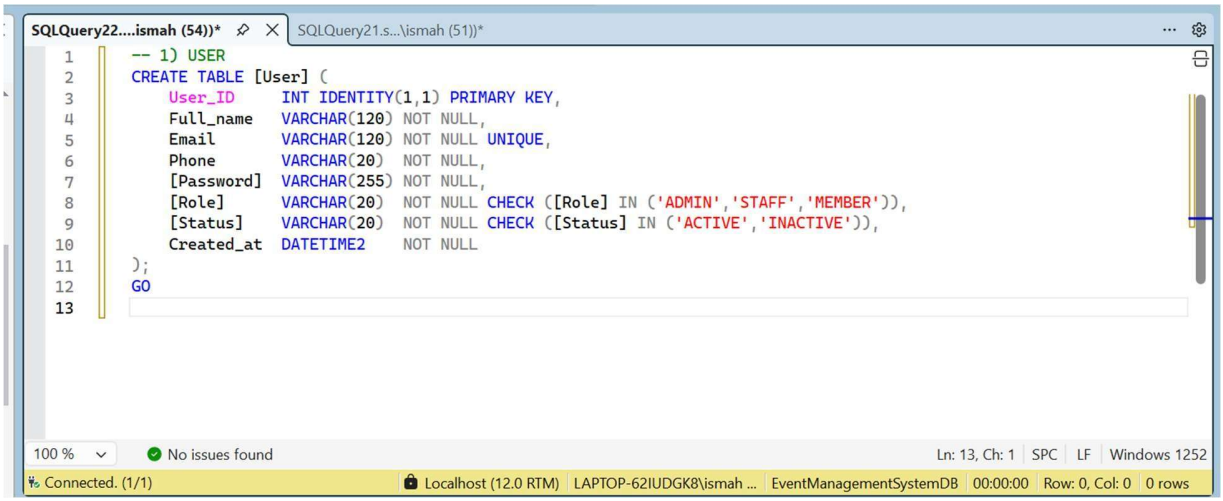


Figure 8: Table creation script - User

2. Venue Table

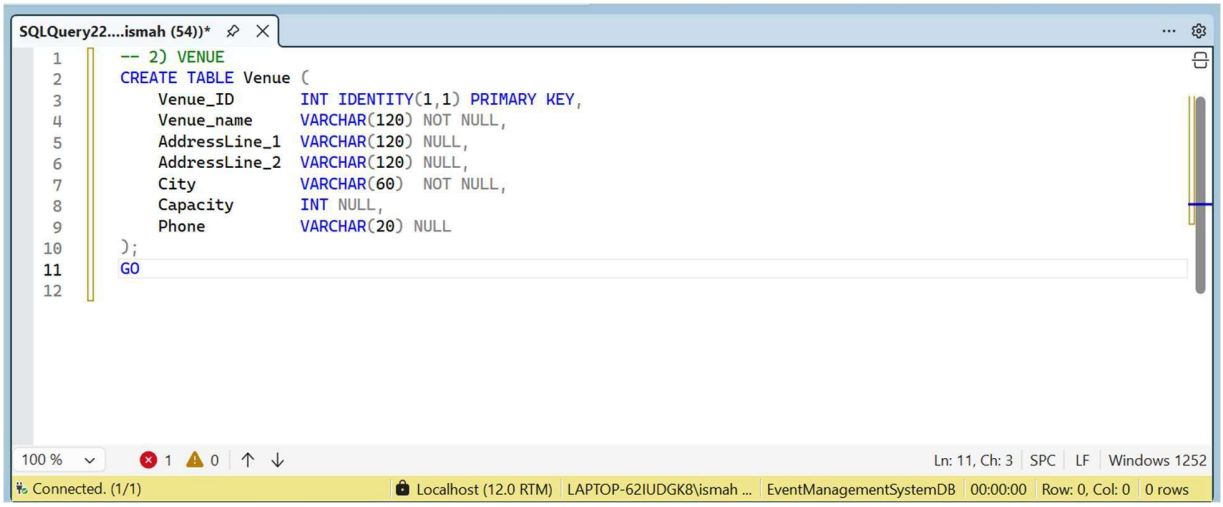


Figure 9: Table creation script -Venue

3. Category Table

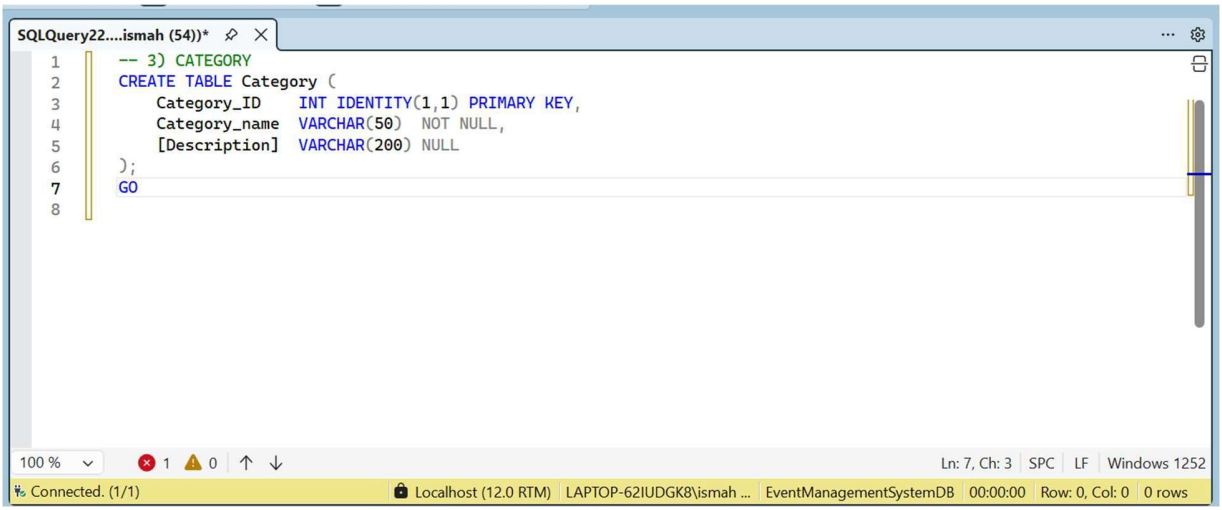


Figure 10:Table creation script - Category

4. Event Table

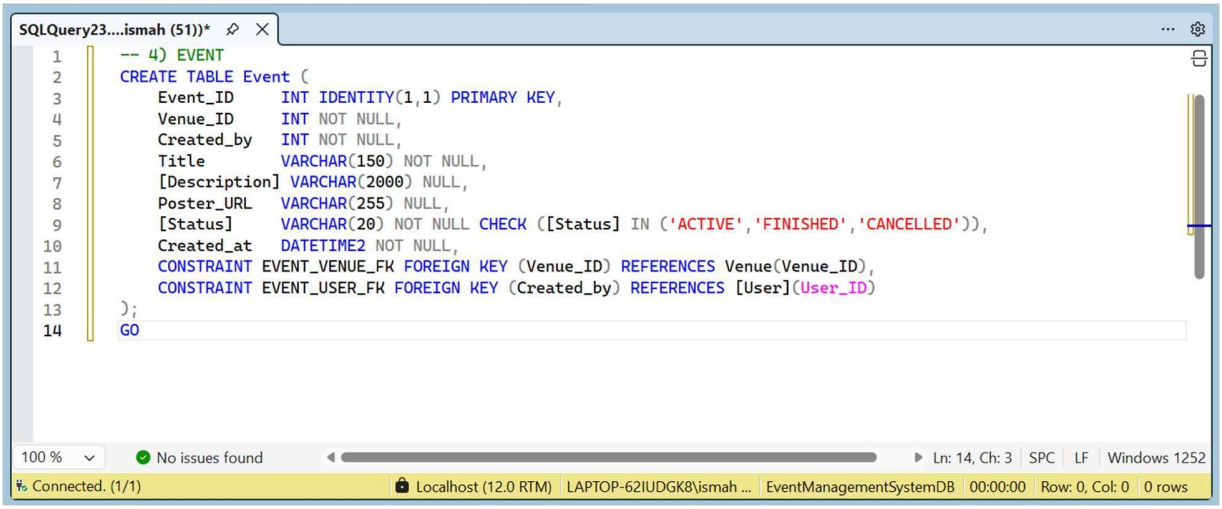


Figure 11:Table creation script -Event

5. Event Category Table

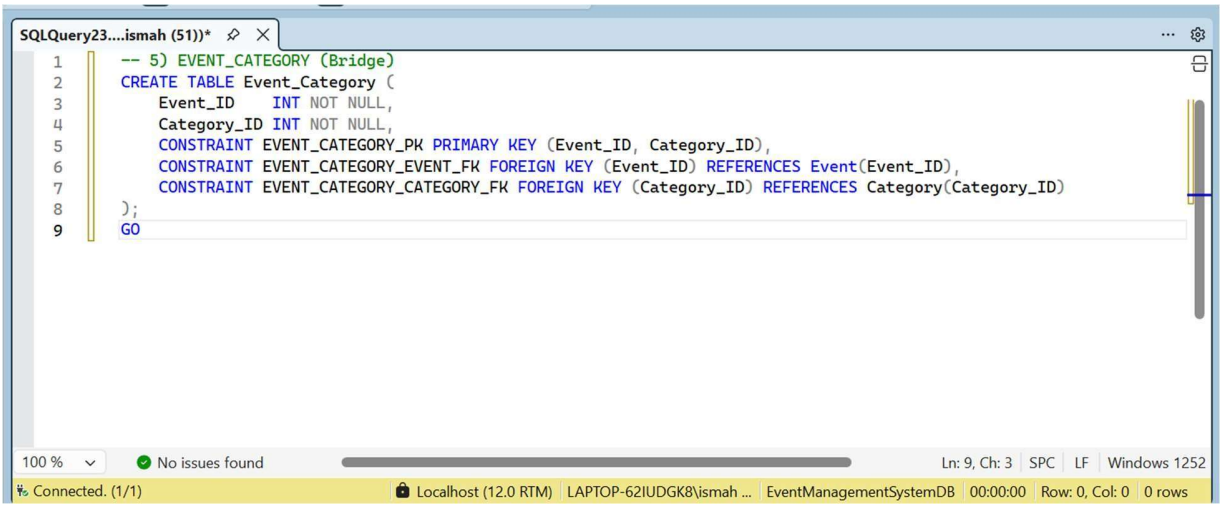


Figure 12:Table creation script -Event category

6. Event Session Table

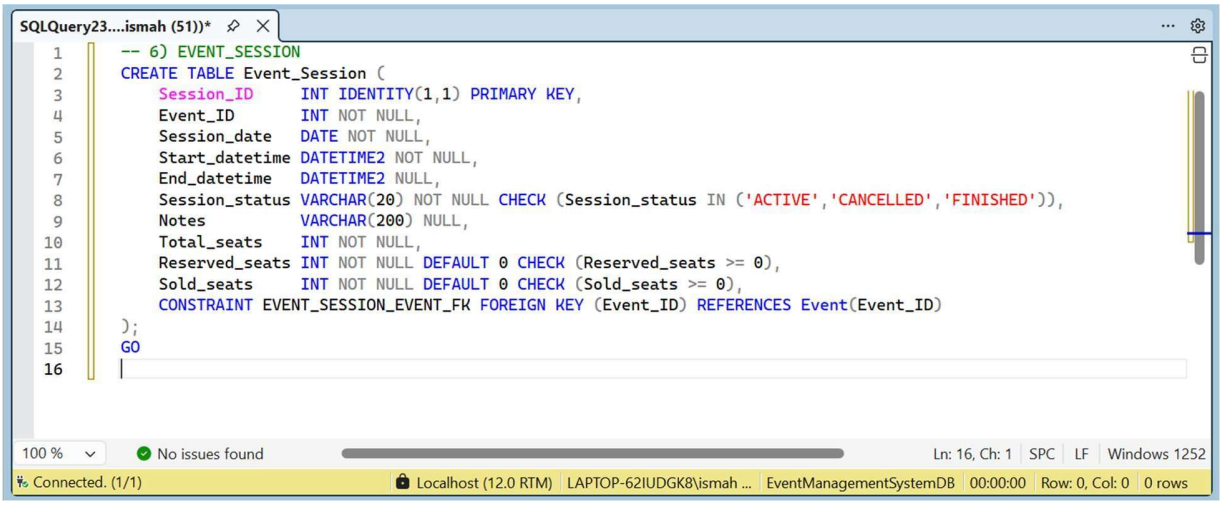


Figure 13:Table creation script -Event session

7. Ticket Type Table

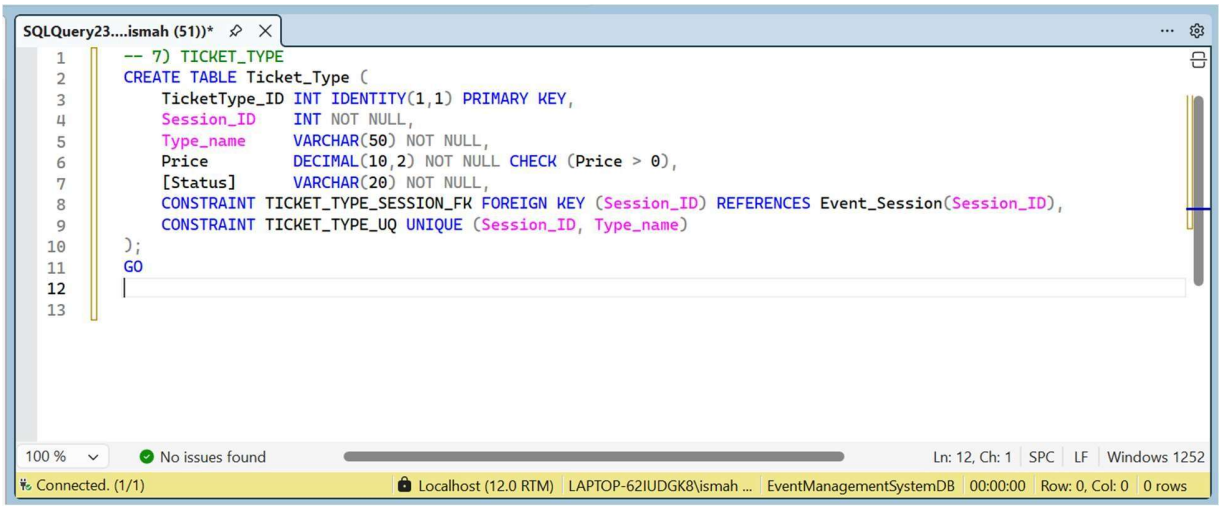


Figure 14:Table creation script -Ticket type

8. Booking Table

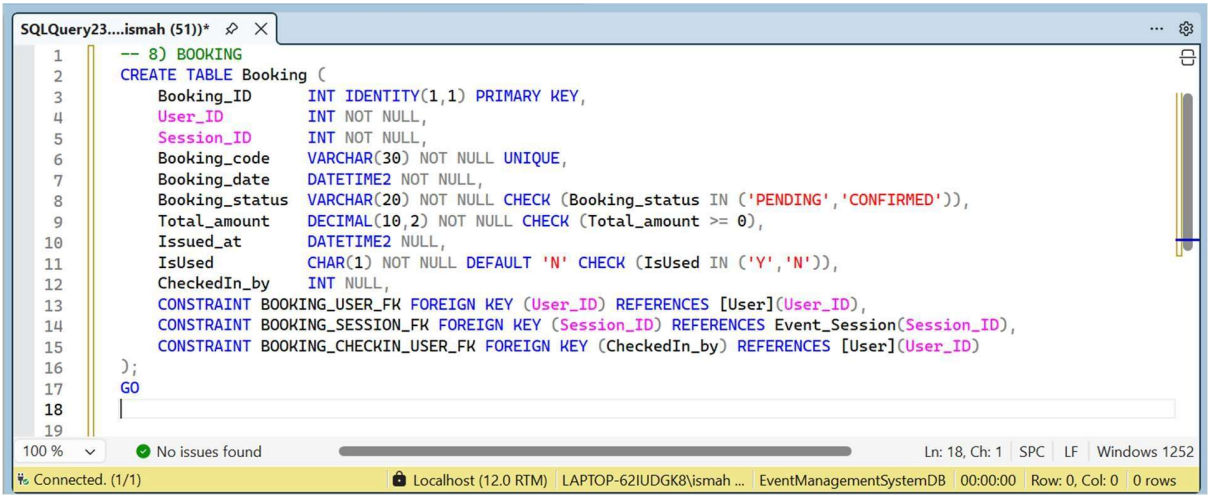


Figure 15:Table creation script -Booking

9. Booked Tickets Table

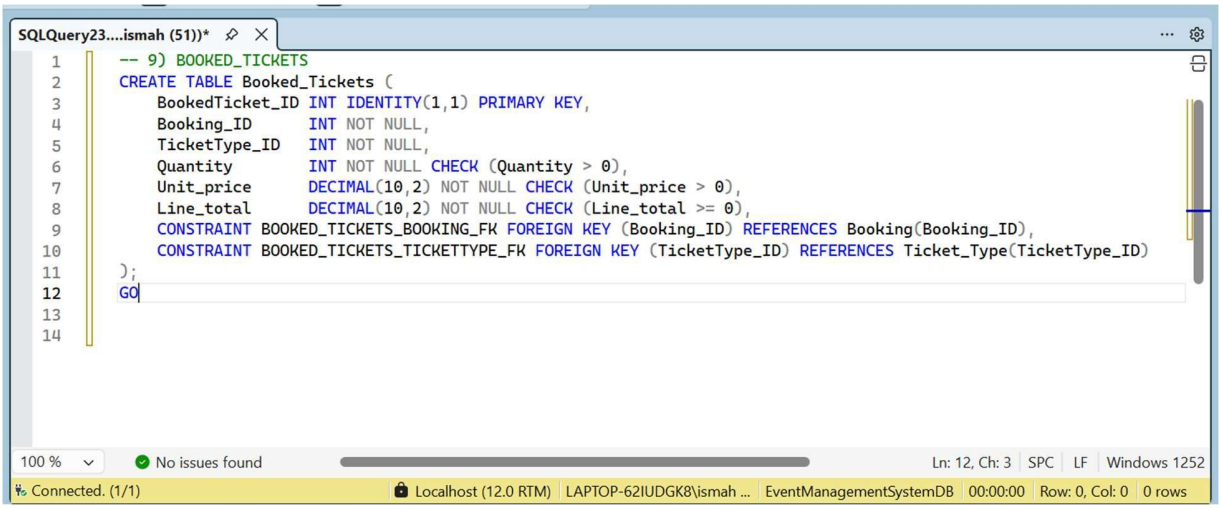


Figure 16:Table creation script -Booked ticket

10. Payment Table

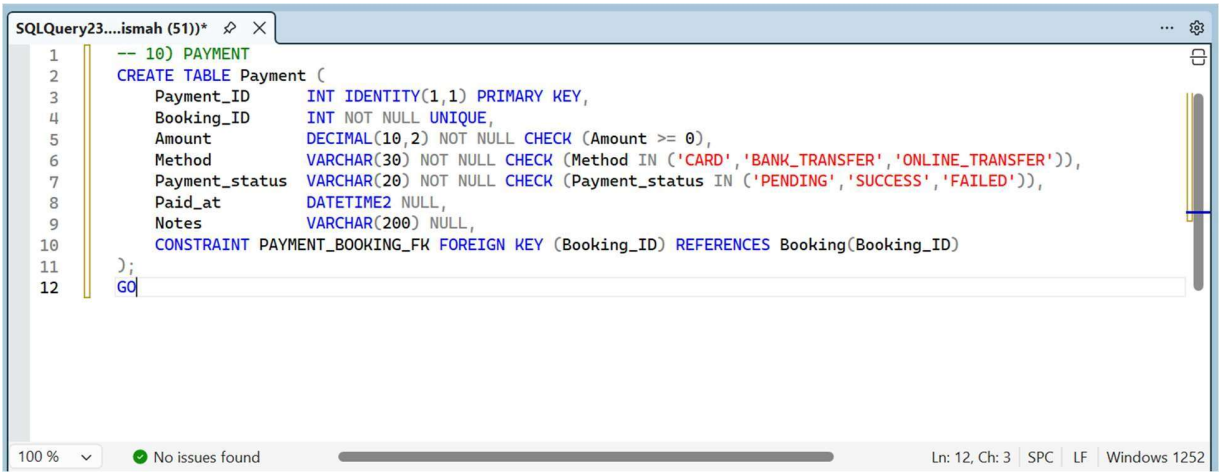


Figure 17:Table creation script -Payment

11. Review Table

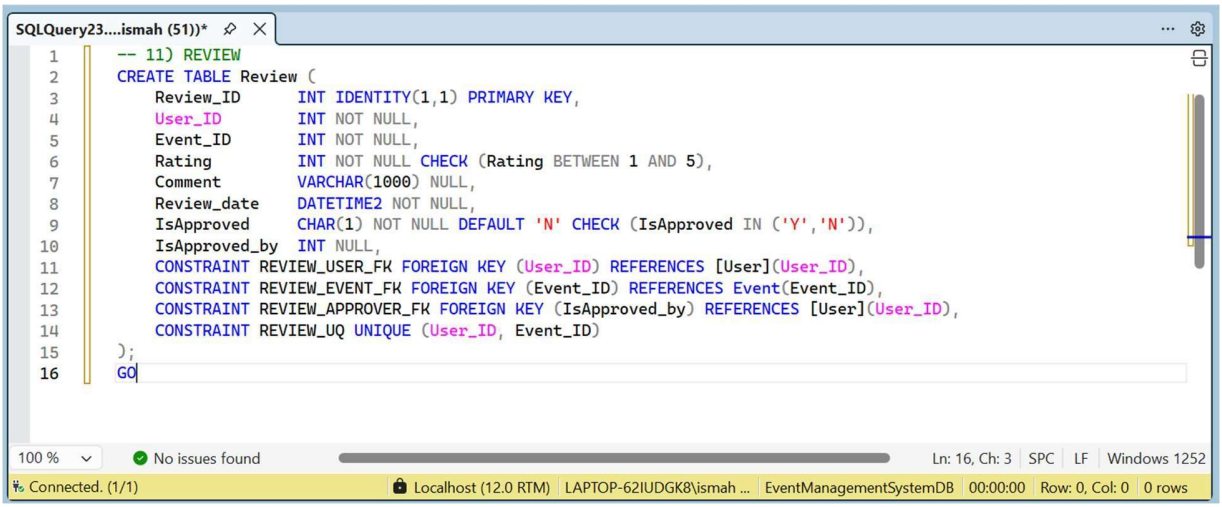


Figure 18:Table creation script -Review

12. Inquiry Table

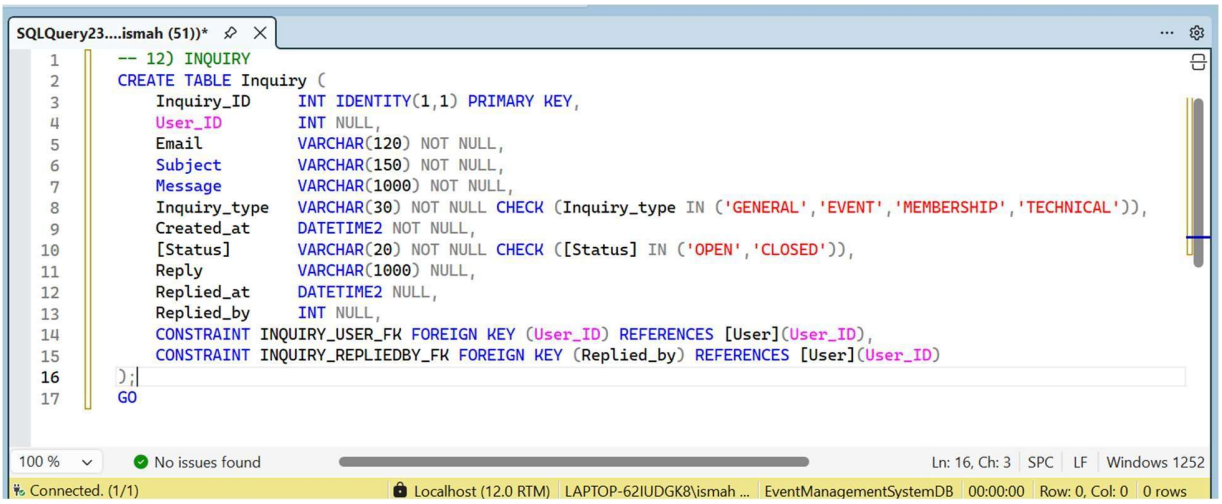


Figure 19:Table creation script -Inquiry

4.3 Data Models with Constraints

1. Users.cs

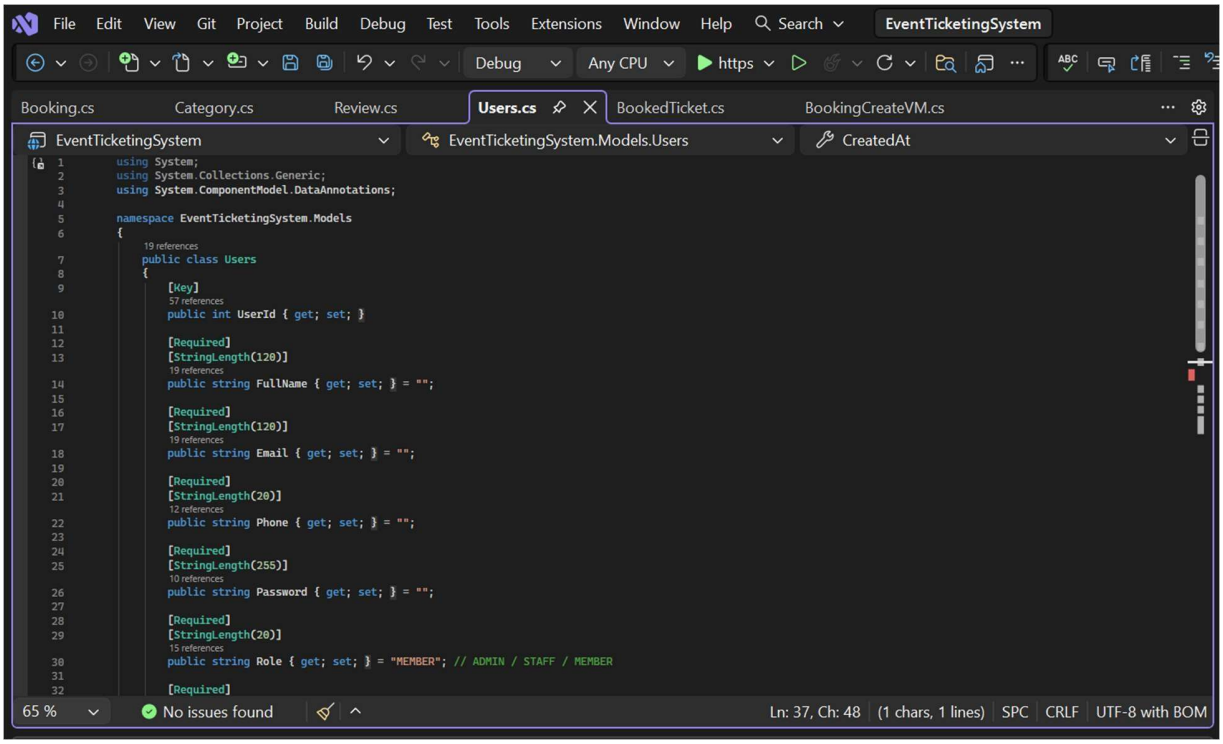


Figure 20: Data model - User

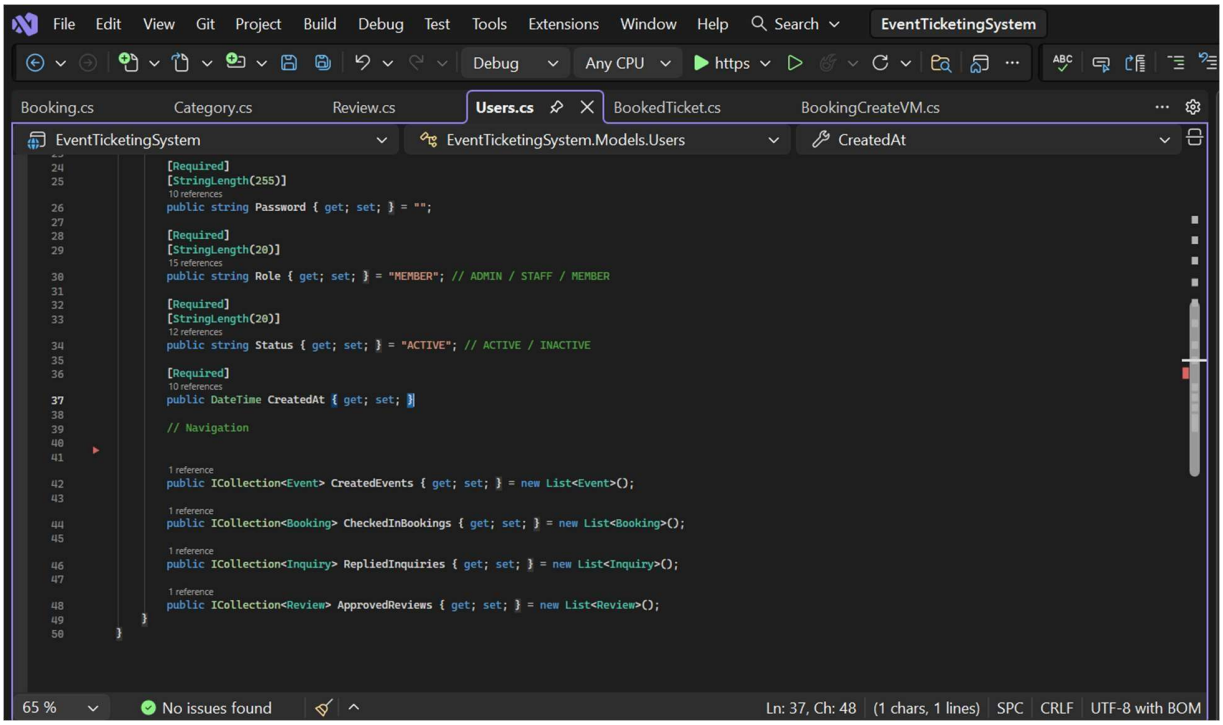


Figure 21:Data model - User

2. Venue.cs

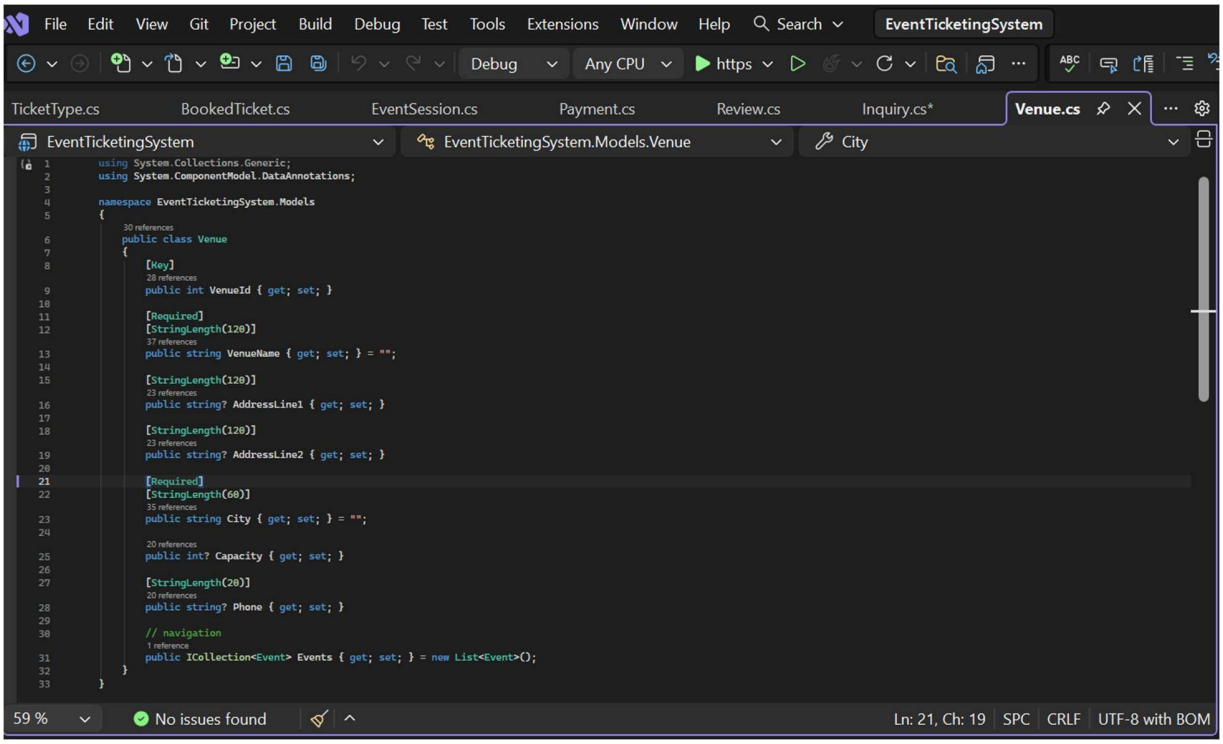


Figure 22:Data model - Venue

3. Category.cs

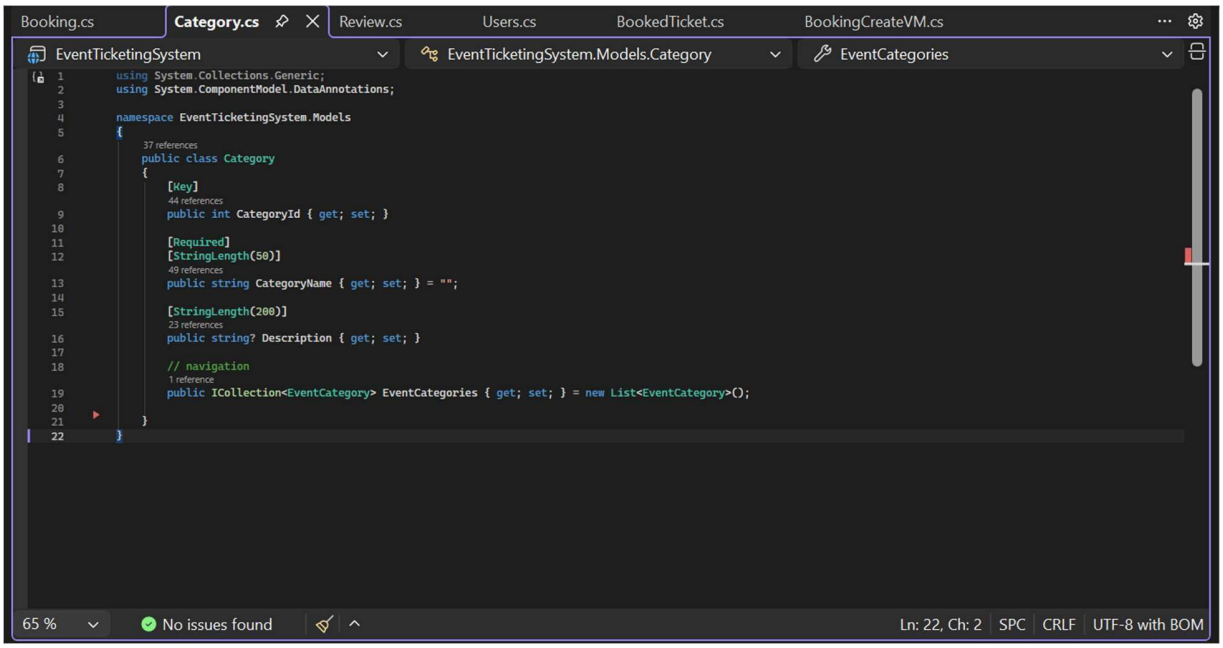


Figure 23:Data model - Category

4. Event.cs

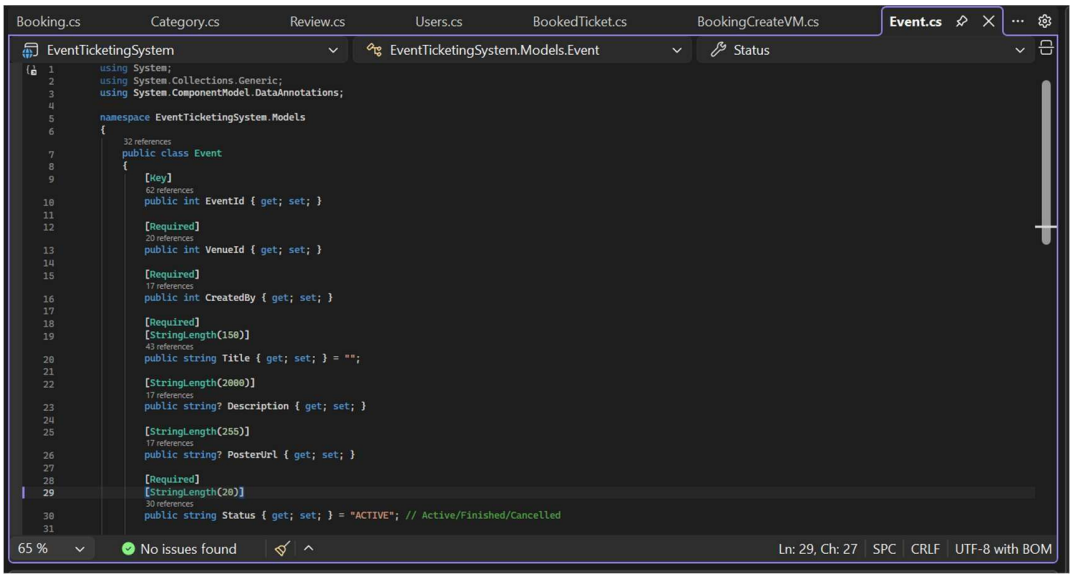


Figure 24: Data model - Event

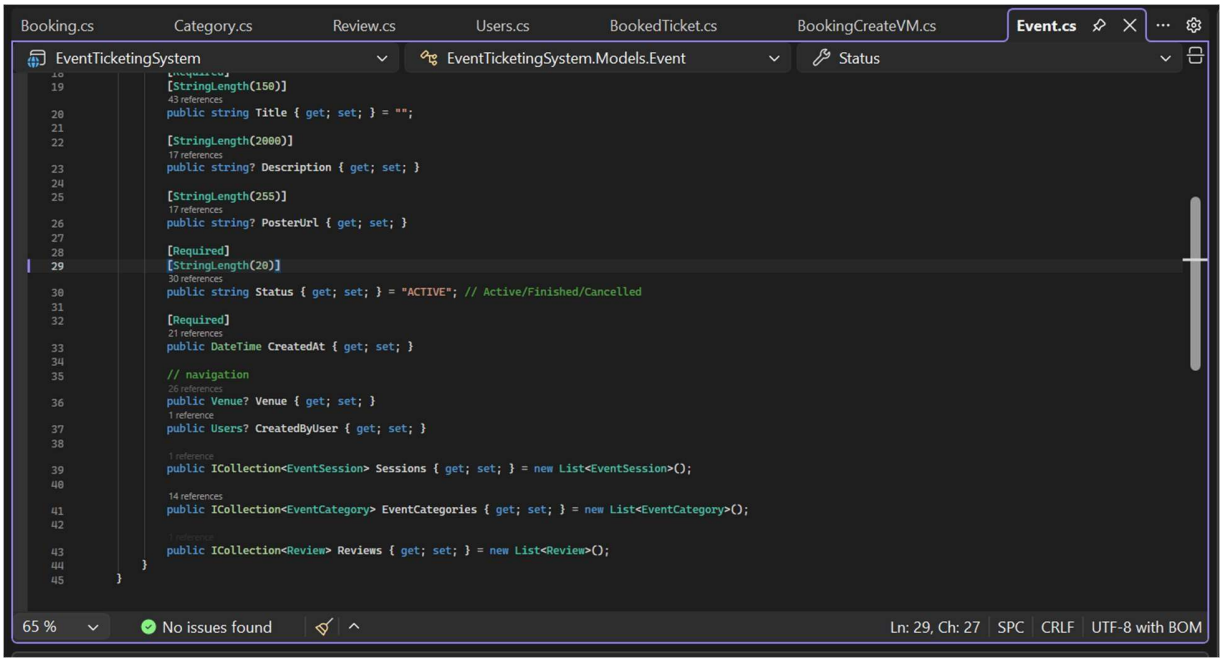


Figure 25:Data model - Event

5. EventCategory.cs

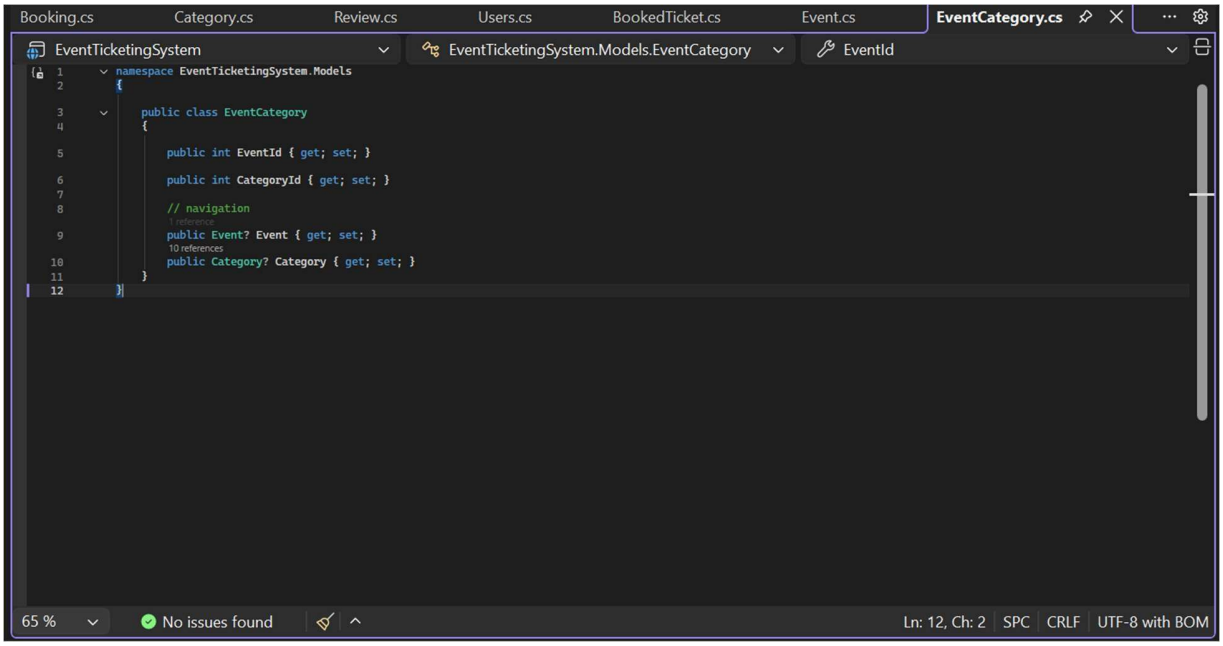


Figure 26:Data model - Event category

6. EventSession.cs

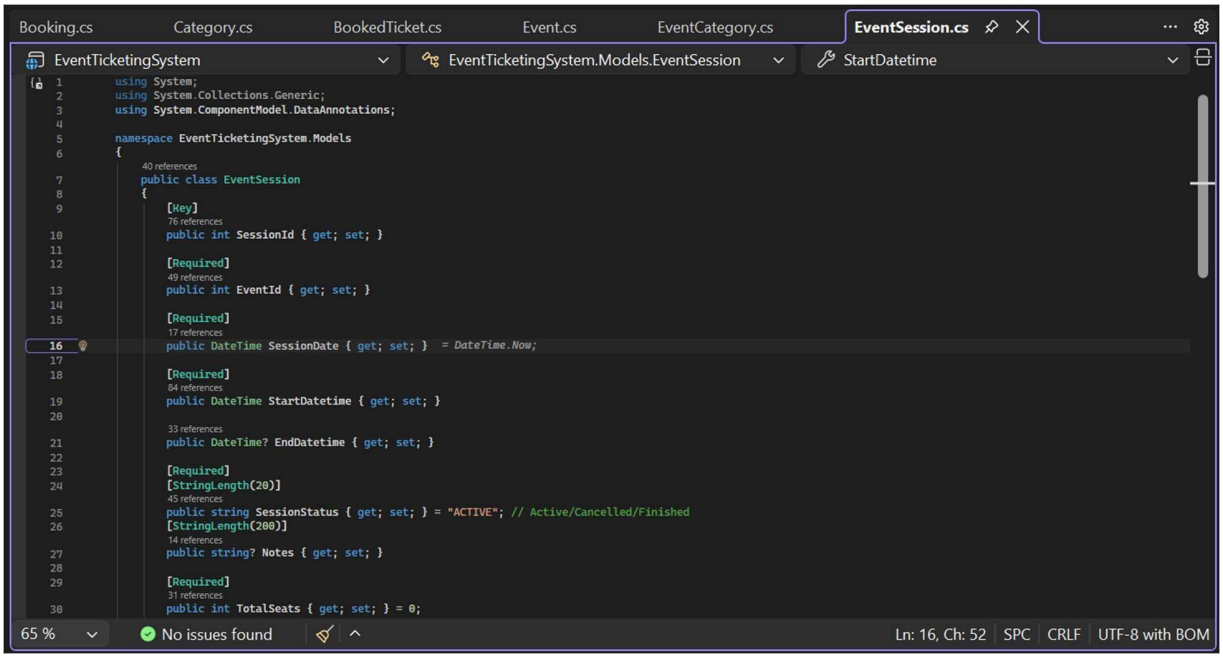


Figure 27: Data model - Event session

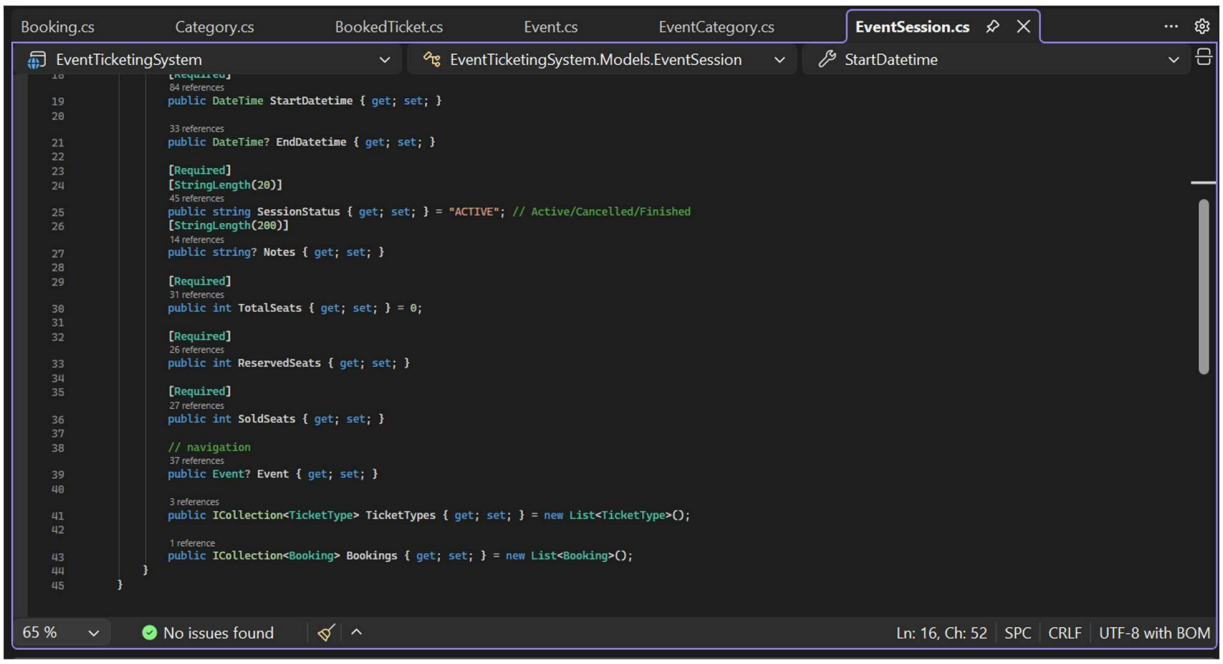


Figure 28: Data model - Event session

7. TicketType.cs

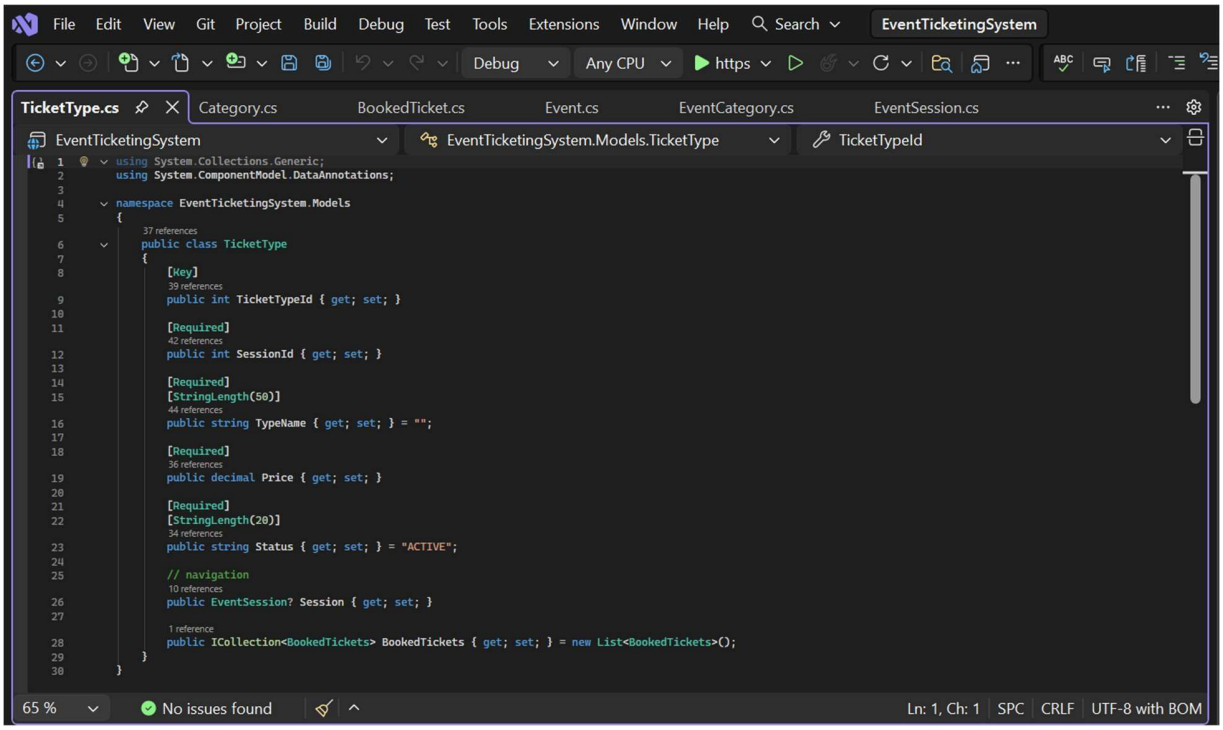


Figure 29:Data model - Ticket type

8. BookedTicket.cs

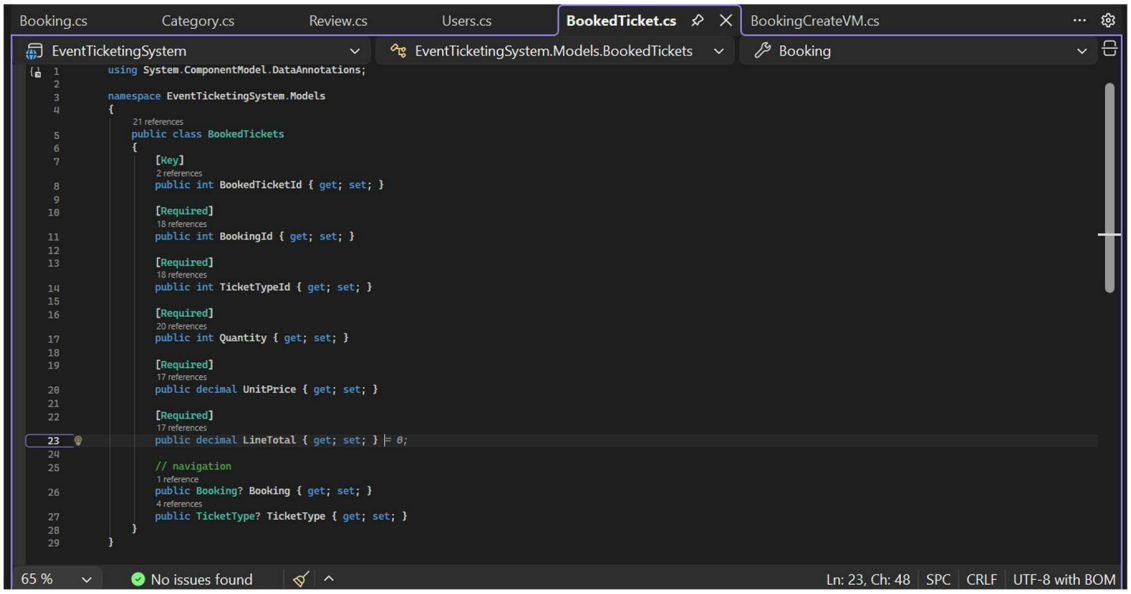


Figure 30:Data model - Booked ticket

9. Booking.cs

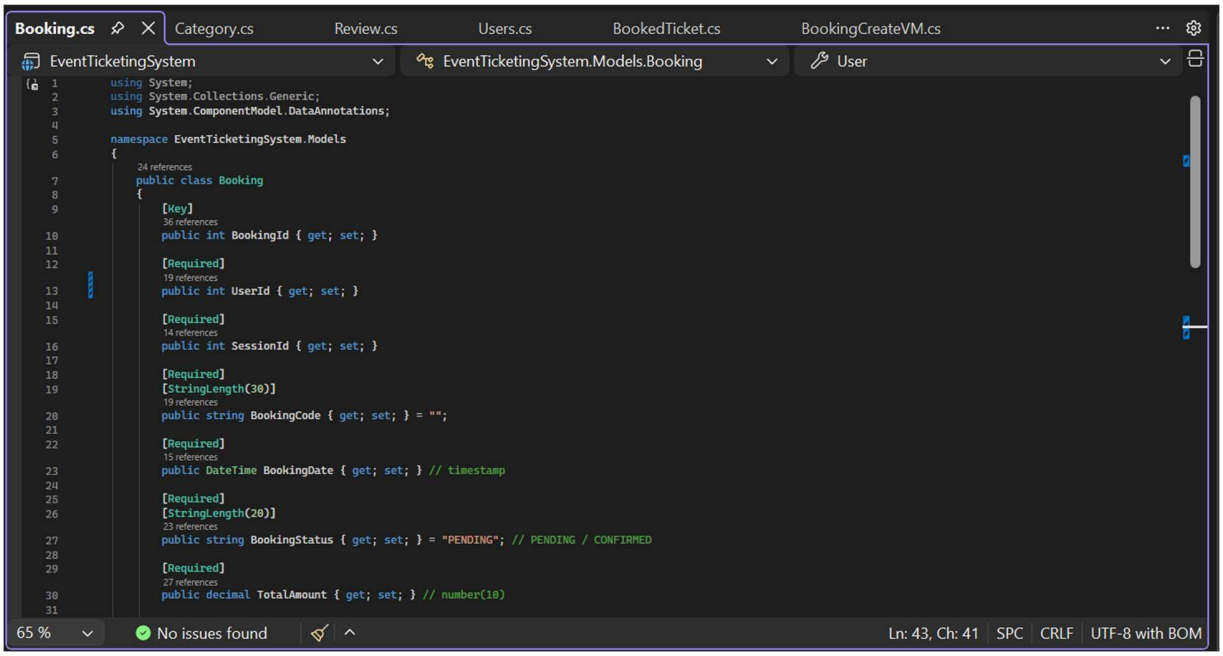


Figure 31:Data model - Booking

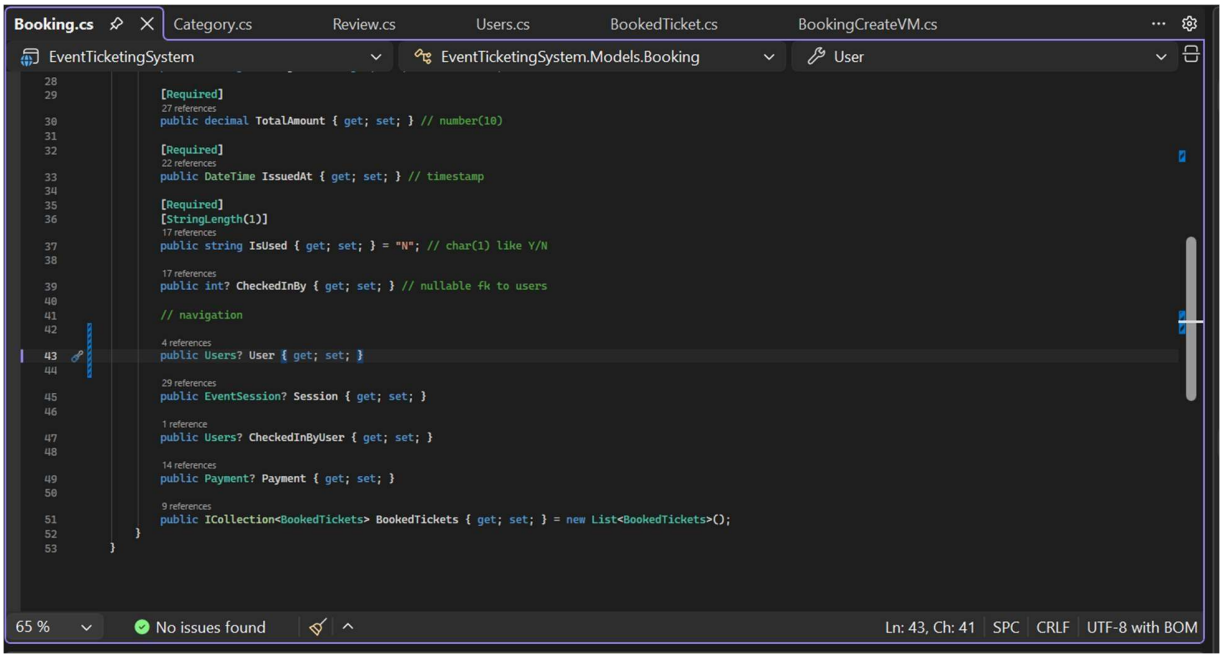


Figure 32: Data model - Booking

10. Payment.cs

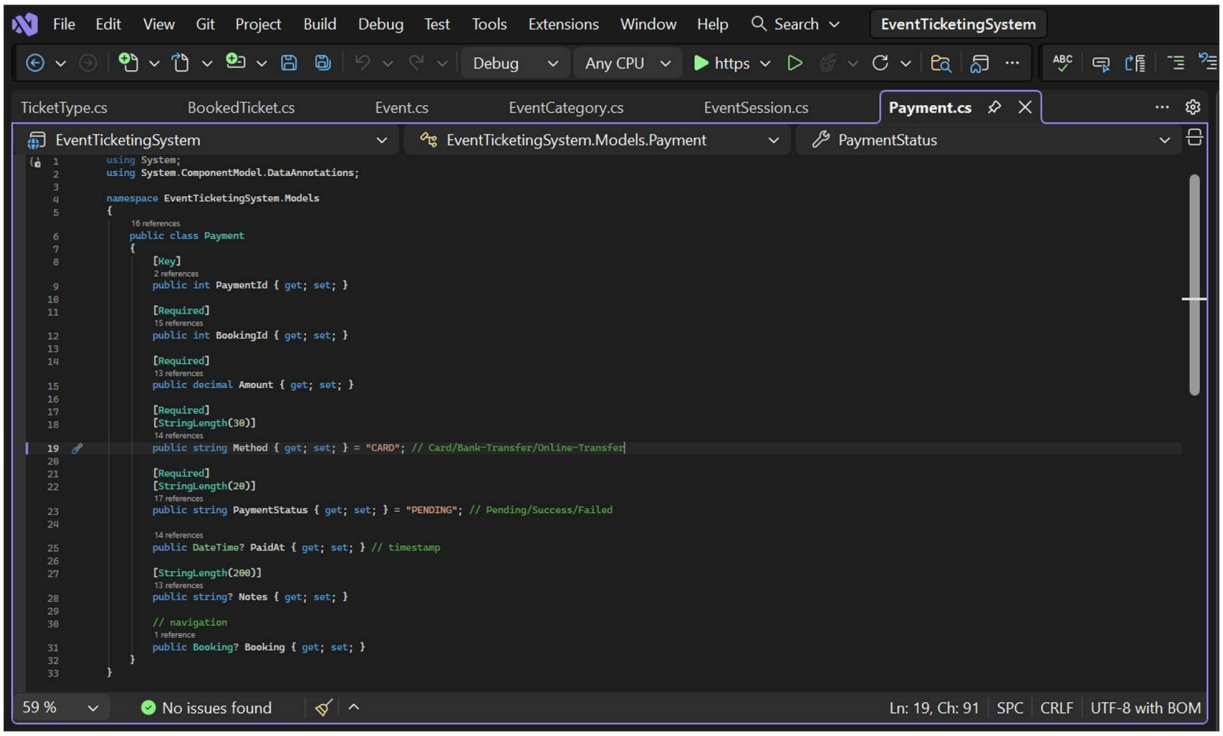


Figure 33: Data model - Payment

11. Review.cs

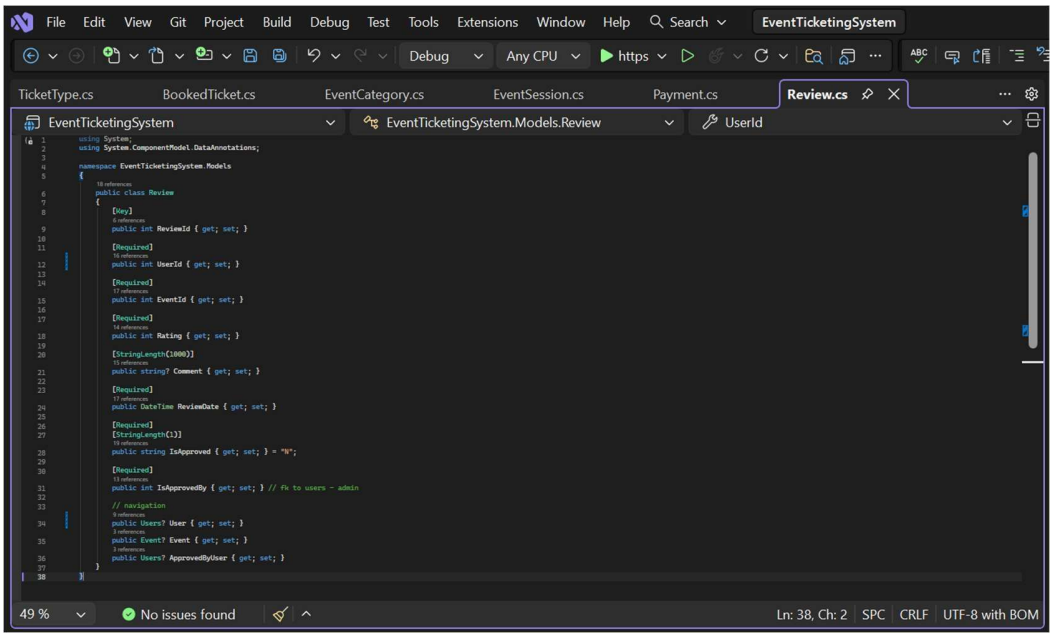


Figure 34: Data model - Review

12. Inquiry.cs

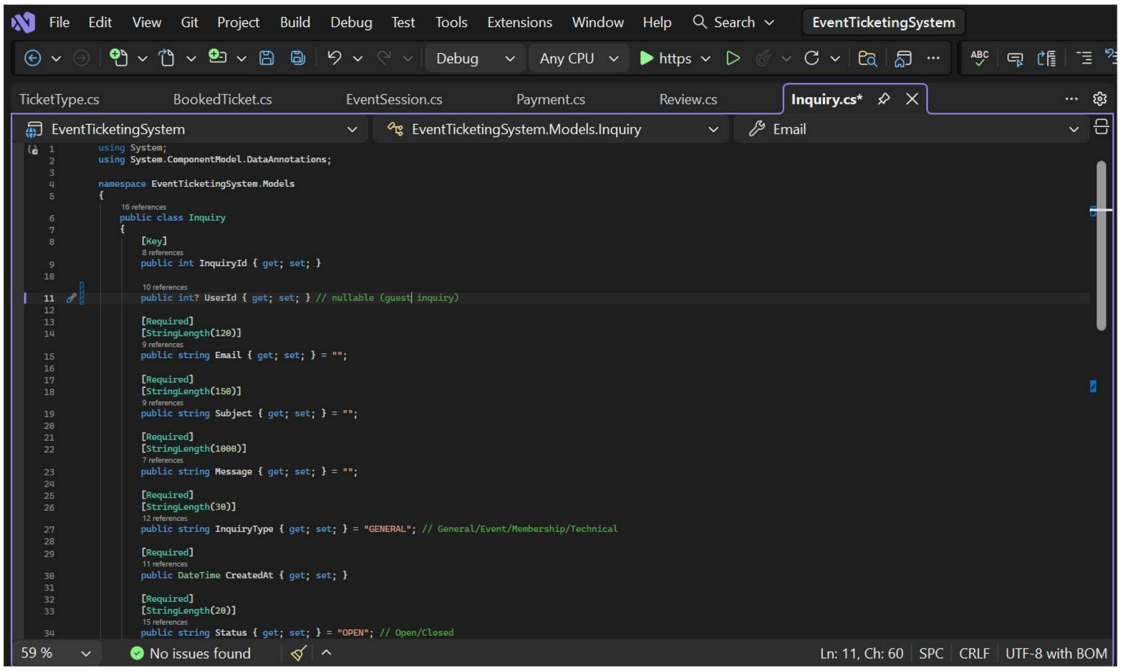


Figure 35:Data model - Inquiry

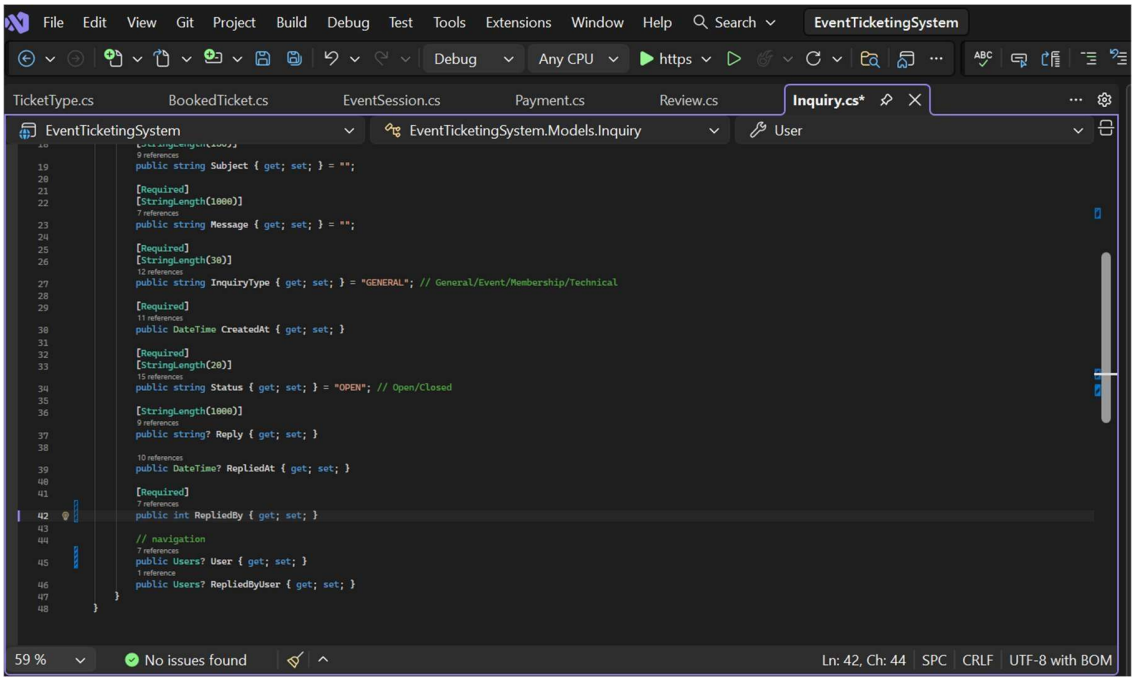


Figure 36: Data model - Inquiry

Utilized Database Constraints

Database constraints are regulations implemented at the schema level to guarantee data veracity, consistency, and integrity. In a transactional event management and ticketing context, constraints

are essential for enforcing business rules, preventing erroneous data entry, and ensuring dependable relationships across entities. The subsequent sections delineate all constraints found in the database design.

1. Primary Key Constraint

The primary key constraint distinctly identifies each entry within a table. Each principal entity in the database, including bookings, events, sessions, categories, and tickets, is allocated a primary key. This guaranteed that each record may be cited unequivocally and precludes duplicate entries inside a table. Primary keys serve as the basis for indexing, enhancing query performance, and are crucial for creating associations with other tables via foreign keys. Without primary key constraints, the database would lack structural dependability and consistency.

2. Foreign Key Constraint

Foreign key constraints maintain referential integrity between related tables by making sure a value in a child table corresponds to an existing value in a parent table. These constraints establish logical links between entities such as events and sessions, bookings and users, bookings and payments, and events and categories. Foreign keys prevent the generation of orphan records, such as bookings without valid sessions or payments without matching bookings. By maintaining referential integrity, foreign key constraints guarantee that the database appropriately reflects real-world linkages and dependencies.

3. Composite Key Constraint

Composite key constraints are applied to associative (junction) tables used to model many-tomany relationships. Instead of a single-column primary key, a combination of two or more foreign keys uniquely identifies each record. This ensures that duplicate associations, such as giving the same category to an event more than once, cannot occur. Composite keys assure uniqueness across a combination related items and preserve a clean, normalized relational structure.

4. Data Type Constraint

Data type constraints require rigorous typing for each column, ensuring that values recorded in the database match their intended format. Numeric types are used for quantities and monetary values, date and time types are used for timestamps, and string types are used for textual data. These

constraints prohibit inaccurate data, such as text in numeric fields or badly formed dates, from being stored. Proper data typing promotes accurate calculations, comparisons, and efficient query execution.

5. Not Null Constraint

The Not Null constraint ensures that necessary attributes always contain a value. Critical fields such as identities, timestamps, status values, and financial sums are declared as non-nullable to avoid incomplete or invalid entries from being retained. This constraint enforces fundamental business requirements, such as guaranteeing that a booking must have a date, a payment must have an amount, and an event should have a title and status. By preventing missing data, Not Null constraints increase data quality, reduce application failures, and enhance the dependability of reporting and analysis.

6. String Length Constraint

String length limits restrict the maximum number of characters that can be recorded in text-based columns. Attributes such as names, titles, status fields, codes, and descriptive text are assigned suitable length constraints. This provides consistency between database storage and user input, prevents excessive data storage, and protects the system from performance degradation caused by extremely long values. String length limitations also contribute to data validation and assist prevent against wrong or inadvertent input.

7. Default Constraint

Default constraints automatically assign default values to columns when no explicit value is provided during record insertion. Fields such as status indicators, flags, counters, and timestamps are initialized with relevant defaults to ensure that records are always produced in a proper condition. This lowers the need on application-level code for initialization and assures consistency across all records. Default constraints are particularly critical in transactional systems, where consistent initial conditions are necessary for accurate workflow execution.

8. Nullable Constraint (Optional Attributes)

Nullable constraints allow specific columns to store Null values, suggesting that the data is optional or may not yet be accessible. Fields such as descriptions, notes, end times, and

administrative references are designed to be nullable to enable real-world scenarios where material may be added at a later stage. This flexibility enables the database to reflect incomplete but legitimate states without violating data integrity. Nullable constraints mix stringent validation with practical usage.

9. Domain (Value) Constraint

Domain constraints confine column values to a preset set of permissible alternatives that indicate allowed business states. Examples include status fields, payment states, session states, and usage indications. These constraints prevent inaccurate or inconsistent values from being saved and provide uniform interpretation of data across the system. Domain constraints simplify querying, reporting, and application logic by ensuring that only recognized values exist in essential areas.

10. Business Rule Constraints (Logical Constraints)

Certain constraints represent business rules that govern system behaviour rather than mere structural principles. Examples include ensuring that ticket amounts are positive, limiting seat bookings above available capacity, allowing admission only after successful payment confirmation, and restricting administrative operations to authorized users. These constraints may be enforced using database Check constraints, application logic, or a combination of both. Together, they ensure that system activities follow stated business policies while preserving transactional integrity.

4.4 Insert Statements (Sample Data Population)

Sample data was populated using two approaches:

1. Seeder method (DbSeeder.Seed()) executed at application startup to insert a complete dataset to populate sample records for testing.
2. Manual INSERT statements executed in SQL Server for additional testing and validation of relationships.

A. Seeder Method Data Insertion and Outputs

DbSeeder File:

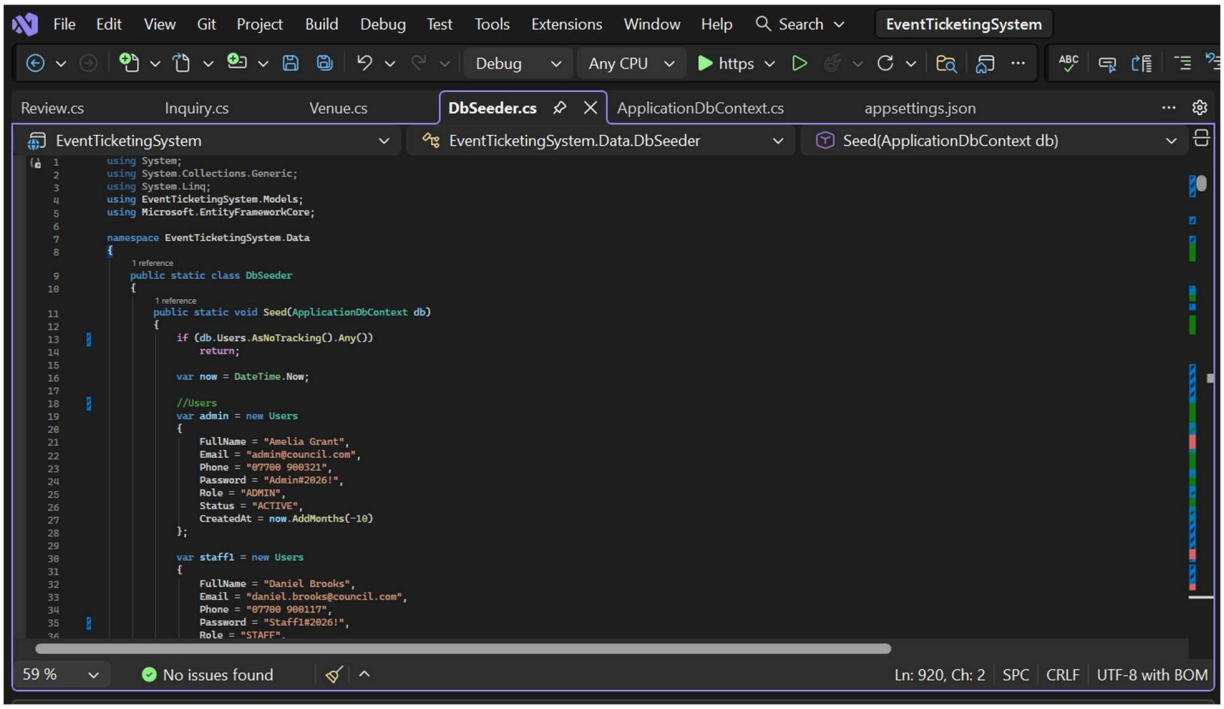


Figure 37: DbSeeder File

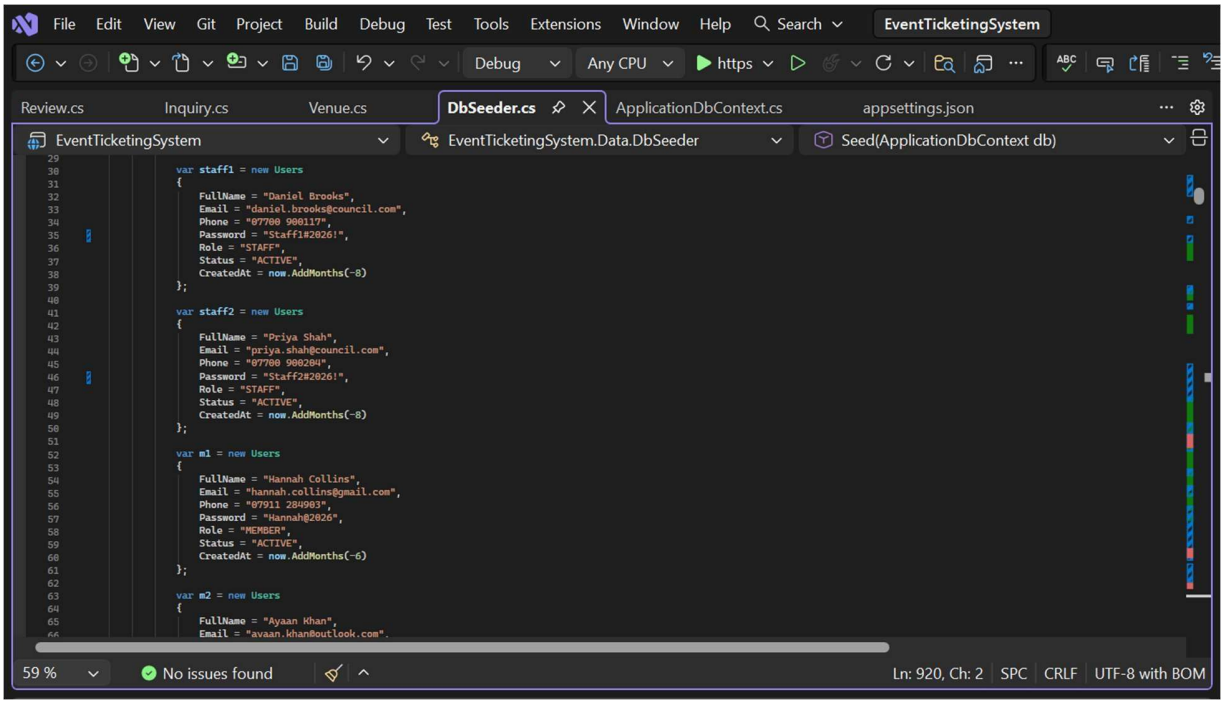


Figure 38: DbSeeder File

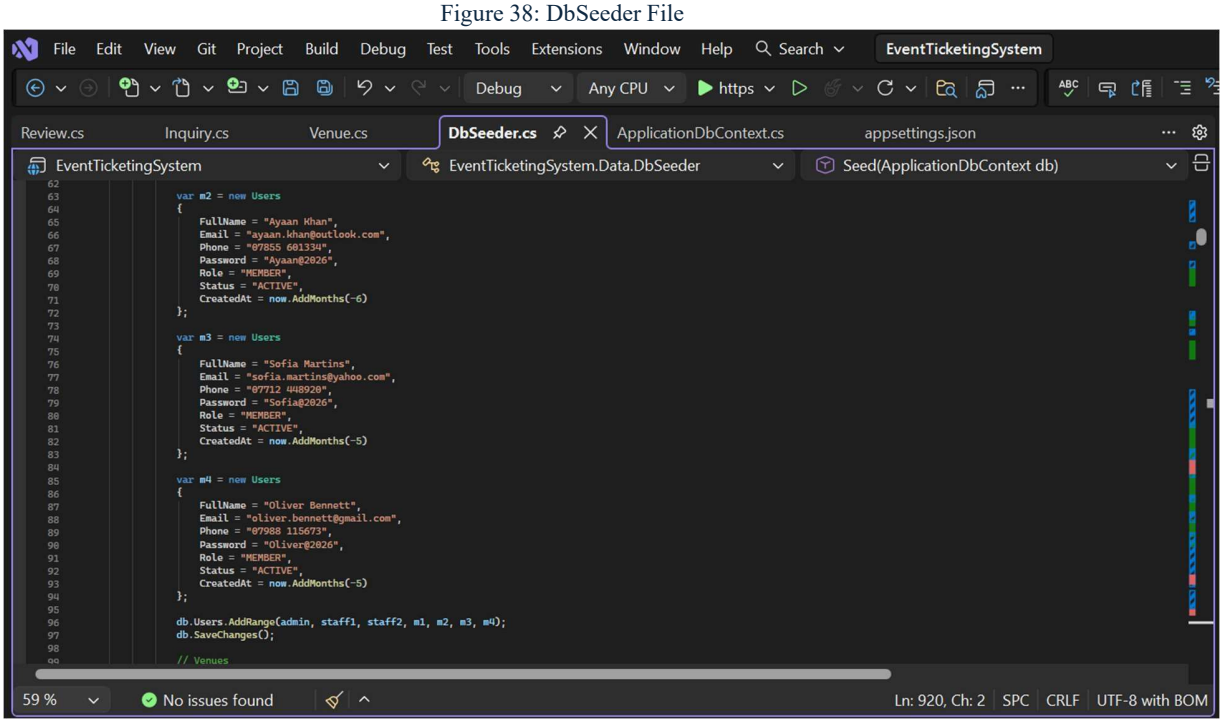


Figure 39: DbSeeder File

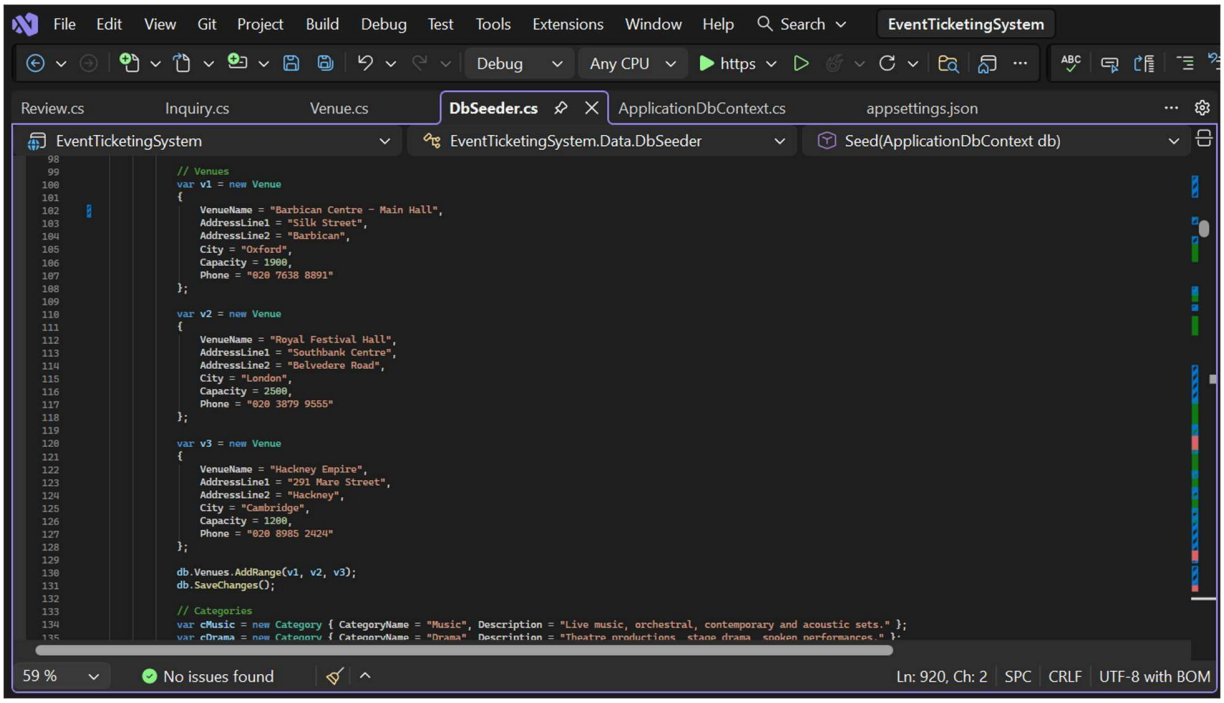


Figure 40: DbSeeder File

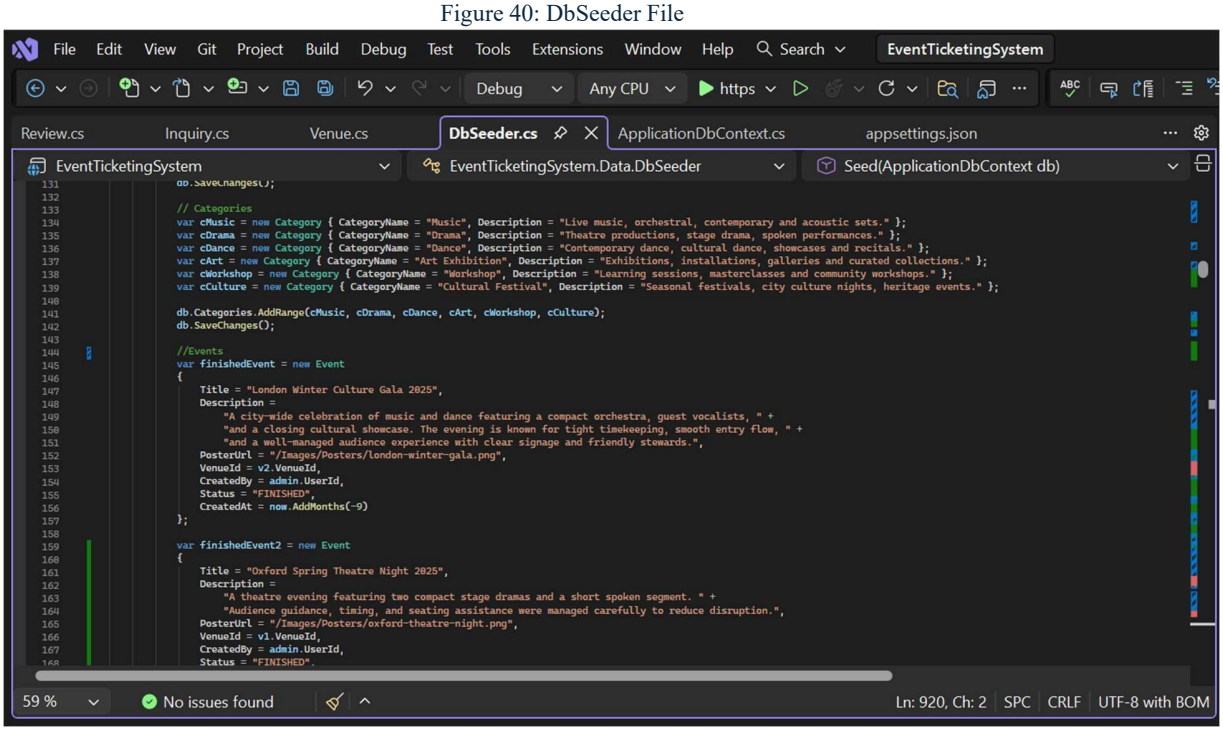


Figure 41: DbSeeder File

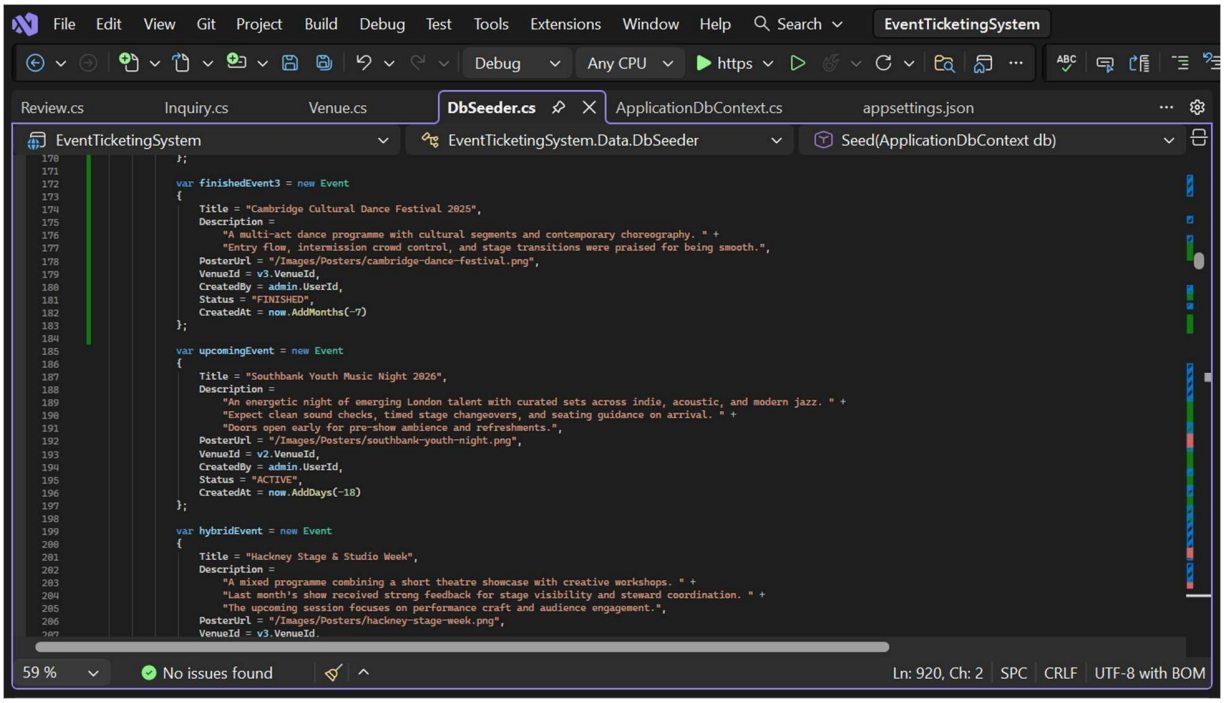


Figure 42: DbSeeder File

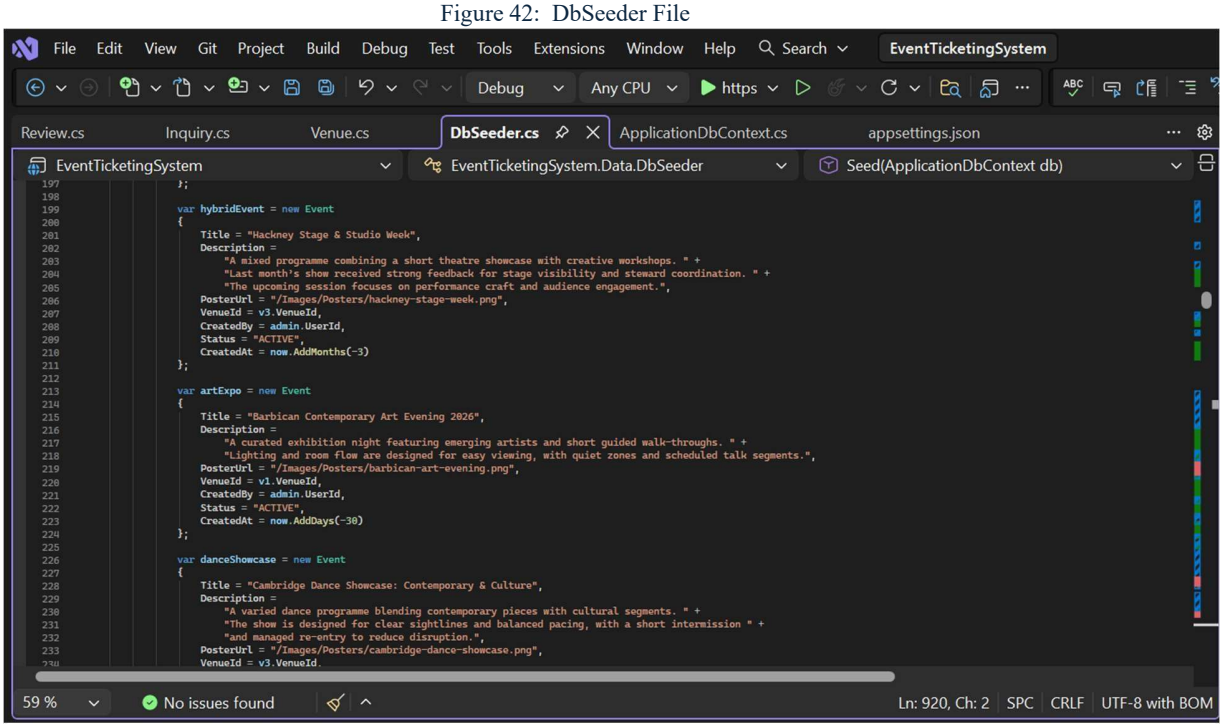


Figure 43: DbSeeder File

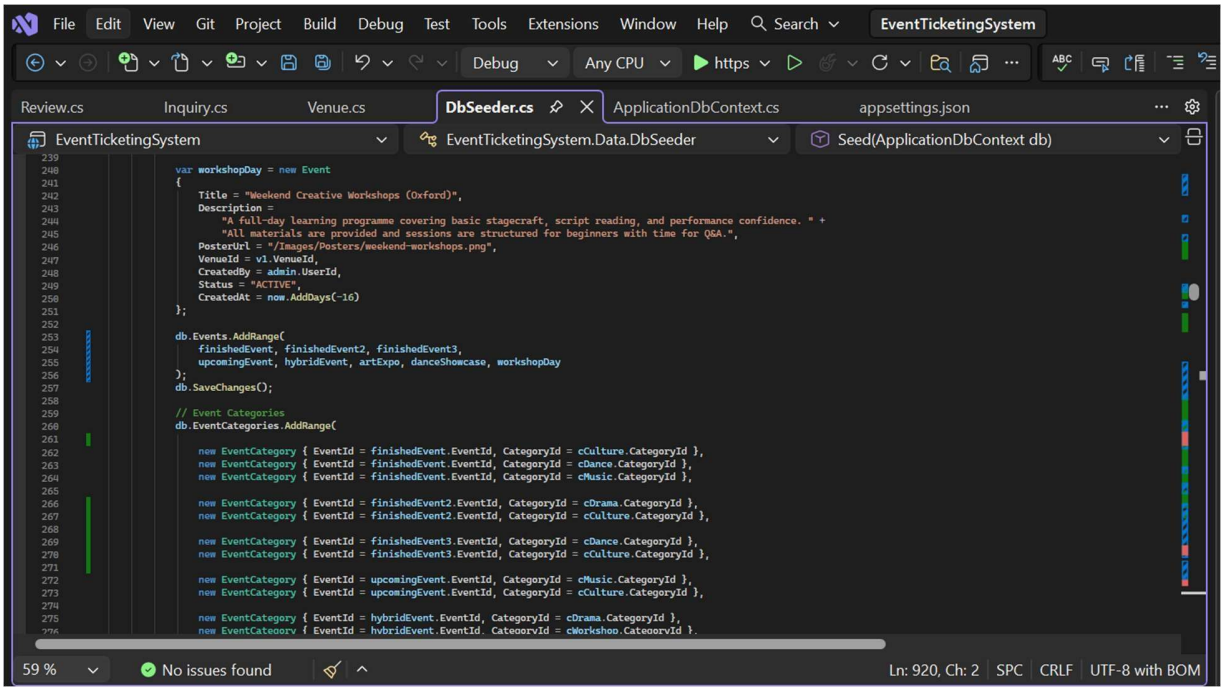


Figure 44: DbSeeder File

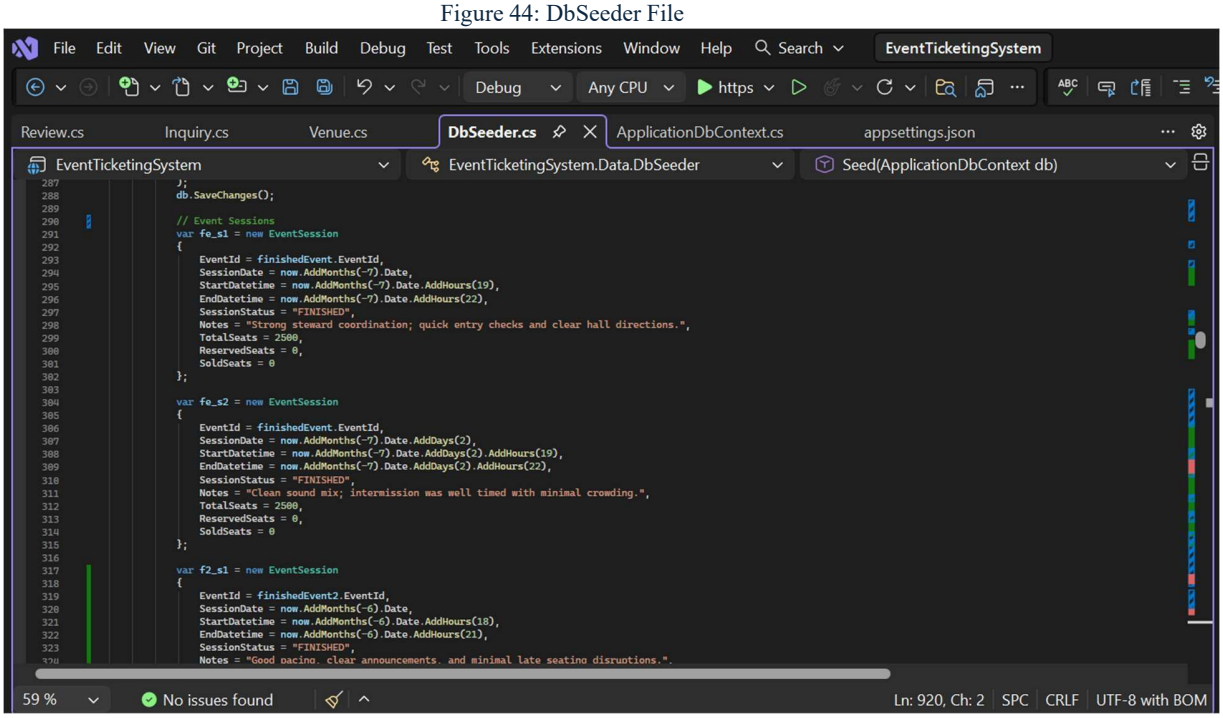


Figure 45: DbSeeder File

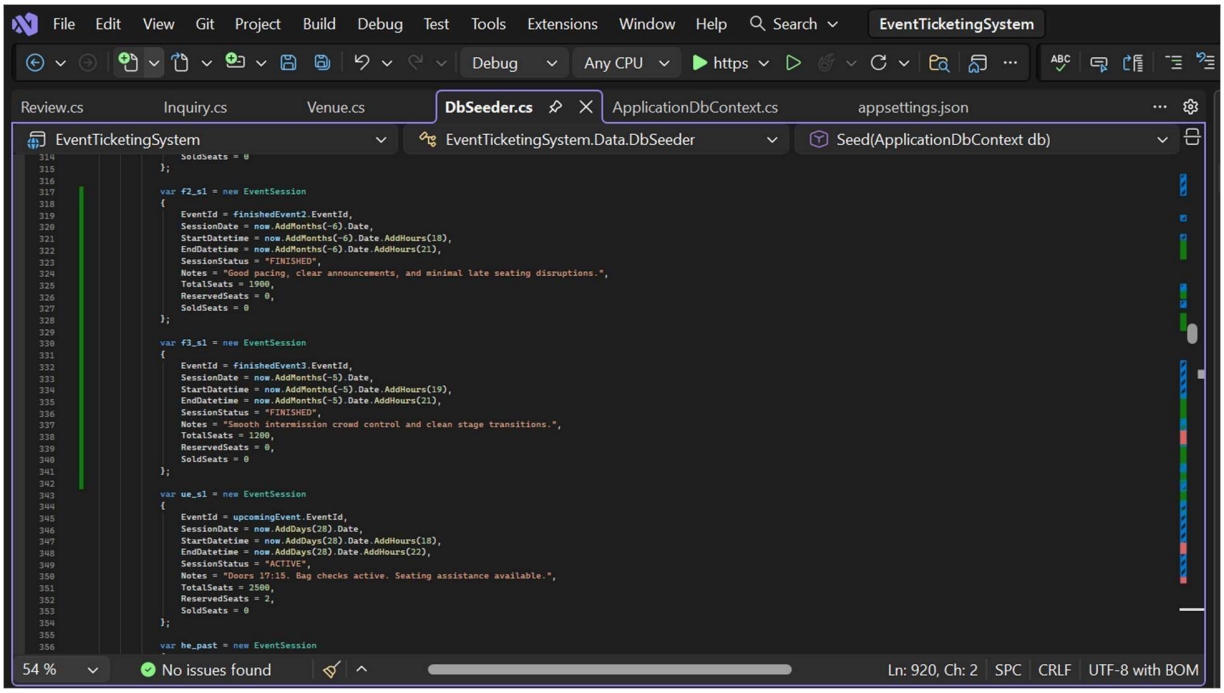


Figure 46: DbSeeder File

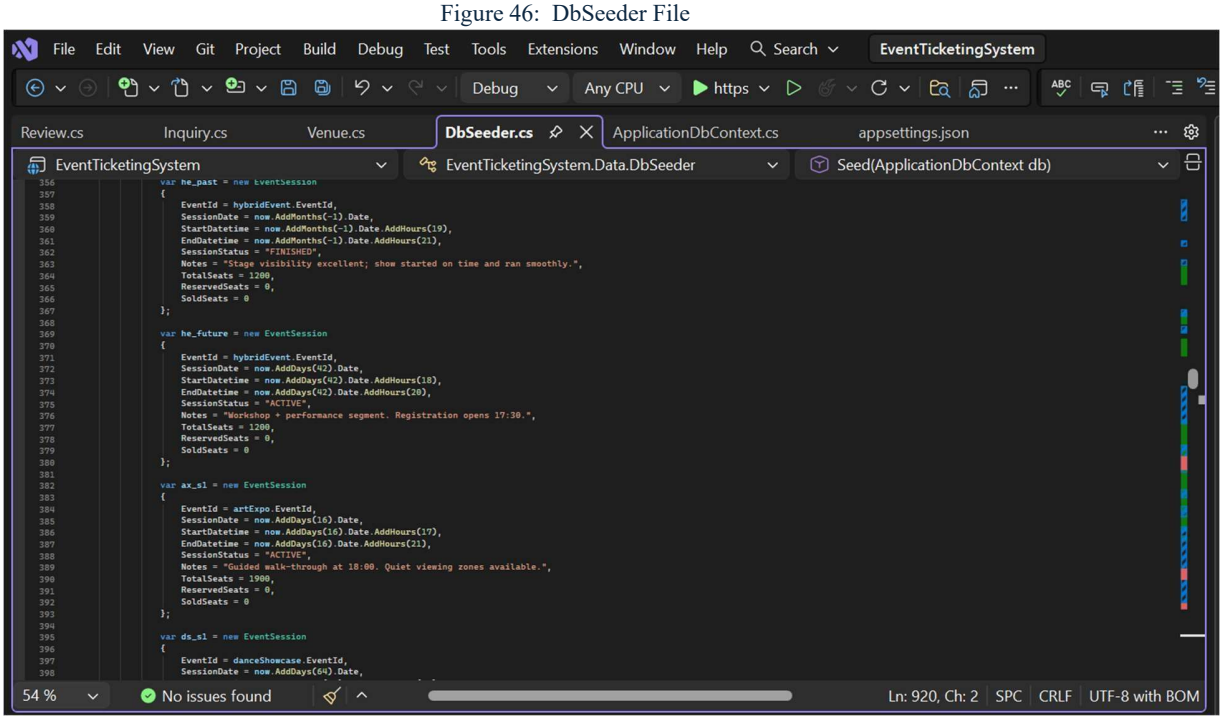


Figure 47: DbSeeder File

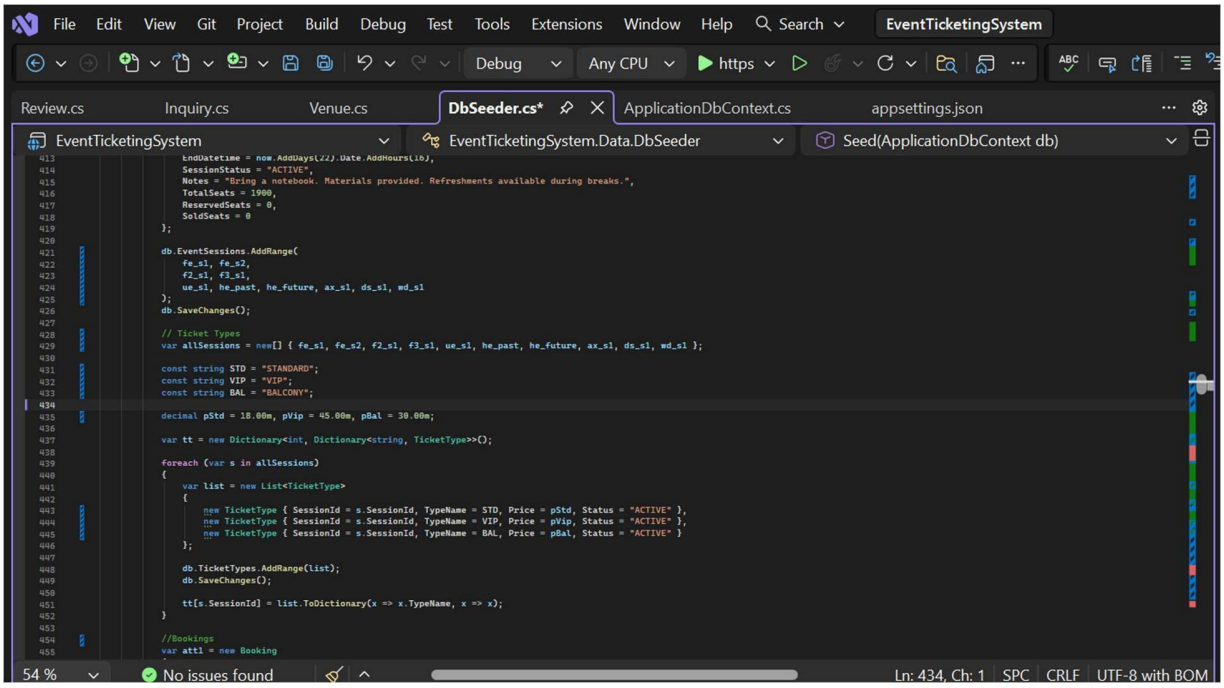


Figure 48: DbSeeder File

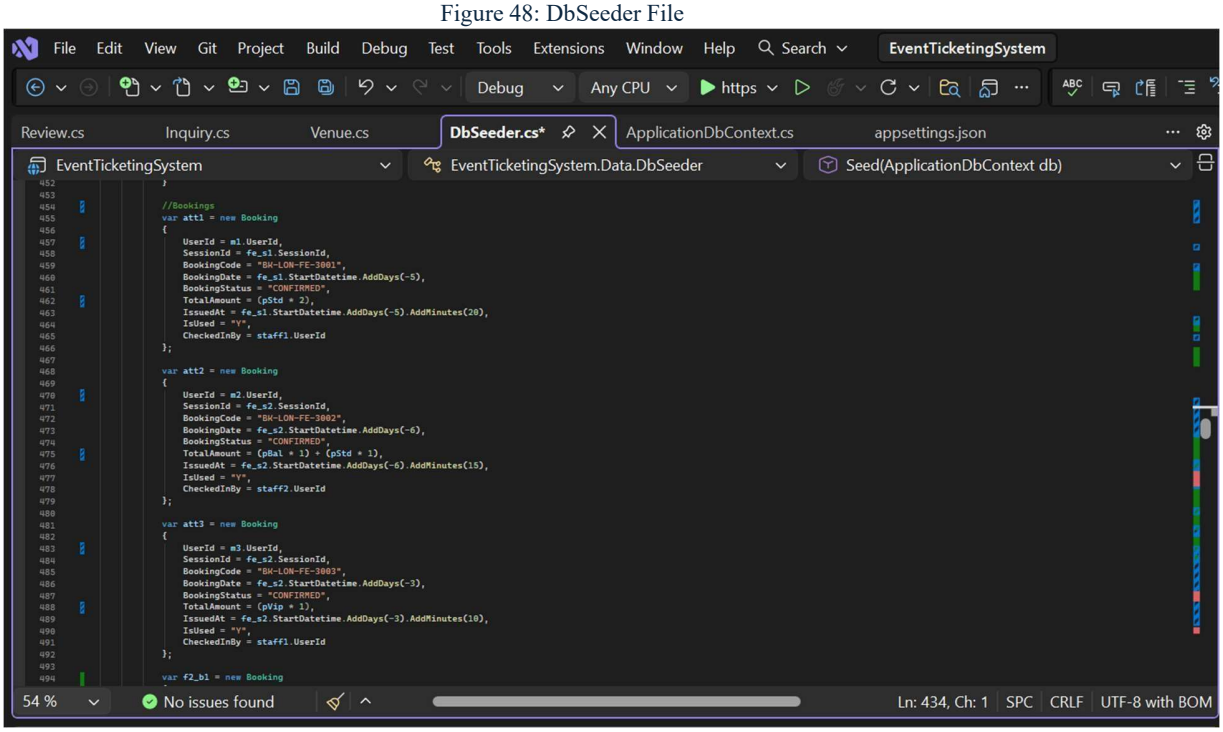


Figure 49: DbSeeder File

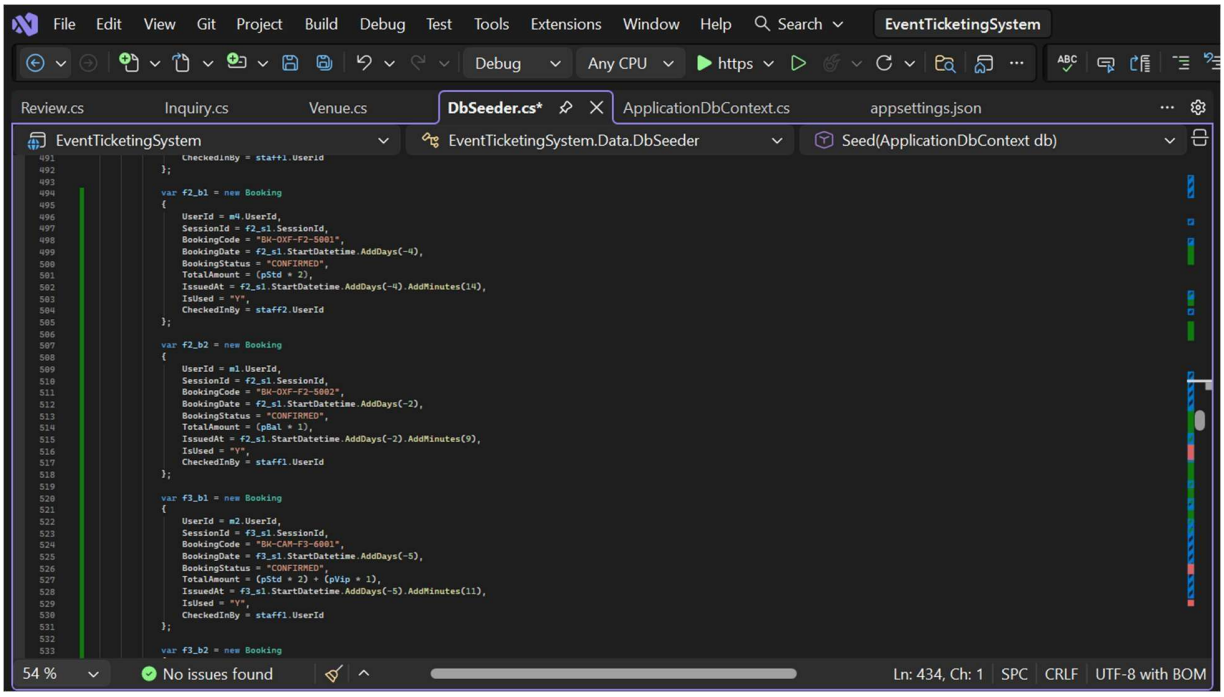


Figure 50: DbSeeder File

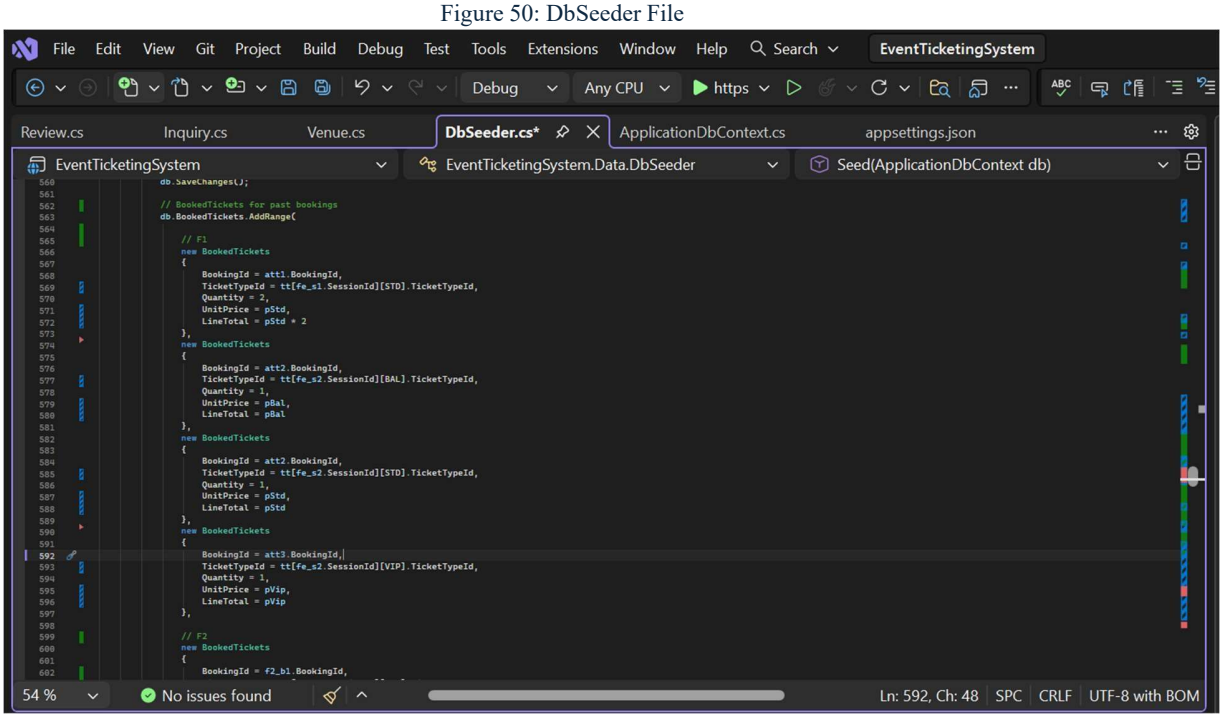


Figure 51: DbSeeder File

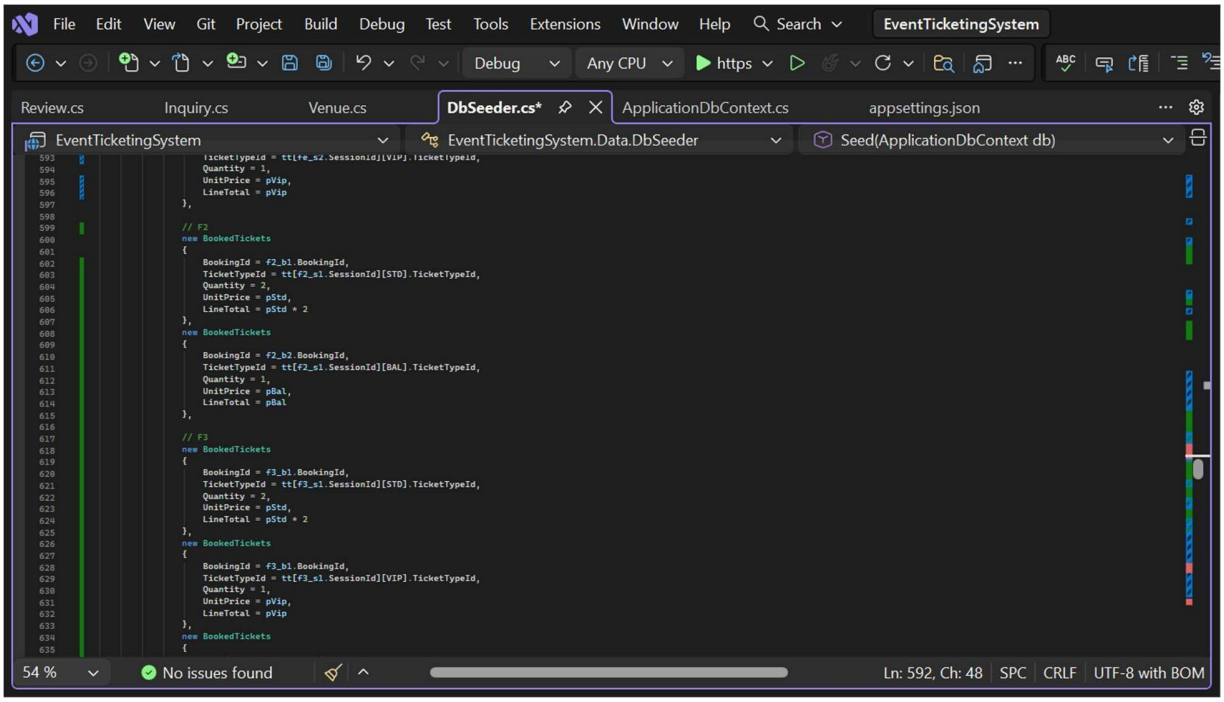


Figure 52: DbSeeder File

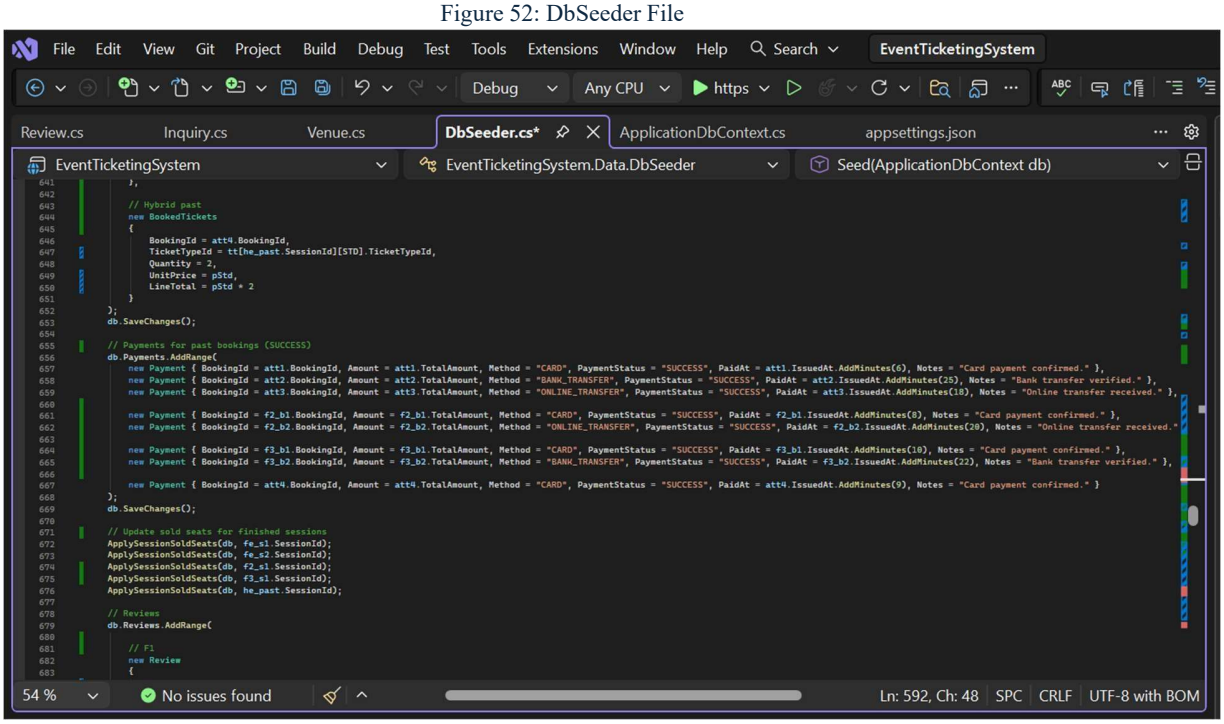


Figure 53: DbSeeder File

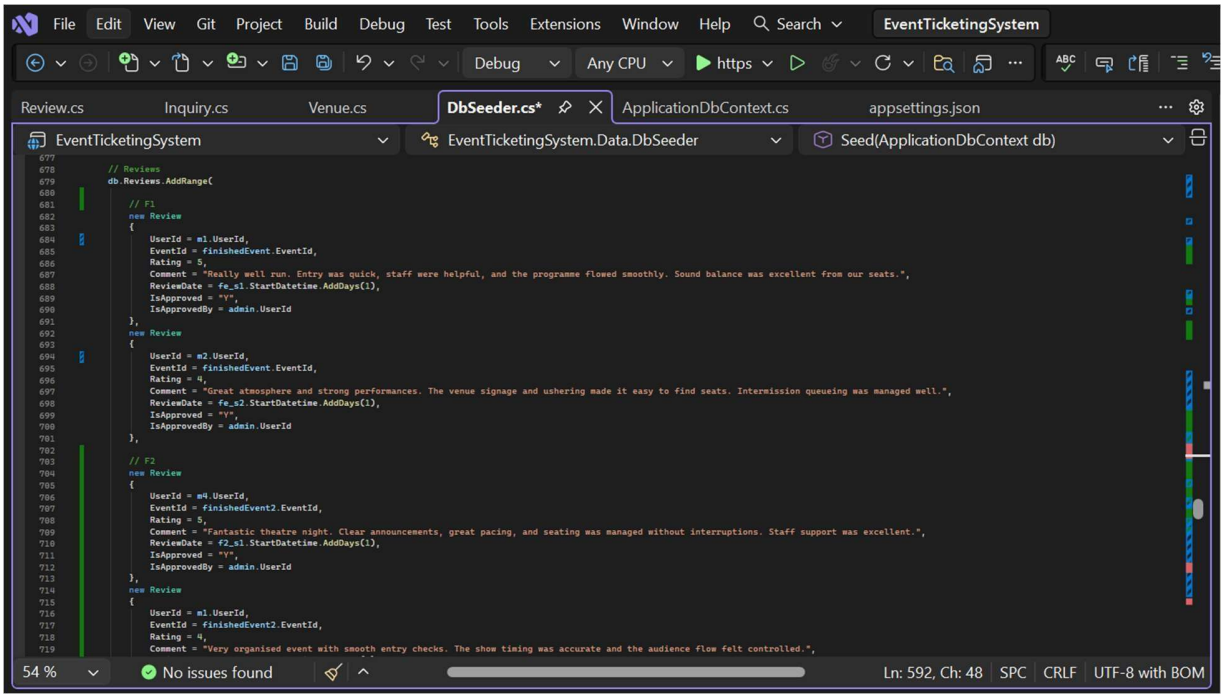


Figure 54: DbSeeder File

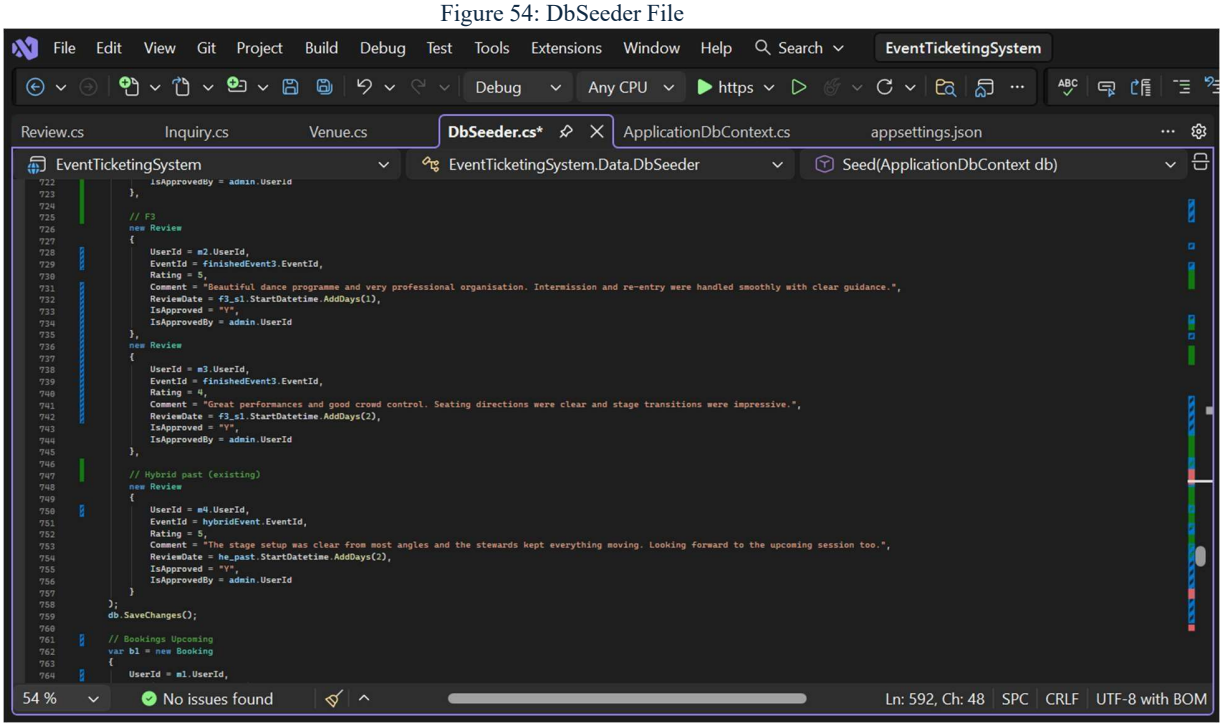


Figure 55: DbSeeder File

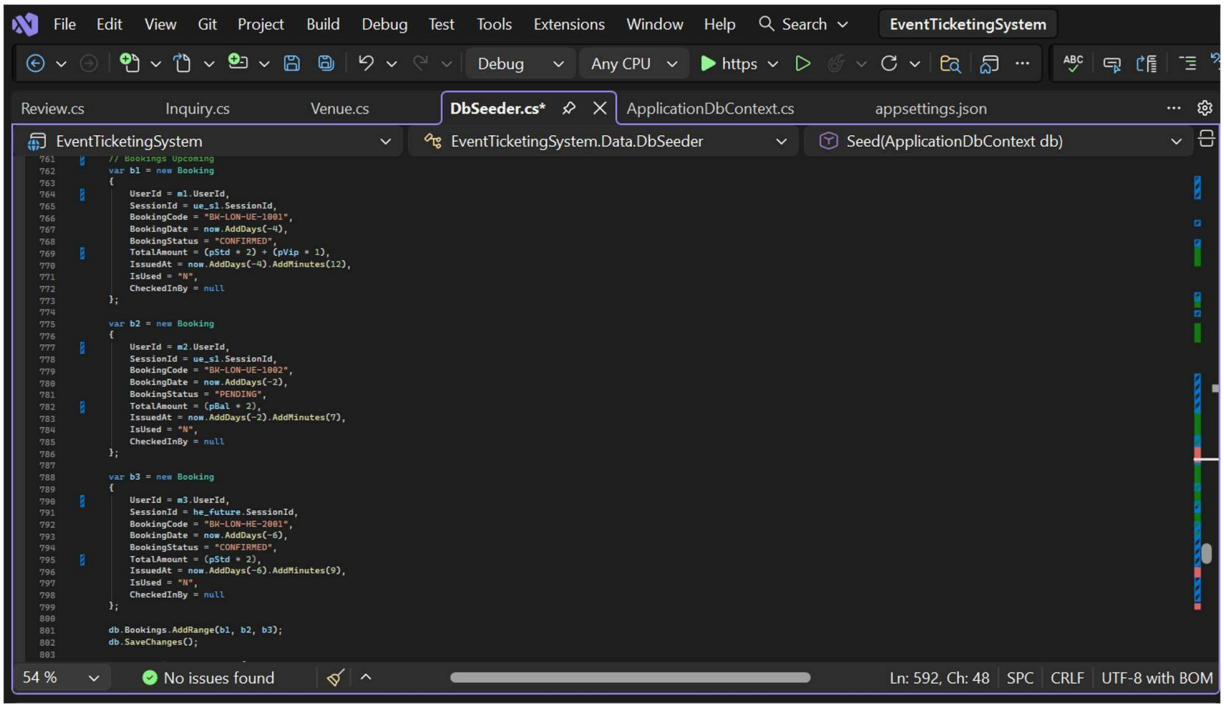


Figure 56: DbSeeder File

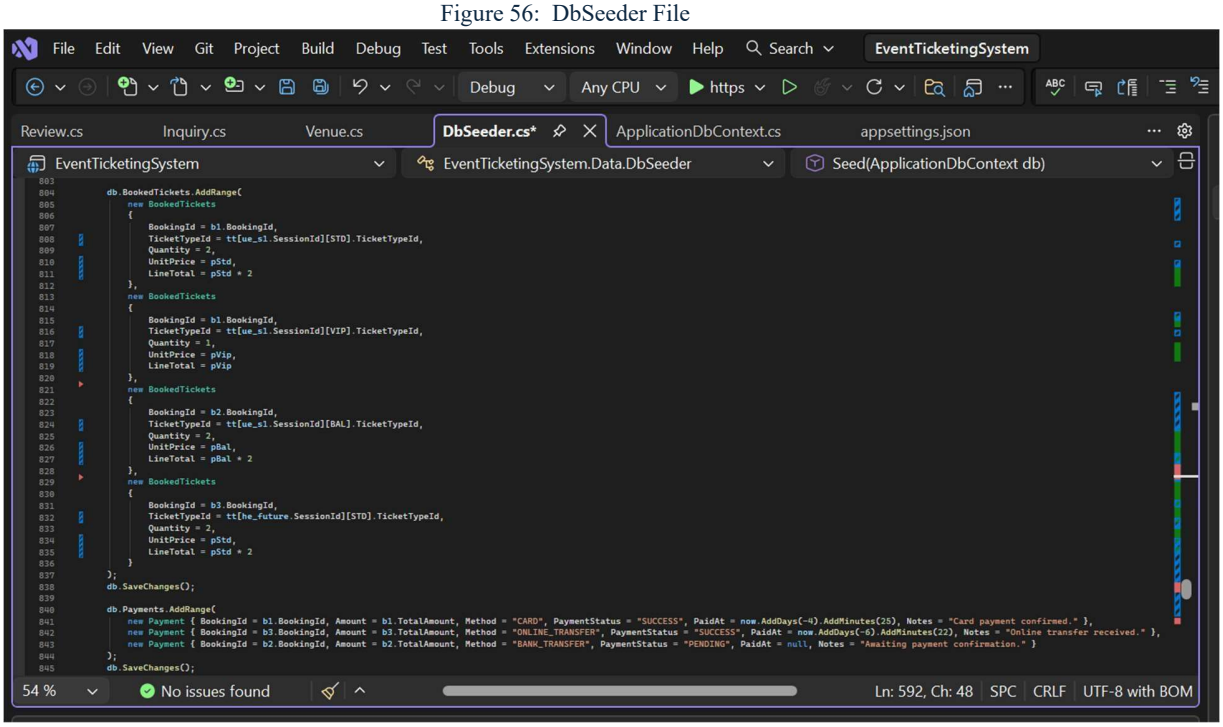


Figure 57: DbSeeder File

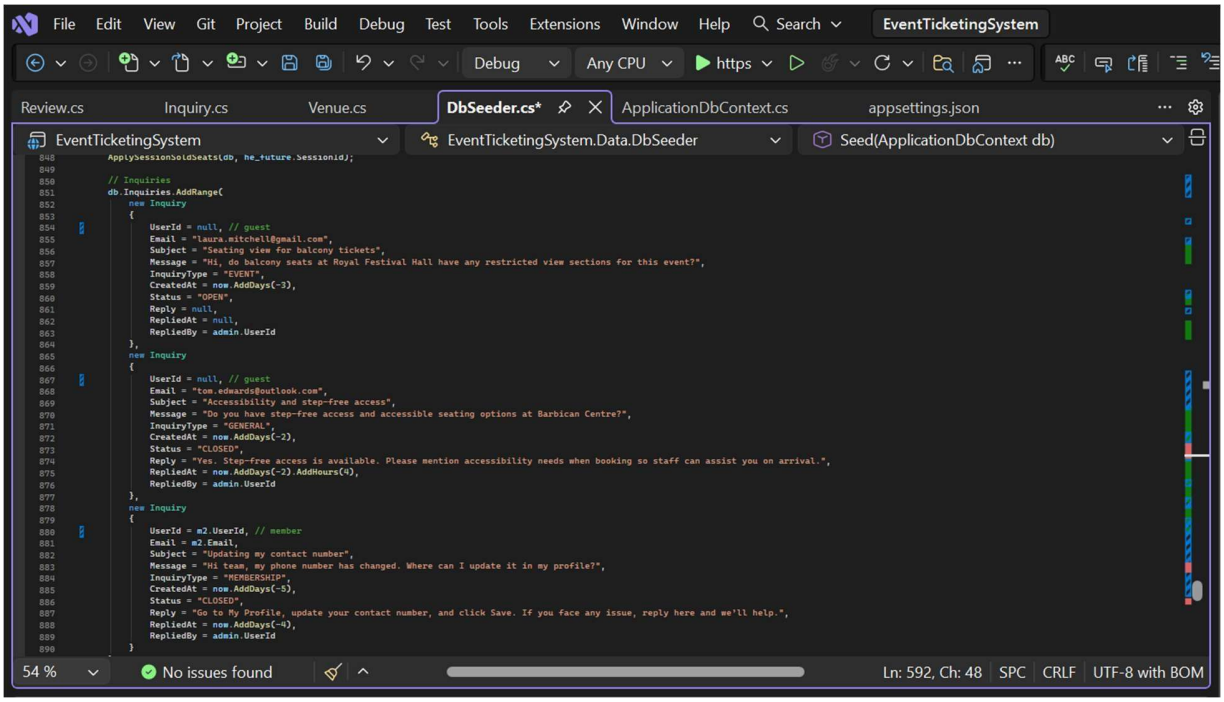


Figure 58: DbSeeder File

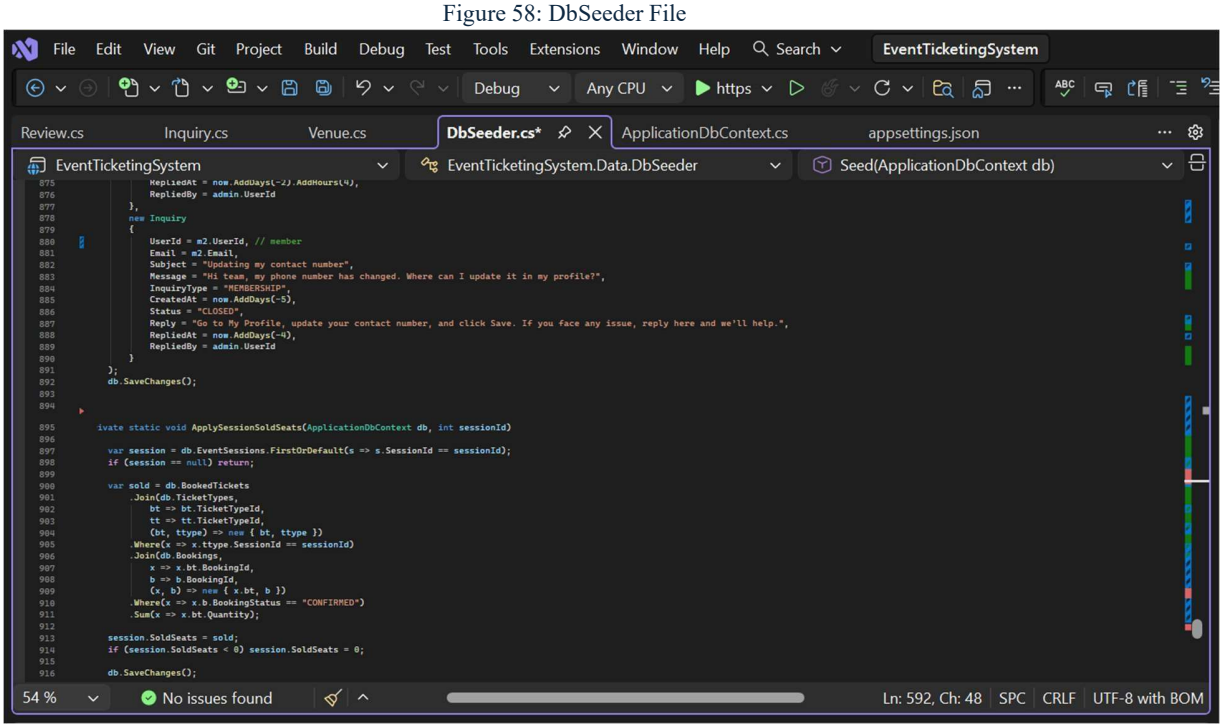


Figure 59: DbSeeder File

Seeded Data Outputs in SQL Server Tables:

1. User

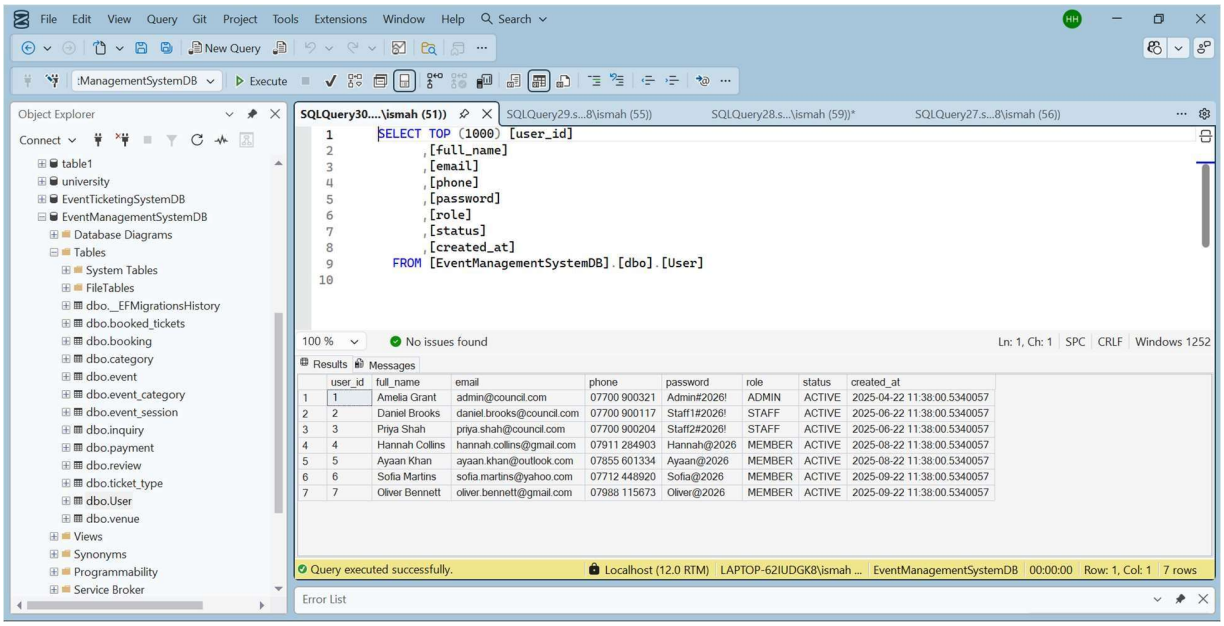


Figure 60: Output Table.

2. Venue

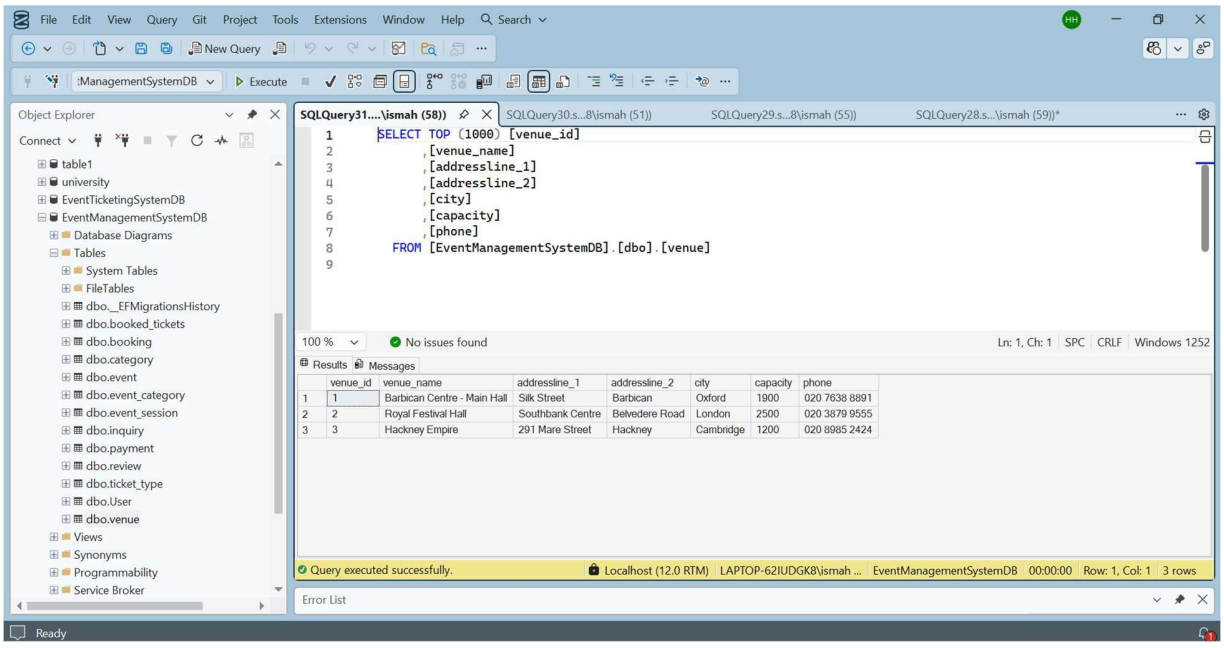


Figure 61:Output Table

3. Category

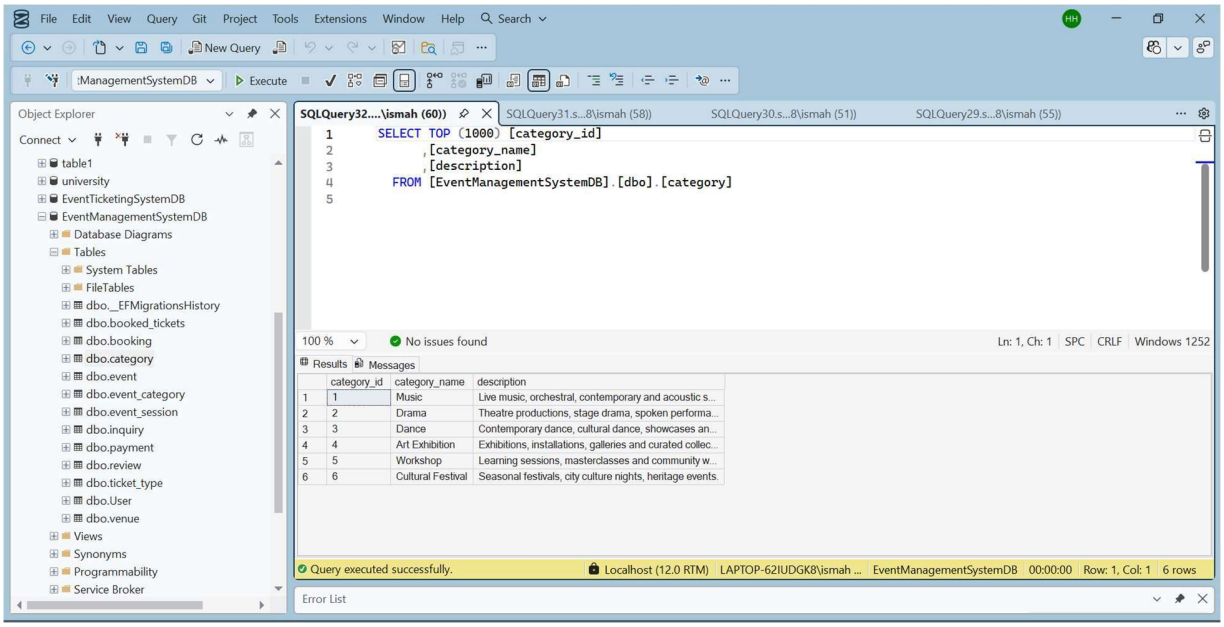


Figure 62: Output Table.

4. Event

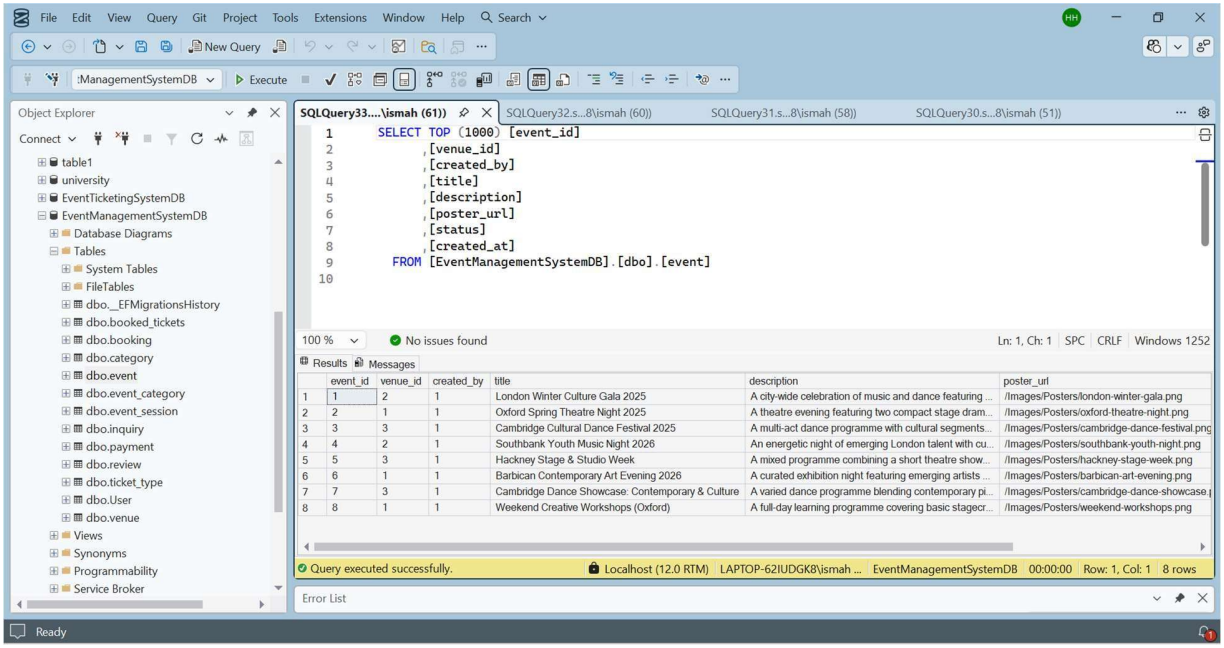


Figure 63: Output Table.

5. Event Category

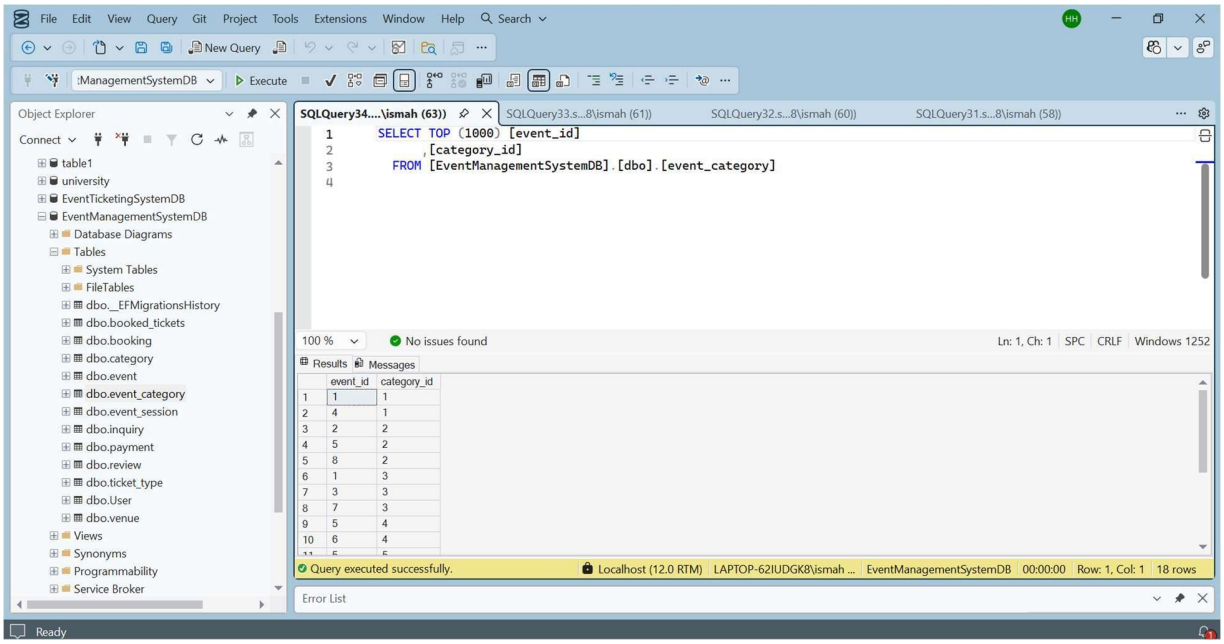


Figure 64: Output Table.

6. Event Session

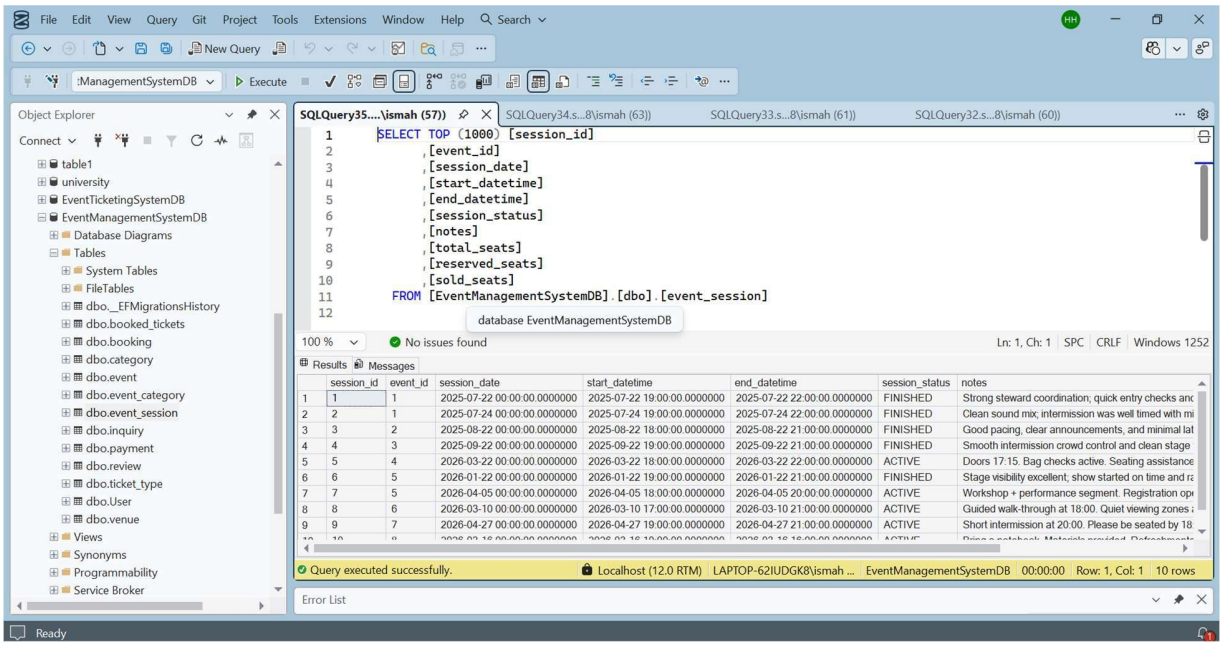


Figure 65: Output Table.

7. Ticket Types

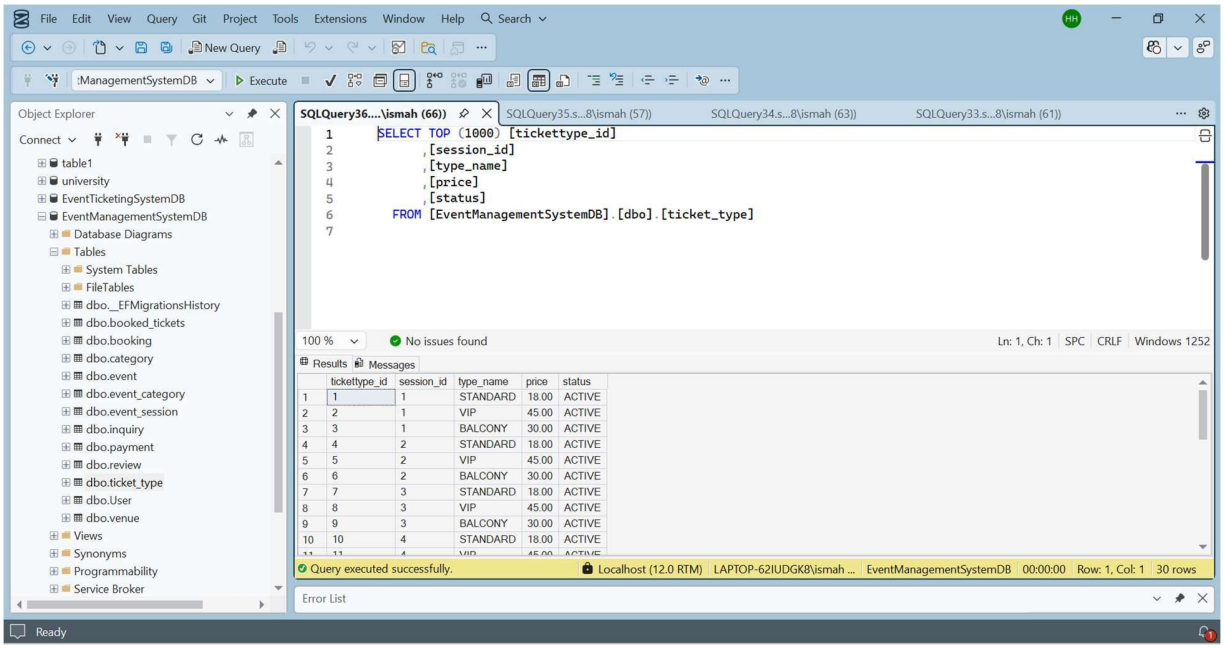


Figure 66: Output Table.

8. Booked Tickets

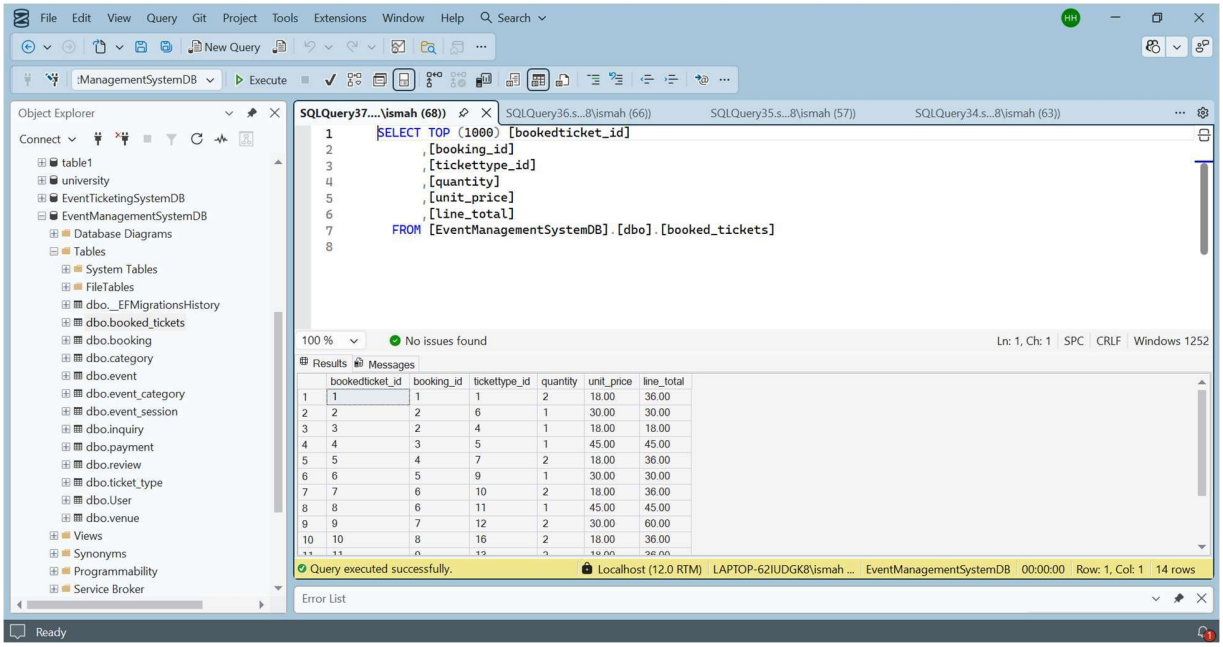


Figure 67: Output Table.

9. Booking

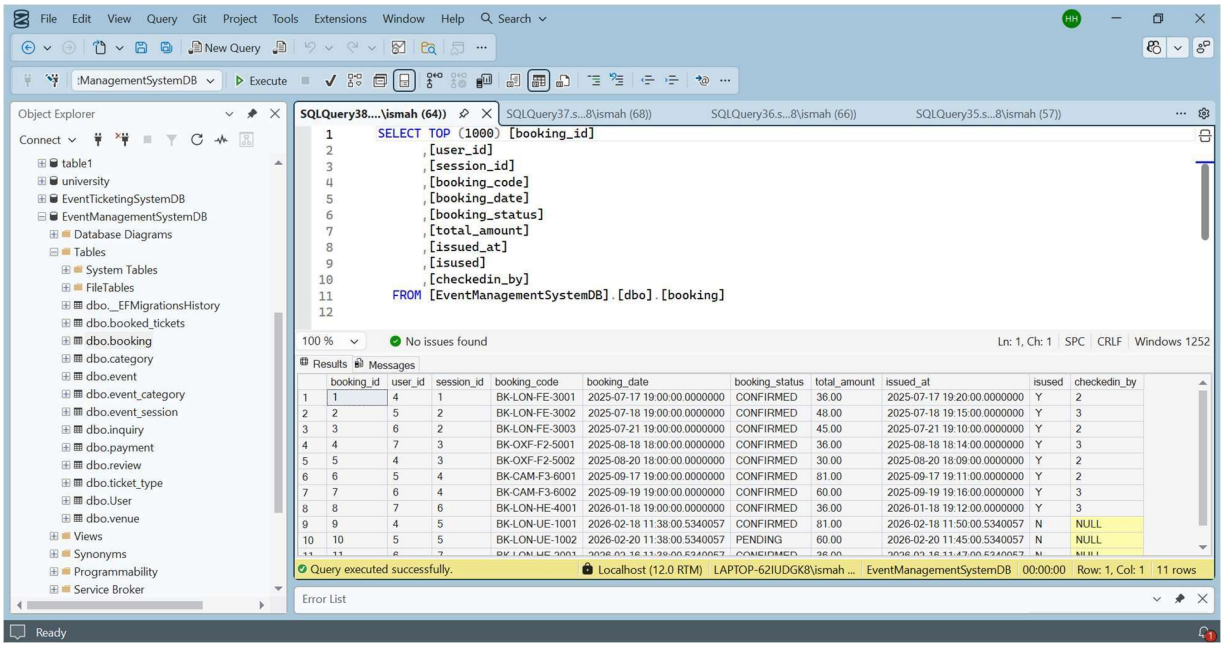


Figure 68:Output Table.

10. Payment

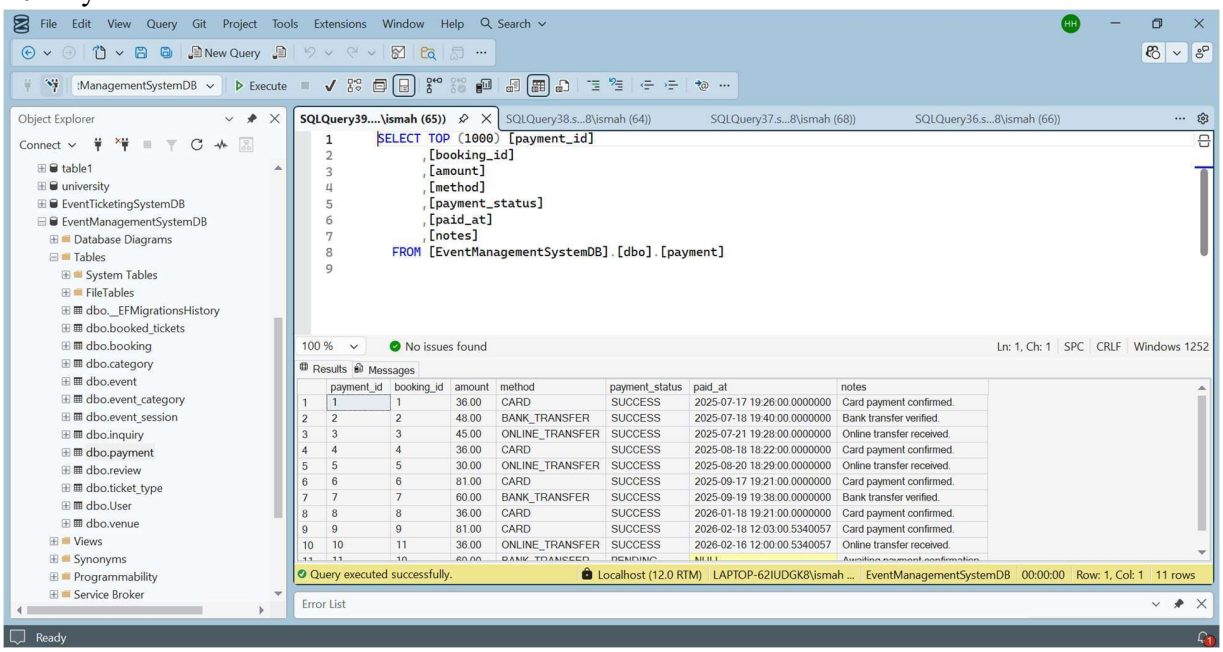


Figure 69: Output Table.

11. Review

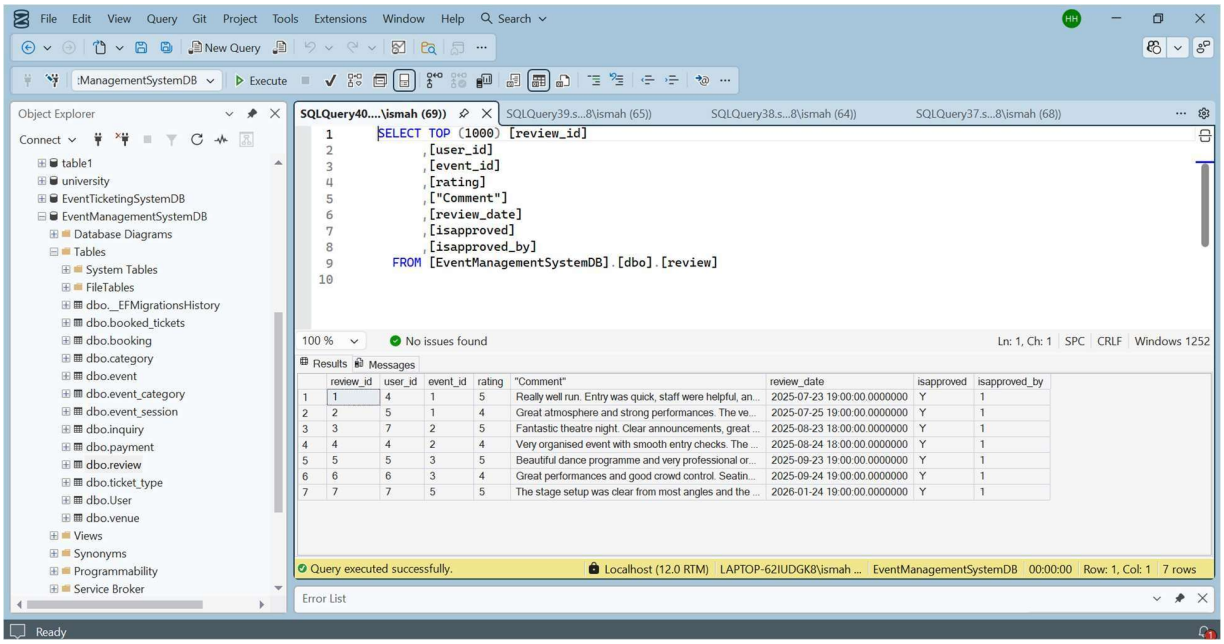


Figure 70: Output Table.

12. Inquiry

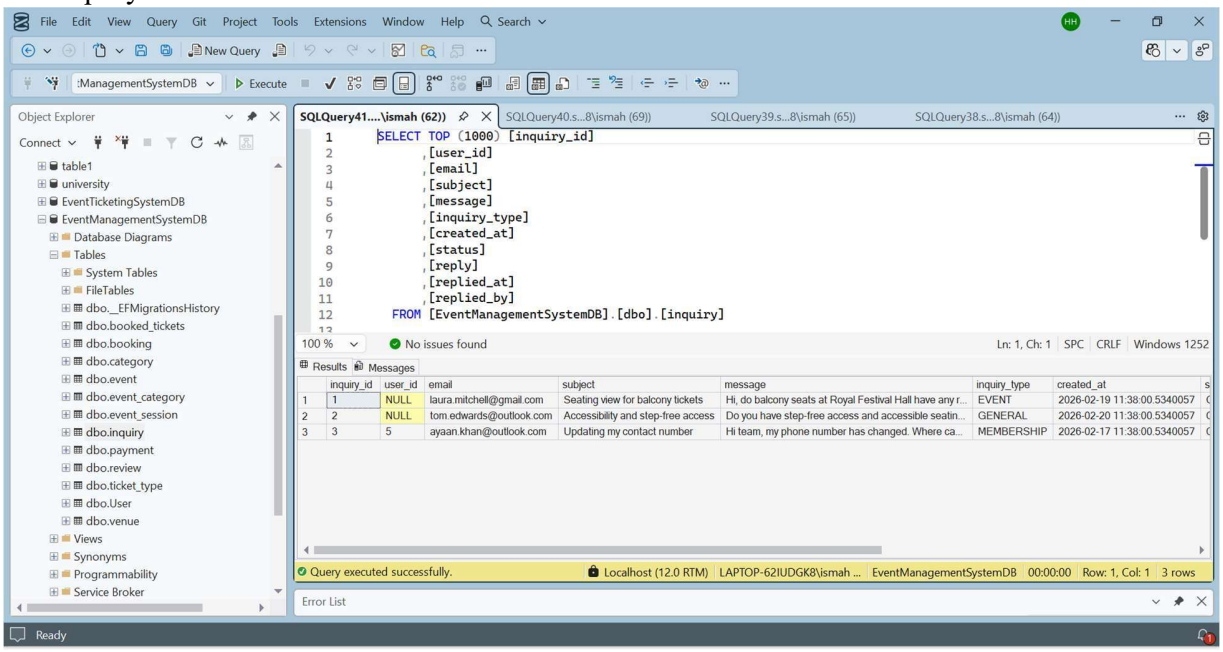


Figure 71: Output Table.

B. Inserting Data using INSERT Method Inserting a

Venue:

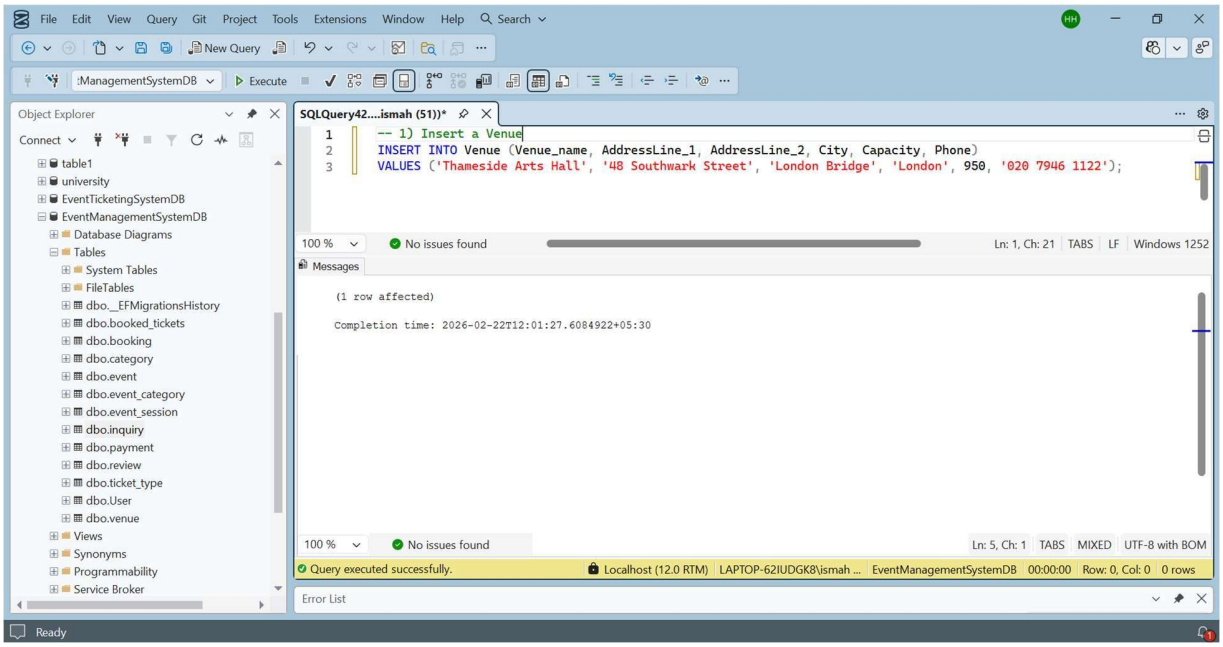


Figure 72: INSERT data.

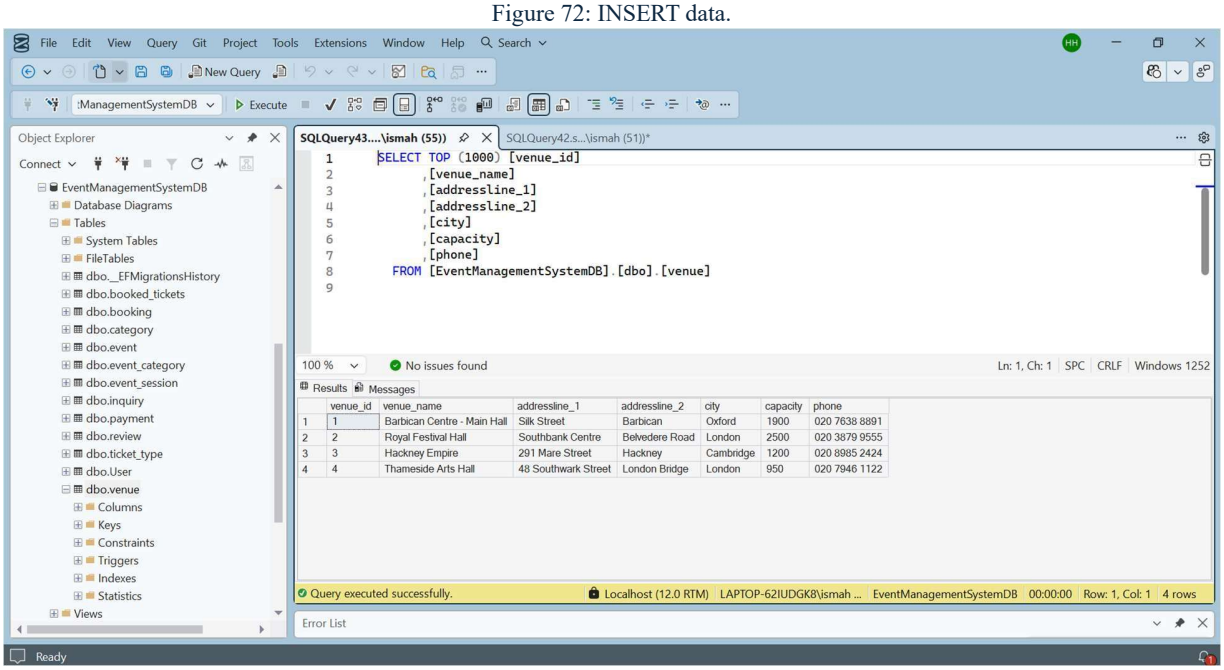


Figure 73: Output Table.

Inserting a Category:

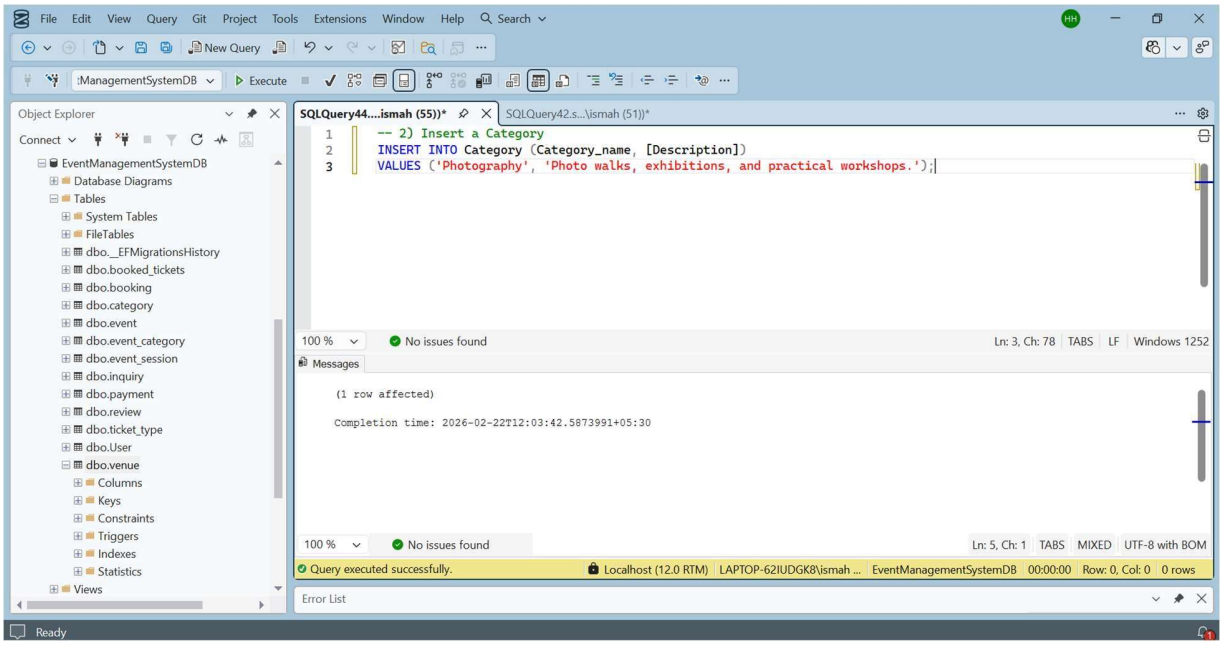


Figure 74: INSERT data.

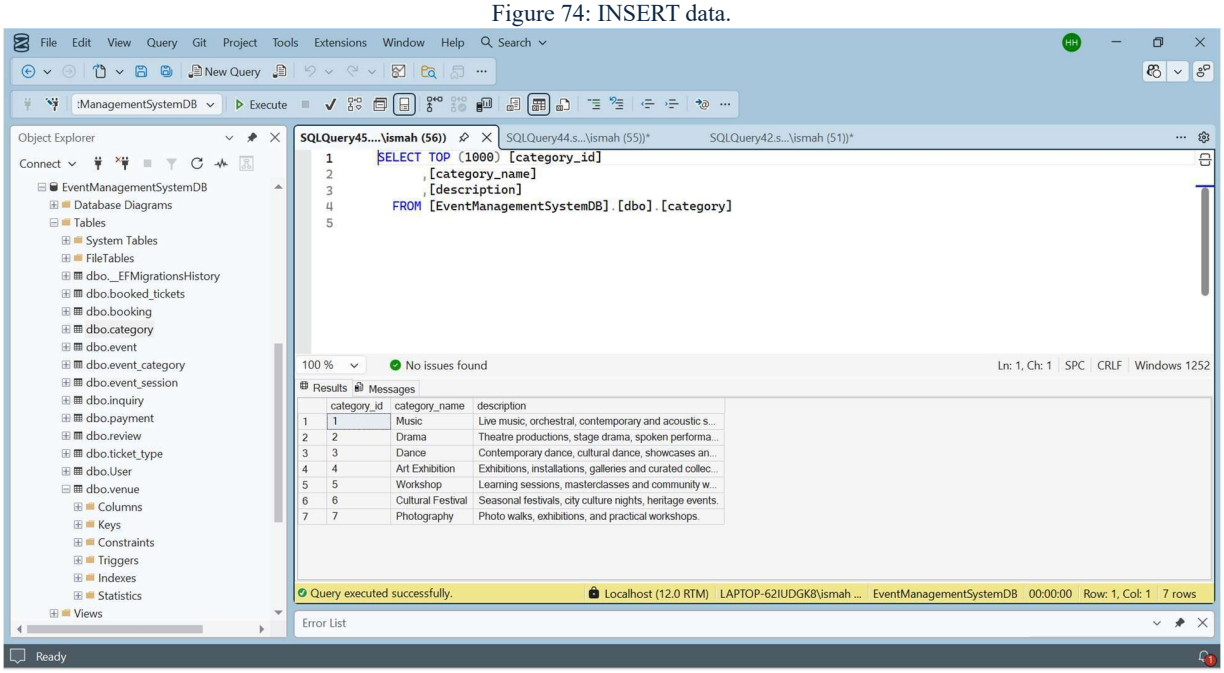


Figure 75:Output Table

Inserting a Member User:

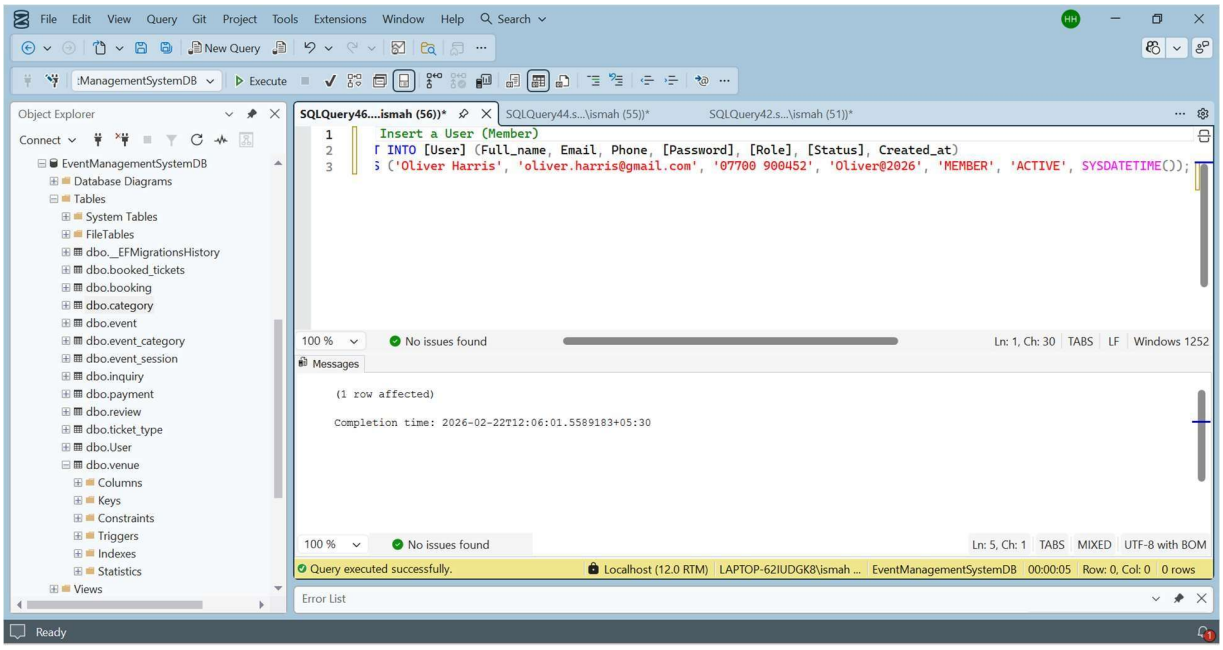


Figure 76: INSERT data.

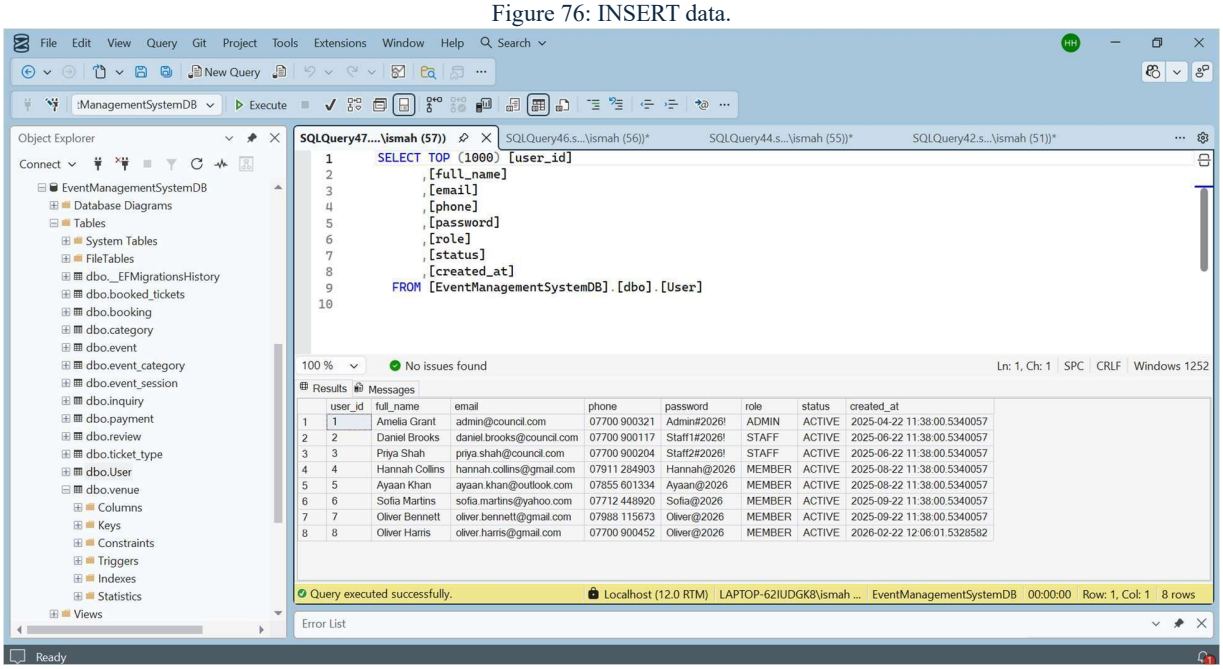


Figure 77:Output Table

Inserting an Event:

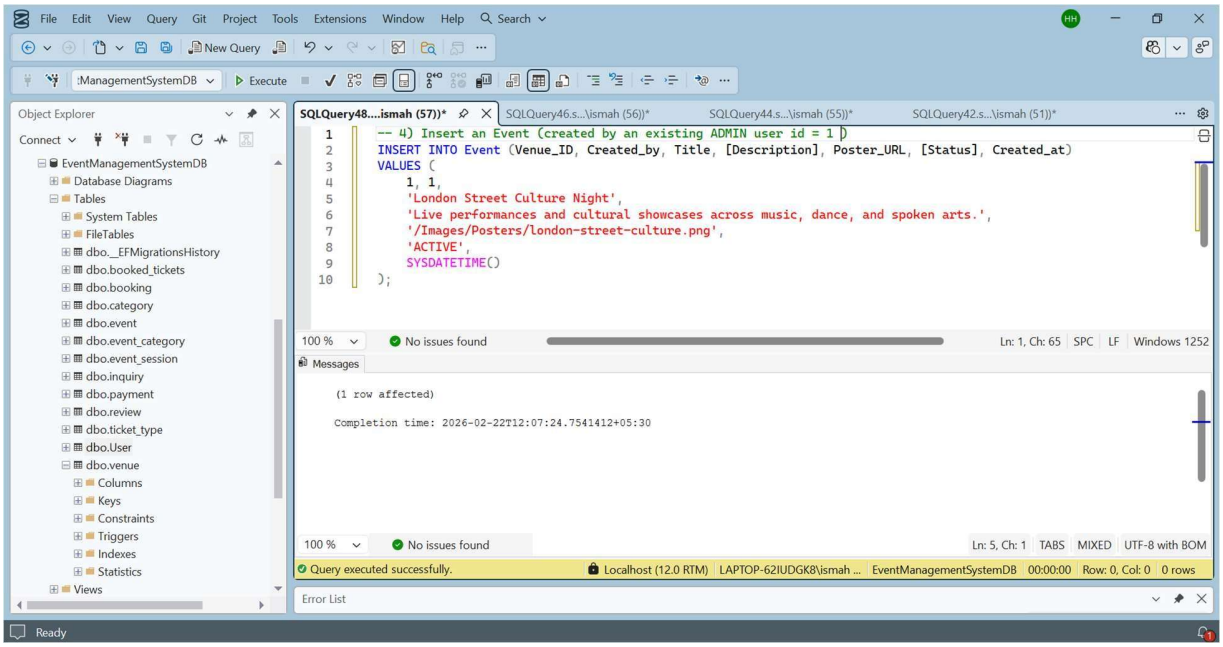


Figure 78: INSERT data.

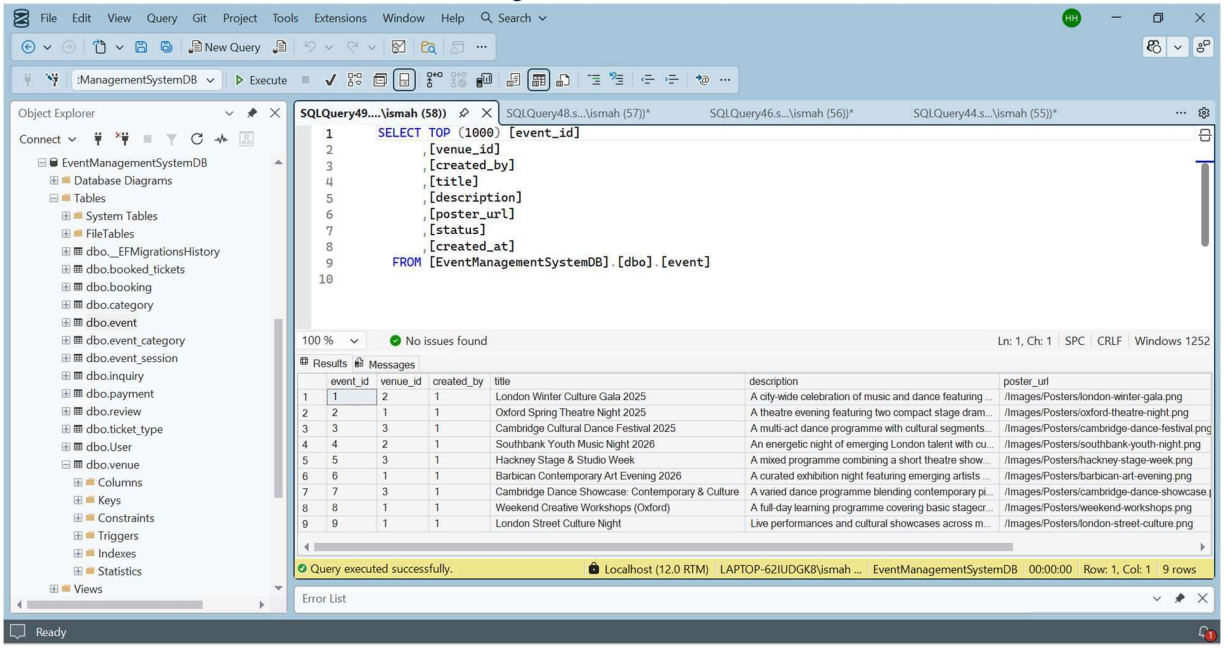


Figure 79: Output Table

Inserting an Event Session:

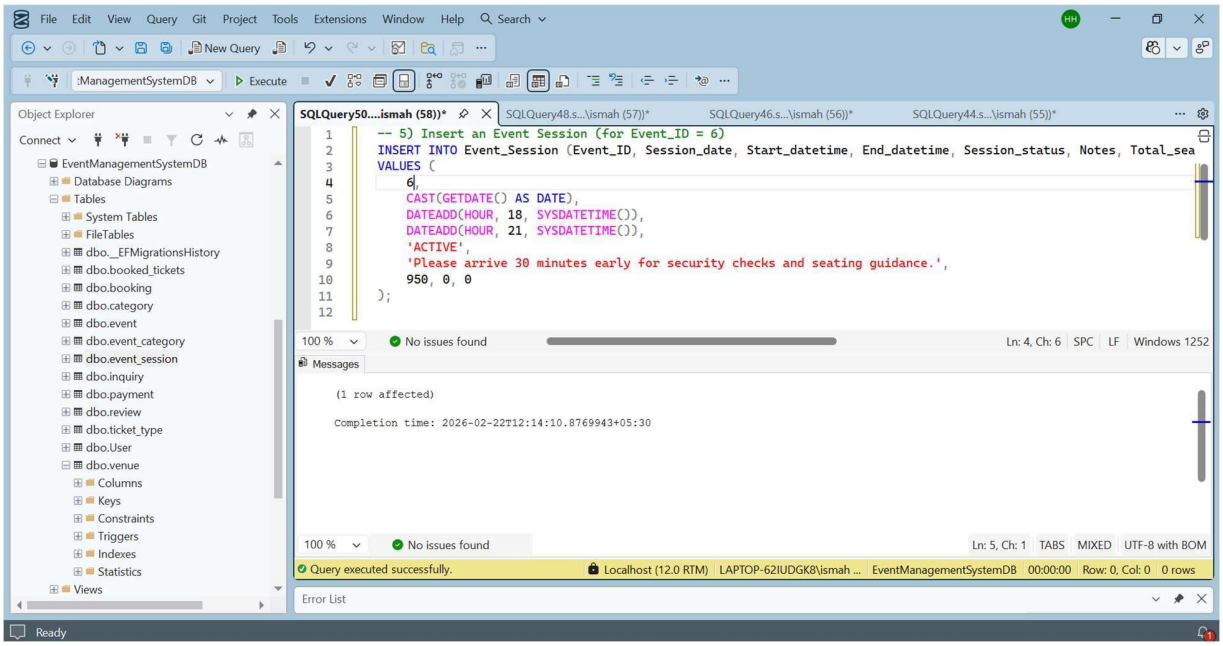


Figure 80: INSERT data.

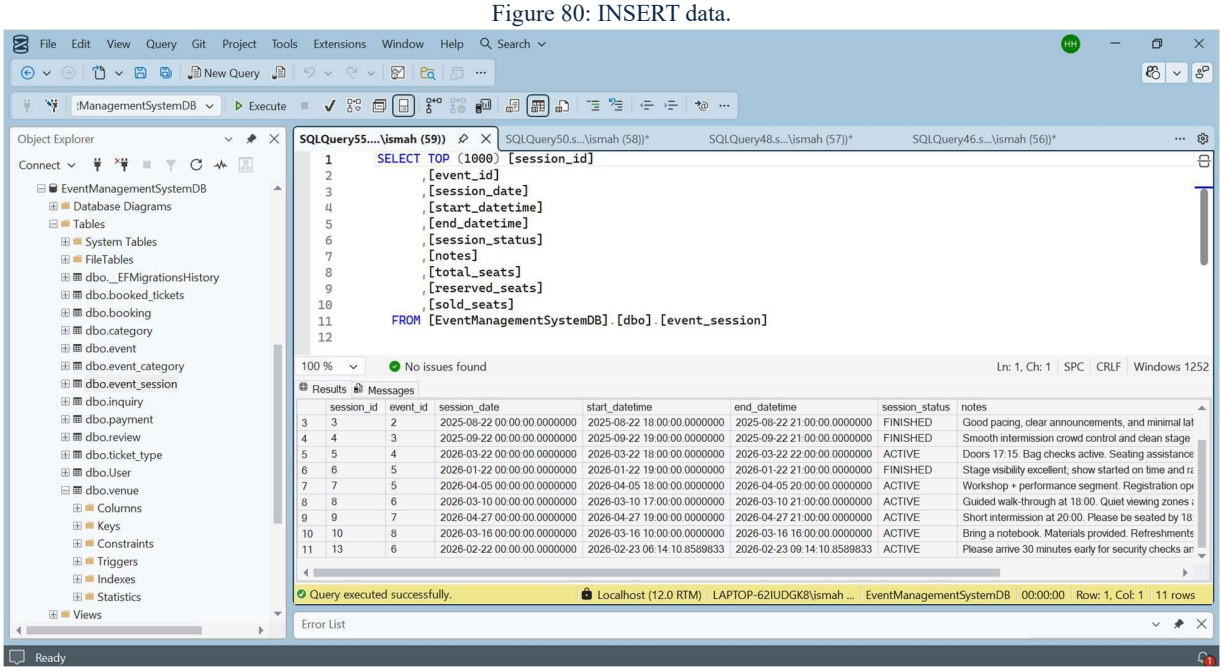


Figure 81:Output Table

Inserting an Inquiry:

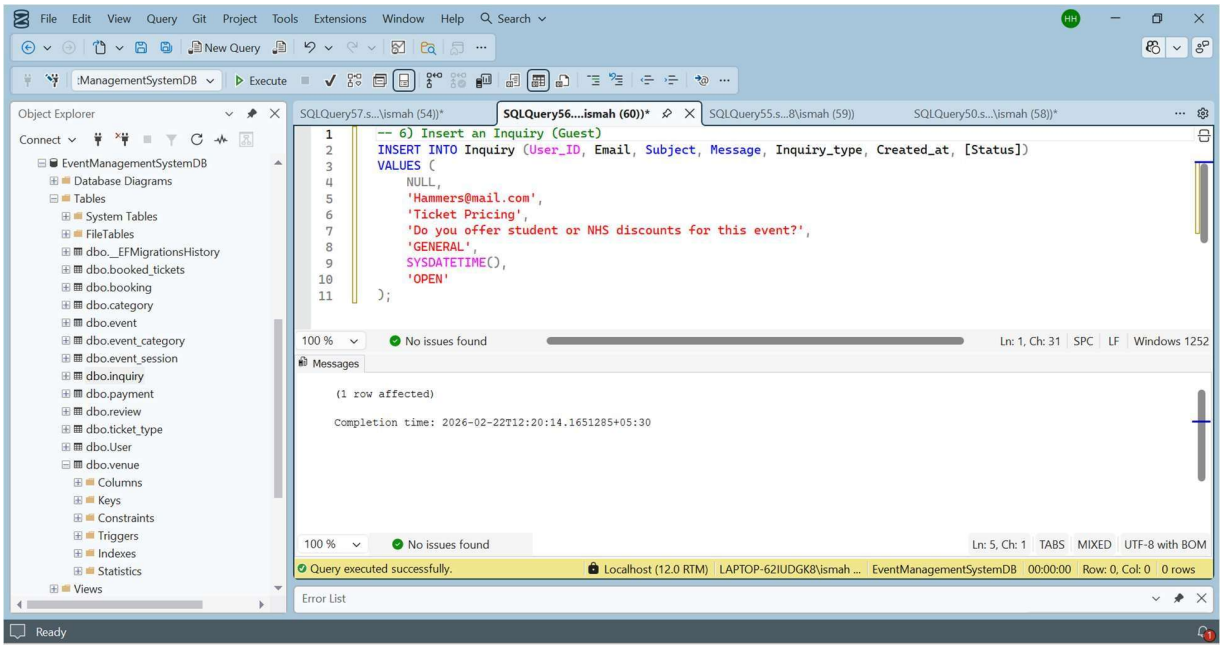


Figure 82: INSERT data.

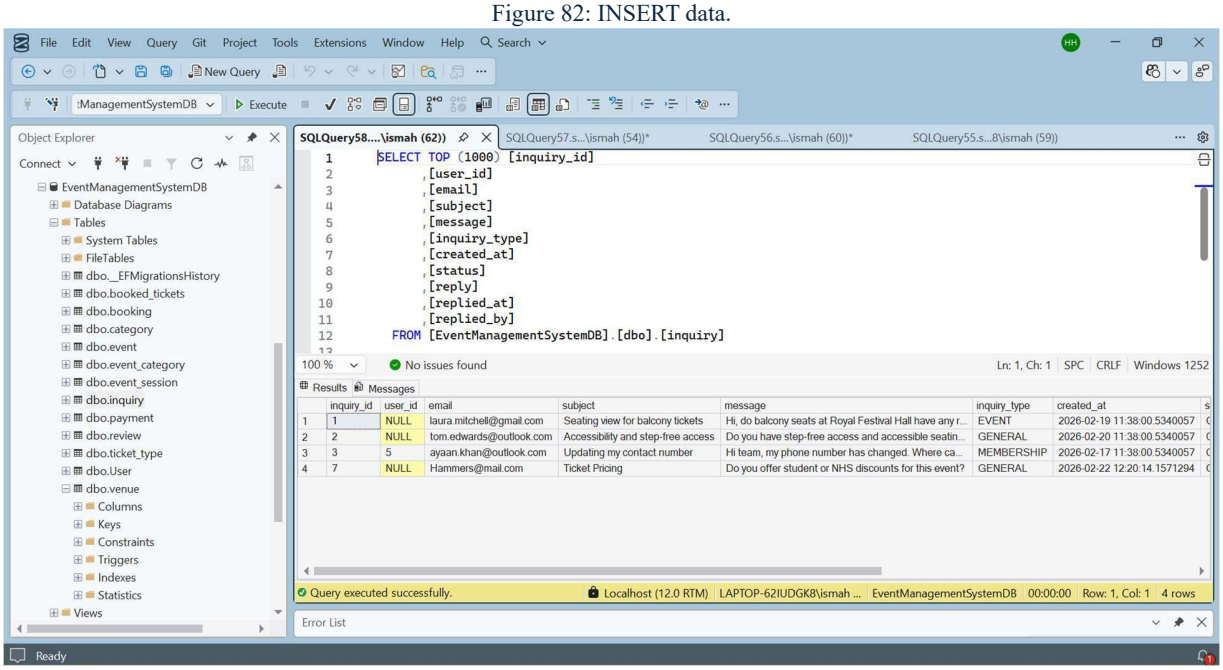


Figure 83: Output Table

4.5 Select Queries and Data Retrieval

1. Upcoming Events with the venue and next session:

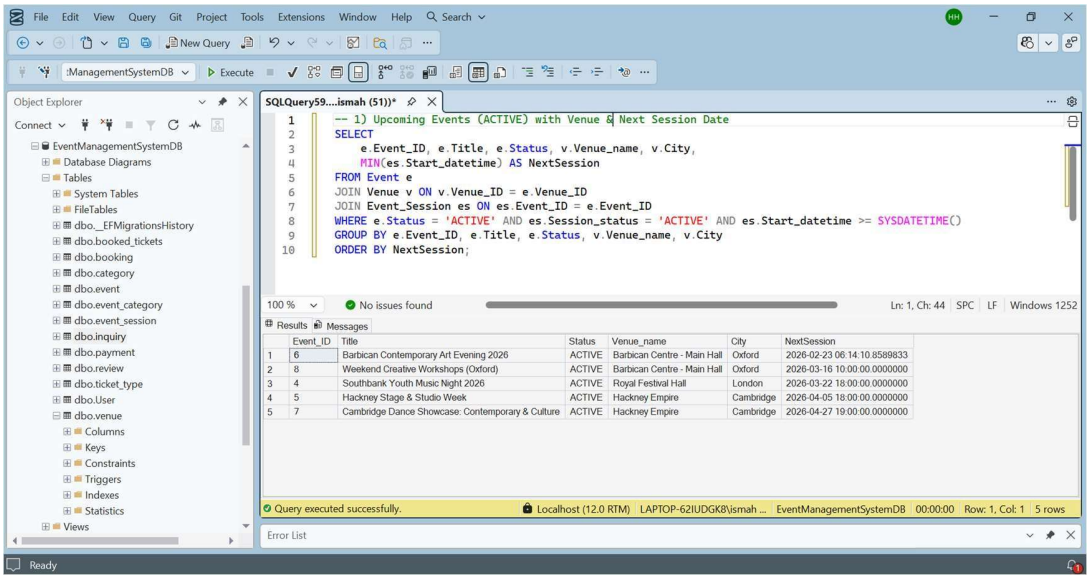


Figure 84:Select Queries and Data Retrieval

2. Member Booking by a User:

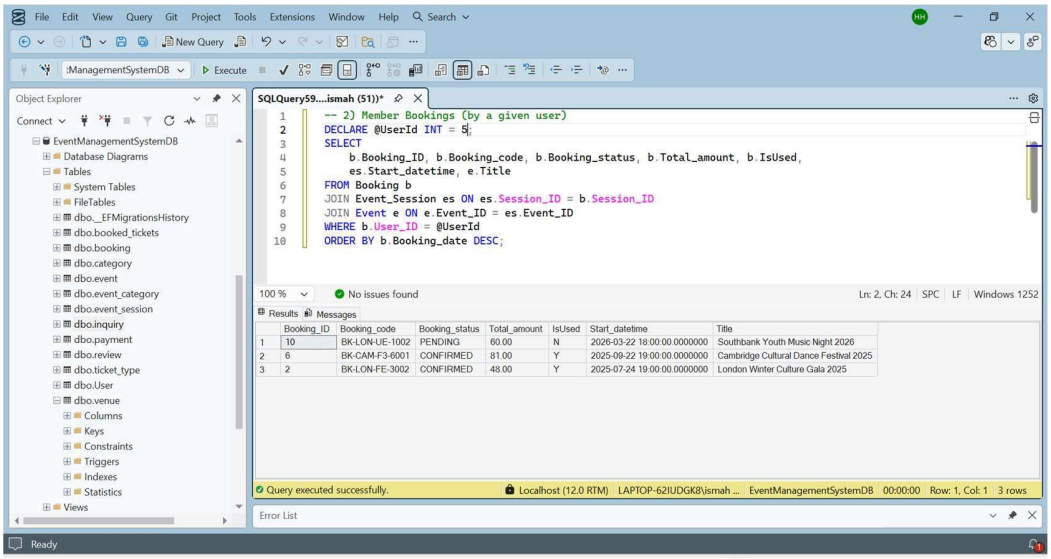


Figure 85:Select Queries and Data Retrieval

3. Approved Reviews:

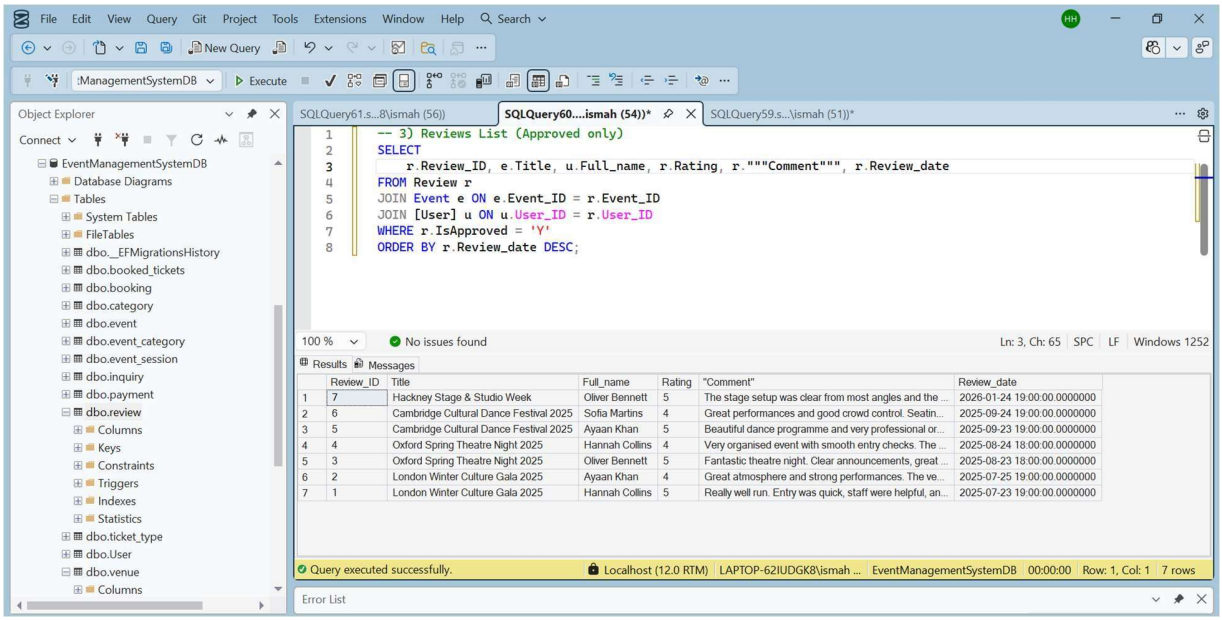


Figure 86:Select Queries and Data Retrieval

4. Ticket Availability for each session:

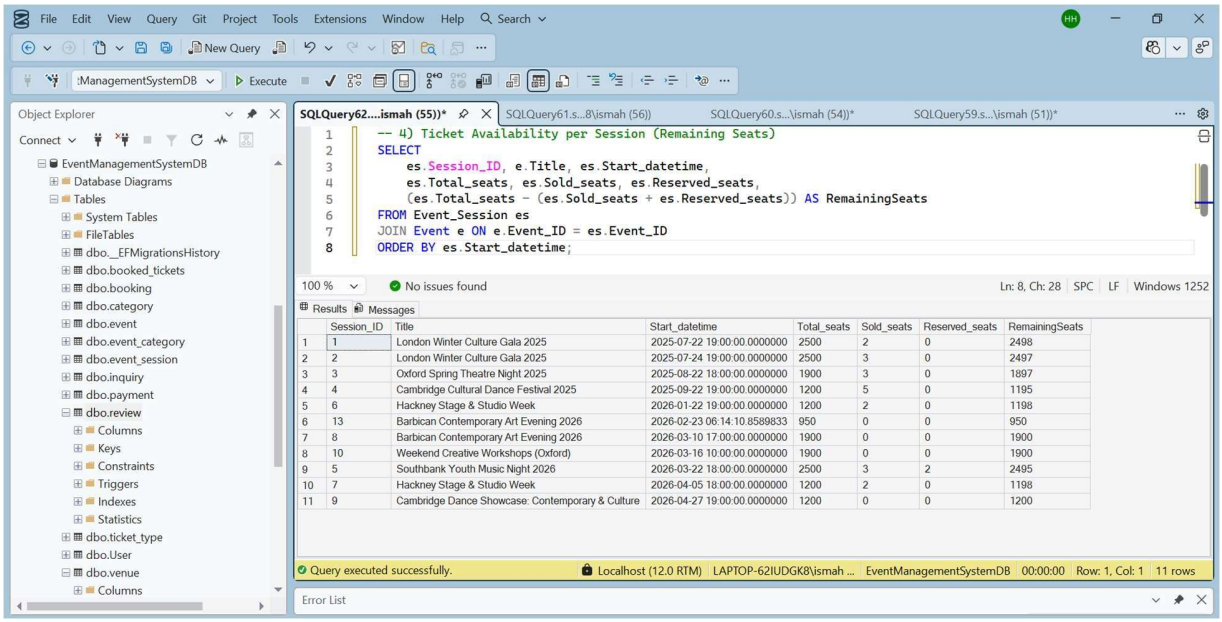


Figure 87:Select Queries and Data Retrieval

5. Booking Breakdown:

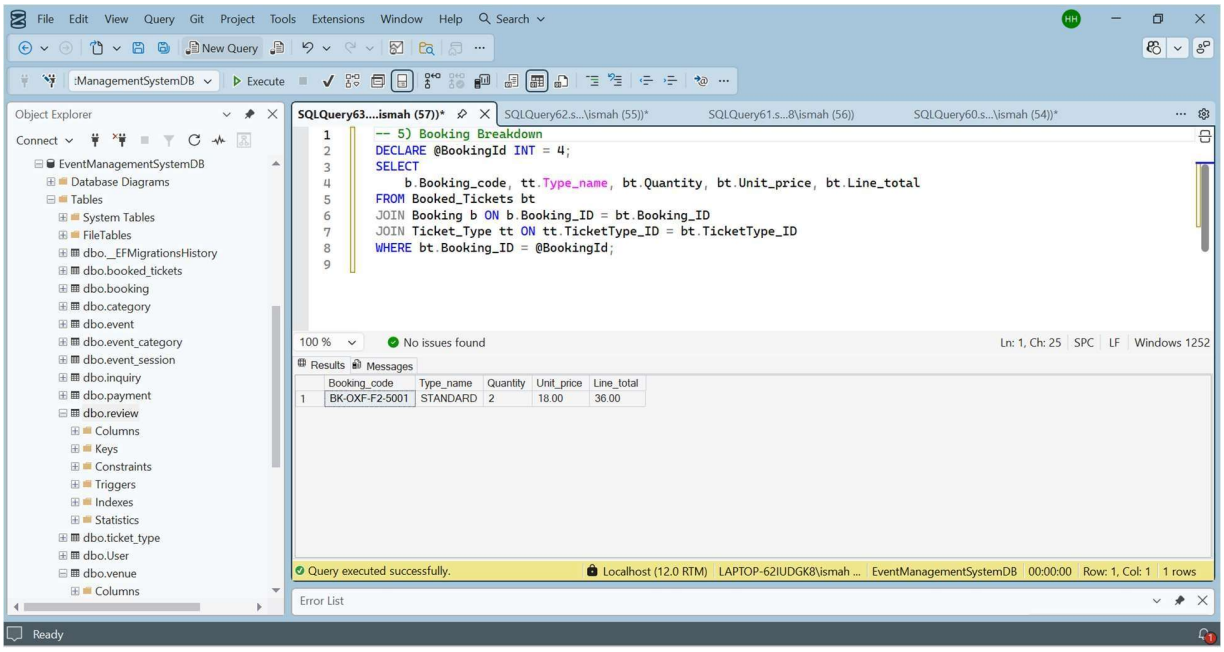


Figure 88:Select Queries and Data Retrieval

6. Payment Status by Booking:

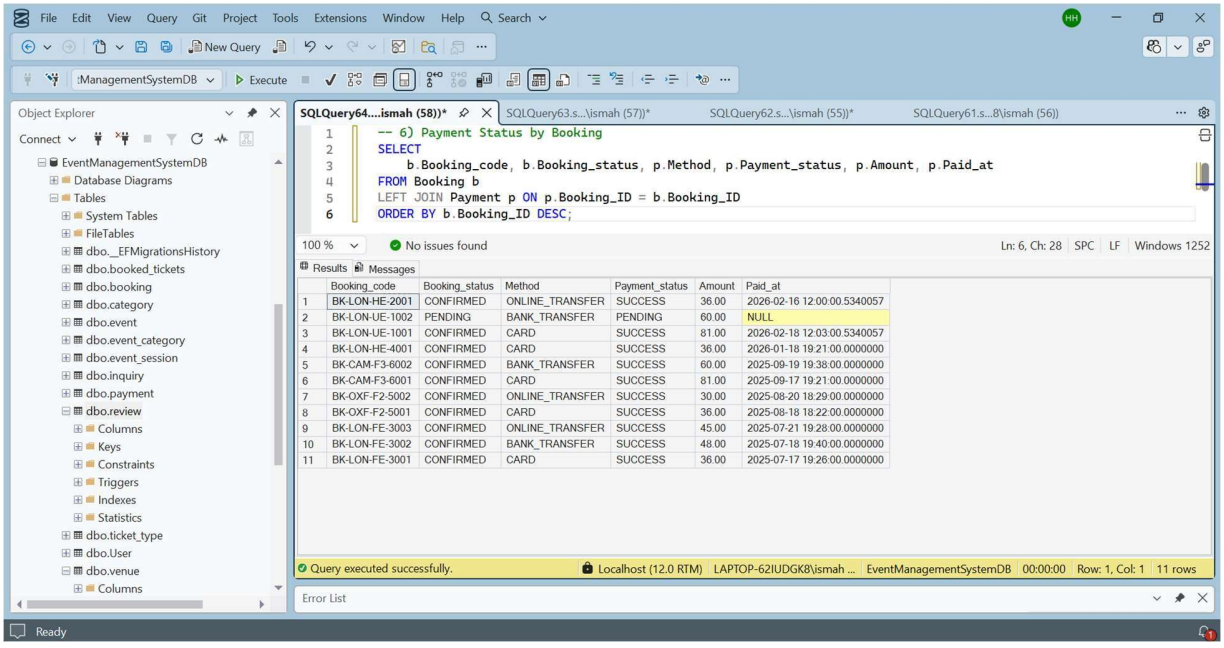


Figure 89:Select Queries and Data Retrieval

7. Inquiries with pending replies

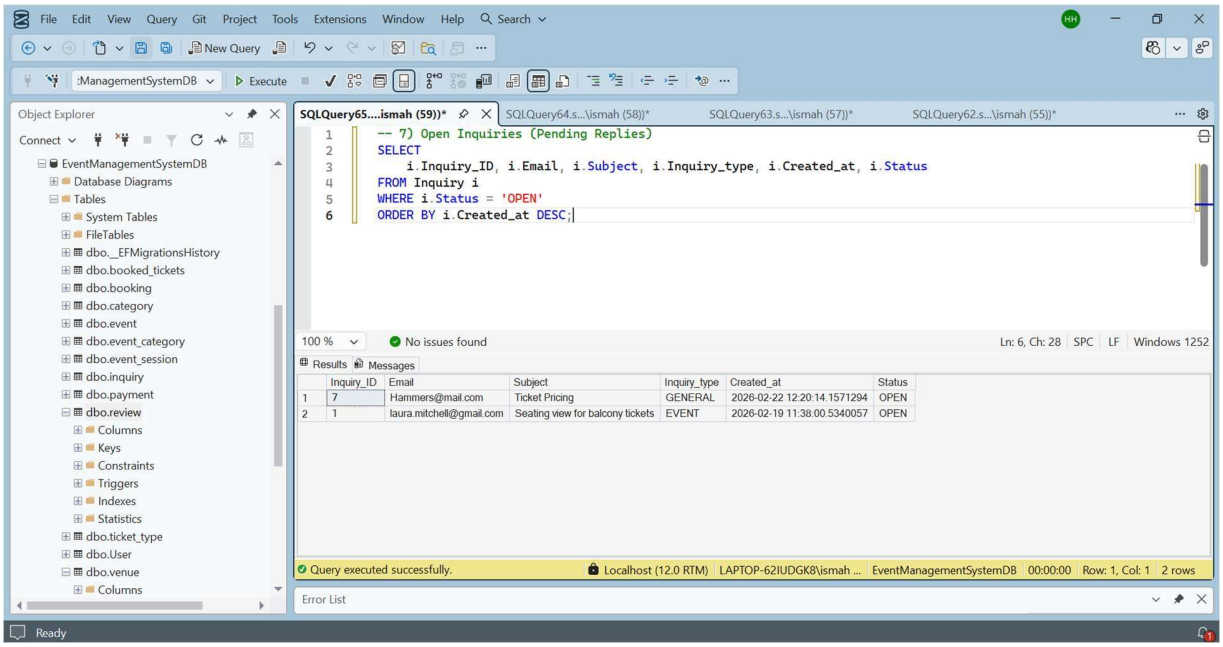


Figure 90:Select Queries and Data Retrieval

4.6 Database Execution

a) Create database manually in SQL Server.

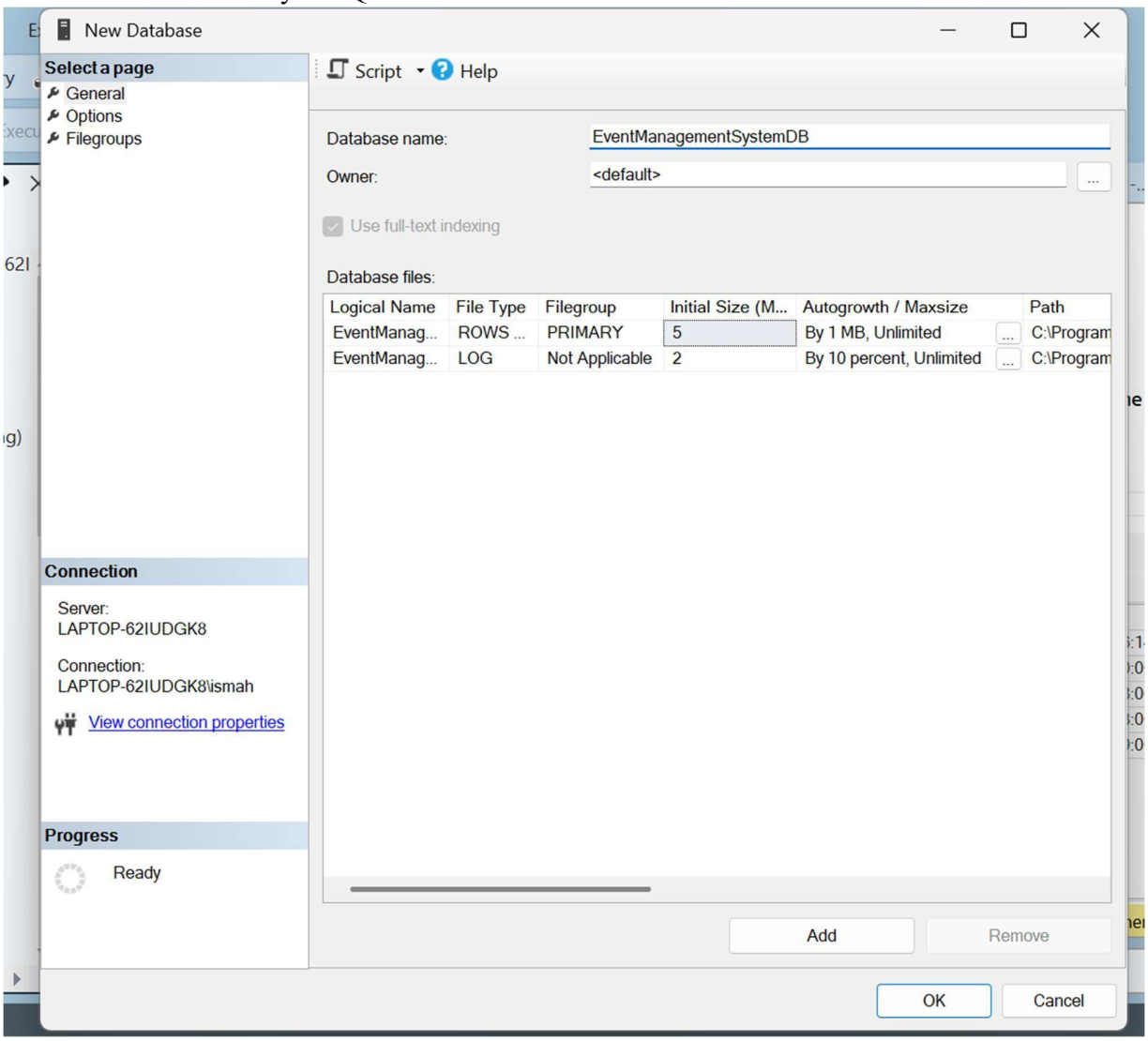


Figure 91: Database creation

b) Update the connection string in the ASP.NET application (appsettings.json).

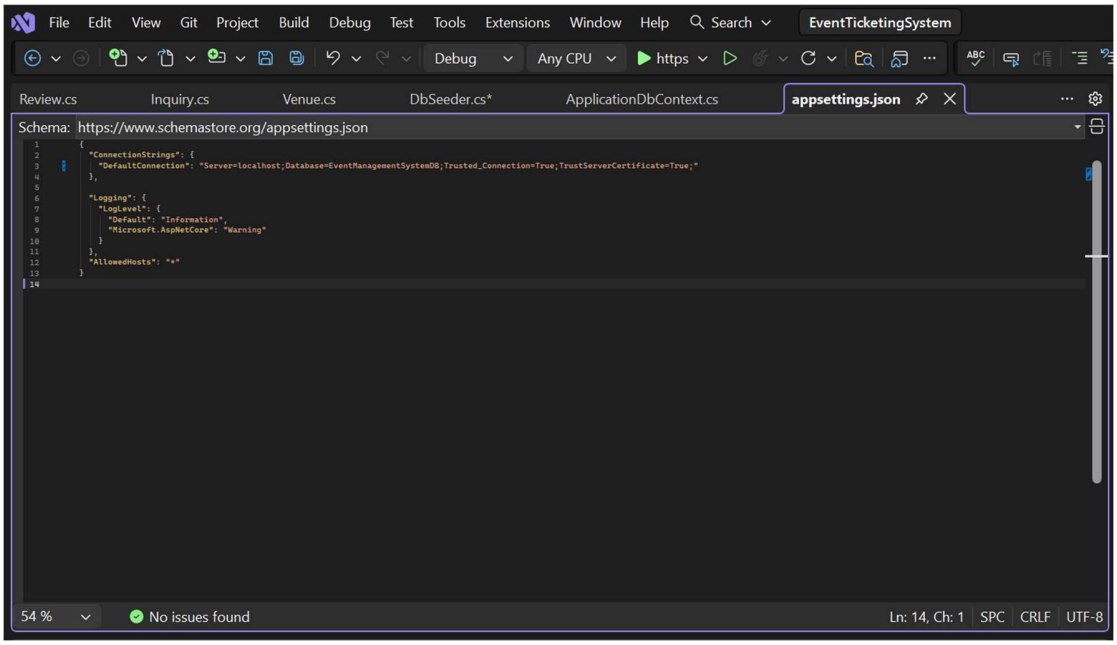


Figure 92: Update connection string

c) Create model classes for each table.

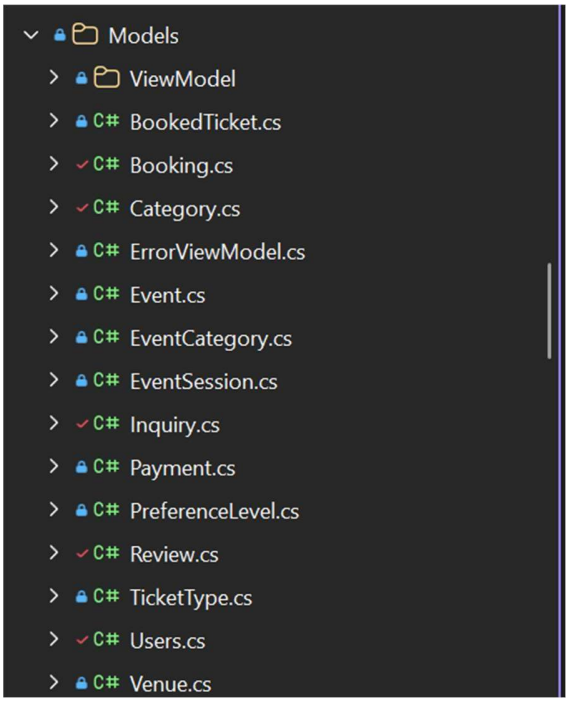


Figure 93: Create model classes

d) Update ApplicationDbContext by adding DbSet<> properties and relationships.

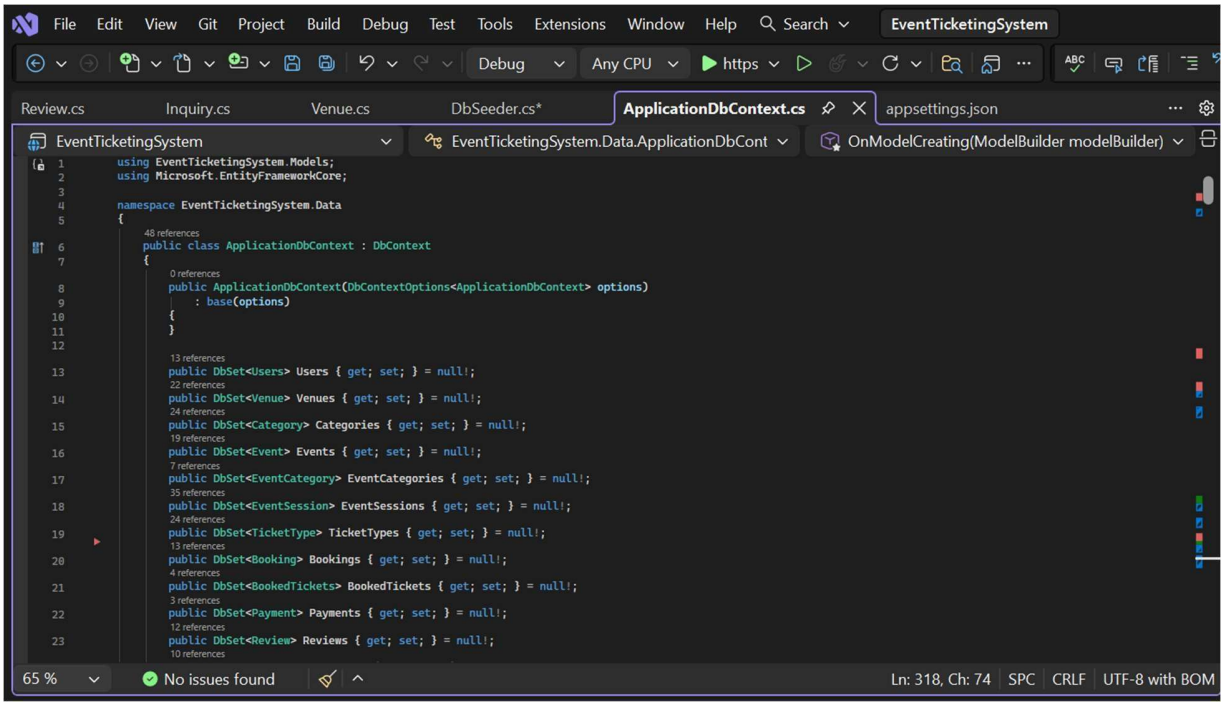


Figure 94: Update ApplicationDbContext

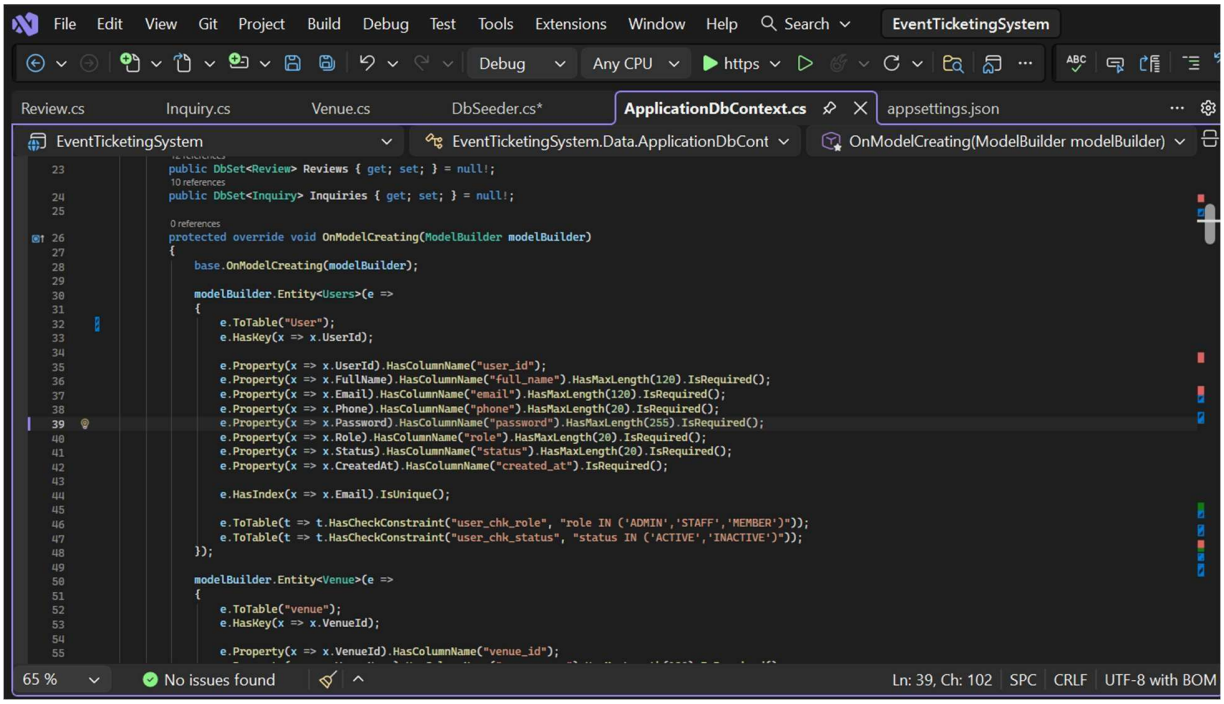


Figure 95:: Update ApplicationDbContext

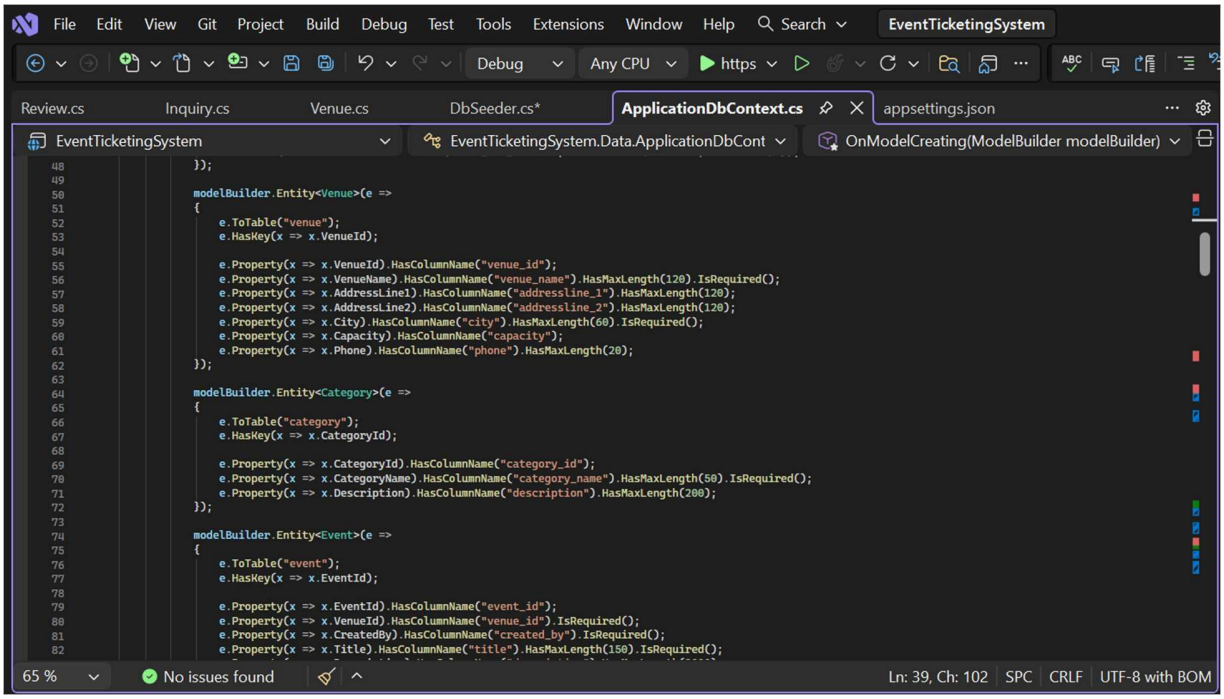


Figure 96:: Update ApplicationDbContext

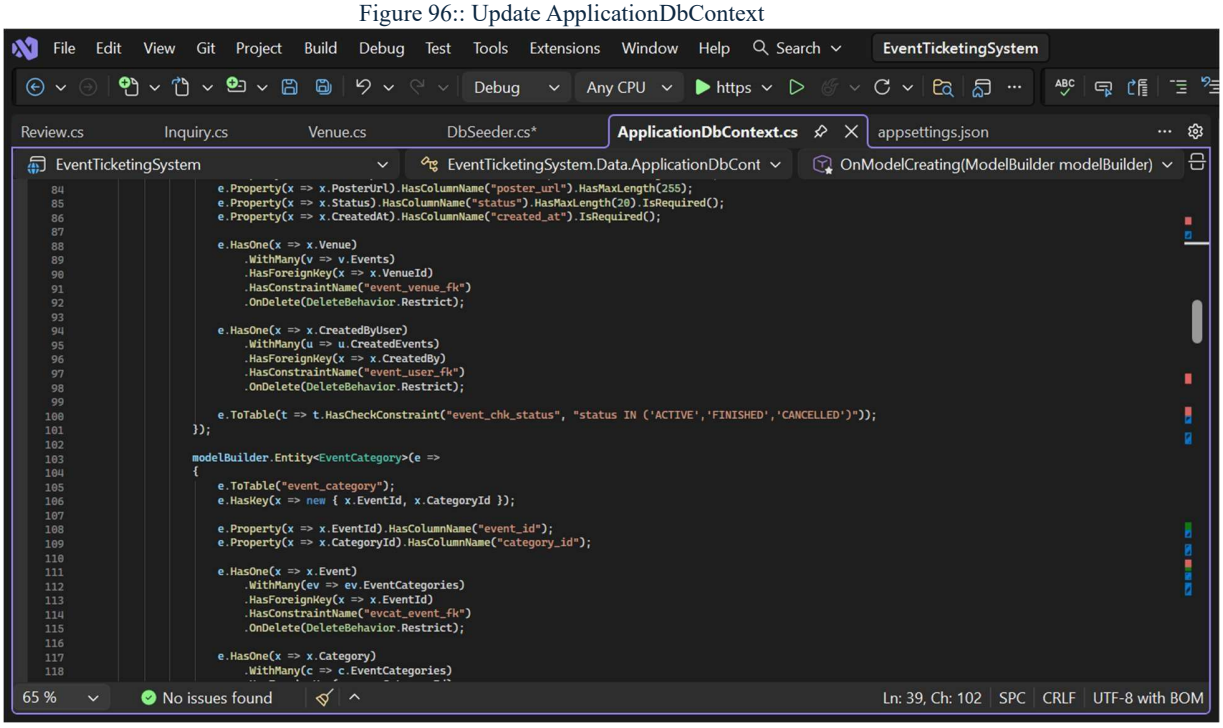


Figure 97:: Update ApplicationDbContext

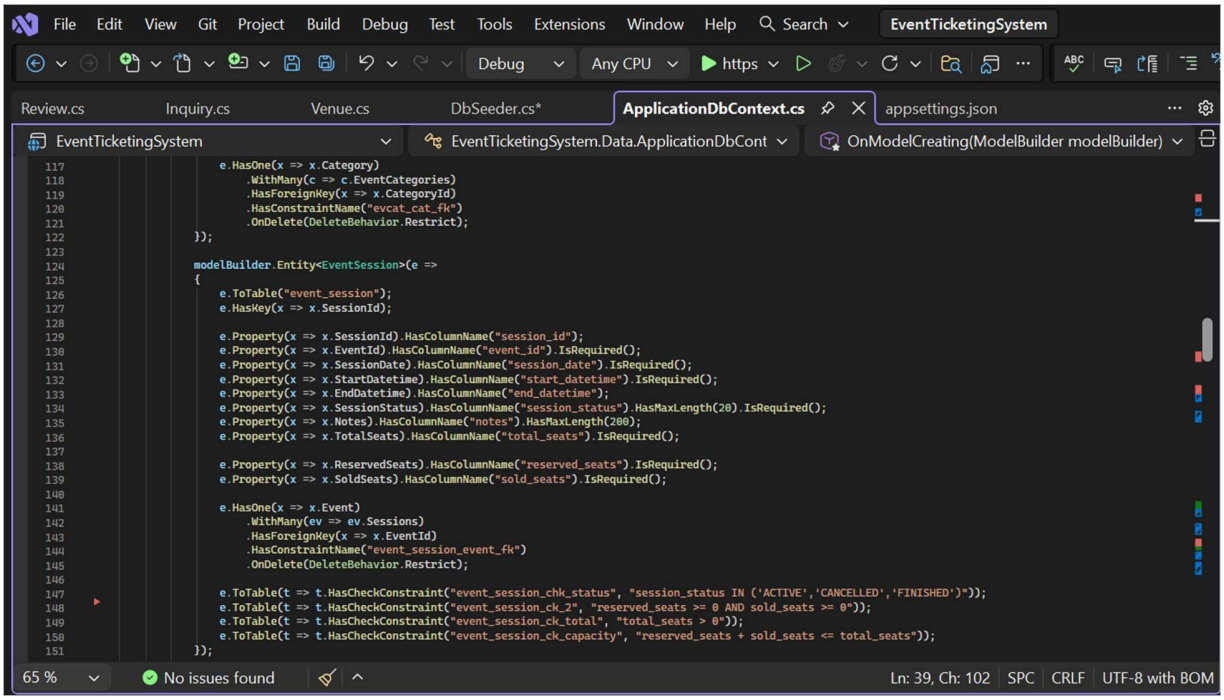


Figure 98: Update ApplicationDbContext

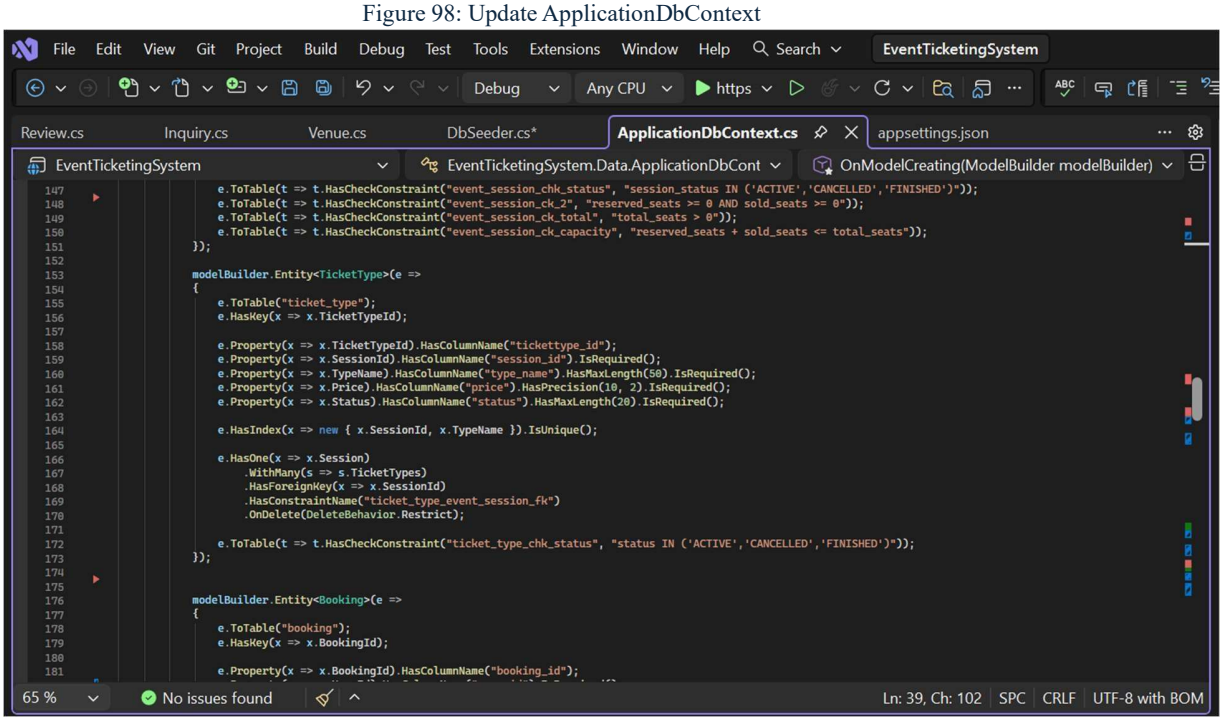


Figure 99: Update ApplicationDbContext

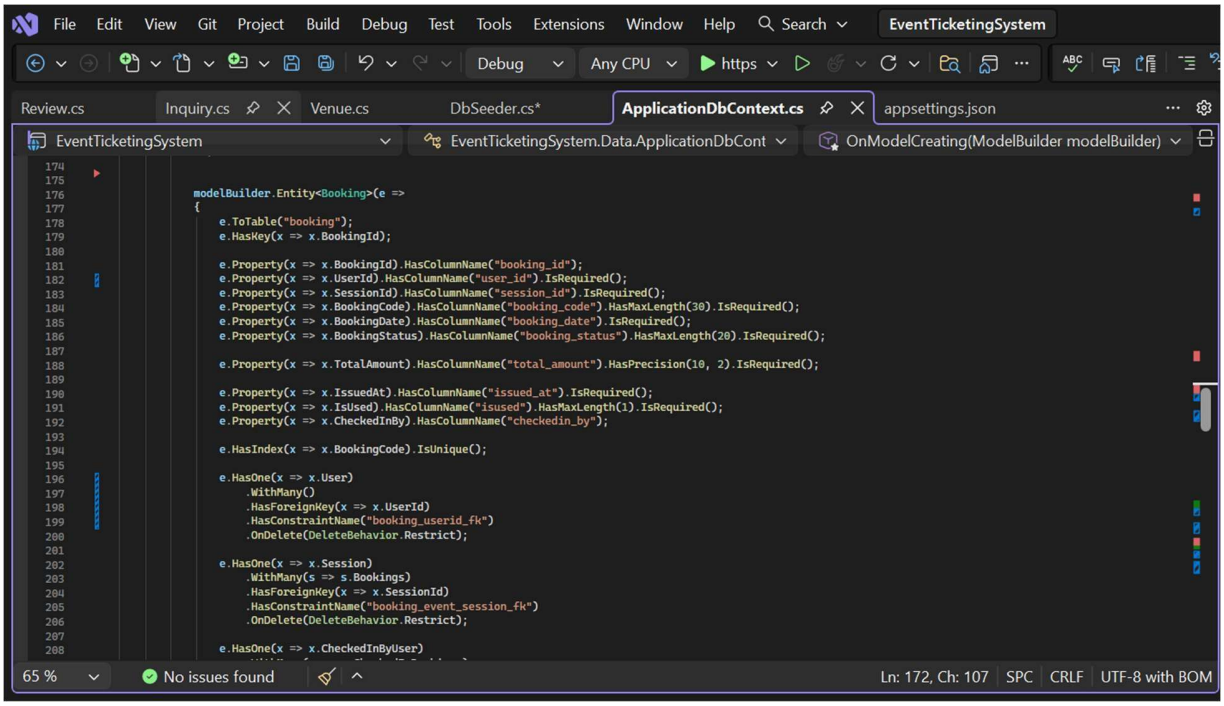


Figure 100: Update ApplicationDbContext

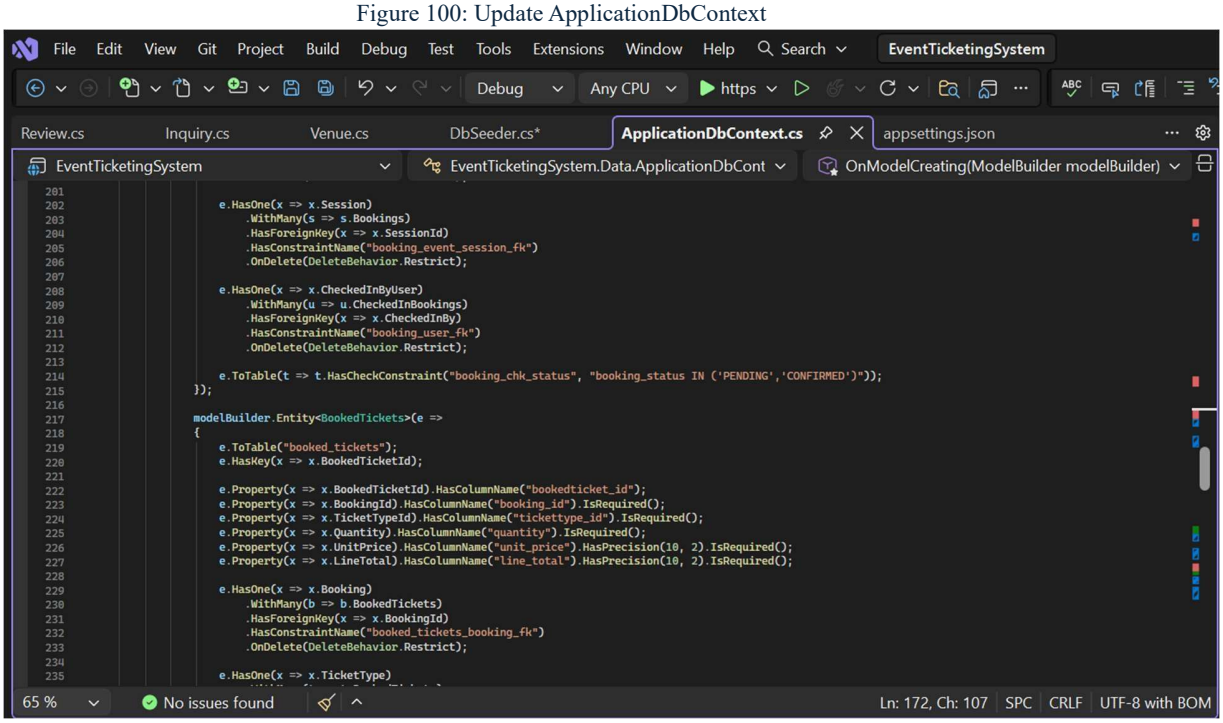


Figure 101: Update ApplicationDbContext

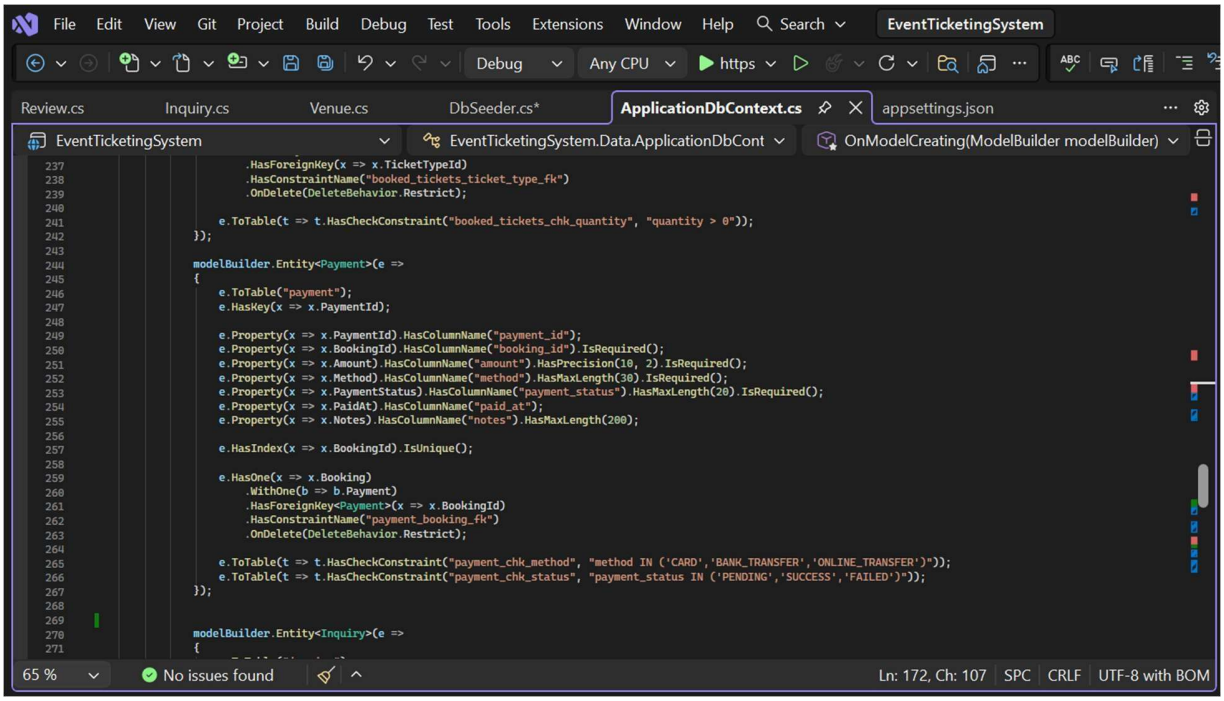


Figure 102: Update ApplicationDbContext

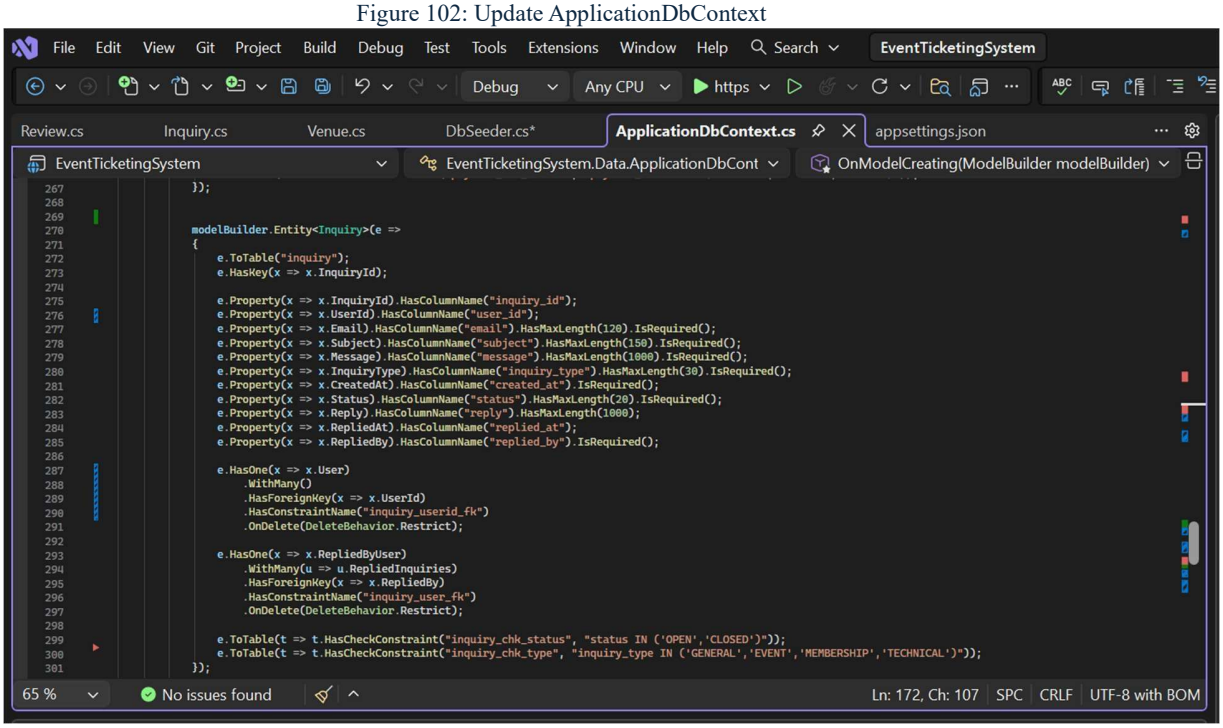


Figure 103: Update ApplicationDbContext

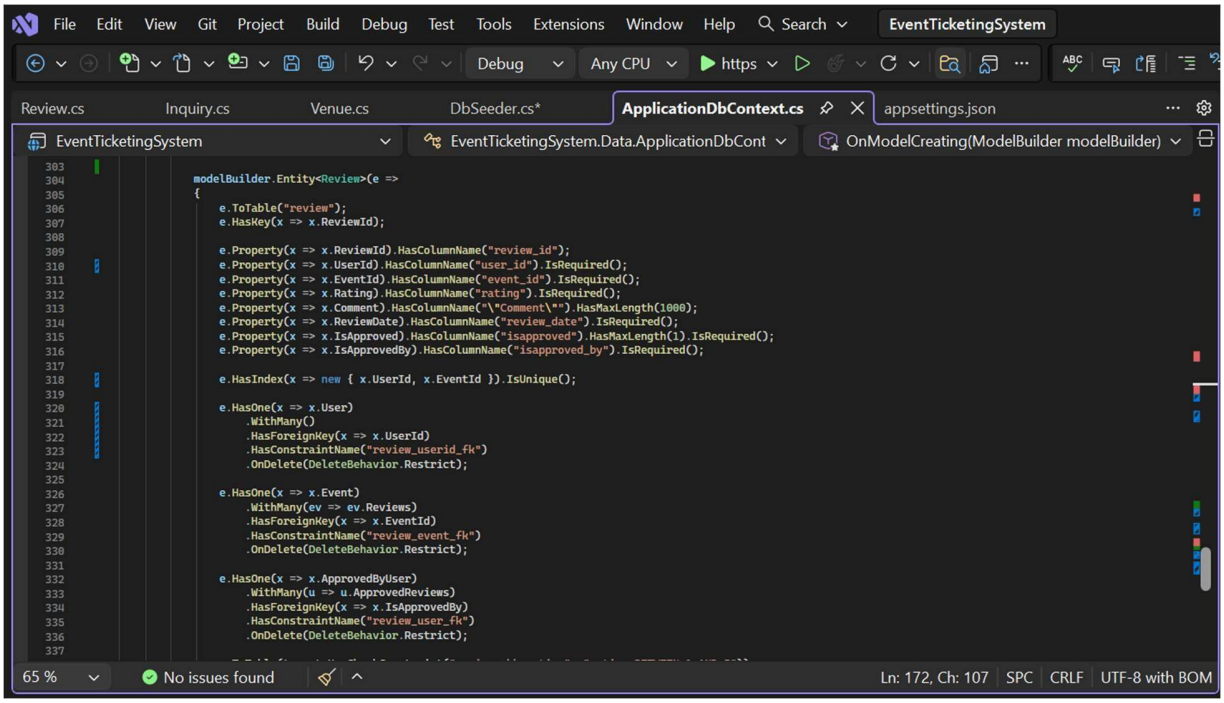


Figure 104: Update ApplicationDbContext

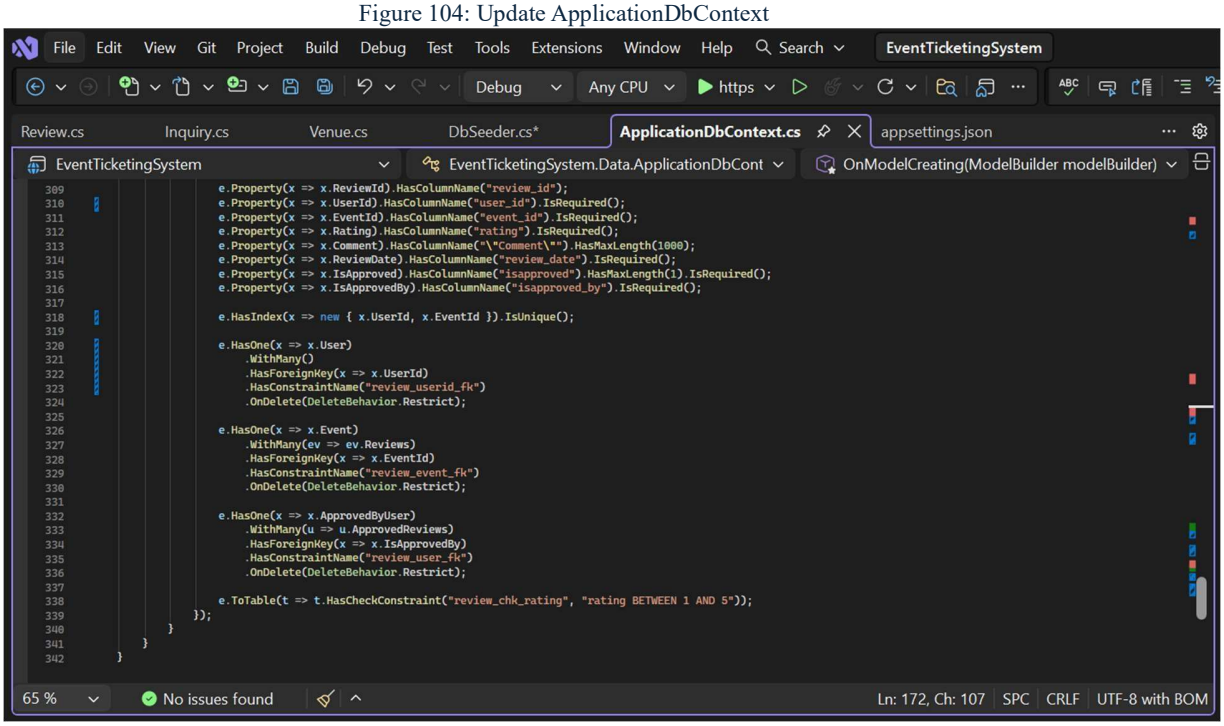


Figure 105: Update ApplicationDbContext

e) Run migrations (Add-Migration) to generate schema changes.

```
PM> Add-Migration InitialCreate
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
```

Figure 106: Run migrations

f) Update database (Update-Database) to apply the schema to SQL Server.

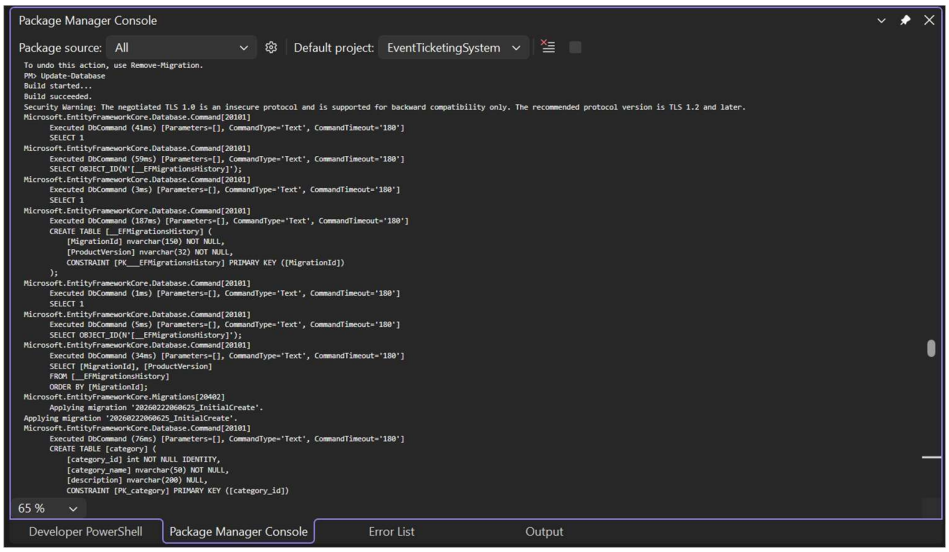


Figure 107:Update database

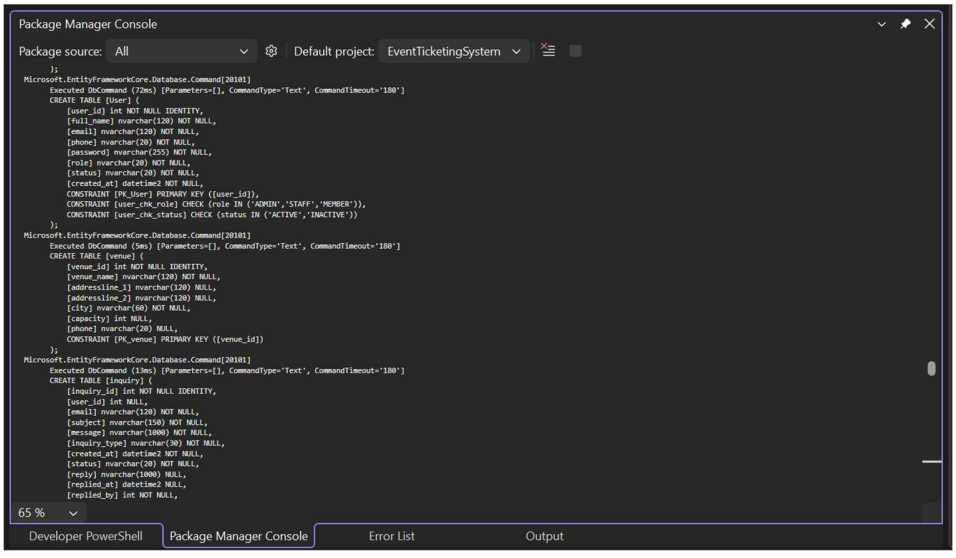


Figure 108:Update database

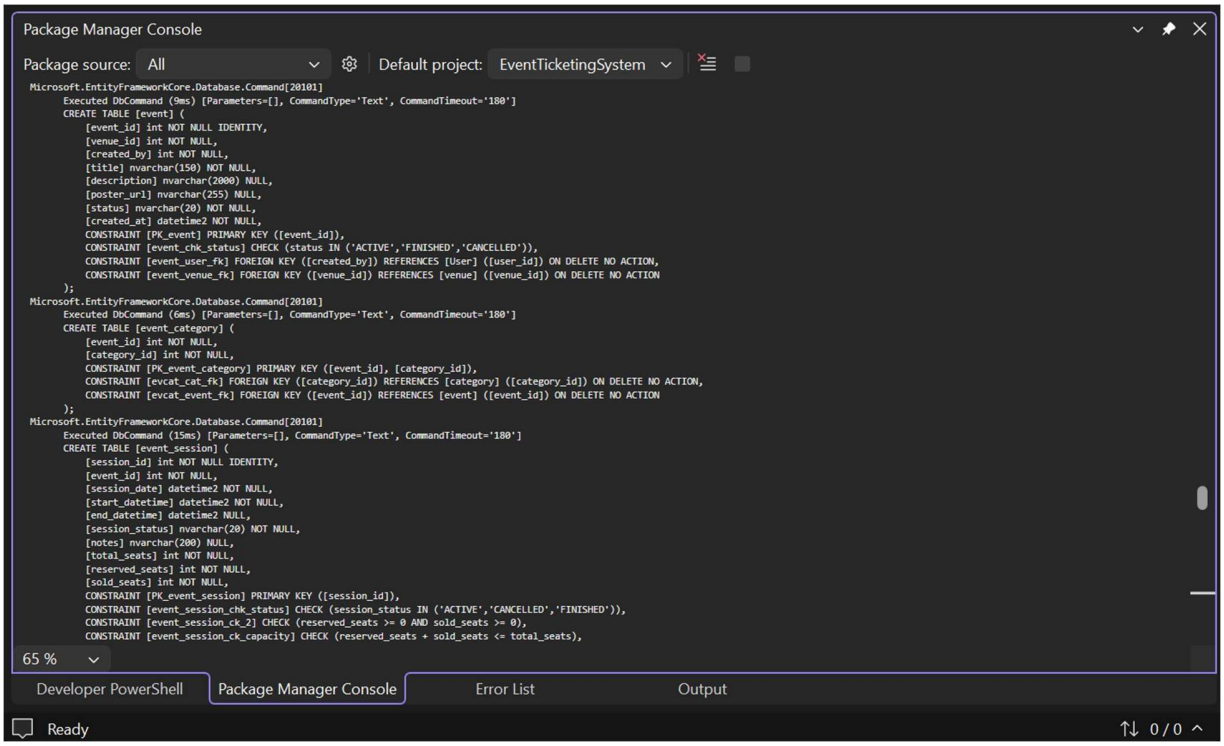


Figure 109Update database

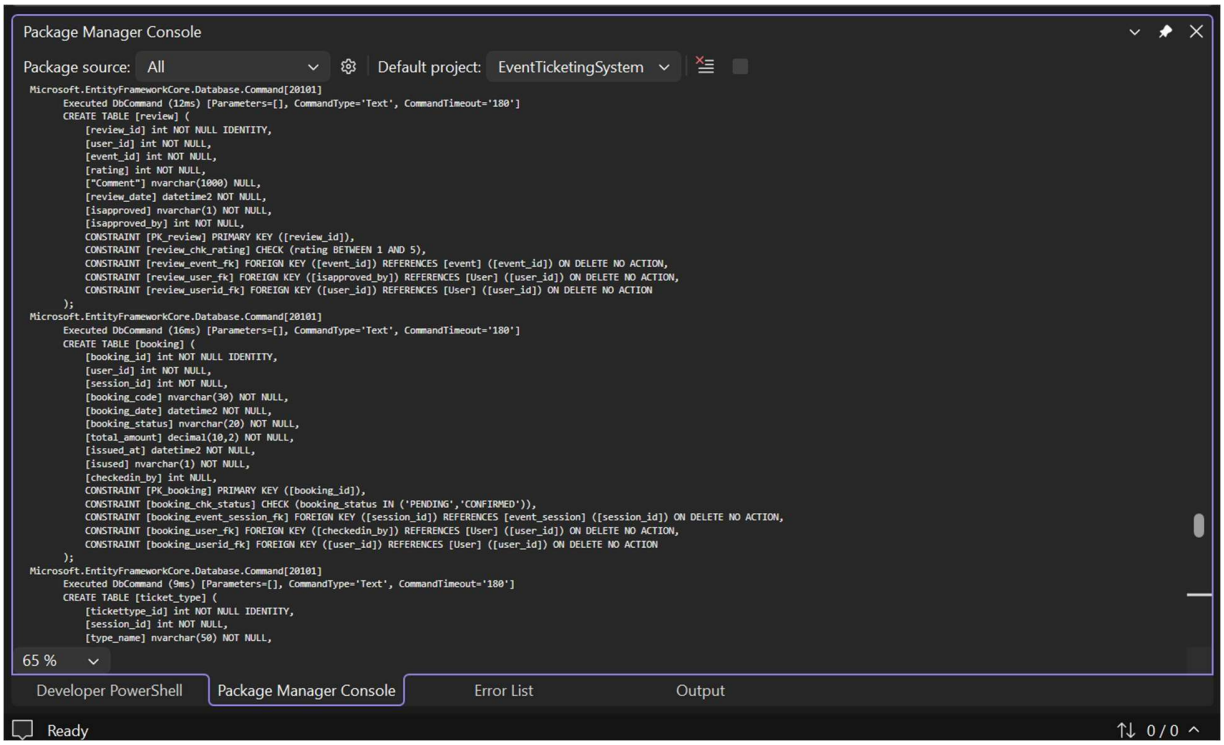


Figure 110:Update database

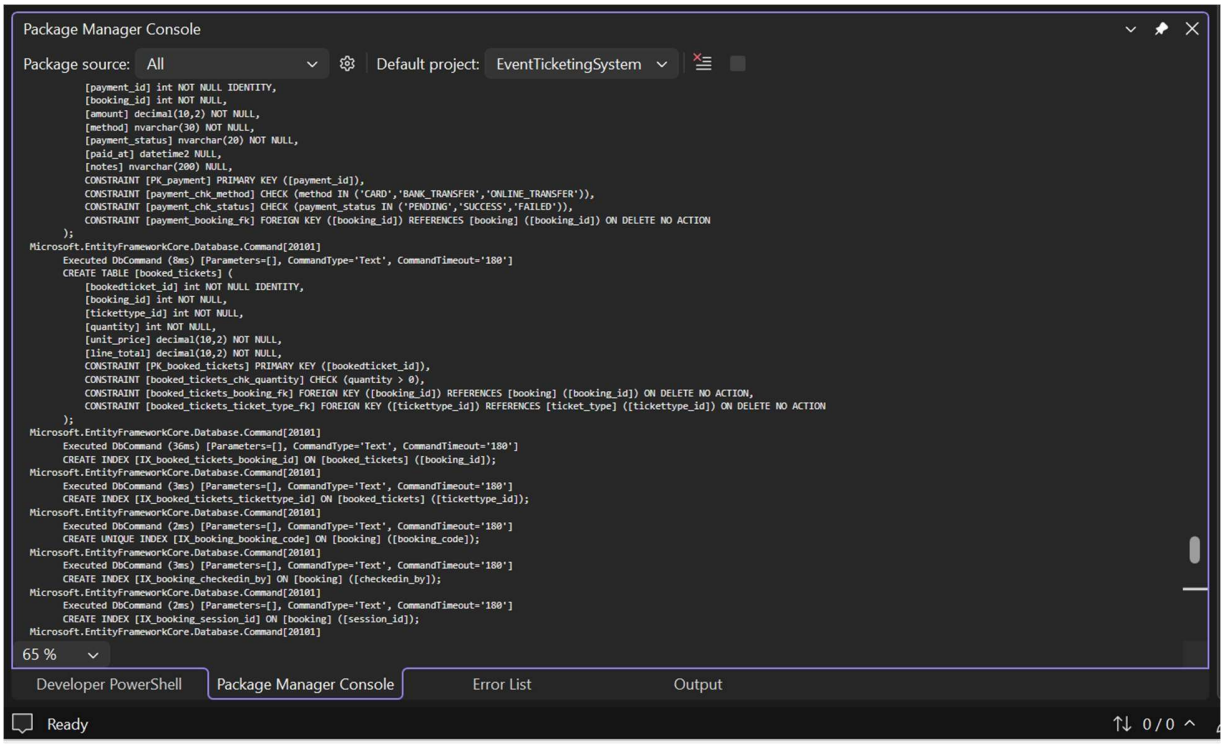


Figure 111:Update database

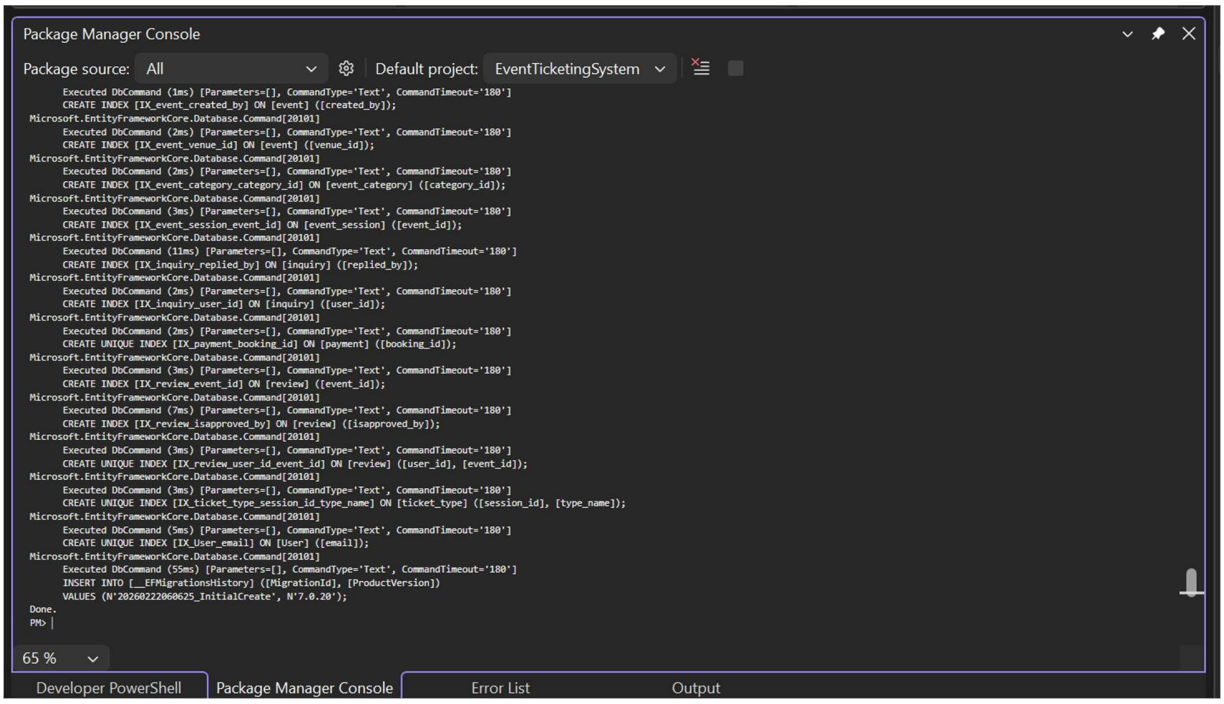


Figure 112:Update database

5. Web Application Development – ASP.NET MVC

5.1 System Architecture Overview

The Smart Event Management and Ticketing System has a three-tier architecture, with the application divided into three layers: display, application logic, and data. This architectural approach increases maintainability, scalability, and system stability by assigning each layer a clear duty. Changes to one layer have little impact on the other layers, making the system easy to extend and manage.

The presentation layer displays the system's user interface and is built with ASP.NET MVC views, HTML, CSS, Bootstrap, and JavaScript. This layer controls all user interactions, such as event browsing, booking forms, dashboards, and administrative panels. It ensures consistent visual design and role-based navigation by presenting just the features that are relevant to the logged-in user, boosting usability and security.

The application logic layer is built with ASP.NET MVC controllers and business logic components. This layer handles incoming requests, applies validation rules, enforces role-based permissions, and oversees workflows like booking generation, payment status management, review moderation, and staff verification. This layer serves as a mediator between the user interface and the database, ensuring that all activities adhere to stated business rules.

The data layer consists of Microsoft SQL Server, which stores and manages all permanent system data. This layer ensures data integrity by utilizing primary keys, foreign keys, unique constraints, and check constraints. SQL Server Management Studio is used to administer databases, execute queries, and validate results. Separating data storage from application logic allows the secure and dependable administration of key information such as users, events, bookings, and payments.

5.2 Project Structure (MVC Pattern)

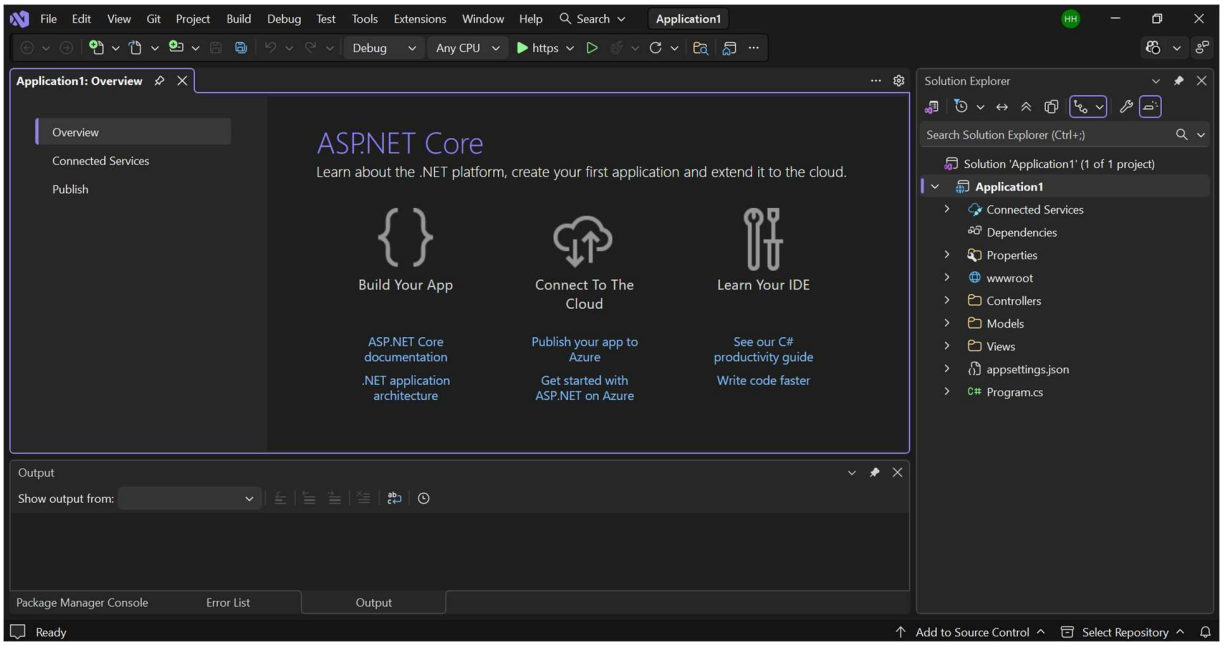


Figure 113:ASP.NET MVC

The Model-View-Controller (MVC) pattern is a software architecture pattern often used in web application development to structure code by splitting tasks into discrete components. The basic purpose of the MVC pattern is to optimize application structure by explicitly segregating data administration, user interface presentation, and request processing logic. This separation helps developers to construct systems that are easier to comprehend, maintain, and extend over time.

The Model component represents the data and business logic of the system. It determines the structure of application data, enforces validation criteria, and controls interfaces with the database or other data sources. The Model is responsible for assuring data consistency and integrity by applying constraints and rules before data is stored or accessed. By isolating business logic within the Model layer, the system reduces duplication of rules and guarantees that all application activities adhere to the same data standards.

The View component is responsible for displaying information to the user in a structured and legible fashion. It gets data from the Controller and renders it into user interfaces using markup and server-side rendering techniques. The View does not include business logic; instead, it concentrates exclusively on displaying data, gathering user input, and maintaining a consistent user experience. This obvious separation allows changes to the user interface, such as layout or

visual design modifications, to be done without altering the underlying application logic or data structure.

The Controller component works as the core coordinator inside the MVC pattern. It handles incoming user requests, processes input parameters, and selects the necessary actions to take. The Controller interacts with the Model to perform data operations and then selects the appropriate View to provide the answer. By managing application flow and request routing, the Controller guarantees that user interactions are transformed into controlled and predictable system activity.

Overall, the MVC pattern establishes a systematic workflow in which user requests are received by the Controller, processed using the Model, and presented through the View. This architectural approach boosts code readability, promotes modular development, and supports scalability by letting each component to expand independently. As a result, the MVC paradigm is widely embraced for constructing resilient, maintainable, and extendable online applications.

5.3 Home Page

Figure 114:Homepage

Figure 115:Homepage

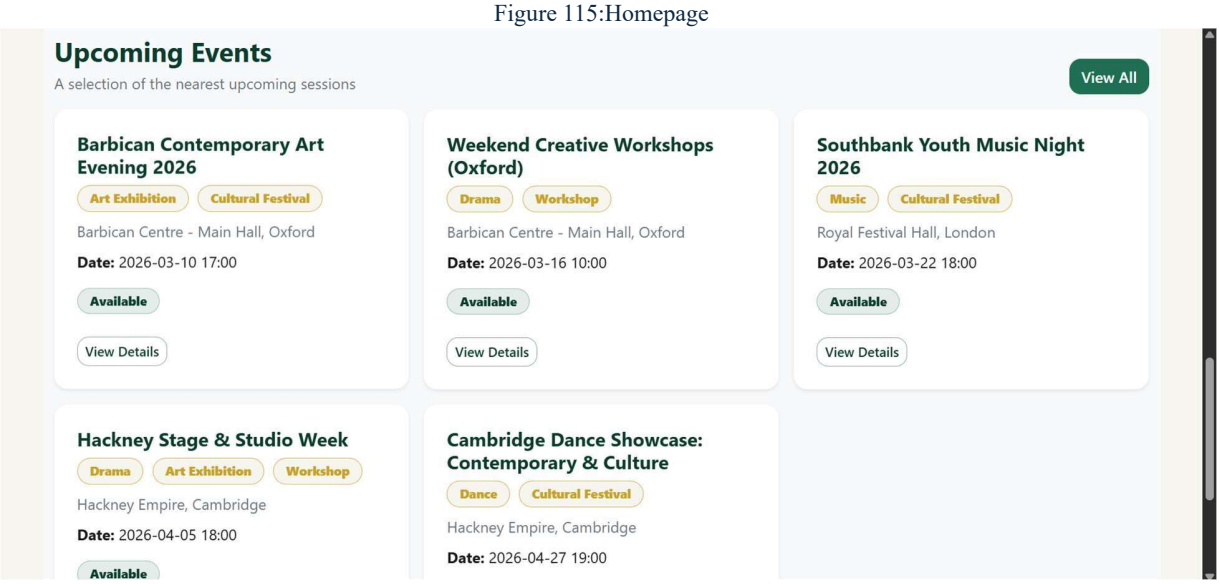


Figure 116:Homepage

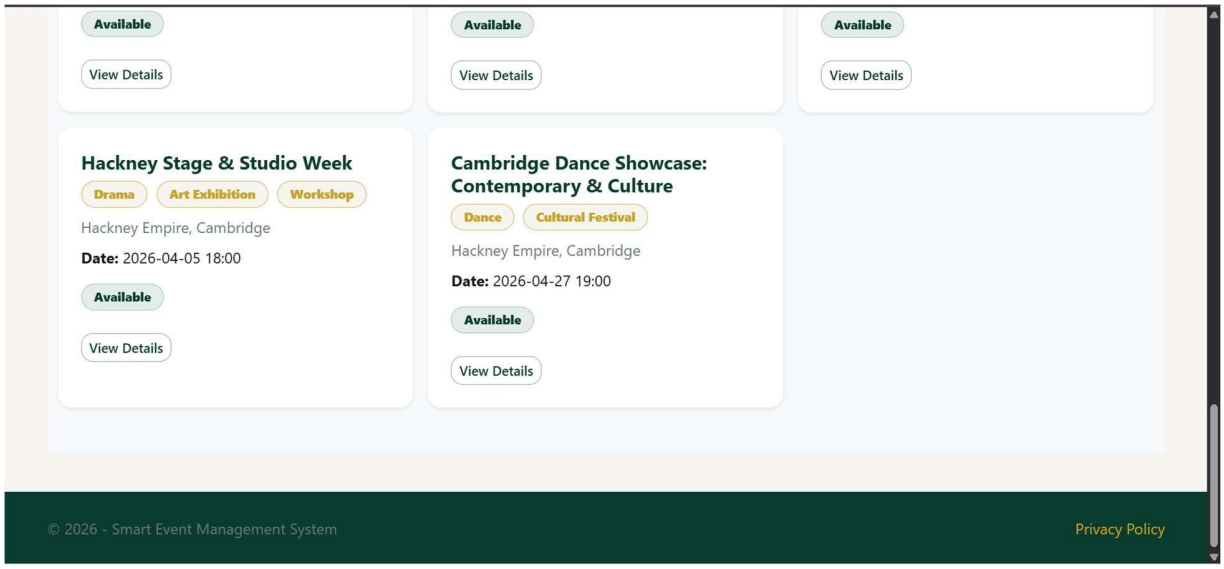


Figure 117: Homepage

The homepage serves as the principal landing page for the system and is available to all users, including guests, members, staff, and administrators. It provides a consistent and welcoming entry point that communicates the objective of the platform and the role of the council in arranging and managing cultural programmes across London and adjacent cities. Guests are able to read general information, examine upcoming events, and grasp the extent of activities offered without requiring an account, ensuring open access to event discovery.

A crucial aspect of the homepage is the presenting of future events in a simple, cardbased arrangement. Each event card includes crucial facts such as the event title, category, venue, date, and current availability status. This allows guests to explore schedules and venues freely; but, sensitive information such as detailed price, ticket selection, and booking actions are prohibited. Guests are requested to register or log in when seeking to access booking-related features, ensuring that transactions and personal data stay secure.

The homepage also encourages user guidance and engagement through concise introductory material, easy navigation links, and a conspicuous inquiry option for assistance. By segregating public access (event browsing and information) from restricted features (pricing, bookings, and account management), the homepage successfully integrates accessibility for guests with controlled functionality for registered users, preserving a unified experience for all roles within the system.

5.4 Member Module

The Member Module serves registered users by giving regulated access to ticket booking, pricing information, reviews, questions, and personal account administration. While general event information is accessible to all visitors, this module offers improved interactivity for verified members, ensuring that sensitive transactional functions and price details are confined to authorised individuals only. This design enforces role-based access control and maintains system integrity.

5.4.1 Member Registration

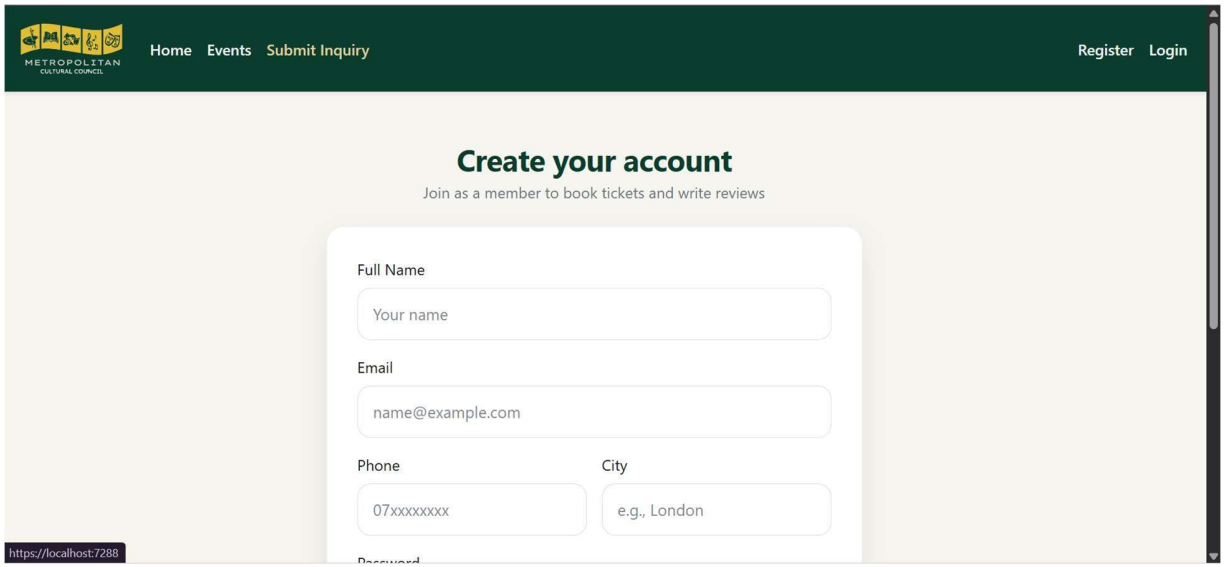


Figure 118: Member Registration

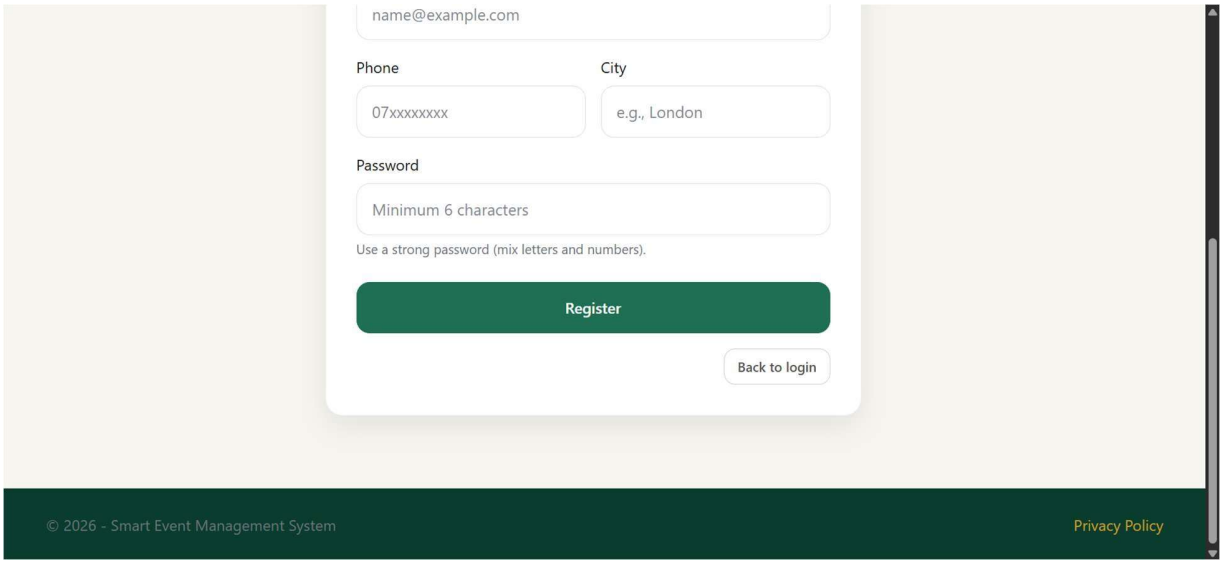


Figure 119: Member Registration

The Member Registration component allows new users to create an account in order to access member-only services. The registration form includes mandatory input areas like complete name, email address, phone number, city, and password. System-level validation guarantees that all fields have been filled out and that each email address is unique. Administrative and staff accounts cannot be created through this interface; thus, all users who successfully register through the system are immediately granted the member role.

5.4.2 Member Login

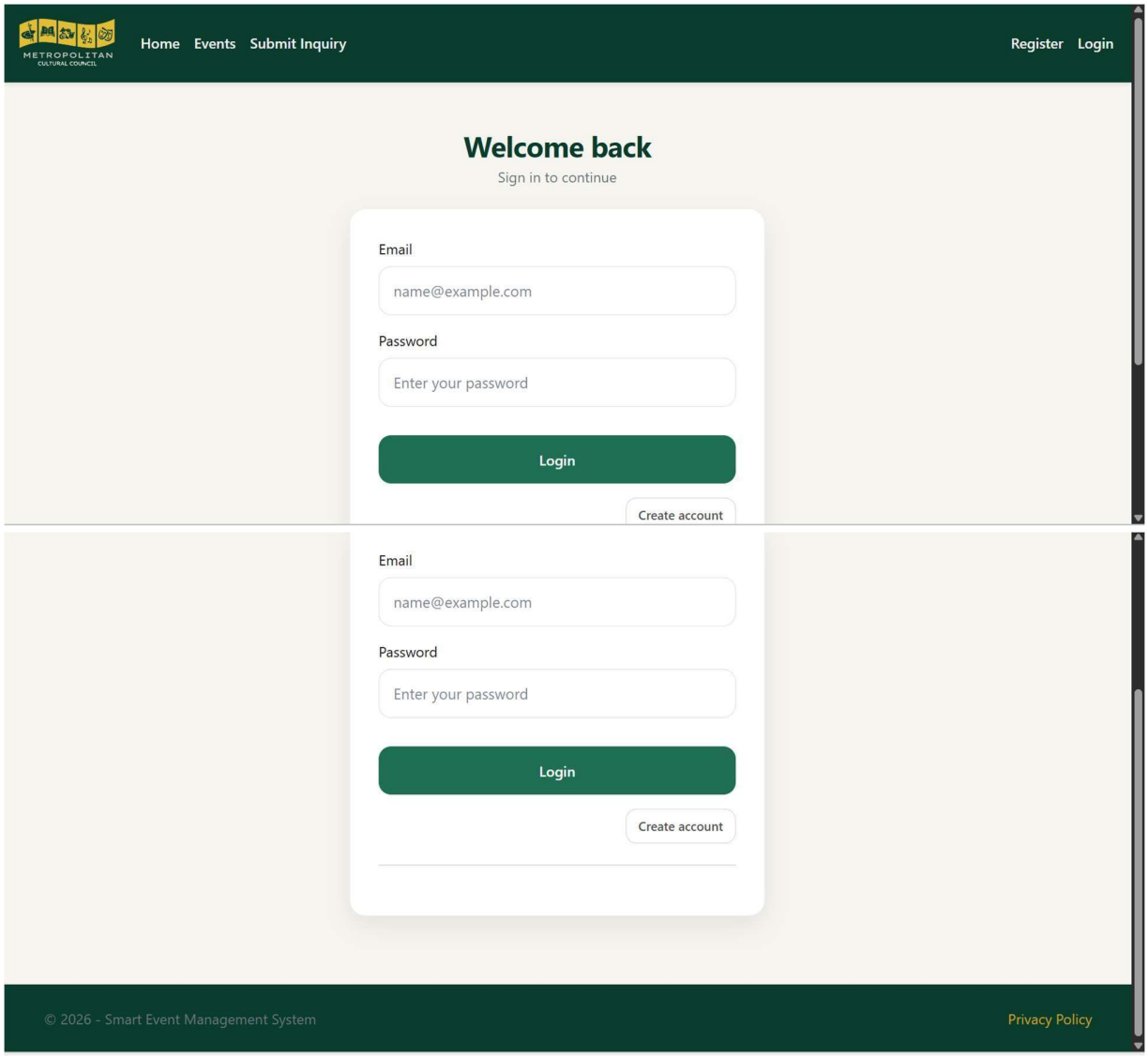


Figure 120: Member Login

The Member Login function offers safe login using registered email credentials and passwords. Upon successful login, users receive access to booking, price, and personalized options that are not available to guests. Invalid login attempts are handled by validation checks, preventing unauthorised access. This technique ensures that only authenticated members can browse ticket prices and execute bookings.

5.4.3 Browse & Search Events

All Events:

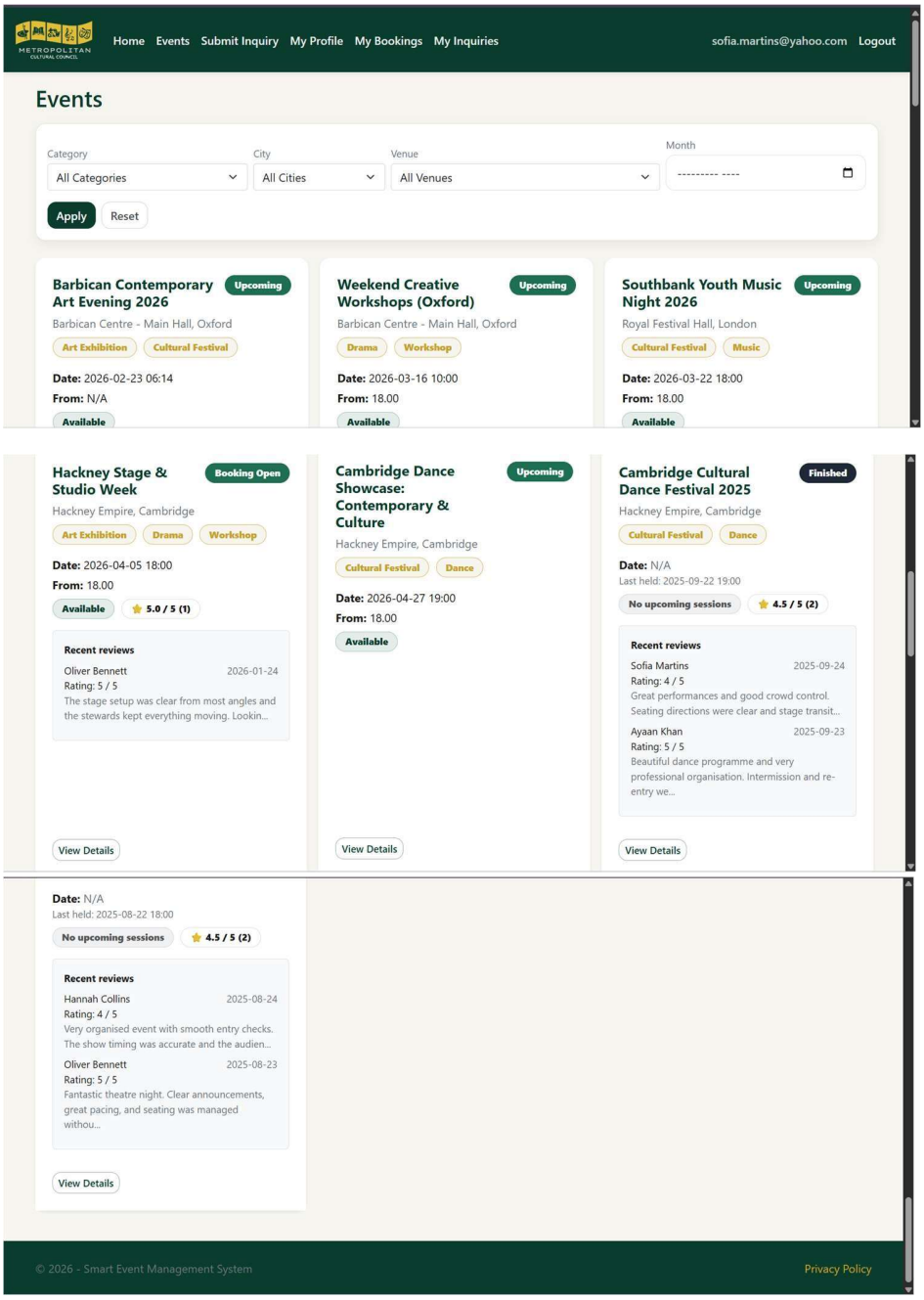


Figure 121:Events

Filter Events:

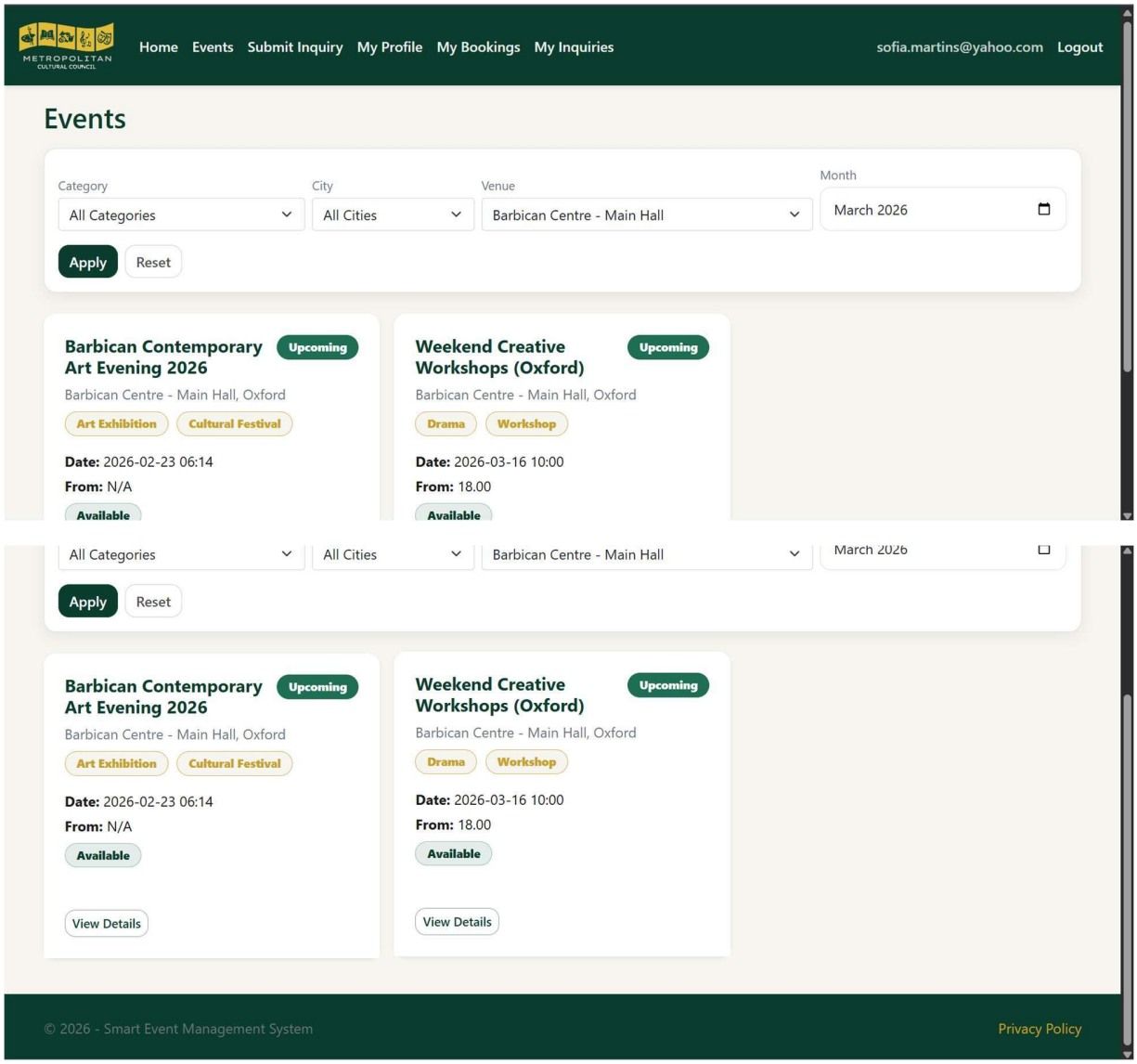


Figure 122:Filter events

Event Details:

Barbican Contemporary Art Evening 2026

Upcoming

Barbican Centre - Main Hall, Oxford

About this event

A curated exhibition night featuring emerging artists and short guided walk-throughs. Lighting and room flow are designed for easy viewing, with quiet zones and scheduled talk segments.

Date and booking

Monday, 23 Feb 2026 • 06:14 - 09:14

A curated exhibition night featuring emerging artists and short guided walk-throughs. Lighting and room flow are designed for easy viewing, with quiet zones and scheduled talk segments.

Date and booking

Monday, 23 Feb 2026 • 06:14 - 09:14

Book Tickets | Back to Events

Reviews

No reviews yet.

View all reviews | Write review

Figure 123: Event Details

Members can explore and find upcoming events through a structured event listing interface. Each event card includes crucial information such as the event title, category, venue, date, and availability status. For active and forthcoming events, a “From” label is provided to indicate the starting ticket price, which indicates the lowest available ticket price for that event session. This starting price is not displayed for finished events, ensuring that pricing information remains relevant and correct. Detailed ticket prices and booking choices become apparent only after a member views the event details page.

5.4.4 Ticket Booking System

Selecting Ticket Type:

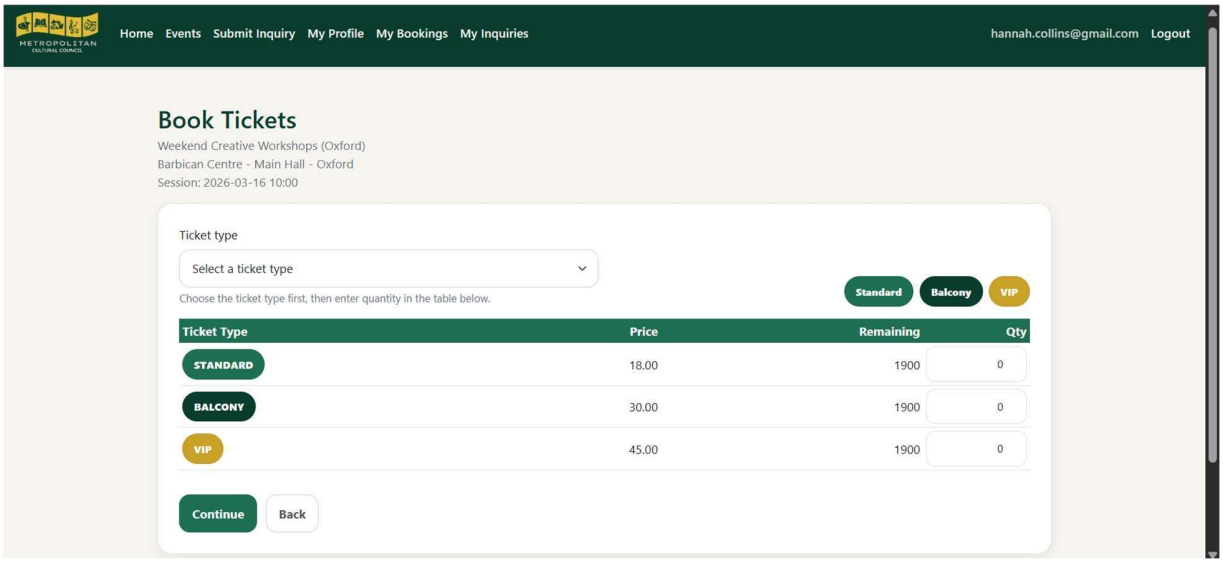


Figure 124: Selecting tickets

Before Payment:

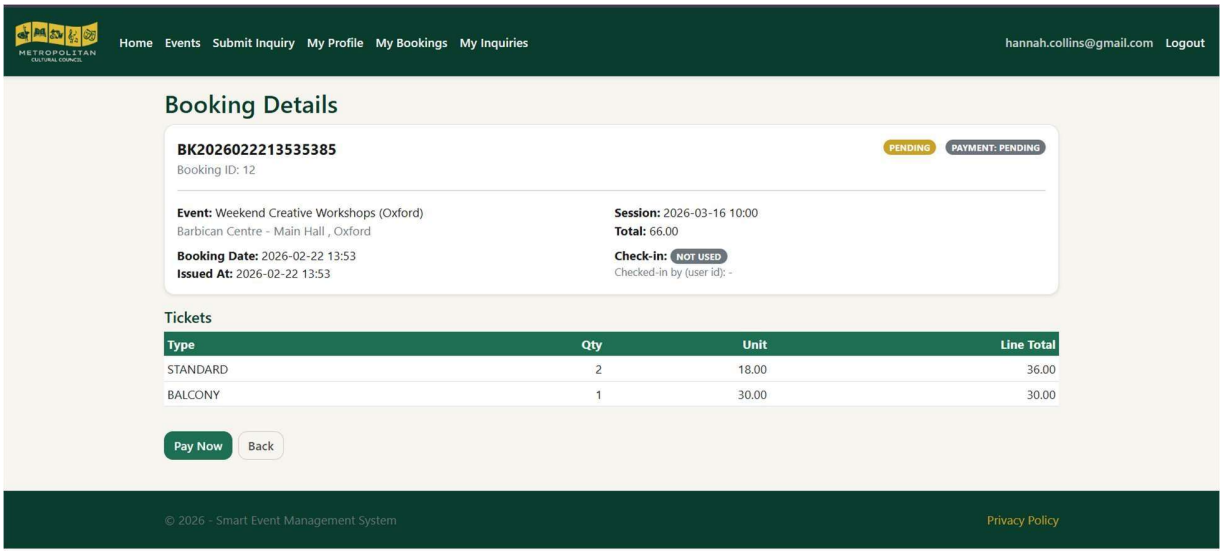


Figure 125: Booking - before payment

After Payment:

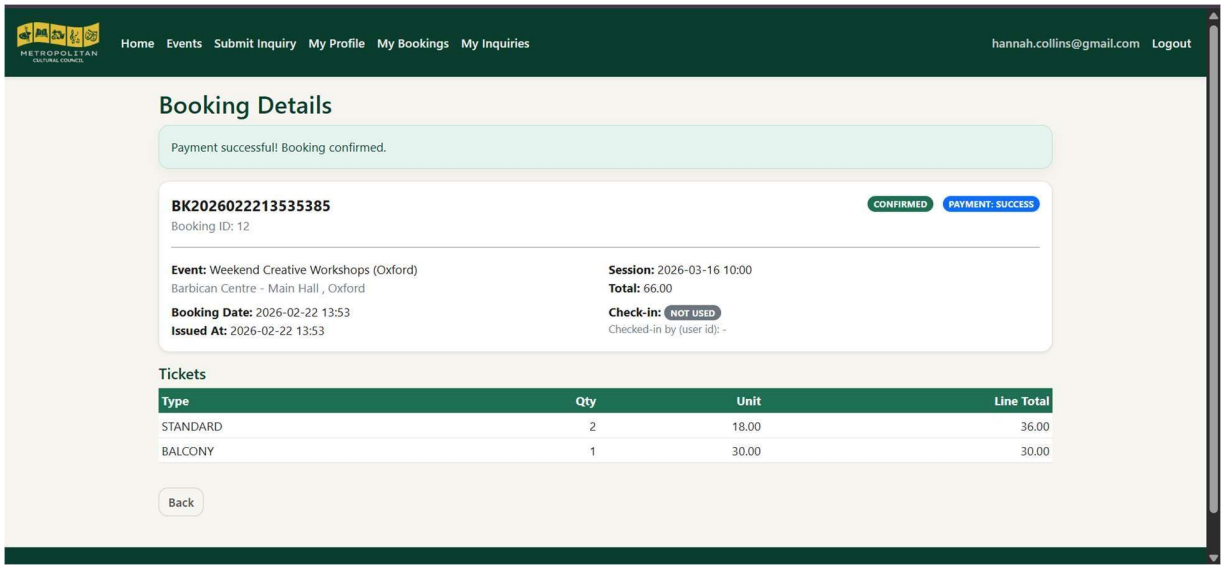


Figure 126: Booking - after payment

Ticket purchasing and full pricing details are restricted solely to registered members. Once logged in, members can browse available ticket kinds, prices, and seat availability for each active event session. Guests are needed to register and log in before accessing this information. The booking process allows members to select ticket numbers and proceed through validation and payment phases, ensuring that bookings are made only within available capacity restrictions.

5.4.5 Review Submission

If didn’t attend the event:

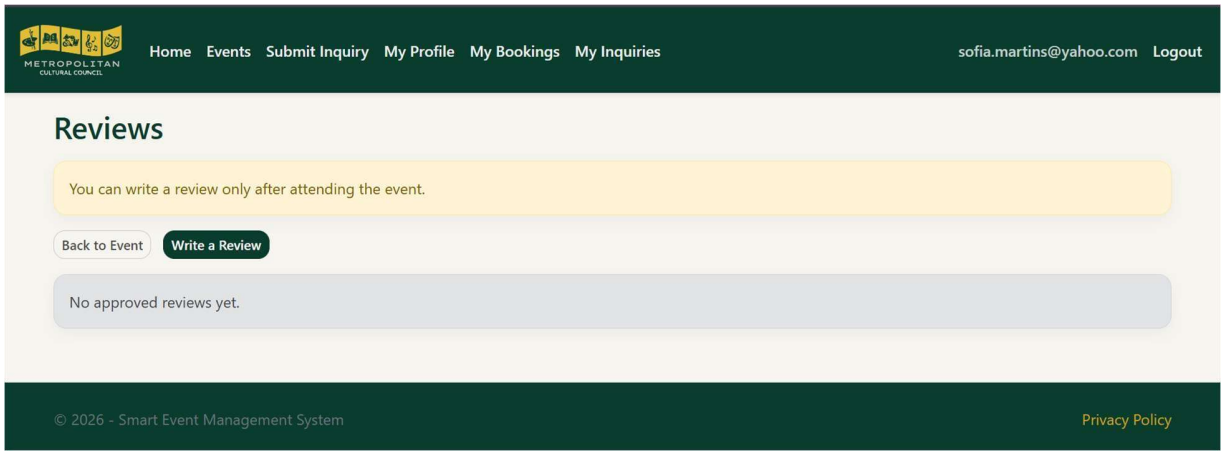


Figure 127: Review submission - if not attended the event.

If attended the event:

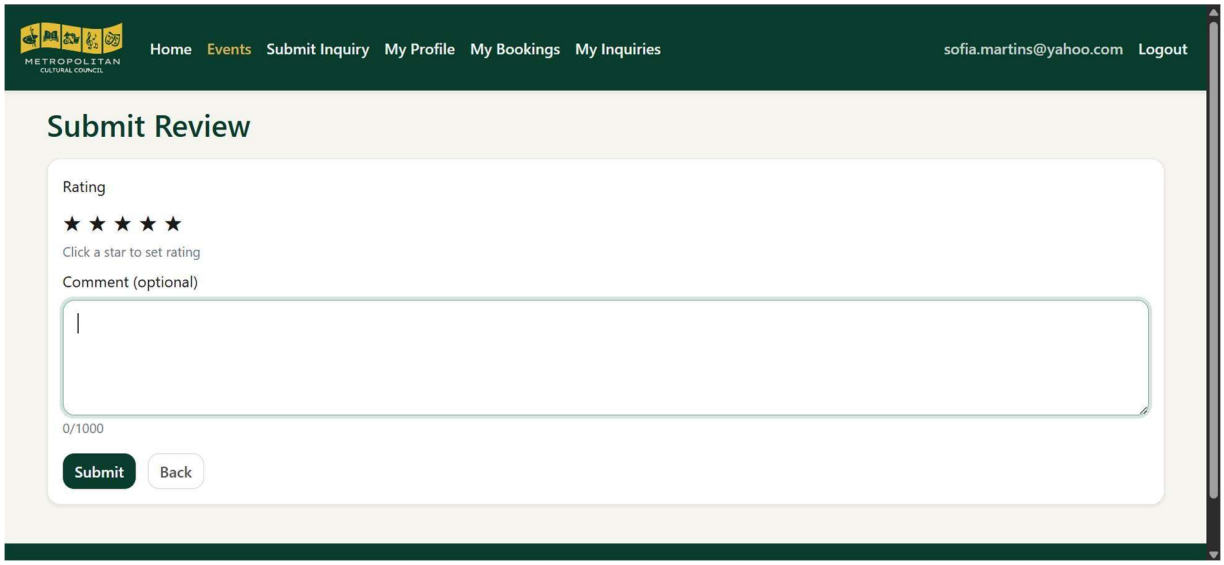


Figure 128: Review submission - if attended the event

The Review Submission tool allows users to submit ratings and feedback for events they have attended. Review access is restricted to completed bookings, guaranteeing that only verified attendees can leave reviews. This limitation preserves the integrity and reliability of feedback displayed within the system and prevents unauthorised or deceptive reviews.

5.4.5 Inquiry Submission

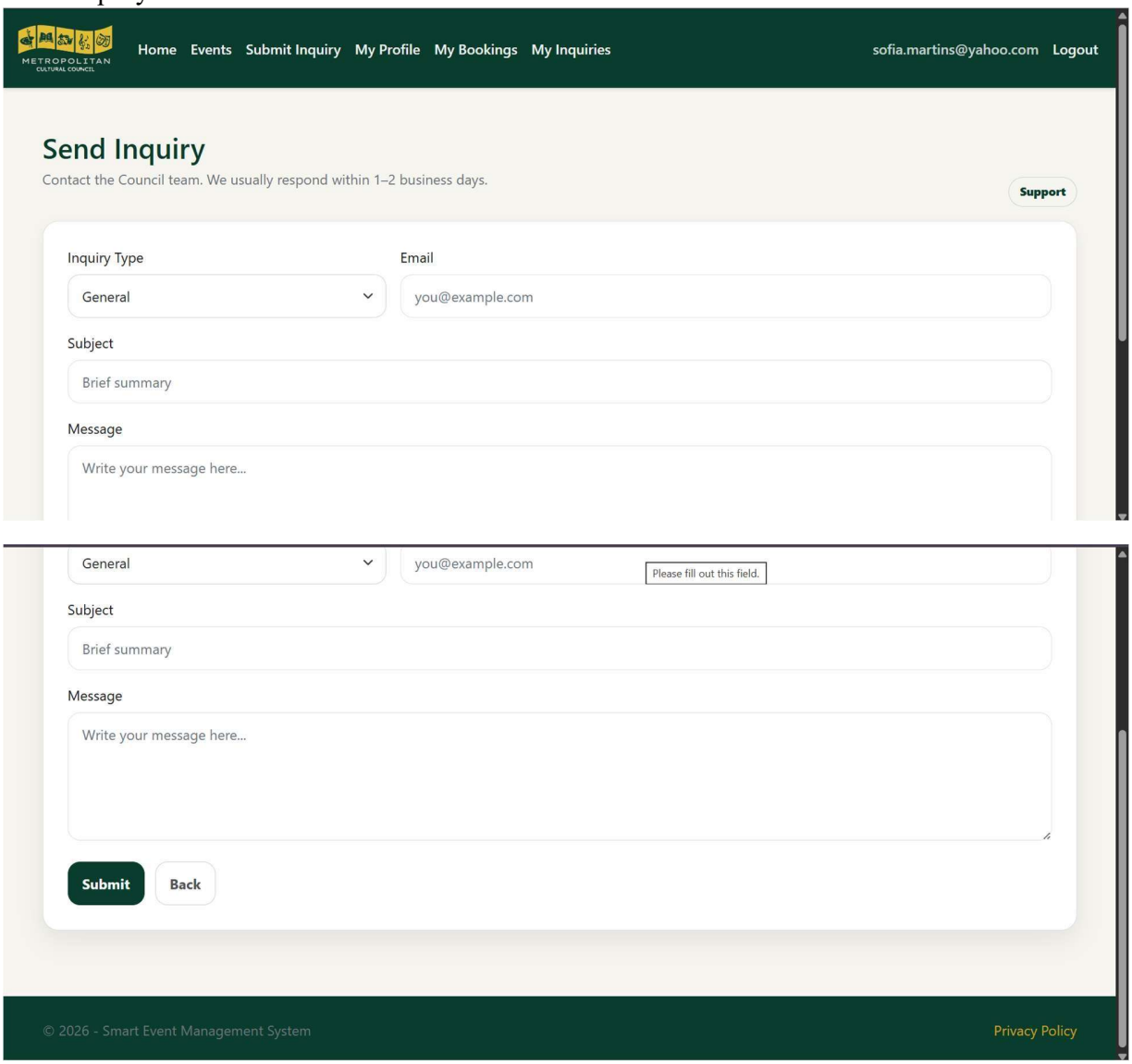


Figure 129:Inquiry Submission

Members can send inquiries directly through the system by selecting an inquiry type and entering a subject and message. This tool provides structured communication pertaining to events, bookings, payments, or general concerns. All inquiries are recorded for administrative review and response, allowing efficient handling and follow-up.

5.4.7 Profile Page

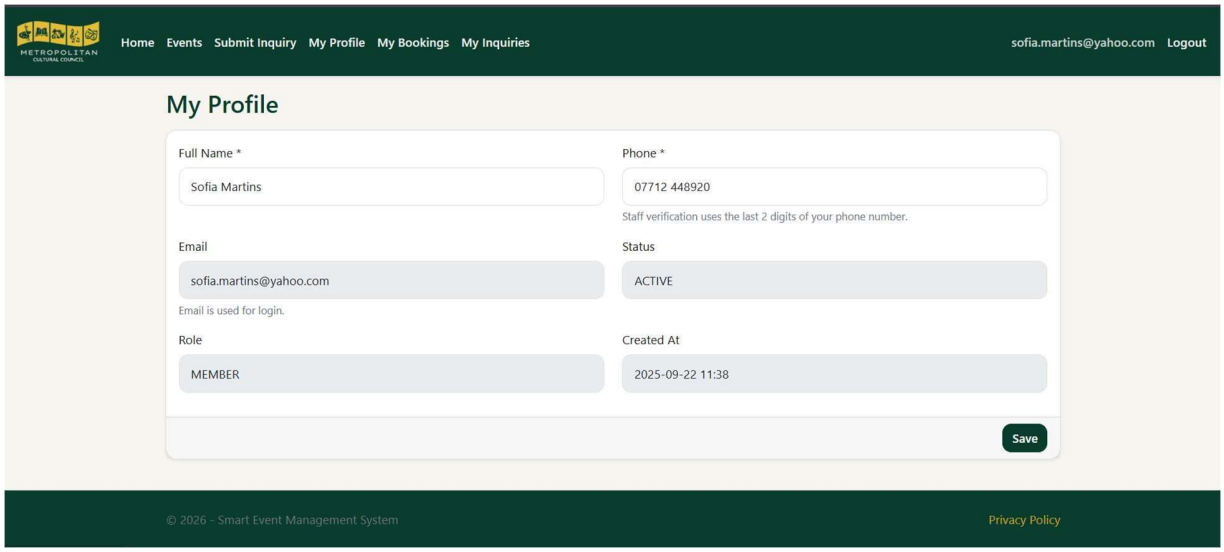


Figure 130: Profile page

The Profile Management tool allows members to view and update certain personal details such as name and phone number, while keeping sensitive fields such as email address and role noneditable. This strategy assures data integrity while allowing members to preserve accurate contact information.

5.4.8 My Bookings Page:

Summary:

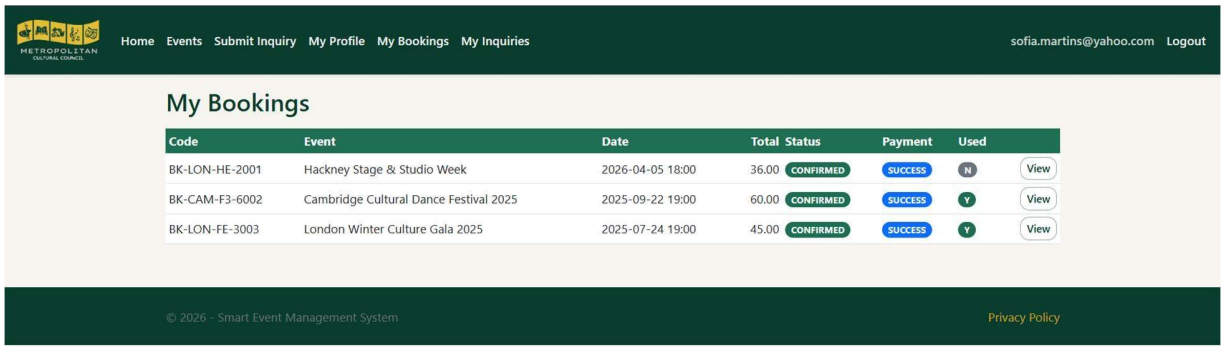


Figure 131: Summary of bookings

Booking Details:

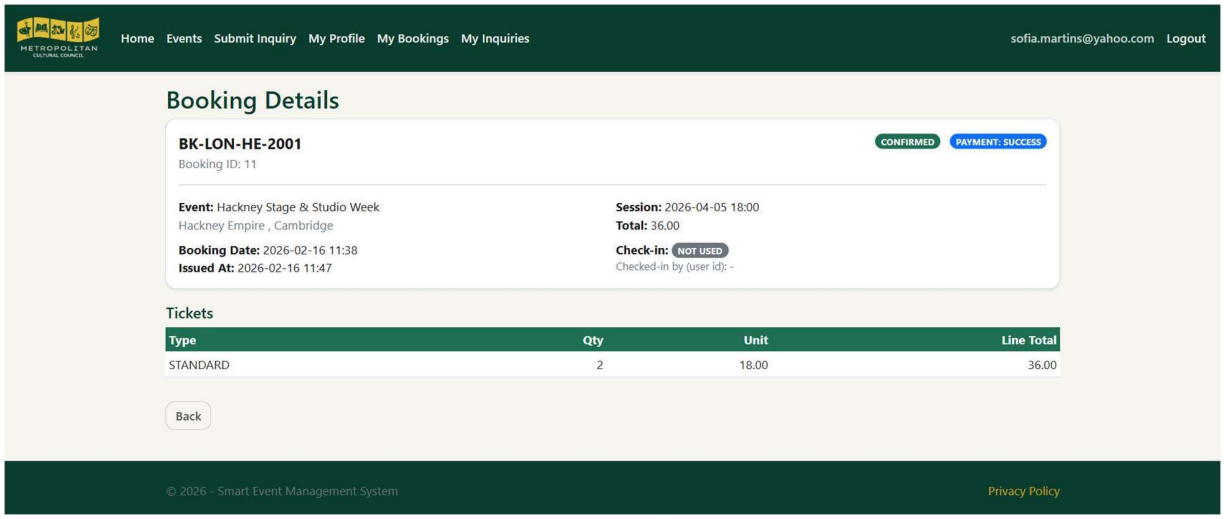


Figure 132: Booking details

The My Bookings area offers users with a detailed overview of their booking history. It displays facts such as event name, session date, booking status, payment status, ticket quantities, and total cost. Members can view both past and upcoming bookings, promoting transparency, recordkeeping, and simple access to booking information.

5.5 Guest Module

The Guest Module gives unregistered users read-only access to the system. Its major goal is to allow users to discover available cultural events, view public feedback, and contact with the

council, while limiting access to sensitive transactional functions like price and ticket purchase. This regulated access promotes user registration while providing transparency about planned programs.

5.5.1 Browse Events

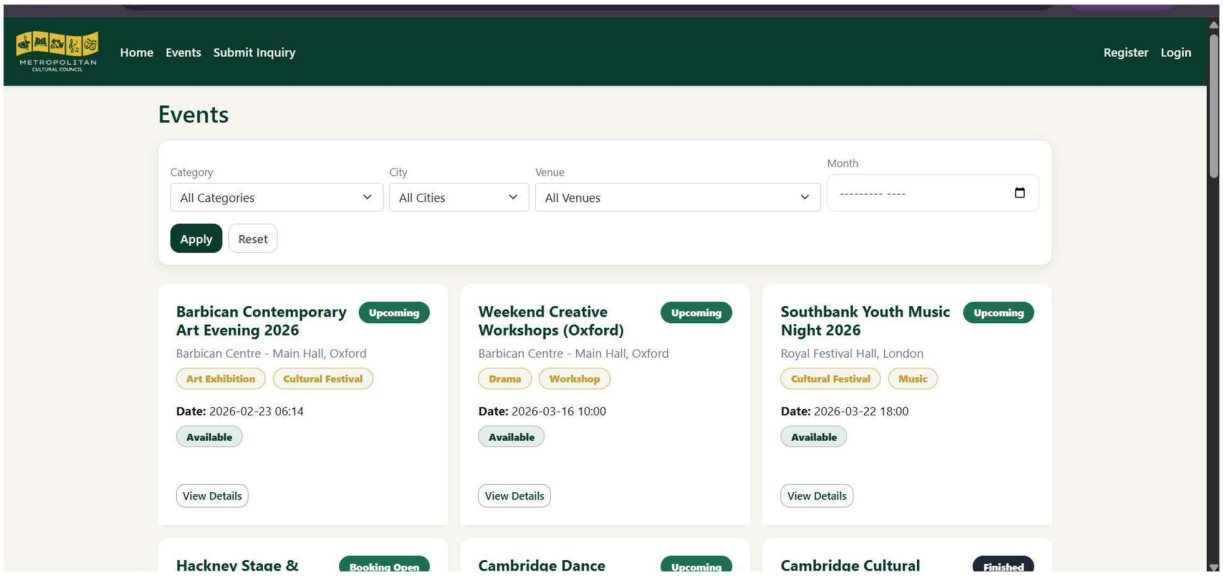


Figure 133: Browse events

The public events interface allows guests to explore a list of scheduled and active events. Each event listing includes key details such as the event title, category, venue, date, and availability status. However, guests are not able to see ticket prices and booking options. This protects price information for authorized members while allowing others to view event schedules and details before registering.

5.5.2 Browse Event Reviews

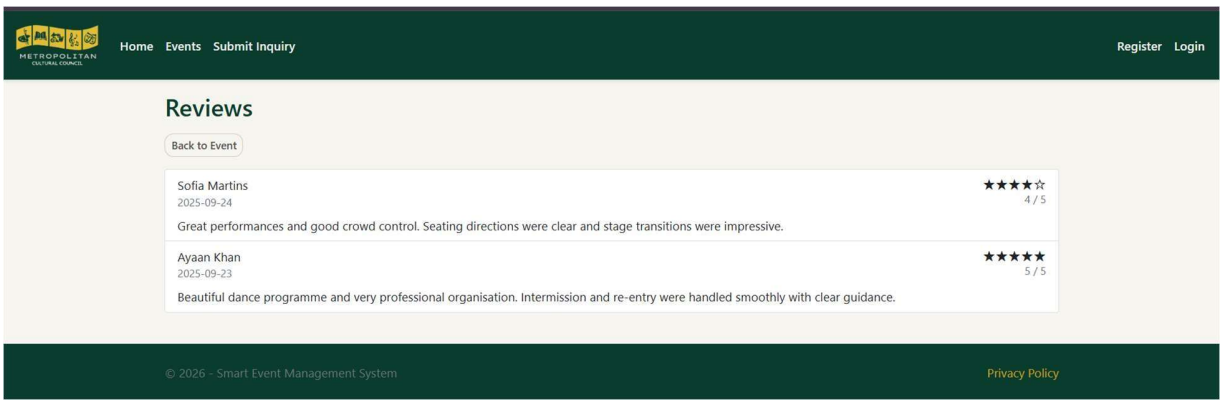


Figure 134: Browse event reviews

Guests can view event reviews made by registered members. This tool enables unregistered users to evaluate event quality and reliability based on previous attendee input. Reviews are shown in a read-only style, and guests are unable to write their own reviews. This strategy increases trust and engagement while maintaining review authenticity.

5.5.3 Restricted Event Search

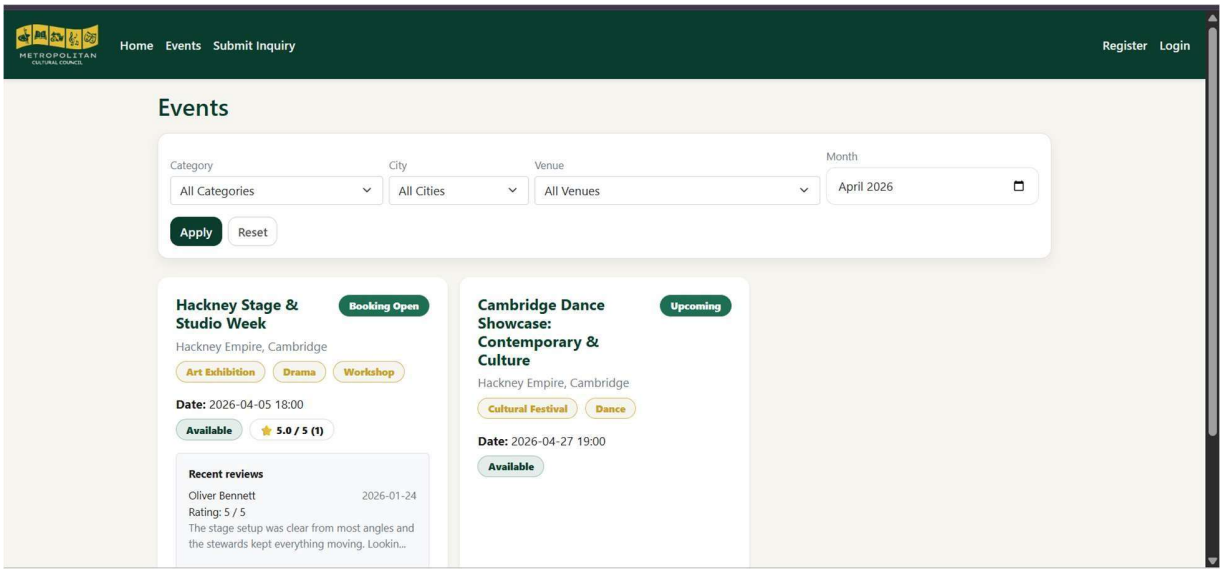


Figure 135: Restricted search

The event search functionality is available to attendees with limited access. Guests can filter events by category, city, venue, and date, but the search results do not include ticket pricing or booking information. This restriction allows guests to discover relevant events without accessing member-only transactional data.

5.5.4 Inquiry Submission

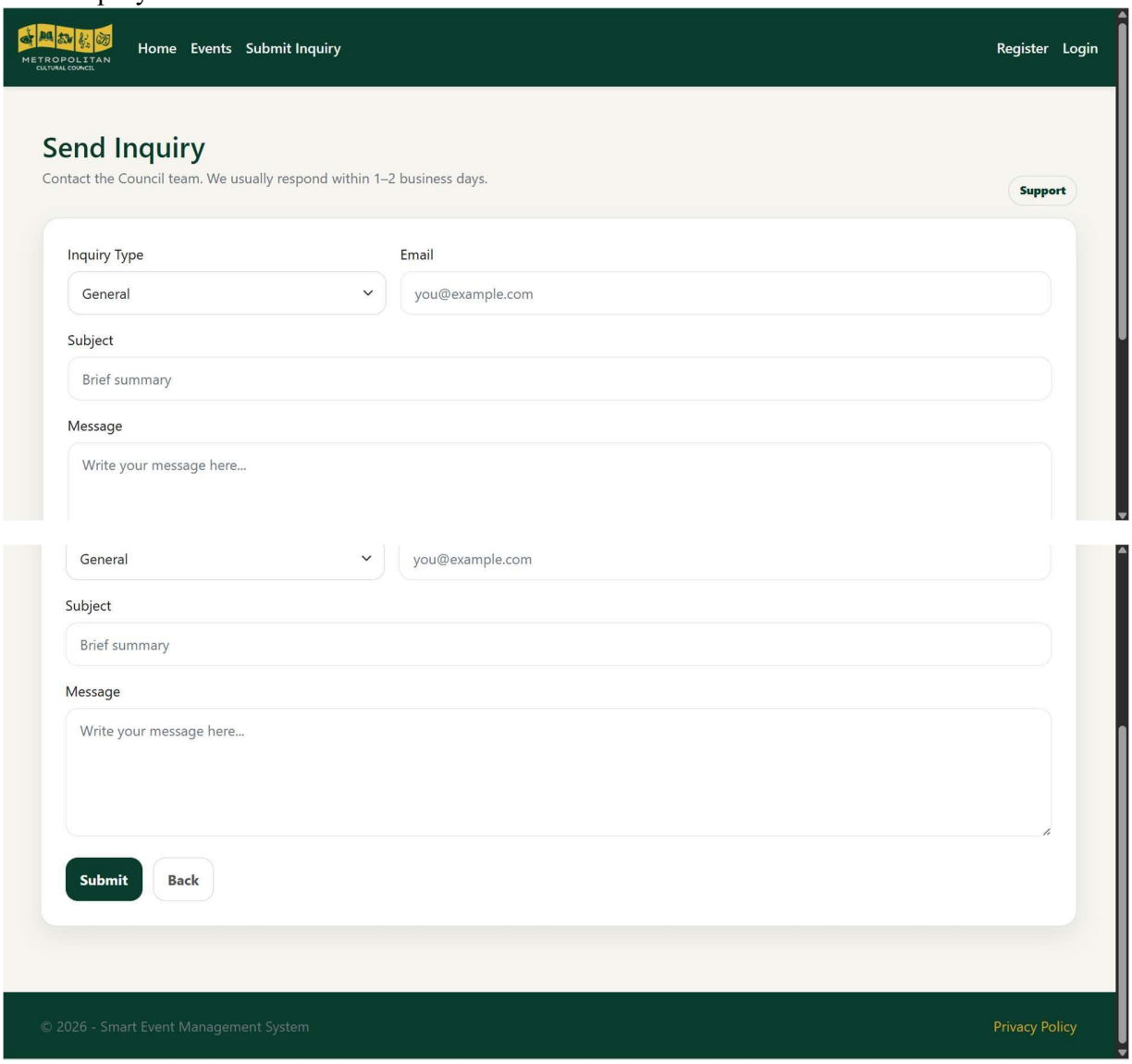


Figure 136: Inquiry submission

The Inquiry Submission function enables visitors to contact the council without registering. Guests can send queries by entering an email address, choosing an inquiry type, and specifying a subject and message. These requests are stored in the system for administrative review and response, allowing communication while limiting booking and account functions.

5.6 Admin Module

The Admin Module grants complete administrative control over the system and is only accessible to authorized administrative users. It enables the development, management, monitoring, and

moderation of all basic system entities, such as venues, categories, events, sessions, reviews, and queries. This module ensures that cultural programs are properly maintained, bookings are tracked, and user-generated content is controlled.

5.6.1 Admin Homepage

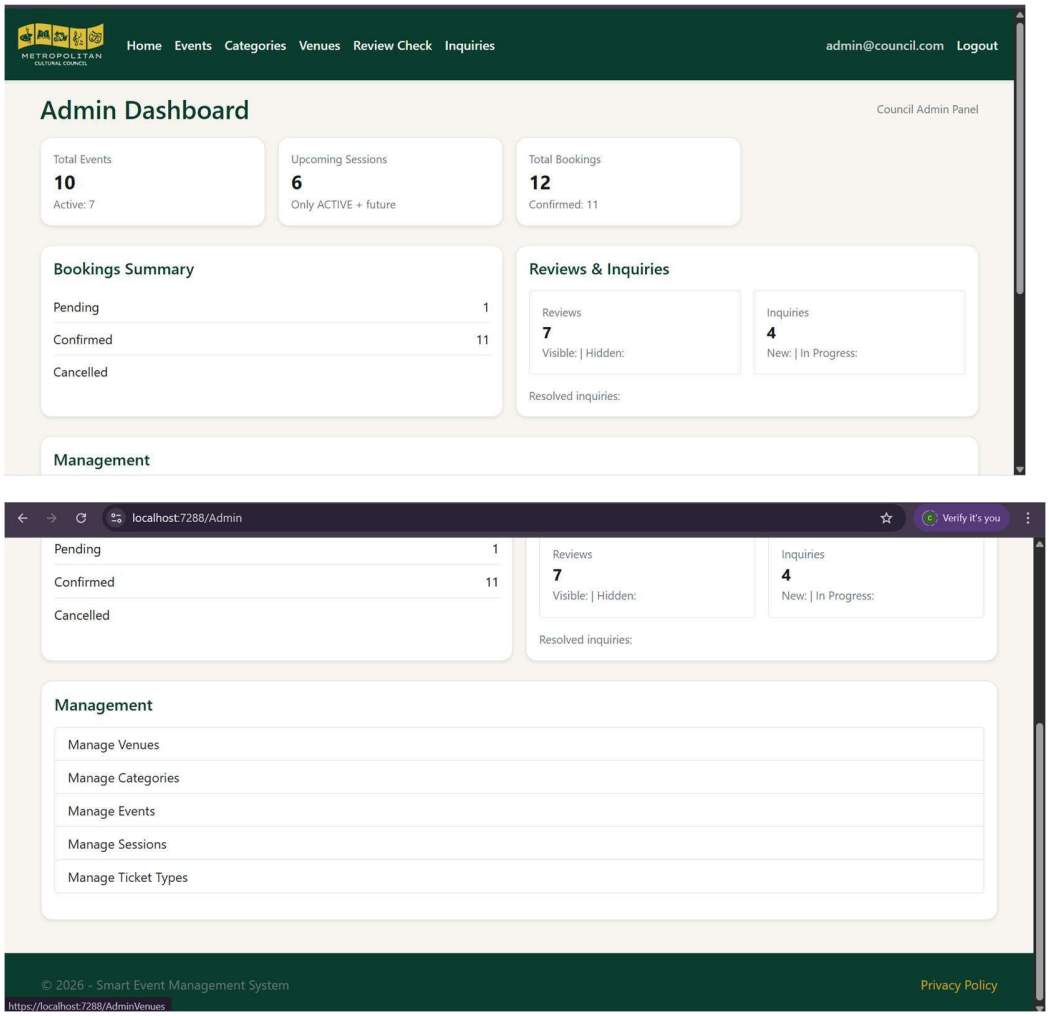


Figure 137: Admin homepage

The Admin Homepage functions as a primary dashboard, providing an overview of system activities and operating condition. It provides crucial statistics such as total events, forthcoming sessions, booking counts, and review and inquiry summaries. This integrated view allows administrators to easily assess system utilization, identify pending activities, and monitor overall performance without having to navigate through each management panel.

5.6.2 Add/Update Venues

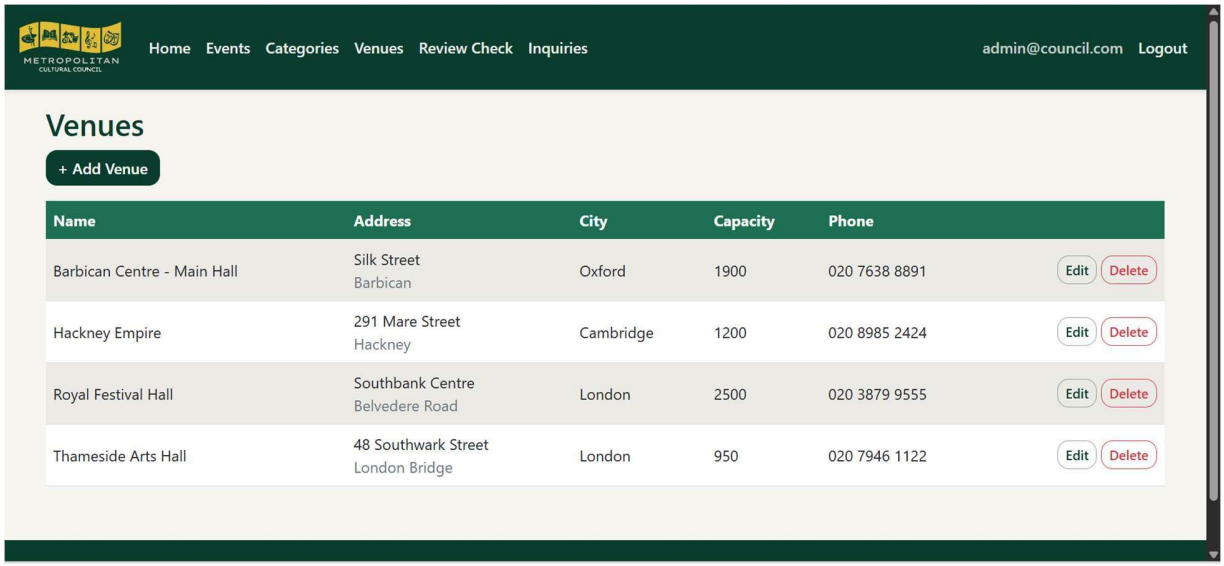


Figure 138: Add/update venues

The venue administration function enables administrators to add new venues and amend existing venue information. Each venue record includes information such as the venue name, address, city, seating capacity, and phone number. Administrators can modify or remove venues as needed, ensuring that event locations are correct and up to date. This functionality enables accurate capacity planning and dependable event scheduling.

5.6.3 Add/Update Categories

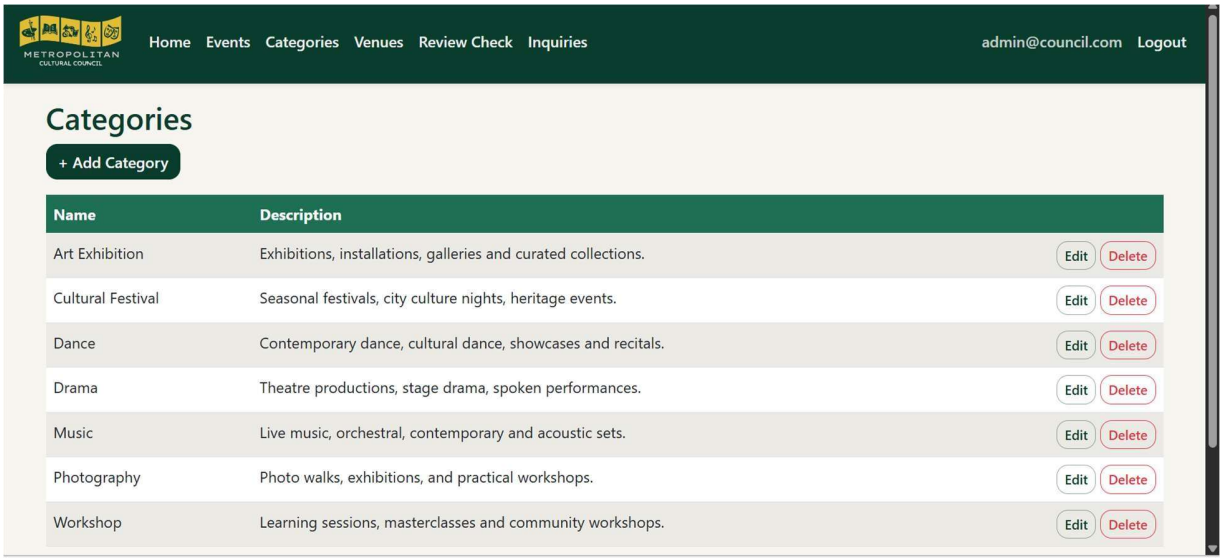


Figure 139: Add/update categories

This functionality allows administrators to manage event categories across the system. Administrators can create new categories or change existing ones by setting their names and descriptions. Categories are used to classify events and provide filtering and searching capabilities. Proper category management increases event discoverability while maintaining consistency in event classification.

5.6.4 Add/Update Events

All events:

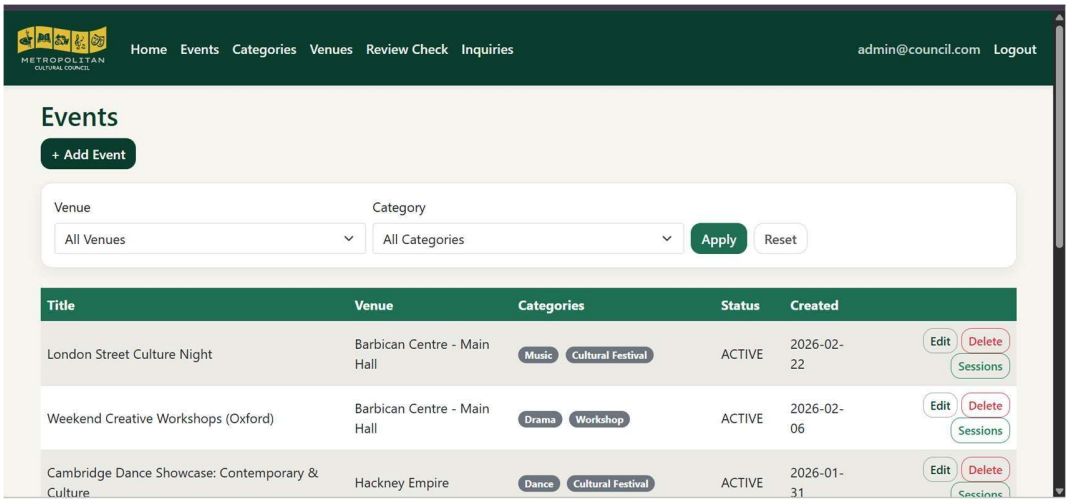


Figure 140: Add/update events

Filter Events:

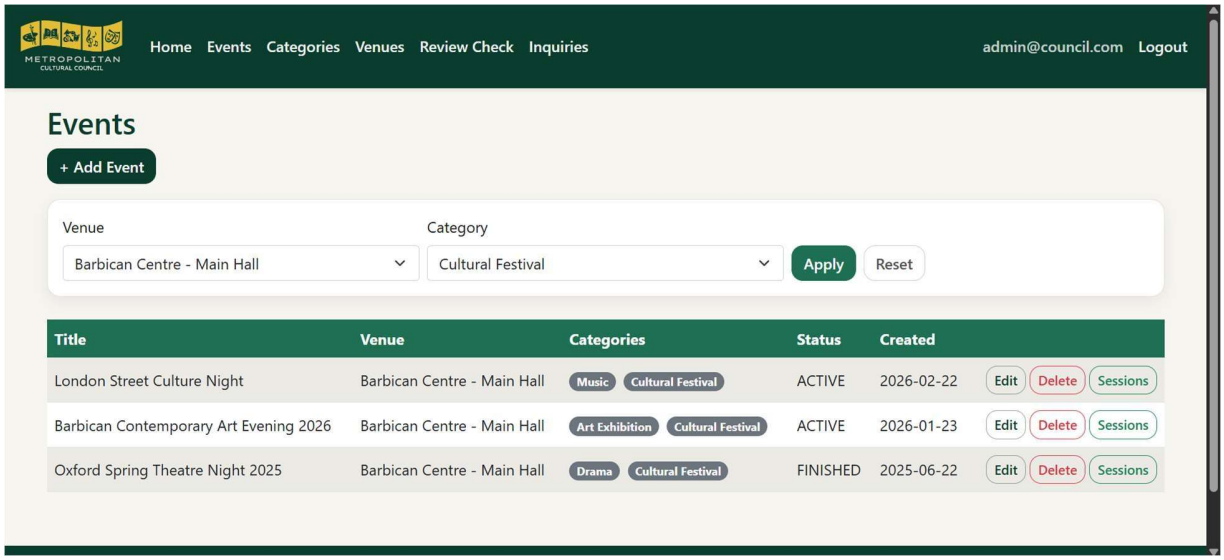


Figure 141: Filter events

The event management component enables administrators to create, change, and manage event records. Each event has information such as the title, description, venue, connected categories, status, and creation date. Administrators can select events by location or category, modify event details, manage event status (active or completed), and remove events as needed. This guarantees that only correct and relevant events are displayed to users. Administrators are also in charge of controlling the event sessions linked with each event. This involves specifying session dates, beginning and ending times, total seat capacity, and session status. Sessions can be added, edited, and cancelled as needed. Proper session management provides precise scheduling, seat availability tracking, and seamless integration with ticket booking and personnel check-in operations.

5.6.5 Review Moderation

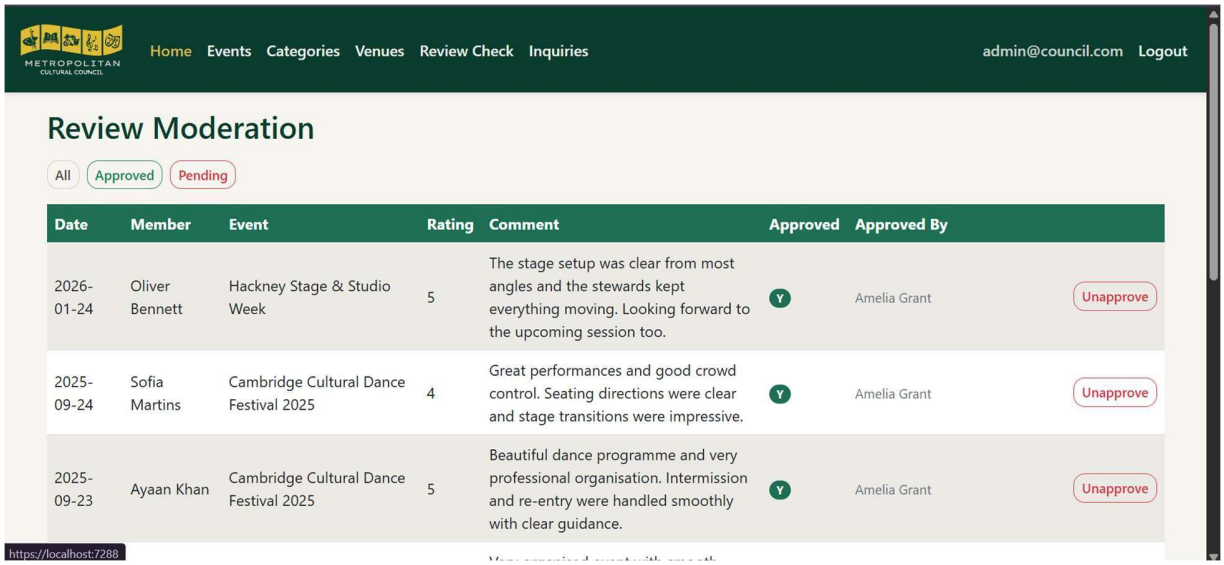


Figure 142: Review Moderation

The Review Moderation tool allows administrators to unapprove reviews submitted by members. Each review is displayed with information such as the event name, rating, comment, and submission date. Administrators can monitor reviews to ensure that the published input is suitable, relevant, and follows system requirements. This procedure ensures the credibility and quality of user-generated material.

5.6.8 Inquiries Handling

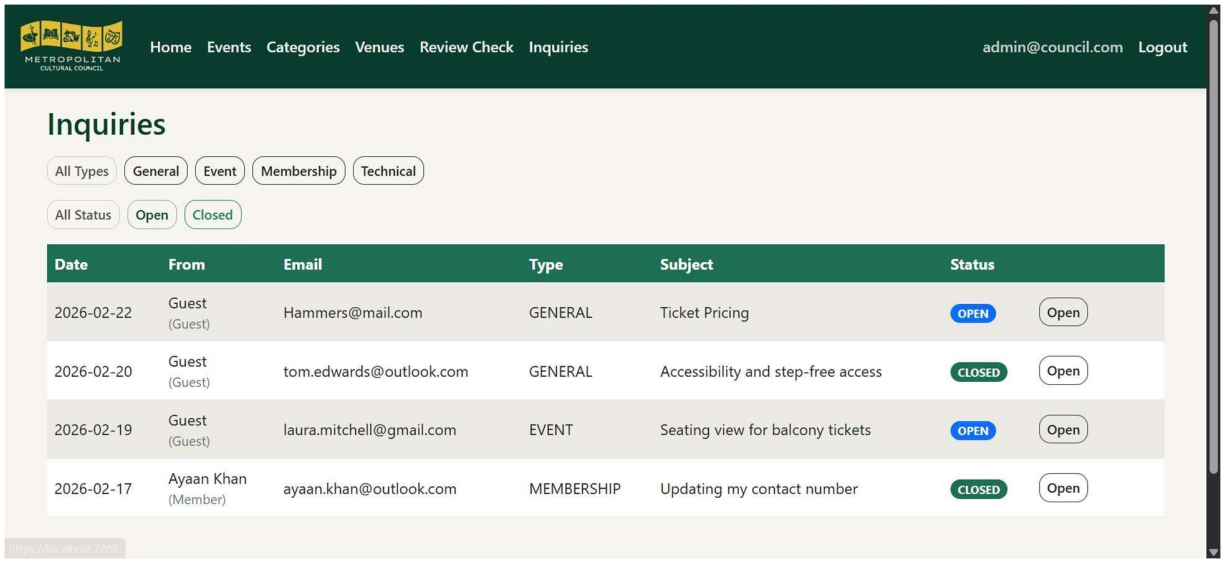


Figure 143: Inquiries handling

The inquiries handling function enables administrators to examine and manage inquiries filed by both guests and members. Inquiries are classified by category and status, allowing for effective filtering and prioritizing. Administrators can review inquiry details, alter the status (open or closed), and respond as needed. This promotes systematic communication and efficient issue resolution.

5.7 Staff Module

The Staff Module is intended to support operational activities during live event sessions. Administrators appoint staff users, who are responsible for on-site ticket verification and attendee check-in. This module separates operational and administrative tasks by preventing staff from creating bookings or modifying event data.

5.7.1 Verify Check-ins

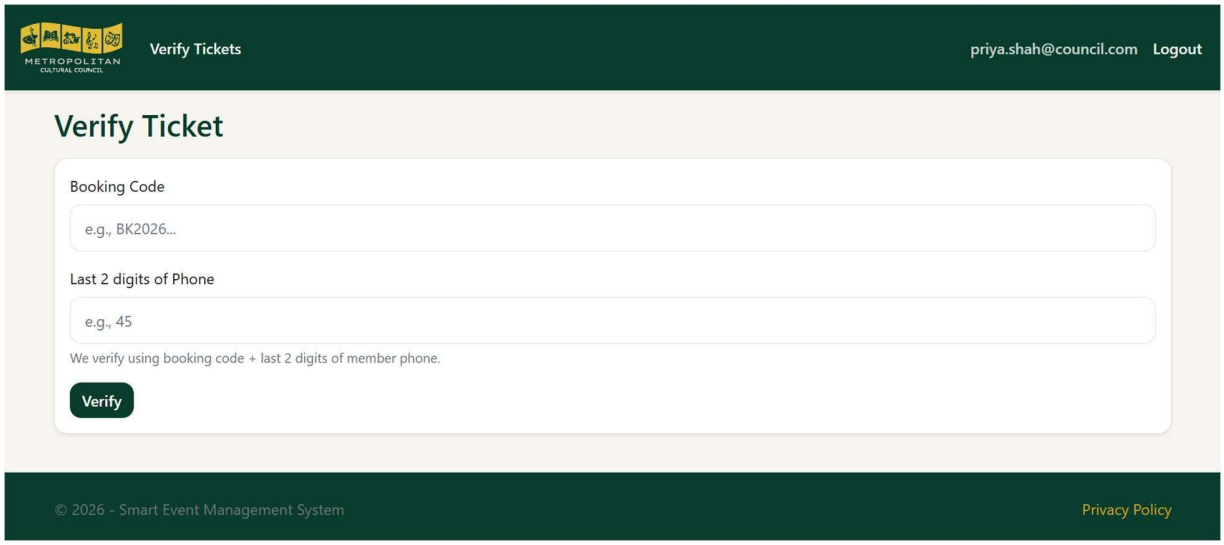


Figure 144: Verify check-ins

The Verify Check-ins tool allows personnel to authenticate attendee access into the venue for a specific event session. Verification is done using a two-factor matching mechanism that includes the booking code generated at the time of booking and the last two digits of the registered phone number. Both values must match the records in the database. The validation logic is implemented at the database level to ensure accuracy, consistency, and protection from manual tampering. Only when both the booking code and phone number digits match, and the booking and payment statuses are genuine, is the attendee designated as verified. This procedure ensures safe and

efficient entry control while prohibiting unauthorized access to event sessions. Staff members execute this verification on-site at the event venue to provide real-time attendance management.

5.8 UI Design and Navigation Flow

The user interface is sleek, consistent, and role-aware, making it easy to use for guests, members, staff, and administrators. A consistent layout structure is maintained across the system, with navigation menus dynamically altering dependent on the authorized user's role. This method makes navigation easier while restricting unauthorized access to restricted elements.

For guest users, the navigation flow is centered on event discovery and information access. Guests can explore events, check event details, and send queries without logging in.
Ticket pricing and booking choices are buried, urging people to join and log in before accessing member-only services like booking and price viewing.

Members may now browse ticket prices, select event sessions, manage bookings, and write reviews through the expanded navigation menu. The booking process is organized and simple, helping members through session selection, ticket selection, and confirmation. Administrative users have specialized dashboards for managing venues, events, sessions, reviews, and queries, whilst staff users have a simpler interface that focuses on check-in verification during events. Overall, the UI design and navigation flow promote effective engagement, clarity, and role-specific functionality throughout the system.

6. Testing and Validation

6.1 Testing Strategy

The testing strategy was developed to guarantee that the system is reliable, secure, and meets established functional and non-functional requirements. Functional testing, role-based access testing, and database-level validation were used to assess both user-facing procedures and data integrity. Testing was performed on all user roles- guest, member, staff, and administrator; to ensure that each position could only access approved features and that unauthorised actions were properly restricted.

End-to-end functional testing was carried out via the web interface to validate critical user experiences such as authentication, registration, event discovery, booking, review submission, administrative administration, and staff check-in verification. Negative testing cases were also included to ensure the handling of invalid inputs, failed login attempts, and rule breaches including overbooking or unauthorized access. In parallel, SQL Server Management Studio was used to test insert operations, query correctness, and the enforcement of constraints such as primary keys, foreign keys, unique constraints, and check constraints. This layered testing strategy provided consistency between application behaviour and stored data, reduced the possibility of data anomalies, and increased overall system reliability.

6.2 Functional Testing

- Login and Authentication Test: Login and authentication testing ensures that users can successfully authenticate with valid credentials and are denied access if they give invalid credentials. It guarantees that verified users are forwarded to the proper interface for their function, such as member, staff, or administrator.
- Registration Test: Registration testing guarantees that all essential fields are completed before an account can be created. Validation checks guarantee correct input forms and ensure email uniqueness. This testing further verifies that public self-registration only creates member accounts and cannot assign staff or administrator roles via the registration interface.

- Event Browsing and Search Test: Event browsing and search testing confirms that users can browse and search events by category, location or city, and date. It guarantees that search

results are accurate, uniform in format, and represent actual event availability. Access limitations are confirmed, allowing people to explore events but not view pricing or booking information.

- Booking and Pricing Access Testing: Booking and pricing access testing has confirmed that only authenticated members may view ticket prices and make bookings. It confirms successful booking creation when there are enough seats available and guarantees that booking codes are generated correctly. Negative scenarios examine overbooking efforts to ensure that the system prevents bookings that exceed available capacity.
- Payment and Booking Confirmation Test: Payment and booking confirmation testing ensures that successful transactions accurately update the payment and booking statuses. It ensures that bookings with pending or unsuccessful payments are prohibited and cannot be used as entry verification. This testing demonstrates that booking confirmation is completely contingent on successful payment completion.
- Review Submission and Moderation Test: Review submission and moderation tests ensure that only qualified members who have attended events may submit reviews. It verifies that reviews are initially unapproved and remain concealed from public view until evaluated by an administrator. This approach ensures the authenticity and quality of published feedback.
- Admin Management Feature Test: Admin administration feature testing ensures that administrators have the ability to administer venues, categories, events, and sessions, as well as moderate reviews and questions. It confirms that these functionalities are exclusively available to administrators and are restricted to other roles, guaranteeing strong role-based access control.
- Staff Check-In Verification Testing: The staff check-in verification exam certifies the onsite entry verification process. It makes sure that check-ins require both the booking code and the last two digits of the member's registered phone number. Verification logic is enforced at the database level, and successful verification accurately updates the booking usage status.

6.3 Database Testing

- Insert Testing: Insert testing ensures that valid entries may be successfully inserted to core database tables and that all required fields are completed. It ensures that the database accepts properly organized input while rejecting incomplete insert operations.
- Unique Constraint Testing: Unique constraint checking ensures that duplicate entries be avoided, such as numerous user accounts sharing the same email address. The database rejects attempt to input duplicate values, ensuring that the data is unique and consistent.
- Foreign Key Constraint Test: Foreign key constraint testing demonstrates that child records cannot refer to non-existing parent data. Insert attempts with invalid foreign key values are refused, preventing orphan records and ensuring relational integrity.
- Query Accuracy Test: Query accuracy testing ensures that common retrieval queries produce accurate and relevant results. This contains queries for forthcoming event sessions, member booking history, and authorized reviews, as well as accurate filtering and joins between related databases.
- Data Validation Test: Data validation testing guarantees that CHECK constraints and domain rules do not allow improper values, such as unsupported status codes or negative numeric data. This prevents the database from entering invalid or inconsistent states.
- Referential Integrity Test: Referential integrity testing ensures that dependent relationships are valid following insert and update activities. It checks that entries like booked tickets always correspond to current ticket types and bookings, guaranteeing consistency across related tables.

6.4 Test Cases Table

1. Functional Testing

2. Field	Details
Test ID	ETS-FT01
Test name	Login with valid credentials
Role	Member
Feature	Authentication
Test steps	1) Open Login page 2) Enter valid email and password 3) Click Login
Expected result	User logs in successfully and is redirected to the appropriate role-based interface.
Actual Result	

Status	Pass

Table 14: Testcase Table 01

Field	Details

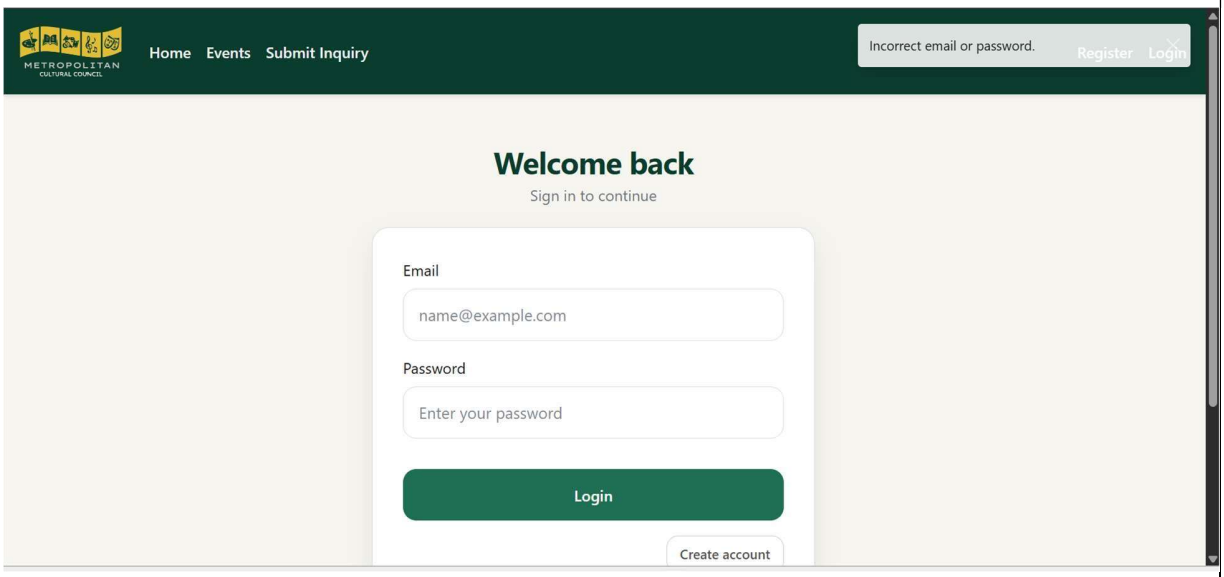
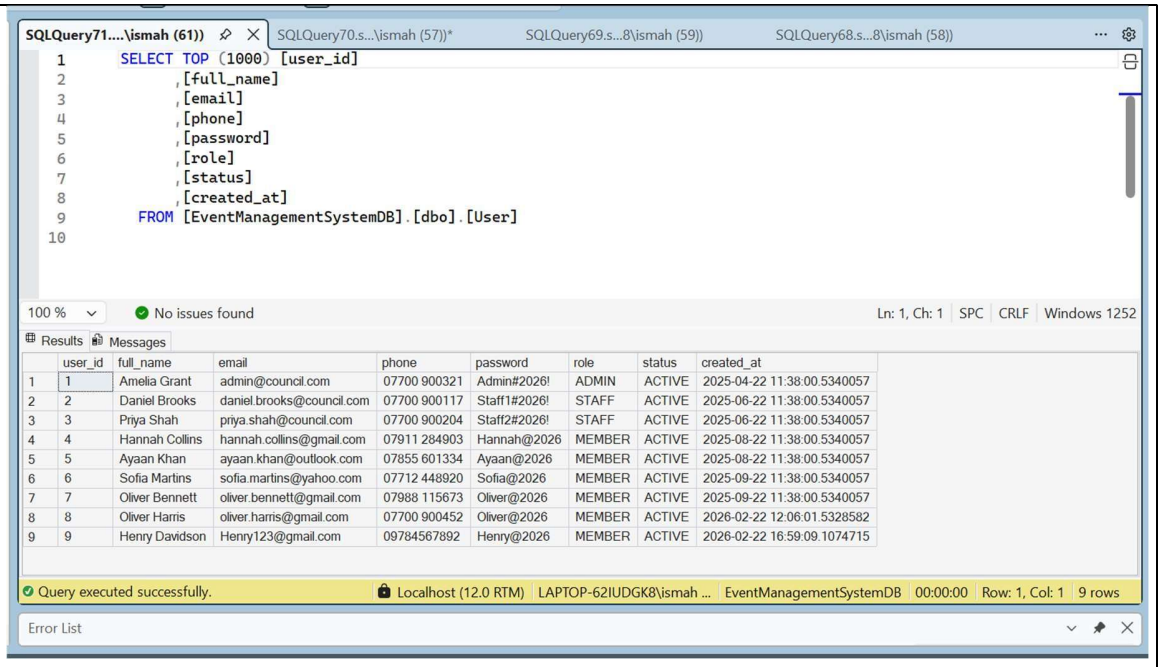
Test ID	ETS-FT02
Test name	Login with invalid credentials
Role	Member / Staff / Admin
Feature	Authentication
Test steps	1) Open Login page 2) Enter invalid credentials 3) Click Login
Expected result	Login is rejected and an error message is displayed.
Actual Result	
	
Status	Pass

Table 15: Testcase Table 02.

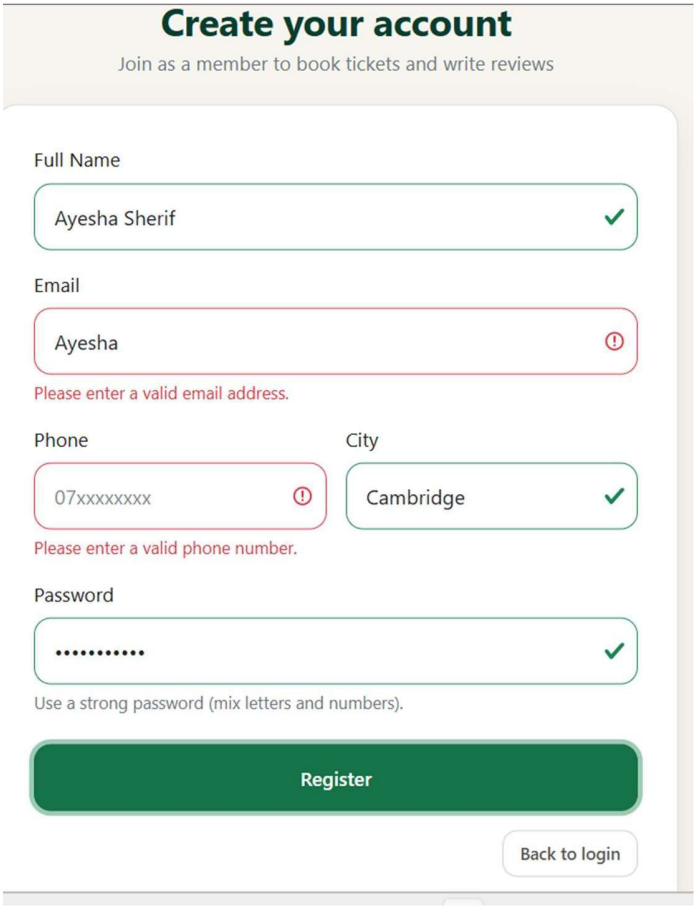
Field	Details
Test ID	ETS-FT03
Test name	Member registration with required fields
Role	Guest
Feature	Registration
Test steps	1) Open Register page 2) Fill all mandatory fields 3) Submit form
Expected result	Account is created successfully with role assigned as MEMBER.
Actual Result	



Status	Pass

Table 16: Testcase Table 03

Field	Details
Test ID	ETS-FT04
Test name	Registration validation for missing fields
Role	Guest
Feature	Registration validation
Test steps	1) Open Register page 2) Leave one or more required fields empty 3) Submit
Expected result	Registration is blocked and validation messages are displayed.
Actual Result	



Status	Pass

Table 17: Testcase Table 04

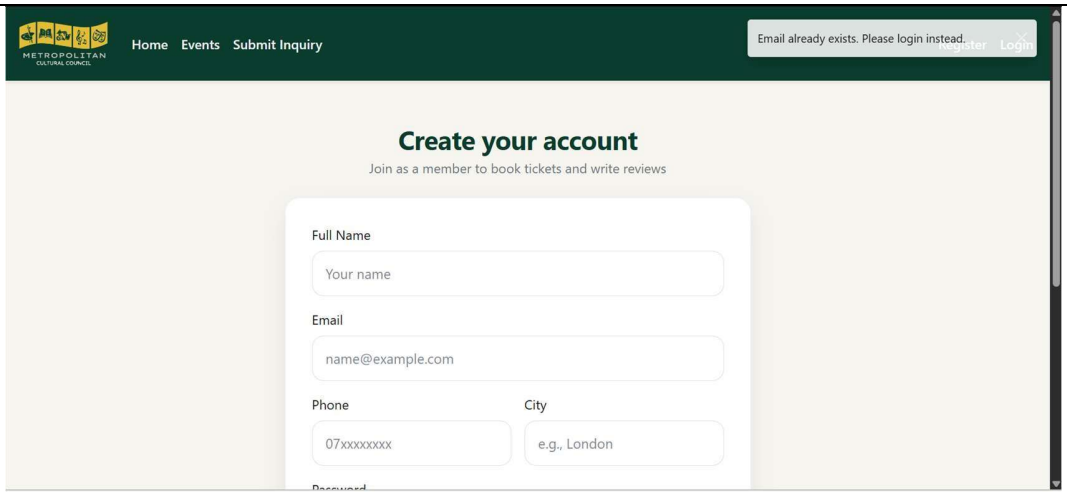
Field	Details
Test ID	ETS-FT05
Test name	Registration email uniqueness enforcement
Role	Guest
Feature	Data validation
Test steps	1) Attempt registration using an existing email 2) Submit
Expected result	Registration fails with an email already exists message.
Actual Result	
	
Status	Pass

Table 18: Testcase Table 05

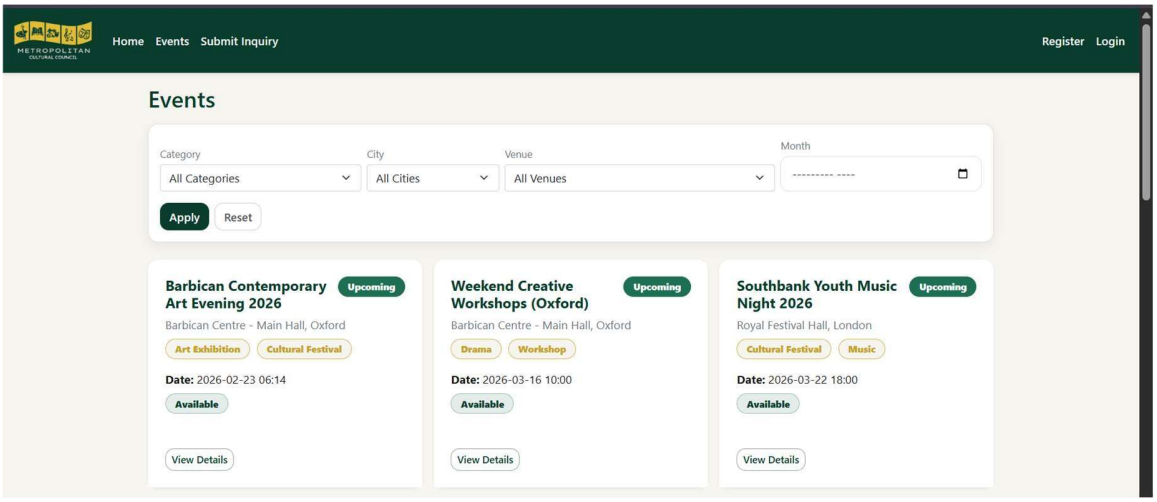
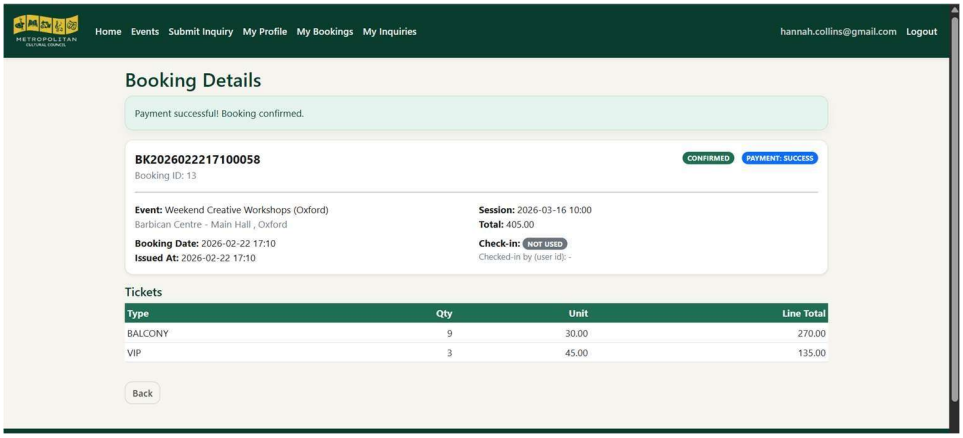
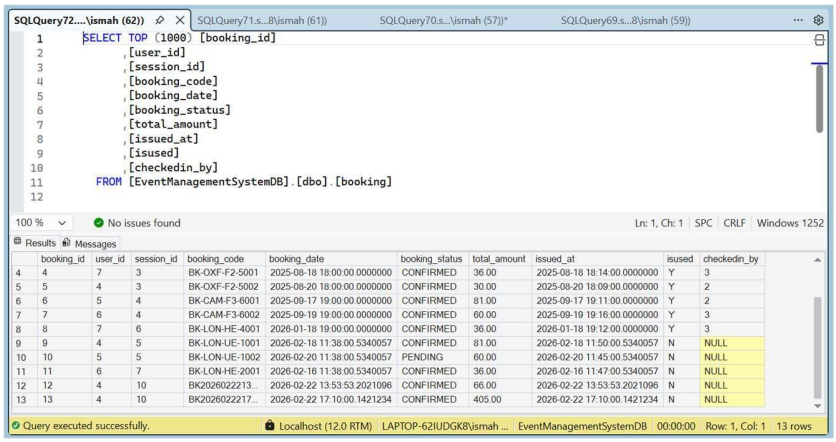
Field	Details
Test ID	ETS-FT06
Test name	Guest event browsing without pricing access
Role	Guest
Feature	Event browsing
Test steps	1) Open Events page as guest 2) View event details
Expected result	Events are visible but ticket prices and booking options are hidden.
Actual Result	
	
Status	Pass

Table 19: Testcase Table 06

Field	Details
Test ID	ETS-FT07
Test name	Member booking with available seats
Role	Member
Feature	Ticket booking
Test steps	1) Login as member 2) Select active session 3) Choose tickets 4) Confirm booking
Expected result	Booking is created successfully and a booking code is generated.
Actual Result	





Status	Pass

Table 20: Testcase Table 07

Field	Details
Test ID	ETS-FT08
Test name	Staff check-in verification success
Role	Staff
Feature	Entry verification
Test steps	1) Enter booking code 2) Enter last two digits of registered phone number
Expected result	Database validates the combination and check-in is successful and database gets updated.

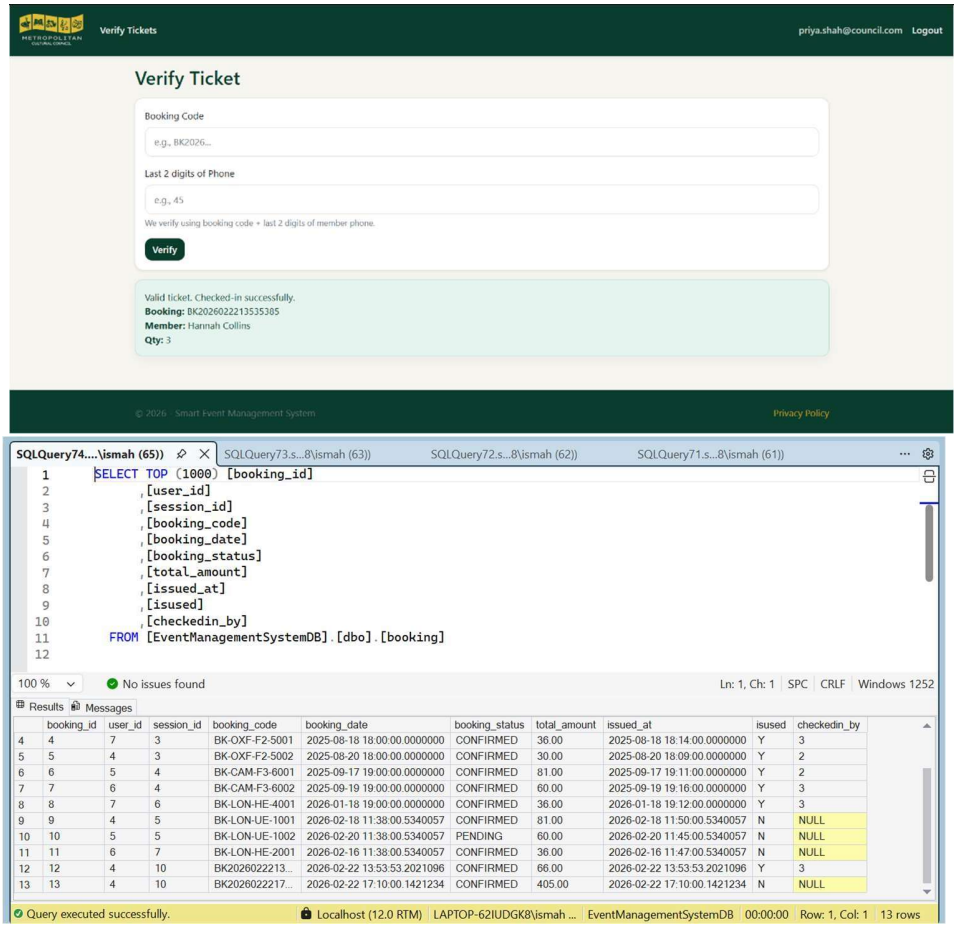
Actual Result

Status | Pass

Table 21: Testcase Table 08

Field	Details
Test ID	ETS-FT09
Test name	Staff check-in verification failure
Role	Staff
Feature	Entry verification
Test steps	1) Enter incorrect booking code or phone digits 2) Attempt verification
Expected result	Verification fails and entry is denied without updating booking status.

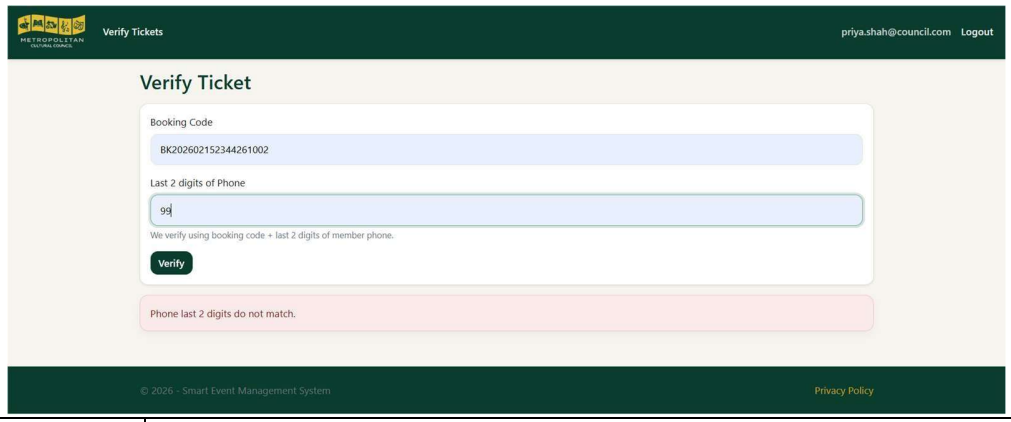
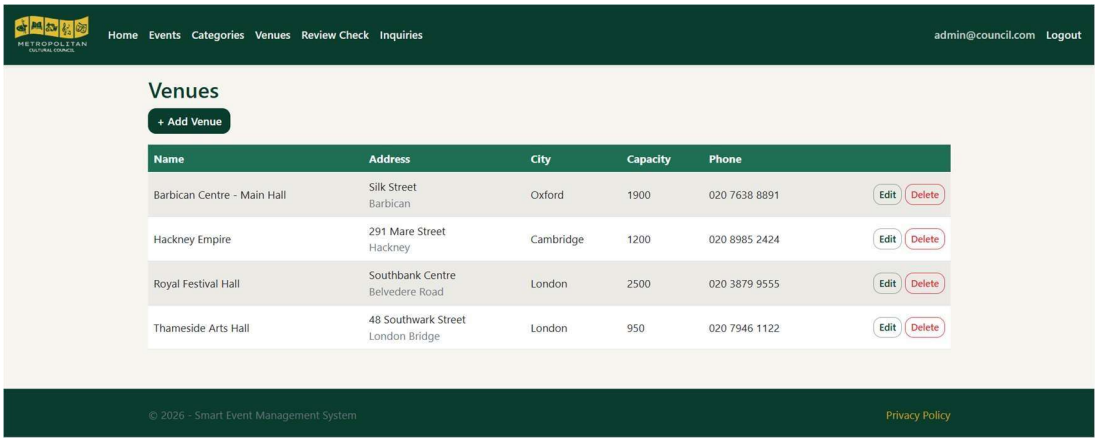
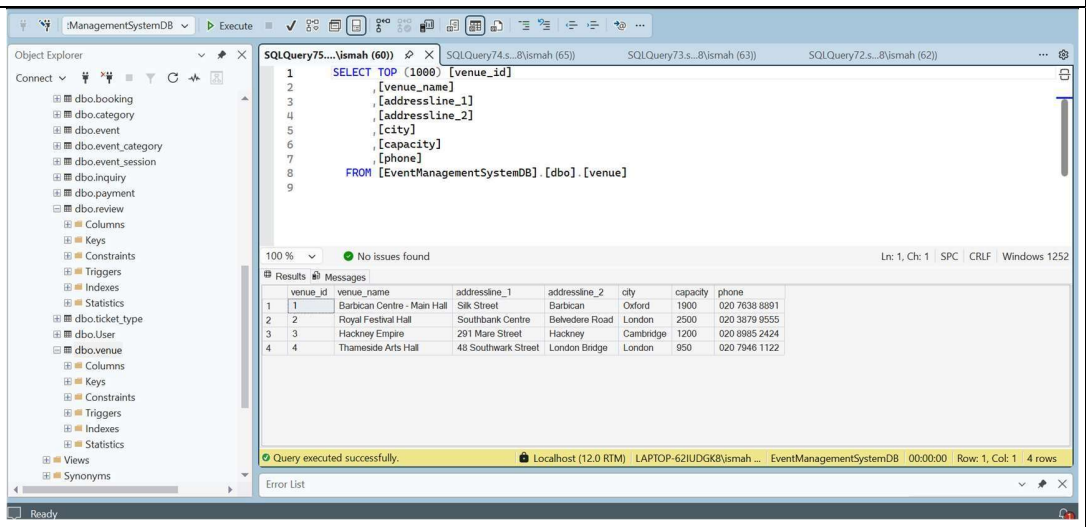
Actual Result	
	
Status	Pass

Table 22: Testcase Table 09

Field	Details
Test ID	ETS-FT10
Test name	Admin deletes a venue
Role	Admin
Feature	Venue management
Test steps	1) Login as admin 2) Select a venue 3) Click Delete

Expected result	Venue is removed or marked inactive based on system rules; dependent data integrity is preserved.
Actual Result	

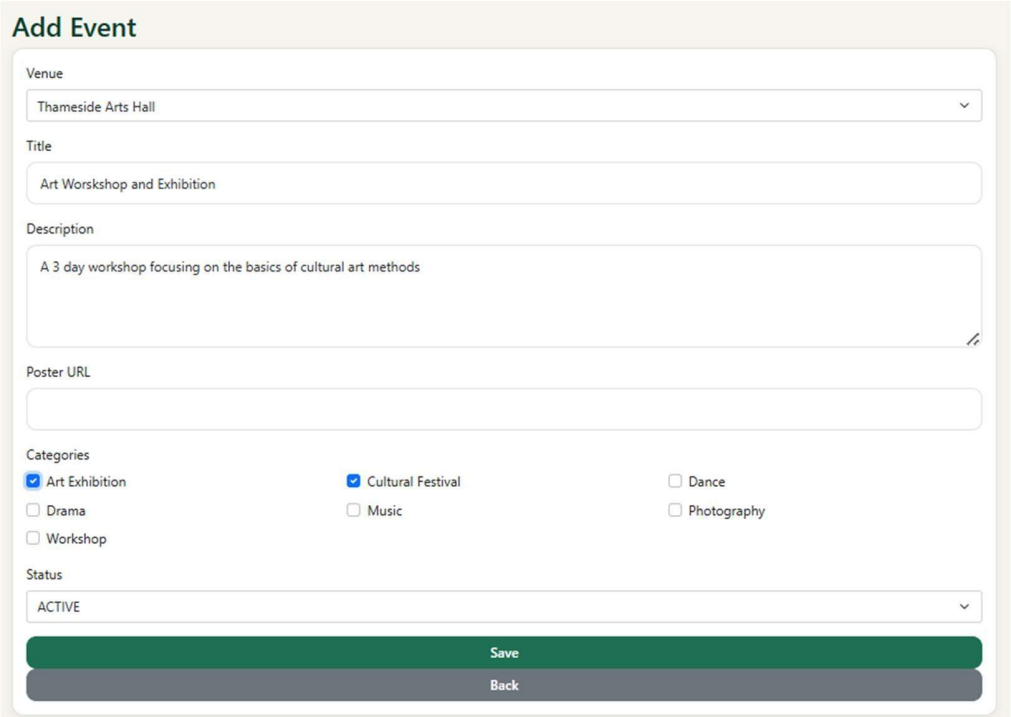


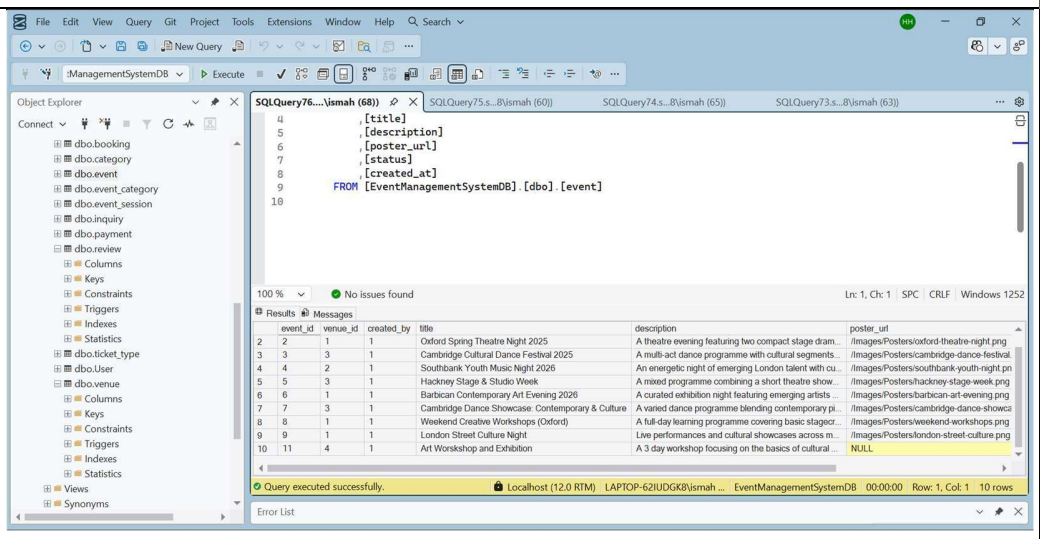


Status	Pass

Table 23: Testcase Table 10

Field	Details
Test ID	ETS-FT11
Test name	Admin creates event with session
Role	Admin
Feature	Event and session management
Test steps	1) Create a new event 2) Add one or more sessions 3) Save
Expected result	Event and associated sessions are stored successfully and appear in event listings.
Actual Result	





Status	Pass

Table 24: Testcase Table 11

Field	Details
Test ID	ETS-FT12
Test name	Admin replies to an unreplied inquiry
Role	Admin
Feature	Inquiry management
Test steps	1) Open inquiries list 2) Select OPEN inquiry 3) Enter reply 4) Submit
Expected result	Reply is saved and inquiry status updates appropriately.
Actual Result	

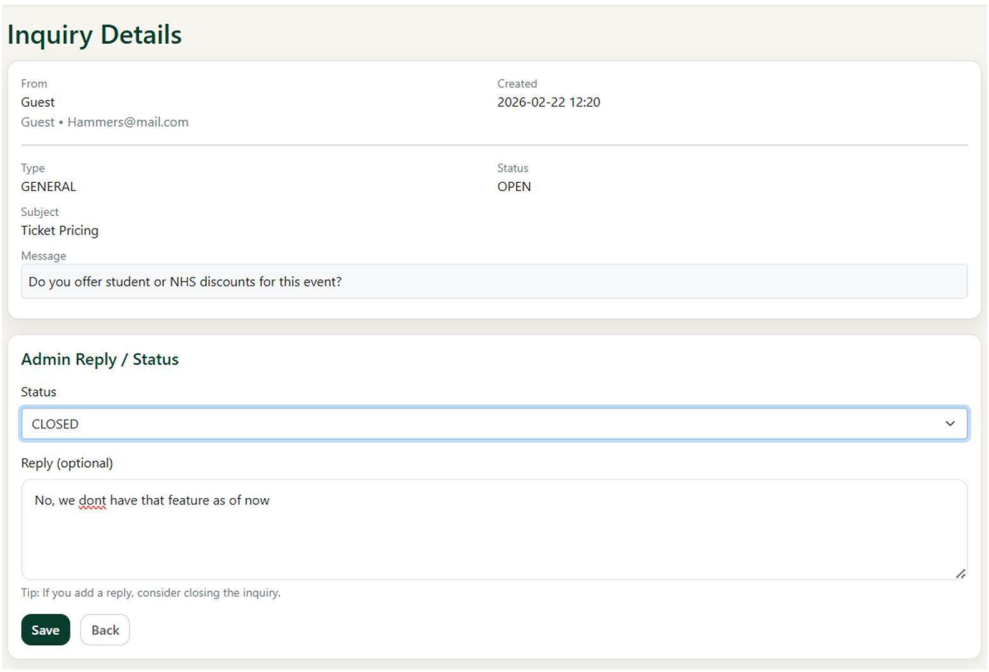
	
Status	Pass

Table 25: Testcase Table 12

2.Database Testing

Field	Details
Test ID	ETS-DBT01
Test name	Insert valid user row
Role	Database (SSMS)
Feature	Insert testing
Test steps	1) Run INSERT into User table with all required fields 2) Execute query

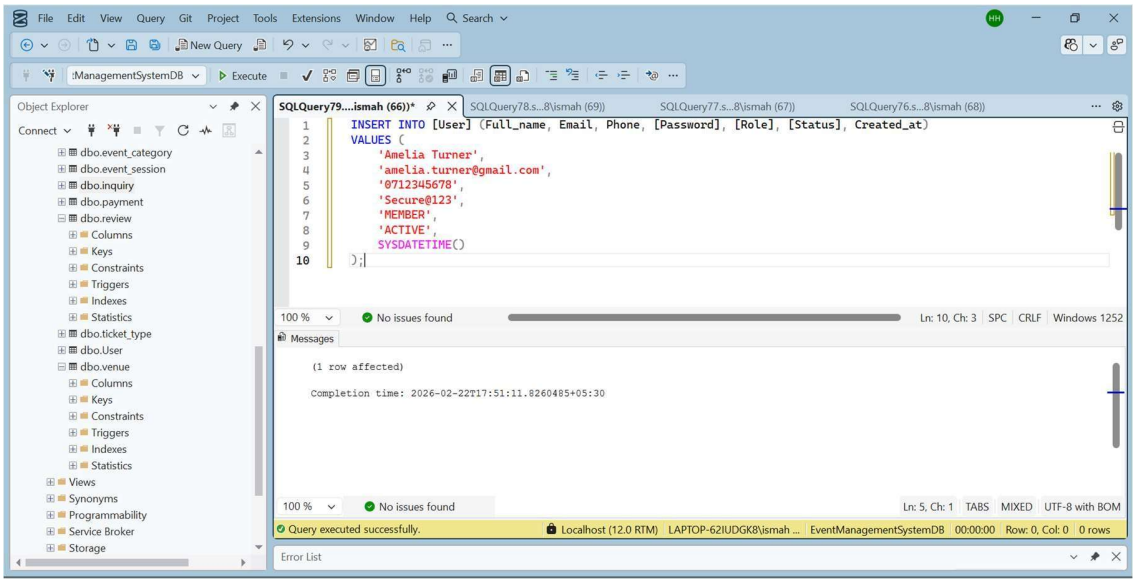
Expected result	Insert succeeds and a new row is created.
Actual Result	
	
Status	Pass

Table 26: Testcase Table 13

Field	Details
Test ID	ETS-DBT02
Test name	Reject insert with NULL in NOT NULL column
Role	Database (SSMS)
Feature	Constraint validation
Test steps	1) Attempt INSERT with a missing required field (NULL) 2) Execute query

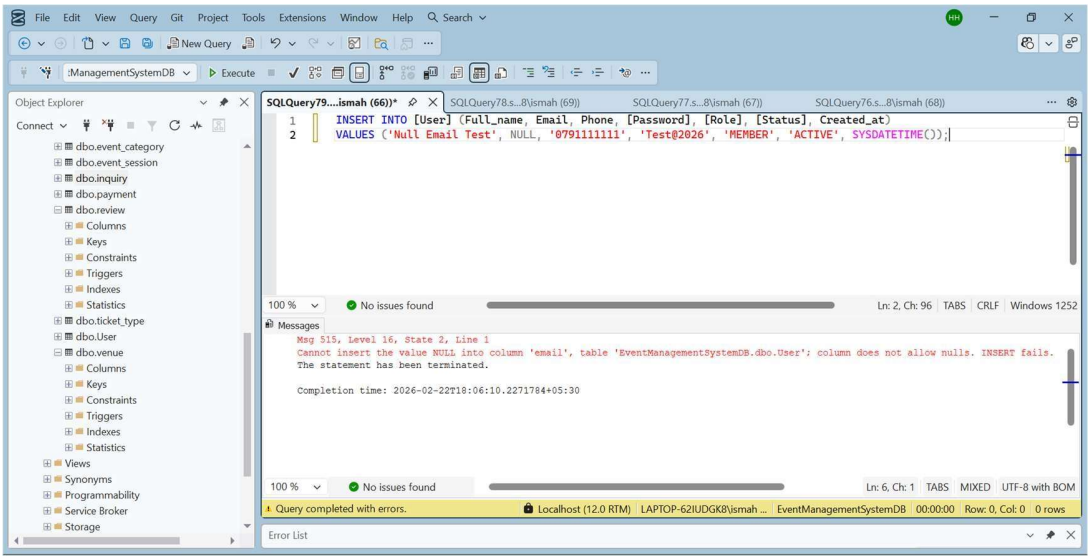
Expected result	Database rejects the insert with a NOT NULL constraint error.
Actual Result	
	
Status	Pass

Table 27: Testcase Table 14

Field	Details
Test ID	ETS-DBT03
Test name	Enforce email uniqueness
Role	Database (SSMS)
Feature	UNIQUE constraint

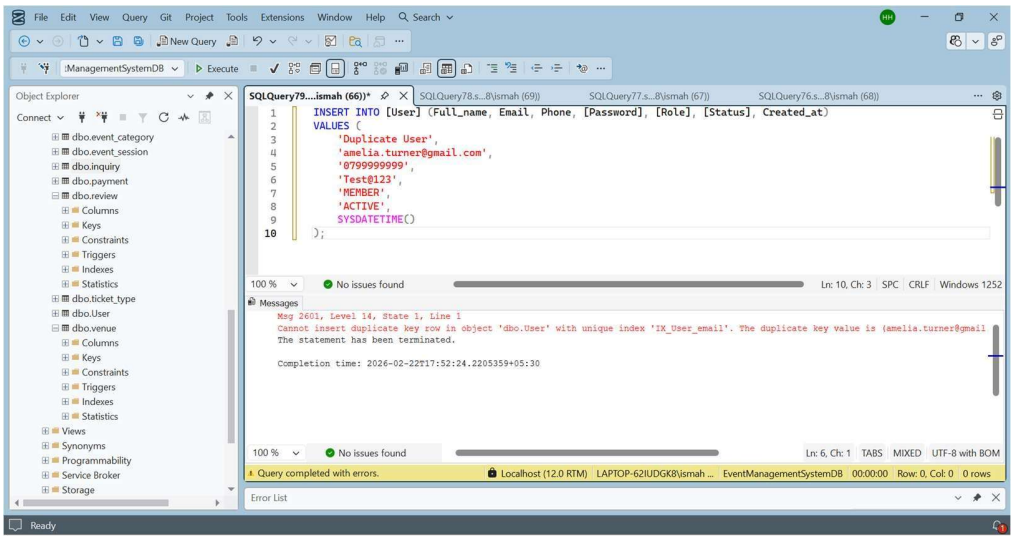
Test steps	1) Insert a user with an existing email 2) Execute query
Expected result	Database rejects the insert due to UNIQUE constraint violation.
Actual Result	
	
Status	Pass

Table 28: Testcase Table 15

Field	Details
Test ID	ETS-DBT04
Test name	Reject orphan foreign key insert
Role	Database (SSMS)

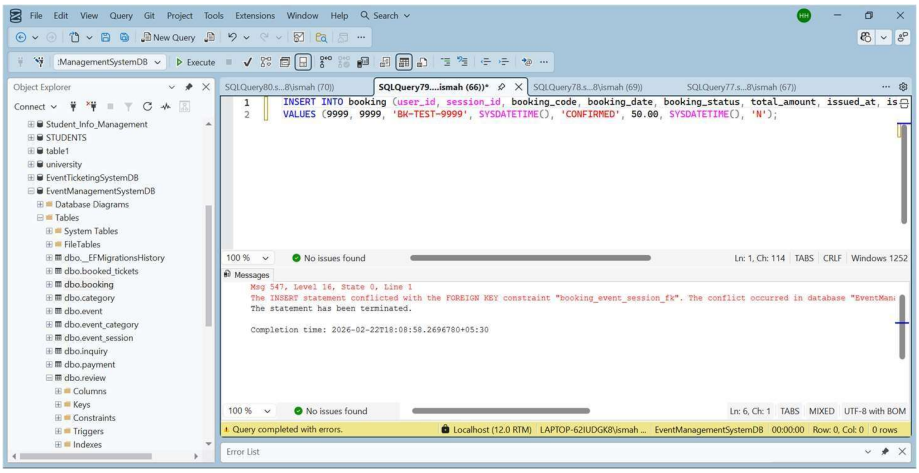
Feature	Foreign key constraint
Test steps	1) Attempt to insert a Booking referencing a non-existing SessionId/UserId 2) Execute query
Expected result	Database rejects the insert due to foreign key constraint violation.
Actual Result	
	
Status	Pass

Table 29: Testcase Table 16

Field	Details
Test ID	ETS-DBT05
Test name	Query upcoming sessions returns correct rows

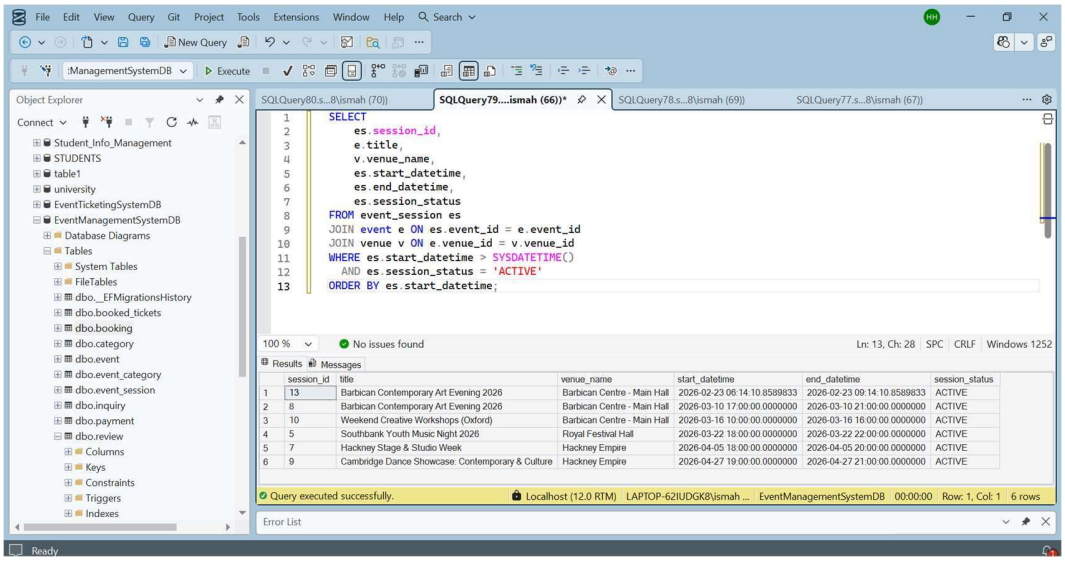
Role	Database (SSMS)
Feature	Query testing
Test steps	1) Run SELECT query filtering sessions by future date and ACTIVE status 2) Execute
Expected result	Query returns only upcoming active sessions with correct joins to events/venues.
Actual Result	
	
Status	Pass

Table 30: Testcase Table 17

Field	Details
Test ID	ETS-DBT06

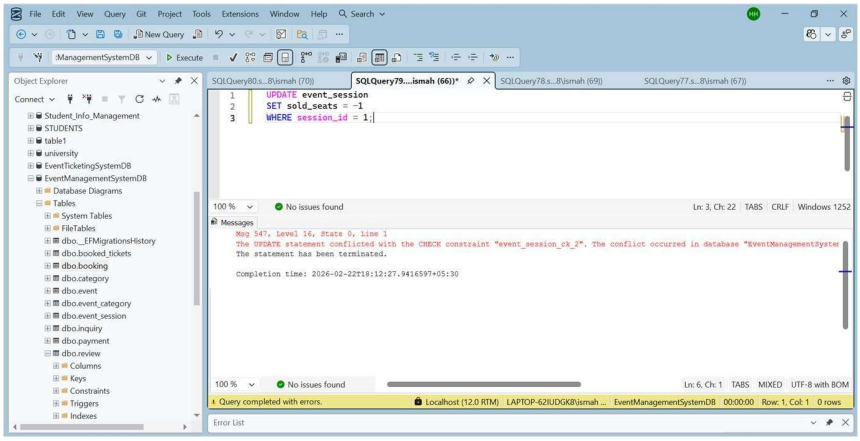
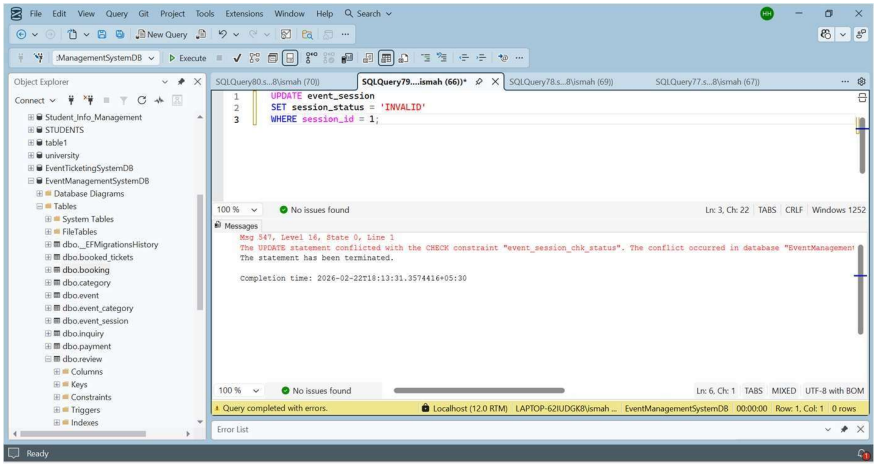
Test name	Validate CHECK constraints for seat counts and statuses
Role	Database (SSMS)
Feature	Data validation
Test steps	1) Attempt to update seats to negative values OR set invalid status text 2) Execute
Expected result	Database rejects invalid updates due to CHECK constraint enforcement.
Actual Result	
	
	
Status	Pass

Table 31: Testcase Table 18

7 Reflection and Further Discussion

7.1 Challenges During Development

One of the most difficult challenges during development is maintaining complex relational data integrity across various interconnected entities such as users, events, sessions, ticket types, bookings, and payments. Maintaining consistency in relationships, especially during booking, payment confirmation, and seat availability updates, necessitates careful design of foreign keys, check constraints, and application-level validation. Any mismatch between application logic and database rules increases the risk of data inconsistency.

Another key challenge is integrating role-based access control throughout the system. Each user role (guest, member, staff, or administrator) has unique rights and workflows that must be properly enforced at both the user interface and controller levels. Preventing unauthorized access to restricted elements including booking, pricing, administrative management, and personnel verification necessitates rigorous session management and conditional rendering.

A further problem is synchronizing application behavior with database constraints. While the database maintains strict rules through constraints, the application must offer useful feedback to users when operations fail. Aligning validation messages, error handling, and business logic with database enforcement increases complexity, especially when dealing with negative circumstances like lost payments, invalid inputs, and overbooking attempts.

7.2 Lessons Learned

The development method emphasizes the significance of clear separation of concerns, especially when utilizing the MVC architectural pattern. Maintaining clear boundaries between models, views, and controllers enhances code readability, simplifies debugging, and facilitates future upgrades. Well-defined duties at each layer make a substantial contribution to a scalable and maintainable system design.

Another critical takeaway is the importance of database-first thinking in transactional systems. Designing strong constraints, associations, and validation rules at the database level offers a solid foundation that protects the system against invalid data even if application-level validation fails. This method boosts confidence in data accuracy and system stability.

The project also emphasizes the value of incremental testing and validation. Testing features as they are implemented, rather than waiting until the end, aids in the early detection of

flaws and reduces integration problems. Role-based testing and database-level testing work together to provide more comprehensive assurance of system correctness.

7.3 System Limitations.

One key disadvantage of the existing technology is the streamlined payment processing technique. While the program allows for payment status tracking and enforces booking confirmation criteria depending on payment success, it does not interact with real-world payment gateways like card processors or digital wallet providers. As a result, payment transactions are simulated, and status updates must be specified or done manually. This reduces the system's suitability for real-world deployment because functionality like real-time payment validation, automated refunds, transaction reconciliation, and fraud prevention are not enabled in the current implementation.

Another constraint is scalability and performance optimisation. The system is intended to perform efficiently under moderate usage settings; nevertheless, it has not undergone significant stress or load testing. High-concurrency instances, such as several customers attempting to book tickets for popular events at the same time, can lead to performance bottlenecks or race conditions. To provide consistent performance in large-scale environments, more optimisation techniques would be required, such as query tuning, indexing schemes, caching mechanisms, and concurrency management.

The system also uses manual administrative workflows for tasks including role assignment, event moderation, review approval, and inquiry processing. While this strategy improves control and security, it may compromise operating efficiency in high-volume settings. As system utilization increases, a lack of automated procedures or prioritization methods may result in delayed replies and increased administrative workloads.

Furthermore, the system is limited to a web-based platform and lacks specialized mobile application support or offline functionality. This may limit accessibility for employees doing onsite check-in activities or consumers who typically access services via mobile devices. To improve usability and real-world deployment, platform support should be expanded and automation improved.

7.4 Potential Future Improvements

One potential future enhancement is the inclusion of a secure online payment channel. Supporting real-time card payments and digital wallet services would improve the system's usability and dependability by allowing for fully automated payment processing. This would enable rapid payment confirmation, automated refund management for cancelled events, and precise transaction records, boosting user trust and making the system more fit for real-world use.

Another proposed change is to improve the staff check-in procedure by incorporating QR code scanning and mobile device support. Replacing manual booking code entry with scannable verification will considerably boost efficiency during high-traffic event sessions while lowering the possibility of human error. Mobile-based authentication would also give workers more flexibility and faster response times at venue access points.

The system may also benefit from the addition of advanced analytics and reporting capabilities. Administrative dashboards that provide booking patterns, income summaries, attendance figures, and event popularity would help with decision-making and strategic planning. Such information would assist event organizers in assessing event performance and optimizing future scheduling and pricing initiatives.

Finally, using automated notification and communication services would boost user engagement and operational efficiency. Email or SMS notifications for booking confirmations, session reminders, event updates, cancellations, and inquiry responses will ensure that users receive timely communication. These features will improve the overall user experience by keeping participants informed and minimizing the need for manual follow-ups.

8.Conclusion.

The Smart Event Management and Ticketing System successfully illustrates a systematic and safe way to administering cultural events using a role-based online application. By merging a relational database with an MVC-based ASP.NET architecture, the system provides explicit separation of responsibilities, consistent data integrity, and limited access to features based on user roles.

The implementation handles critical requirements such event discovery, ticket buying, staff verification, administrative management, and query handling. Strong database constraints and validation procedures assure consistency and accuracy, while systematic testing confirms that both application workflows and database operations function properly.

Overall, the technology lays a solid foundation for a practical event management platform. While some constraints exist, the design allows for future advancements and scalability. The project exemplifies the actual application of software engineering principles, database design methodologies, and safe web development practices in a well-organized solution.

9. Appendix

https://github.com/IsmaHaseemdeen/Smart-Event-Management-and-Ticketing-System

https://drive.google.com/drive/folders/1pWAMhr7kVmlmKmpEQ5WeBdFro3hwPNEl?usp=sharing